La ventana de escritorio

AlgoProject — manual de uso. Actualizado el 2026-10-04 con `tools/manual.py`.

1. 61. Proyectos — la galería de todos los proyectos, y el camino hasta una estrategia
2. 65. Proyecto — el raíl del workflow: mapa, botones de correr y estado
3. 69. Proyecto — filtros sobre un databank, y «Continuar workflow»
4. 68. Estrategia — la ficha de una estrategia: curvas, estadísticas, tests, metadatos y «Archivar»
5. 58. Estrategia — cómo se lee el resultado de un estudio
6. 70. OPERACIÓN — «En marcha», «Registro de búsquedas» y la franja de trabajos
7. 71. Portfolios — las estrategias archivadas, e «Importar»
8. 35. La ventana — cómo se abre, el mapa de sus zonas, y las plantillas
9. 84. La ventana: Investigar — el mapa, la memoria, el director y su propuesta
10. 38. Activos — costes, tramos, MC Retest y Cross Market de cada instrumento
11. 66. Configuración SQX — los ajustes con que nacen los proyectos nuevos
12. 67. Datos — AlgoData, velas, spread real y calidad del feed
13. 60. Cómo se lee la ventana — etiquetas, números, botones, identidad y modo lectura
14. 72. Crear desde la ventana — la plantilla, el proyecto, y el export al terminar cada tarea
15. 95. CRITERIOS — qué tiene que cumplir una estrategia en cada paso para pasar sola

61. Proyectos — la galería de todos los proyectos, y el camino hasta una estrategia

Qué pregunta responde

- **¿Qué proyectos tengo, dónde están y cómo van?** La zona **Proyectos** (grupo PROYECTO de la barra lateral) enseña una tarjeta por proyecto de SQX, esté en el maestro, en el conductor o en el custodio, **tenga informes o no**. Antes, un proyecto recién construido no aparecía en ningún sitio hasta que un estudio le escribía un informe.
- Cada tarjeta dice **símbolo y timeframe**, **cuántas estrategias** tiene en total entre todas sus databanks, **qué plantilla** lo generó y **en qué estado** está: corriendo, parado, terminado o construido.
- **¿Cómo llego de un proyecto a una estrategia?** Clic en la tarjeta → se abre **Proyecto** con ese proyecto cargado (su workflow) → en **Databanks**, que sigue al mismo proyecto, doble clic en una fila del panel → se abre **Estrategia** con su ficha. La barra de arriba (`Proyectos › proyecto › estrategia`) y Ctrl+K llevan a ellas.

Cuándo lo usas, y cuándo no

Lo usas para empezar a mirar cualquier proyecto: es la puerta de entrada al espacio de trabajo.

No sirve para lanzar nada en SQX. La galería lee ficheros; al único worker que está corriendo un proyecto le pregunta su estado con `-project action=status`, que no sincroniza nada. Al maestro no le pregunta nunca: de él solo lee ficheros. Nunca se cuenta una databank con el verbo `count` de SQX, que sincroniza desde disco y borraría lo que el worker solo tiene en memoria.

No enseña los cinco proyectos de fábrica de SQX (Builder, Retester, Optimizer, PortfolioComposer, PortfolioMaster): no son proyectos de nadie (regla 10 de `CLAUDE.md`).

Antes de empezar

Nada. La ventana se abre con `bin/algoui` (o `algoui` desde cualquier terminal, o el icono **AlgoProject** del escritorio). Qué hacer si no abre, y el mapa de todas las zonas de la barra lateral con el capítulo de cada una: capítulo 35.

La primera vez que el demonio responde a la galería tarda **unos 12 segundos**: calcula la identidad de cada estrategia de los tres installs (7 181 el 27-09-2026) para contar las distintas. La ventana no se congela: la galería dice «Leyendo los proyectos de los installs…» y se rellena sola al llegar la respuesta. Las siguientes veces tarda unas centésimas.

Cómo se ejecuta

En la ventana: barra lateral → PROYECTO → **Proyectos**. O Ctrl+K y escribir el nombre del proyecto: la fila dice símbolo, timeframe, estrategias y estado, y Enter abre el proyecto igual que un clic en su tarjeta.

La misma lista, sin ventana:

```
curl -s 127.0.0.1:8765/api/projects/all | python3 -m json.tool
```

Dónde vive una estrategia de un proyecto, por su identidad (lo que usa la ventana al abrir la ficha; la identidad es el SHA-256 que se ve en el tooltip de la estrategia):

```
curl -s "127.0.0.1:8765/api/projects/find?
project=USDJPY&identity=3a34db2c3a4eedb545466b83fb5c190027165e94119f776af00cccd8df0d8d2a"
```

parámetro	obligatorio	qué hace
`project`	sí	el proyecto
`identity`	sí	la identidad de la estrategia
`databank`	no	la databank donde mirar primero; sin él, se miran todas

Nada de esto toca SQX.

Qué produce

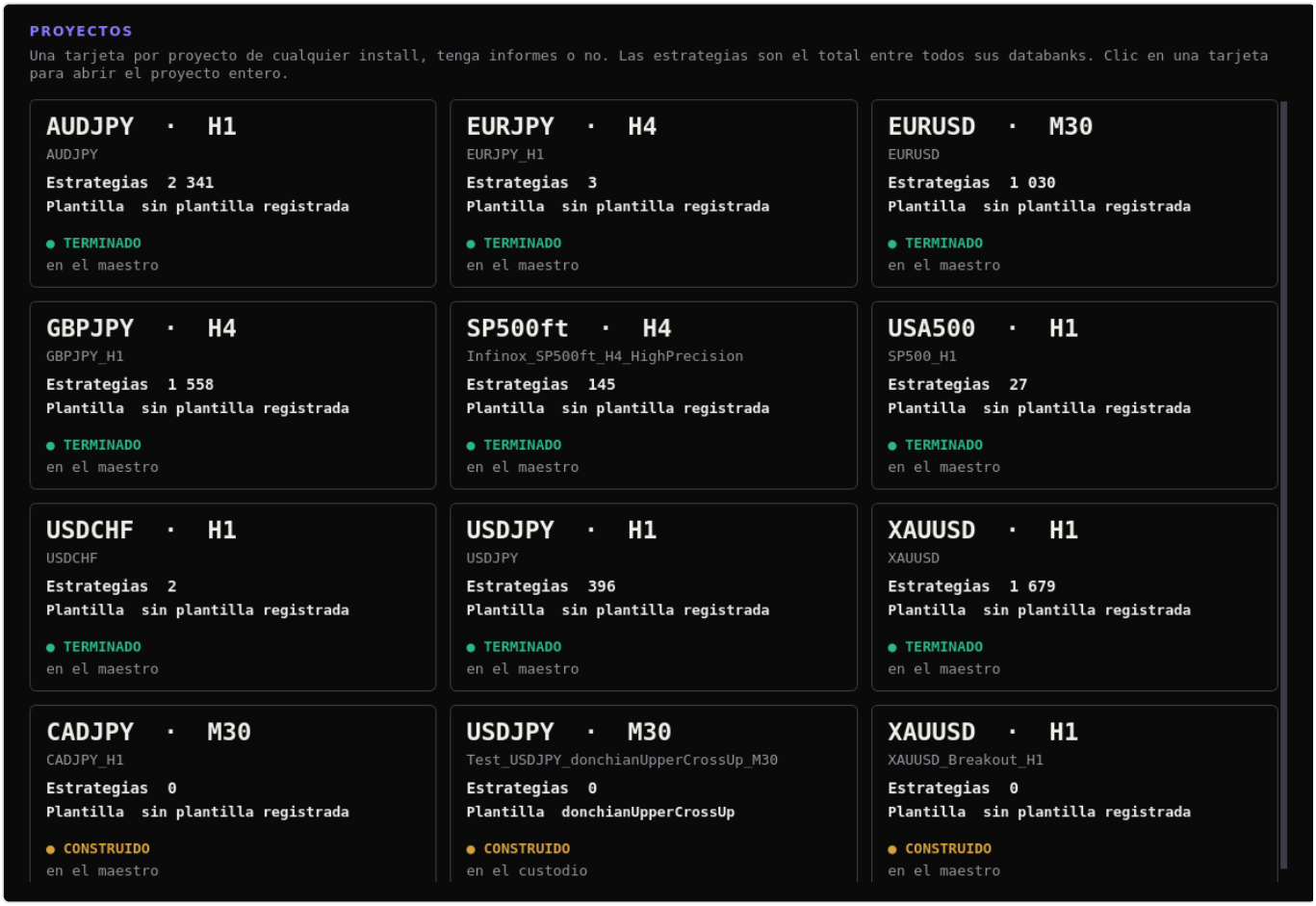
La galería no escribe ningún fichero. Lo que sí hace el **clic en una tarjeta**:

1. **Pregunta antes de cargar.** Una caja lista las databanks con estrategias («SPP OOS (219 estrategias)», …) y recuerda que las operaciones y la cosecha se exportan con `orderstocsv` en el conductor, un trabajo detrás de otro.
2. **Sí:** pide la carga de cada una de esas databanks (`POST /api/load`, lo mismo que elegir una databank a mano; las métricas se leen de los `.sqx` sin SQX), elige el proyecto y, como databank, **la primera que tiene estrategias**, y abre **Proyecto**. La línea de estado dice cuántas se pidieron («USDJPY: carga pedida para 3 de 3 databanks») y la franja de trabajos de abajo va mostrando los exports.
3. **No:** elige solo el proyecto, sin databank (elegir una ya la cargaría), y abre Proyecto sin cargar nada. La línea de estado dice «AUDJPY: elegido sin cargar».

Ctrl+K lista los proyectos que la galería ya ha leído; si la abres justo al arrancar la ventana, antes de que lleguen, dice «Los proyectos aún se están leyendo» en vez de congelarse. Los proyectos recientes se recuerdan solo por su nombre.

El doble clic en una estrategia del panel la elige (proyecto, databank y estrategia) y abre **Estrategia**.

Cómo se lee el resultado



La galería pone tantas tarjetas por fila como caben (de una a cuatro): en una ventana de 1440 de ancho son tres, en una de 1680, cuatro. Nunca aparece barra de scroll horizontal.

Cada tarjeta, de arriba abajo:

- **Símbolo · timeframe**, en grande. Salen de `registry.csv` cuando el proyecto lo creó el builder; si no, del gráfico principal de la primera tarea del `project.cfx`. Por eso `EURJPY_H1` sale **H4**: su primera tarea está configurada en H4, diga lo que diga el nombre.
- **El nombre del proyecto**, en gris.
- **Estrategias**: estrategias **distintas** (por identidad) sumando todas sus databanks. Puede ser menos que los ficheros `.sqx` si hay copias idénticas (XAUUSD: 1 805 ficheros, 1 679 estrategias).
- **Plantilla**: la de `registry.csv`; «sin plantilla registrada» en los proyectos anteriores al registro.
- **El estado**, con su color:

estado	color	qué significa	de dónde sale
CORRIENDO	violeta	SQX lo está corriendo ahora; la tarjeta añade cuántas estrategias lleva procesadas	el log de hoy lo arrancó y no lo ha terminado, y el worker está vivo (su `action=status`)
PARADO	rojo	se paró antes de acabar	el log de hoy dice «Project stopped», o lo dejó a medias y el worker ya no está
TERMINADO	verde	la última corrida acabó	el log de hoy dice «Project finished»; o, si hoy no corrió, sus databanks tienen estrategias
CONSTRUIDO	ámbar	existe, pero ninguna tarea ha dejado estrategias	ninguna databank tiene `.sqx`
AUSENTE	gris	ya no está en ningún install, pero quedan sus informes	solo existe `AlgoData/reports/<proyecto>/`

- **En el maestro / conductor / custodio**: el install donde vive. Si está en un worker y también en el maestro (una copia que el dueño hizo para mirarla), manda la del worker.

Pasa el ratón por una tarjeta: el tooltip dice cómo se supo el estado («El log de hoy dice «Project finished».») y para qué se creó el proyecto (la columna `purpose` del registro).

En **Estrategia**, la barra de arriba dice `Proyectos › USDJPY › Strategy 1.12.28`: cada miga abre su zona. La identidad no se escribe nunca en pantalla: está en el tooltip de la miga de la estrategia. La ficha que se abre ahí (curvas, estadísticas IS/OOS1/OOS2, metadatos y los estudios) tiene su propio capítulo, el 68, con sus capturas.

Un ejemplo completo

El 27-09-2026, con el custodio y el maestro con proyectos:

```
$ curl -s 127.0.0.1:8765/api/projects/all | python3 -m json.tool | head -40
{
    "projects": [
        {
            "name": "AUDJPY",
            "kind": "legacy",
            "purpose": "",
            "template": null,
            "reports": false,
            "symbol": "AUDJPY",
            "timeframe": "H1",
            "strategies": 2341,
            "databanks": {
                "Existing portfolio": 0,
                ...
                "OOS": 726,
                "Results": 1595,
                "RetestMarkets - Structural": 20,
                ...
            },
            "state": "terminado",
            "why": "No corre hoy y sus databanks guardan estrategias.",
            "live": null,
            "install": "master",
            "install_label": "maestro"
        },
```

`AUDJPY` no tiene carpeta en `AlgoData/reports/` (`"reports": false`) y aparece igual: antes no salía en ninguna lista. En la ventana, clic en la tarjeta **USDJPY · H1** → la caja pregunta por SPP OOS, WFM y WFM Stability (las tres con estrategias) → «Sí» → la línea de estado dice «USDJPY: carga pedida para 3 de 3 databanks», la databank elegida es SPP OOS y se abre Proyecto. En Databanks, doble clic en una fila → Estrategia con `Strategy 1.12.28` de la databank WFM.

Qué NO te dice

- **«Terminado» sin log de hoy es una deducción.** SQX no deja un log por proyecto; si el proyecto no corrió hoy, la galería solo sabe que sus databanks tienen estrategias. Pudo pararse otro día.
- **El total no dice nada de calidad**: son estrategias guardadas, no supervivientes. El embudo de Proyecto dice cuántas pasan cada criba.
- **Un proyecto corriendo se cuenta por ficheros**, no por identidad: mientras SQX escribe, un `.sqx` puede estar a medias y no se abre.

Si algo falla

- **La galería se queda en «Leyendo los proyectos de los installs…» más de medio minuto**: el demonio no responde. Ciérrala y ábrela, o arranca el demonio a mano (`python3 -m ui.daemon.serve`) para ver su error.
- **«El demonio no respondió: …» bajo el título**: el demonio se cayó o no está levantado.
- **La línea de estado dice «el demonio no aceptó …» tras un clic**: esas databanks no se pudieron pedir; elígelas a mano y usa el ↻ de la barra de carga.

- **«ninguna databank de guarda esa estrategia»**: la estrategia se borró o se movió de databank desde que se pintó el panel; recarga el proyecto.

65. Proyecto — el raíl del workflow: mapa, botones de correr y estado

Cambio del 2026-09-28. La regla de «no quemar el oos2» ya no se aplica a humanos: en la ventana nada sale sellado y el paso 20 no espera a 17, 18 y 19. Sólo un agente autónomo (`ALGO_AUTONOMOUS=1`) ve los estados «sellado» y la puerta cerrada que describen algunos párrafos y capturas de abajo.

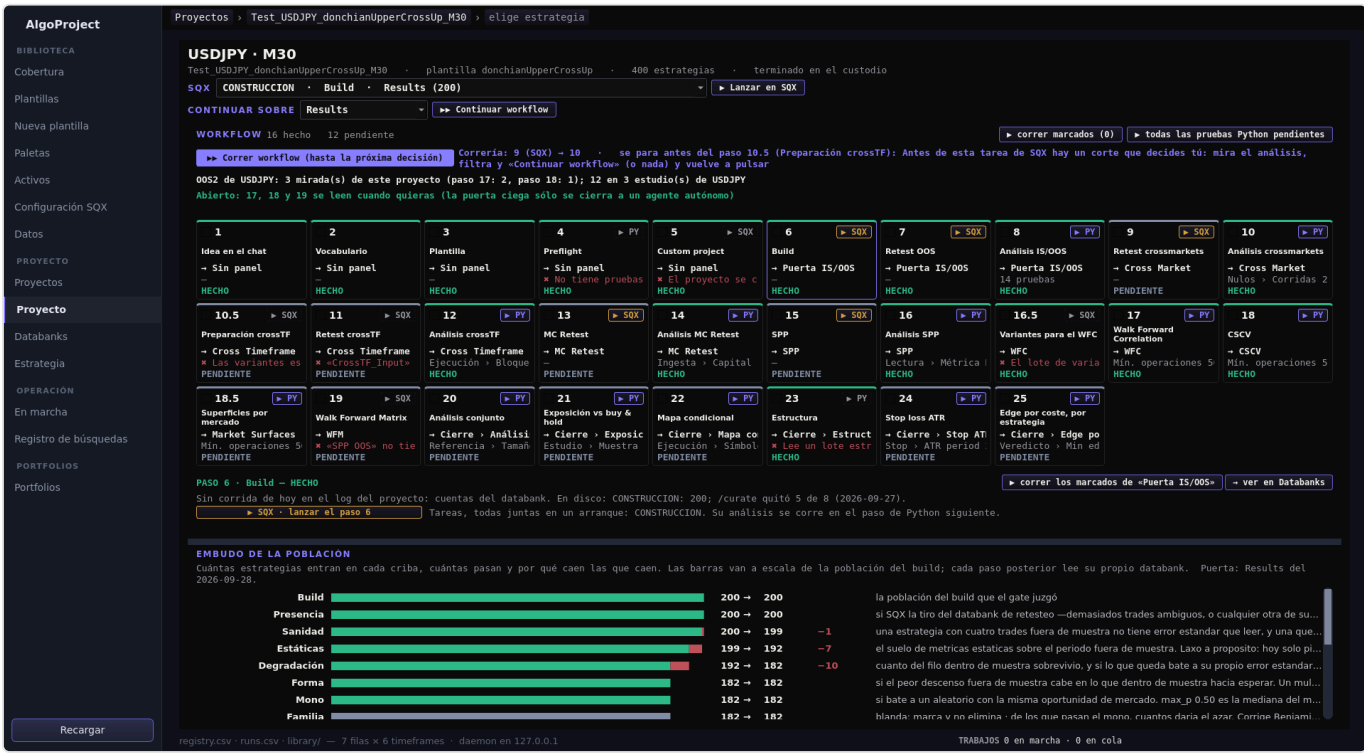
Cambio del 2026-09-28 (tarde): Proyecto se parte en dos zonas. Todo junto no cabía en una pantalla normal. En la barra de la izquierda, el grupo PROYECTO es ahora **Proyectos** → **Proyecto** → **Databanks** → **Estrategia**:

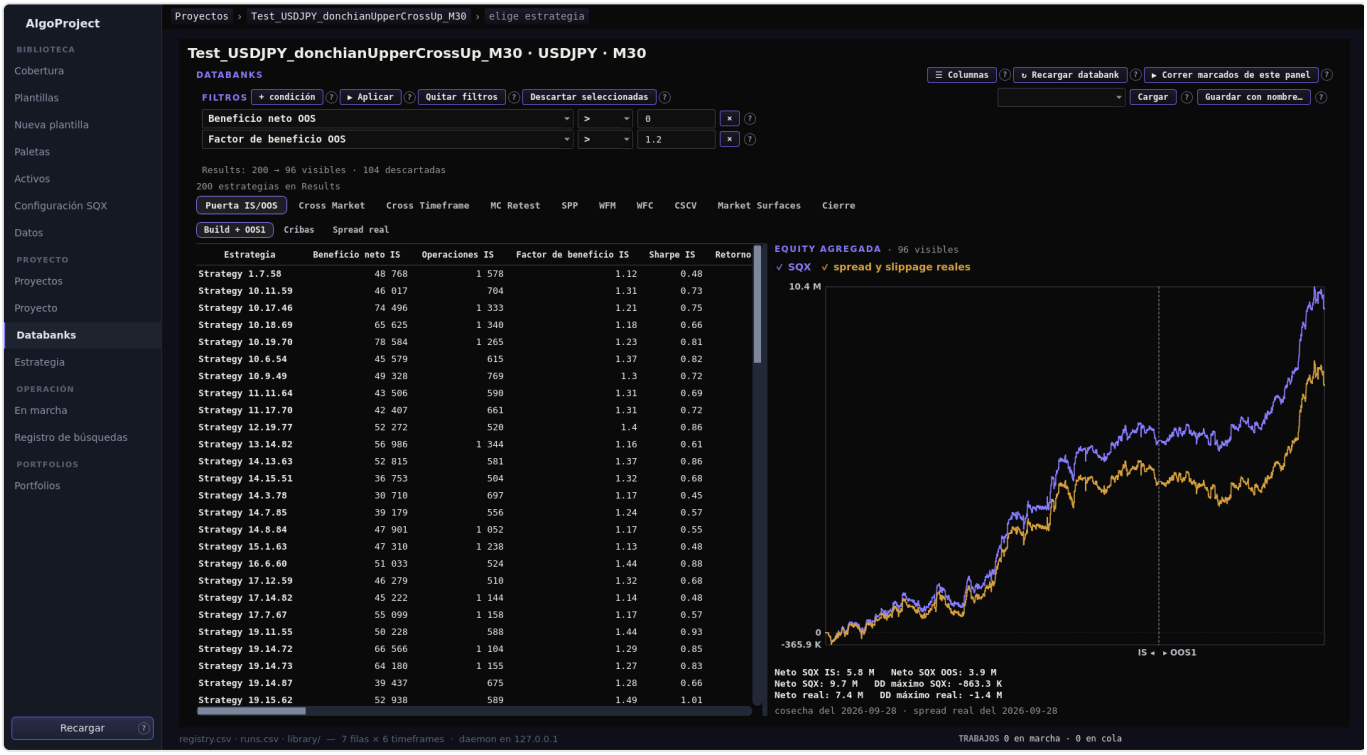
- **Proyecto** es el workflow: el título y sus datos, la fila **SQX** («▶ Lanzar en SQX»), la fila **CONTINUAR SOBRE** (su desplegable de databank y «▶▶ Continuar workflow»), el raíl con su cajón y el **embudo**.
- **Databanks** es sólo el panel de databanks, con toda la altura: la tira FILTROS, las pestañas, la tabla (sin equity agregada en ninguna pestaña desde el 2026-09-29), «↻ Recargar databank» y «▶ Correr marcados de este panel».

Las dos siguen al mismo proyecto: el que eliges en la galería o en la barra de arriba. Un clic en una tarjeta del raíl **no te cambia de zona**: deja elegida la pestaña de ese paso en Databanks, y el botón → **ver en Databanks** del cajón te lleva allí. Donde este capítulo dice «el panel de abajo», ahora es la zona Databanks.

Cambio del 2026-09-28 (noche): lo que pediste sobre la zona Proyecto.

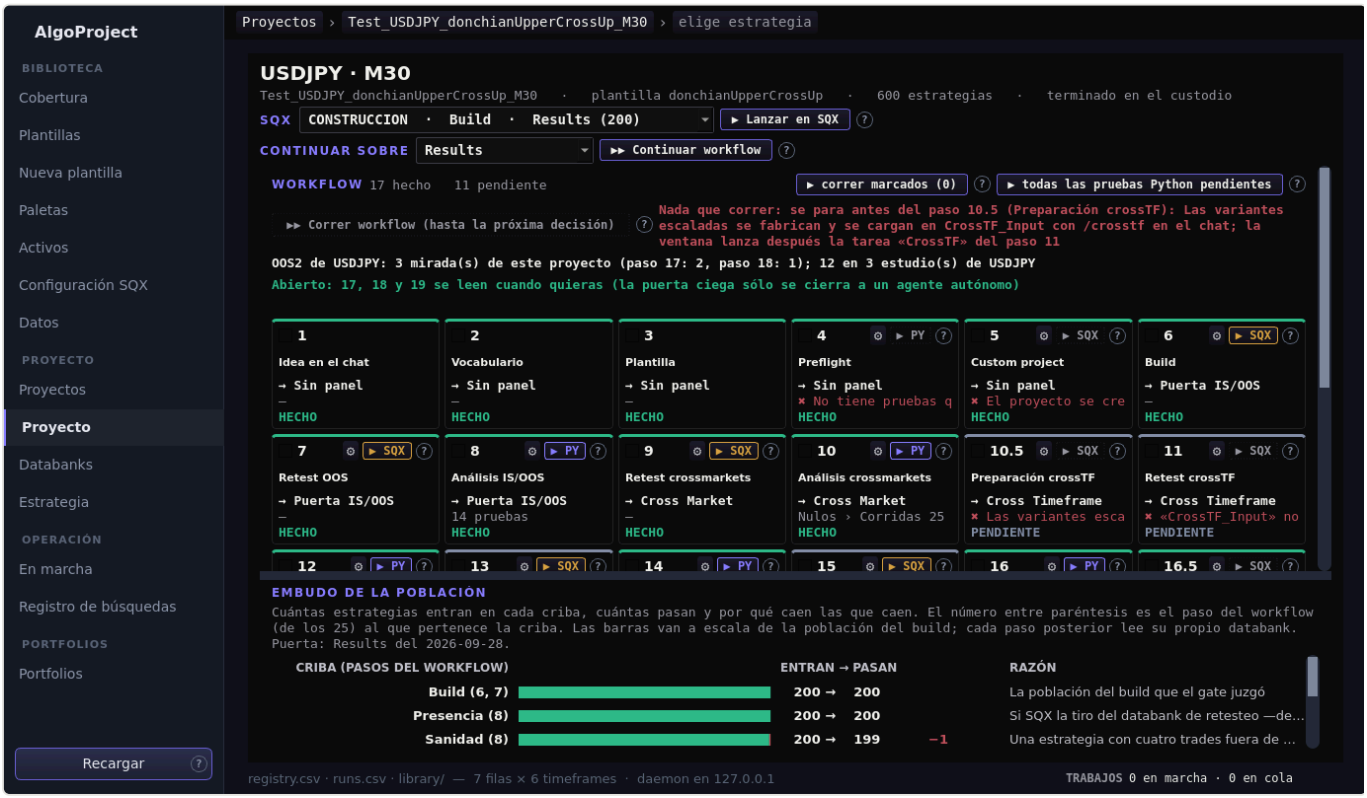
- **⚙ en cada tarjeta**, a la izquierda de «▶ SQX» / «▶ PY»: te lleva a la pantalla donde se edita la configuración de ese paso (ver «El engranaje ⚙» más abajo). La flecha de lanzar sigue igual.
- **Abajo del cajón, qué hay que lanzar antes**: «ANTES HAY QUE LANZAR: paso 10.5 (Preparación crossTF) — pendiente» en rojo, o «REQUISITOS HECHOS: ✓ …» en verde.
- **«▶ Lanzar todos (N)»** encima de las pruebas de un paso que tiene más de una prueba de Python.
- El botón del cajón del paso 6 dice **«▶ SQX · Activar el builder»** (el paso sigue llamándose Build), y su «?» ya no tapa la frase de al lado.
- El paso 4 explica en el cajón **qué es el preflight** y qué comprueba.
- «configuración» enseña los valores **con los que correrá para este proyecto**: el feed y el timeframe del proyecto donde la ventana los pone, y un ⚠ donde no (ver «USDJPY y los XAUUSD del config.yaml»).
- El paso 17 dice en el cajón **qué mira la Walk Forward Correlation**.
- El **embudo** lleva una cabecera («Criba (pasos del workflow) · Entran → pasan · Razón»), el paso de cada criba entre paréntesis — «Estáticas (8)» — y cada frase de la razón empieza en mayúscula.





Qué pregunta responde

- **¿En qué paso está este proyecto?** Arriba de la zona Proyecto hay una tarjeta por cada fila de `WORKFLOW.md` (los 25 pasos más los tres medios: 10.5, 16.5 y 18.5, **28 tarjetas**). Cada una dice su estado con una palabra y un color: **hecho** (verde), **en marcha** (violeta), **pendiente** (gris), **bloqueado** (rojo), **sellado** (ámbar, sólo para un agente autónomo: 17, 18 o 19 terminados cuyo resultado no se enseña hasta que el paso 20 se abra) y **sin dato**. Las tarjetas se reparten en tantas por fila como quepan (de 3 a 10: nueve a 1920 px, seis a 1280) y el raíl se desplaza hacia abajo, así que ninguna se sale de la pantalla (2026-09-28). Al abrir el proyecto el raíl toma la altura de sus tarjetas, hasta el 70 % de la zona; si arrastras la banda, se queda donde la dejes.



- **¿Dónde miro su resultado?** Cada tarjeta dice a qué pestaña del panel de databanks va («→ Puerta IS/OOS», «→ Cierre › Stop ATR»). Al hacer clic en la tarjeta, debajo del raíl se despliega el **cajón** con las pruebas de ese paso, y la zona **Databanks** deja elegida esa pestaña; **→ ver en Databanks**, en el cajón, te lleva a ella. - **¿Qué puedo correr desde aquí, y con qué configuración?** En el cajón, cada prueba (cada estudio de Python del paso) tiene su casilla, su estado, su configuración en una línea, el botón «configuración» que despliega todos sus ajustes con su explicación, y su ▶. - **¿Cuándo puedo leer el paso 20?** Cuando quieras: la línea de arriba sale verde, «abierto: 17, 18 y 19 se leen cuando quieras». Un agente autónomo, en cambio, ve una línea ámbar: «SELLADO — el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19] … (en el estudio USDJPY_M30_donchianUpperCrossUp)». La puerta mira **un solo estudio del ledger**: el de símbolo, timeframe y plantilla del proyecto, el mismo que lee el análisis del paso 20. Filas apuntadas bajo otro nombre (por ejemplo, pruebas con el nombre del proyecto como familia) no la abren. Mientras falte uno, la tarjeta 20 está **bloqueada** y su prueba no se puede lanzar.

Cuándo lo usas, y cuándo no

Lo usas para ver de un vistazo qué le falta a un proyecto y para lanzar sus análisis de Python sin escribir comandos: los estudios se reparten solos en paralelo (un trabajo por carril y, dentro, una estrategia por núcleo físico).

Cada paso se corre desde su tarjeta (cambio del 28-09-2026). Los de Python llevan ▶ **PY** (violeta) y los de SQX ▶ **SQX** (ámbar): el ▶ SQX lanza **todas** las tareas de ese paso juntas en un solo arranque del worker del proyecto (las ocho del MC Retest, las dos del SPP…), y el worker se para al acabar. Un botón que no se puede pulsar sale **gris y con borde de rayas**. Desde el 2026-09-29 el rojo con «✖» se guarda para un fallo de verdad (el worker ocupado, el databank de entrada vacío); un botón desactivado por lo esperado sale con un punto «·» en el color normal de la tarjeta (un botón desactivado no enseña su ayuda, así que la frase igual va debajo). Tres pasos de SQX no se lanzan desde la ventana porque no son una tarea que arrancar: el **5** (el proyecto se crea con el builder), el **10.5** (las variantes escaladas se fabrican solas al lanzar el **11**) y el **16.5** (el lote de variantes se fabrica con `/variants`); su ▶ SQX sale gris con esa frase, en el punto neutro y no en rojo, porque no es un fallo — es cómo funciona ese paso.

Para correr el workflow entero está **▶▶ Correr workflow (hasta la próxima decisión)**, el botón violeta relleno de la segunda fila de la cabecera; a su derecha dice qué correría ahora y dónde se pararía. Está explicado abajo, en «Correr workflow».

La fila **SQX** bajo el título del proyecto («Lanzar en SQX», ver abajo) sigue lanzando **una** tarea suelta, la que tú elijas; «Continuar workflow», en la fila CONTINUAR SOBRE justo debajo, corta lo que filtraste en el databank que elijas en su desplegable y lanza la tarea siguiente (capítulo 69). Todas piden confirmación. El análisis que cuelga de un paso de SQX (la nube de parámetros del 16.5, el análisis del WFM del 19) no se lanza desde el raíl: se lee después de la tarea y se corre desde el panel de databanks.

Los pasos 21 a 25 leen las supervivientes y el raíl no sabe en qué databank están: su **▶ PY** corre sobre **el databank que tenga abierto el panel de la zona Databanks** (y sólo las filas seleccionadas, si hay), lo mismo que el botón «correr los marcados de este panel». Sin databank abierto, dice «elige en el panel el databank que lee».

Antes de empezar

- La ventana abierta (`bin/algoui`) y el proyecto elegido en la galería.
- Lo que cada estudio lee tiene que existir en `AlgoData`: la cosecha, el export de operaciones, el lote de variantes… Si falta, el botón no falla: dice qué falta y con qué skill se consigue.
- Para los estudios que apuntan su mirada en el ledger (SPA/StepM del paso 8, WFC, CSCV, superficies por mercado y el paso 20), el proyecto tiene que tener **plantilla** en `AlgoData/projects/registry.csv`: la familia del ledger es el nombre de esa plantilla. Sin plantilla, esas pruebas salen **bloqueadas** con la frase «el proyecto … no tiene plantilla en projects/registry.csv».

Cómo se ejecuta

No hay comando: son botones.

botón	dónde	qué corre
casilla de la tarjeta	cada tarjeta	marca o desmarca todas las pruebas del paso que se pueden correr
▶ **PY** de la tarjeta	pasos de Python	las pruebas marcadas de ese paso; si no hay ninguna marcada, pregunta primero (con la lista, «No» por defecto) y corre **todas** las suyas que no estén en marcha, salvo las que leen oos2 o escriben en el ledger (en 17, 18 y 20, que sólo tienen de esas, esas, con su pregunta)
▶ **SQX** de la tarjeta, o ▶ **SQX · lanzar el paso N** del cajón (en el 6, ▶ **SQX · Activar el builder**)	pasos de SQX	todas las tareas de ese paso, juntas, en un solo arranque del worker; pregunta antes
casilla y ▶ de una prueba	cajón	solo esa prueba
▶ **Lanzar todos (N)**	cajón, encima de las pruebas, en un paso con más de una de Python	todas las pruebas de ese paso que se pueden correr aquí, a la vez; antes pregunta por las que leen oos2 o escriben en el ledger
⚙ de la tarjeta	pasos de SQX y de Python	no corre nada: abre la pantalla donde se edita la configuración del paso
▶ **correr los marcados de «pestaña»**	cajón, y la cabecera del panel de databanks	las pruebas marcadas de todos los pasos cuyo resultado vive en esa pestaña
▶ **correr marcados (N)**	cabecera del raíl	todas las pruebas marcadas del proyecto
▶ **todas las pruebas Python pendientes** (antes «correr todo»)	cabecera del raíl	toda prueba gratuita de Python que aún no tiene resultado, marcada o no (nunca las que leen oos2 o escriben en el ledger, ni 21-25); no toca SQX
▶▶ **Correr workflow (hasta la próxima decisión)**	segunda fila de la cabecera	SQX y Python en el orden de WORKFLOW.md hasta el primer corte que decides tú; pregunta antes
Rehacer las filas de 17-19	cabecera, solo cuando hace falta	`python3 -m ledger.backfill --blind <proyecto> … --write`

Por defecto salen marcadas solo las pruebas **gratuitas** pendientes de los pasos pendientes. **Nunca salen marcadas, ni entran en «todas las pruebas Python pendientes» ni en «Correr workflow»**: las que leen el oos2 o escriben una fila en el ledger (WFC, CSCV, superficies por mercado, el análisis del WFM, el SPA/StepM del paso 8 y el paso 20), ni las de los pasos 21-25 (el raíl no sabe en qué databank están sus supervivientes). Esas se marcan a mano o se lanzan con su ▶, y antes de correr la ventana **pregunta** con una ventanita que dice qué gasta cada una («lee el oos2 y escribe su fila en el ledger»); si dices que no, no se lanza nada. En el cajón llevan delante el aviso «⚠ …» en mayúsculas.

Cada prueba decide sola en qué databank corre: el de su último resultado; si no tiene, el databank que produjo el paso de SQX anterior (para el 8, el del retest OOS y luego el del build). Un estudio de población corre una vez; uno que lee una sola estrategia corre una vez por estrategia del databank, todas en cola a la vez. WFC, CSCV y superficies corren una vez por madre con lote de variantes.

La cola deja arrancar trabajos mientras quedan núcleos y **al menos 20 GB de RAM libre**, la reserva de Python del reparto del dueño; lo demás espera su turno en el orden pulsado.

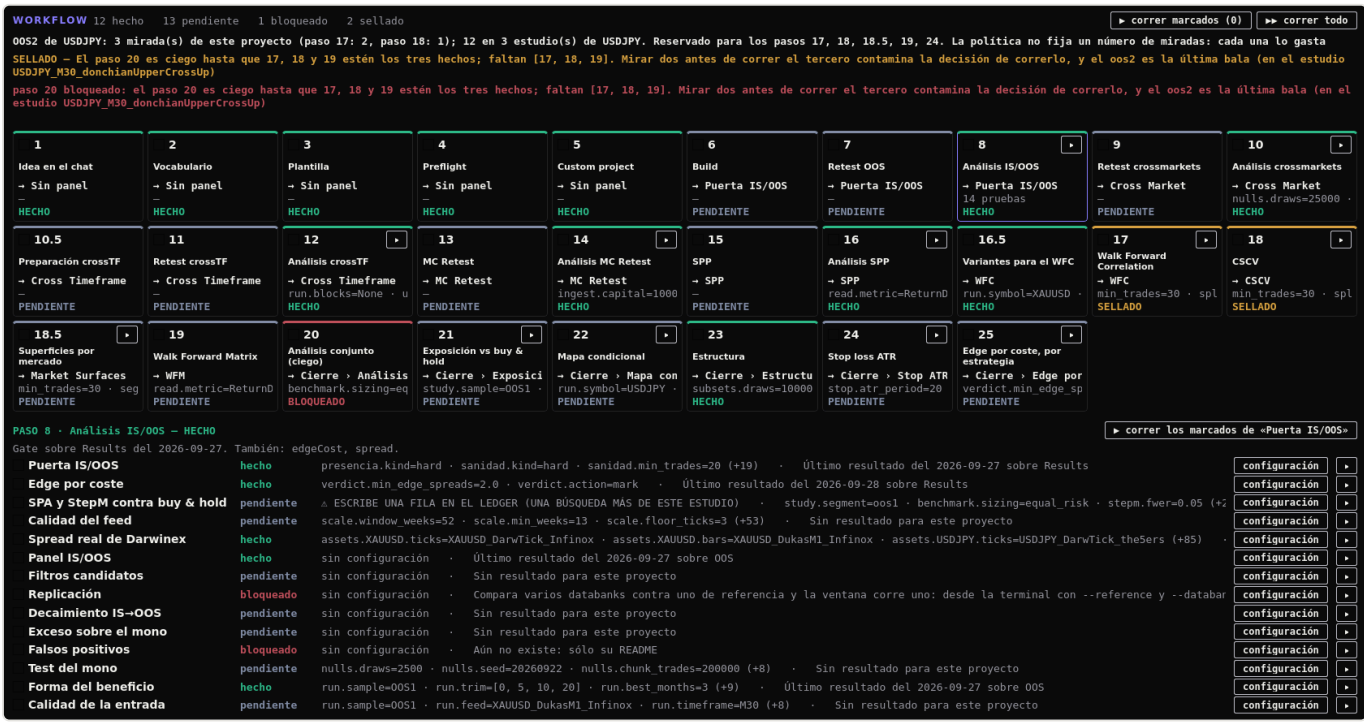
Los botones de Python no tocan SQX. Tarda lo que tarde cada estudio (un `profitShape` de una estrategia, 1-2 s).

Qué produce

Lo mismo que el estudio lanzado a mano: su carpeta en `AlgoData/reports/<proyecto>/<databank>/<día>/<estudio>/` (o en `strategyPermutations/<proyecto>/<madre>/estudios/` para WFC, CSCV y superficies) y el log del trabajo en `AlgoData/logs/ui/`. «Rehacer las filas de 17-19» **añade** filas al ledger del estudio (símbolo × timeframe × familia); no reescribe ninguna, y si ya existen se niega.

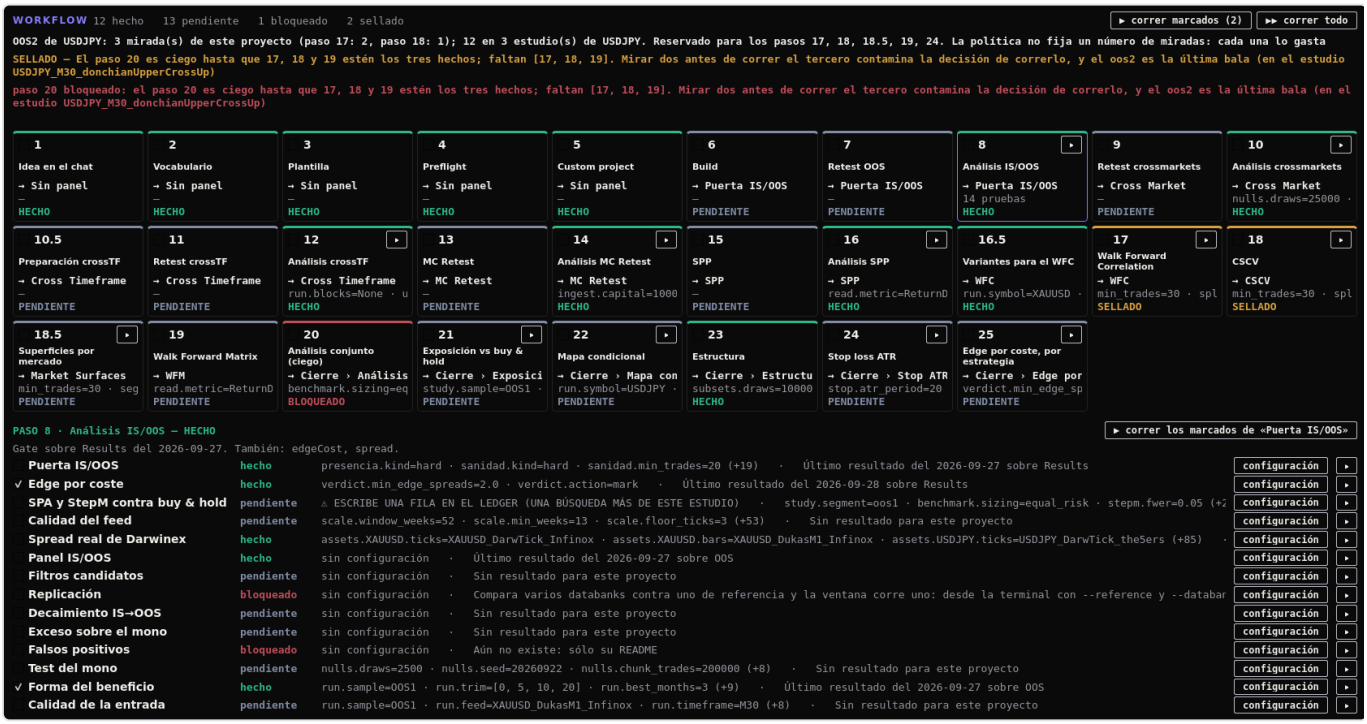
Cómo se lee el resultado

El raíl del proyecto `Test_USDJPY_donchianUpperCrossUp_M30`, con el paso 8 abierto en el cajón:

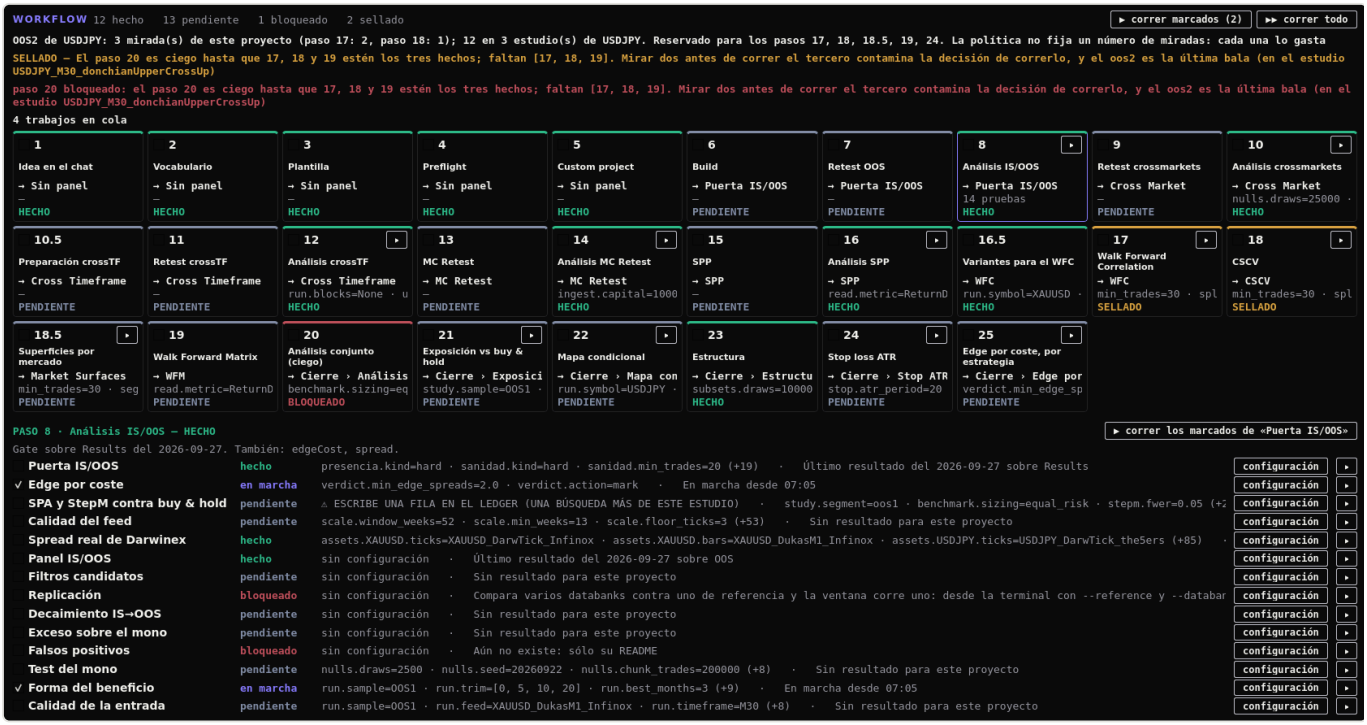


- La primera línea es el presupuesto de **OOS2**: cuántas veces lo ha mirado este proyecto y qué pasos lo tienen reservado para un agente autónomo. Para ti es sólo un contador.
- La segunda es la **puerta del paso 20**: verde y abierta para ti; ámbar sólo para un agente autónomo mientras no estén 17, 18 y 19 estén los tres en el ledger.
- La tercera, en rojo, el motivo de cada paso bloqueado (un botón desactivado no enseña su ayuda, por eso va escrito).
- En cada tarjeta: número, SQX/PY/TÚ, título, la pestaña donde se lee su resultado, su configuración en una línea (o cuántas pruebas tiene) y el estado.

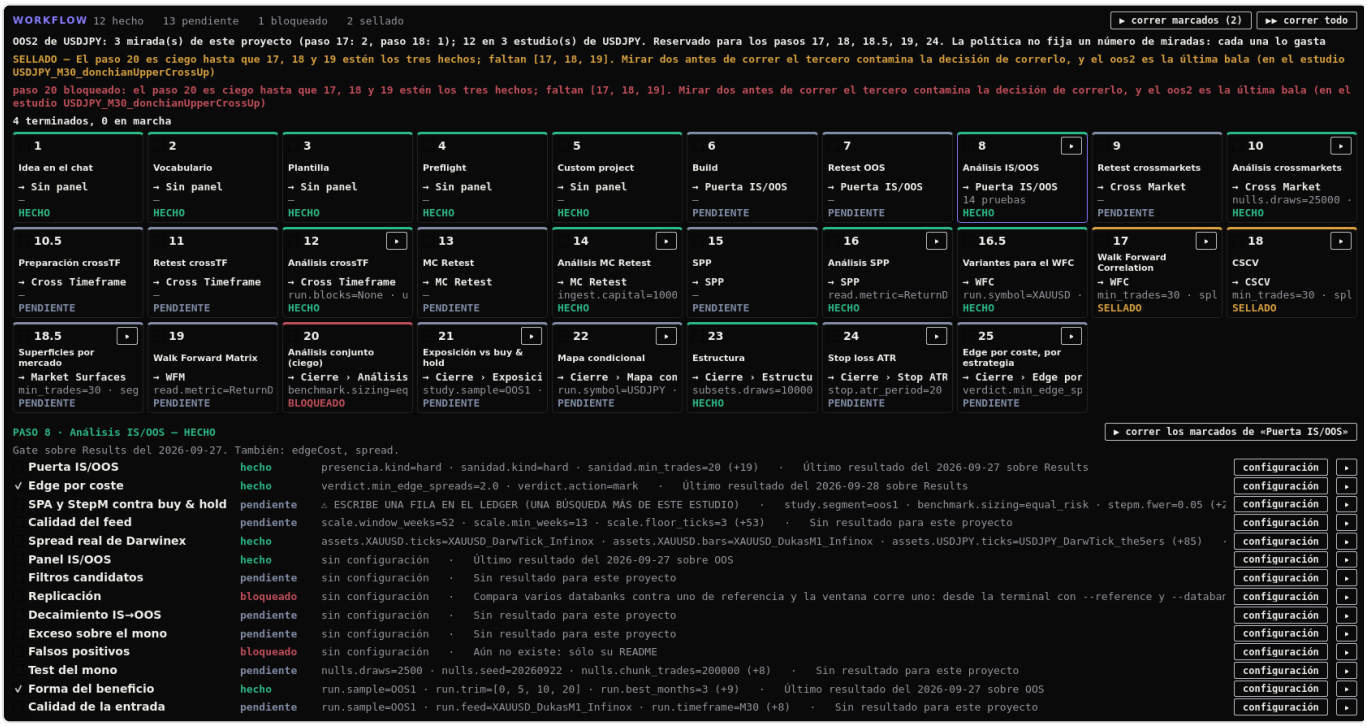
Esta vez no sale nada marcado: lo que queda pendiente o lee oos2 o es del 21-25. Se marcan a mano dos pruebas del paso 8, «Edge por coste» y «Forma del beneficio», y se pulsa «▶ correr los marcados de «Puerta IS/OOS»»:



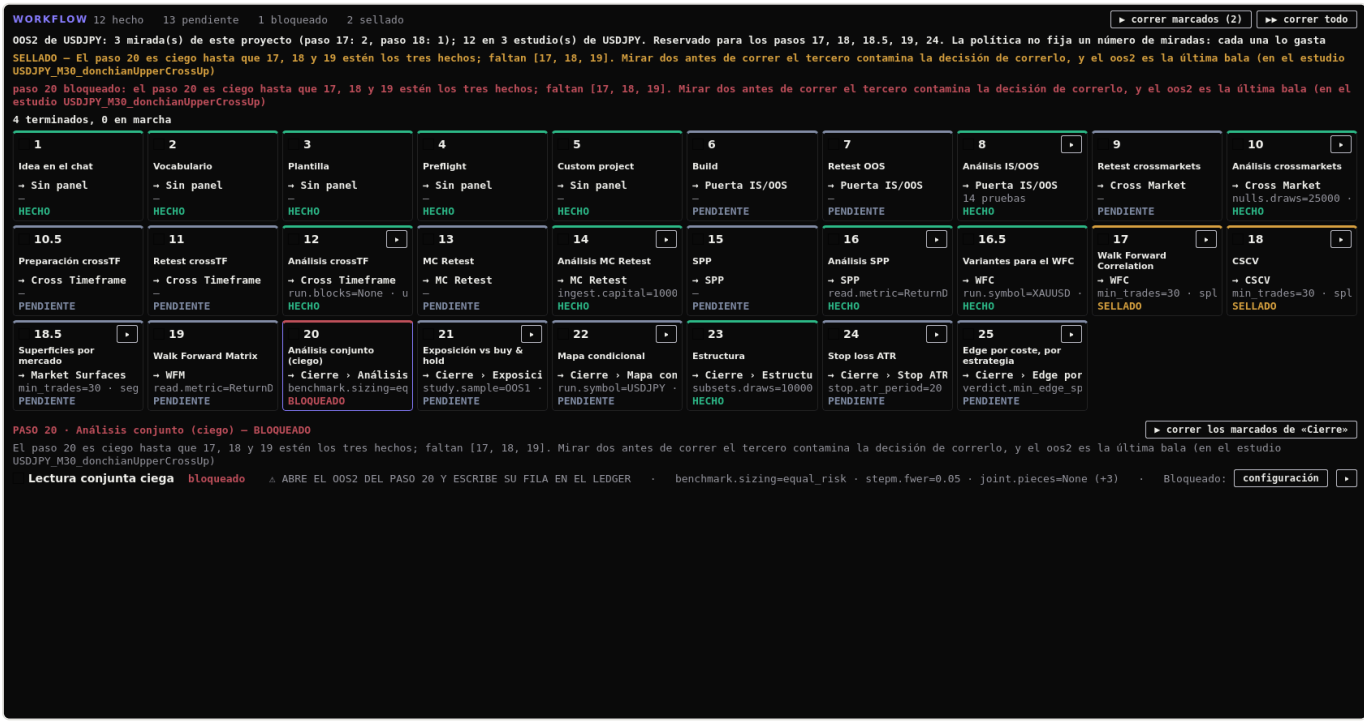
Las dos arrancan **a la vez** (la de edge por coste sobre la cosecha de `Results`, la de forma del beneficio una vez por cada una de las 3 estrategias del retest `OOS`): las dos pruebas dicen **en marcha** con su hora de salida.



Cuando acaban, el raíl se recarga solo y lo dice («4 terminados, 0 en marcha»); las pruebas pasan a **hecho** con el día y el databank de su resultado, que ya está en su pestaña del panel.



El paso 20: la tarjeta y su prueba están **bloqueadas**, con la frase de la puerta del ledger y el estudio que mira. En este proyecto faltan los tres (17, 18 y 19) en el estudio de su plantilla, aunque 17 y 18 tengan resultado en disco: sus filas se apuntaron bajo otro nombre. Nada de OOS2 se enseña.



Un ejemplo completo

Con el proyecto USDJPY elegido, el mismo recorrido que las capturas. Lo que el demonio devolvió en `/api/jobs` medio segundo después de pulsar (los cuatro trabajos arrancados en el mismo segundo, sin cola):

```
20260928-070450-0 edgeCost 8 Results - rc= None queued= None
20260928-070450-1 profitShape 8 OOS Strategy 10.11.79 rc= None queued= None
20260928-070450-2 profitShape 8 OOS Strategy 9.25.72 rc= None queued= None
20260928-070450-3 profitShape 8 OOS Strategy 9.27.83 rc= None queued= None
```

Y dos segundos después, los cuatro con `rc= 0`. La línea del raíl: «4 terminados, 0 en marcha».

Qué NO te dice

- **«Hecho» no es «bueno».** Dice que hay un resultado en disco, no qué dice. El veredicto está en el panel de databanks.
- **El estado de un paso de SQX sale del disco**, no de preguntarle a SQX: el log del install y los `.sqx` de sus databanks. Si `/curate` ya quitó estrategias, las cuentas son las de después. «En marcha» sólo mientras un proceso de SQX vive en ese install: un build cortado a medias (el log dice que empezó y no terminó, y no queda ningún proceso) sale **pendiente**, con esa frase.
- **«Sellado» no dice nada del resultado a propósito.** Mirar el 17 y el 18 antes de correr el 19 contamina la decisión de correrlo.
- La configuración de una línea es la del `config.yaml` del estudio tal y como está hoy; si un resultado viejo se hizo con otra, el panel lo marca como desfasado, no el raíl.

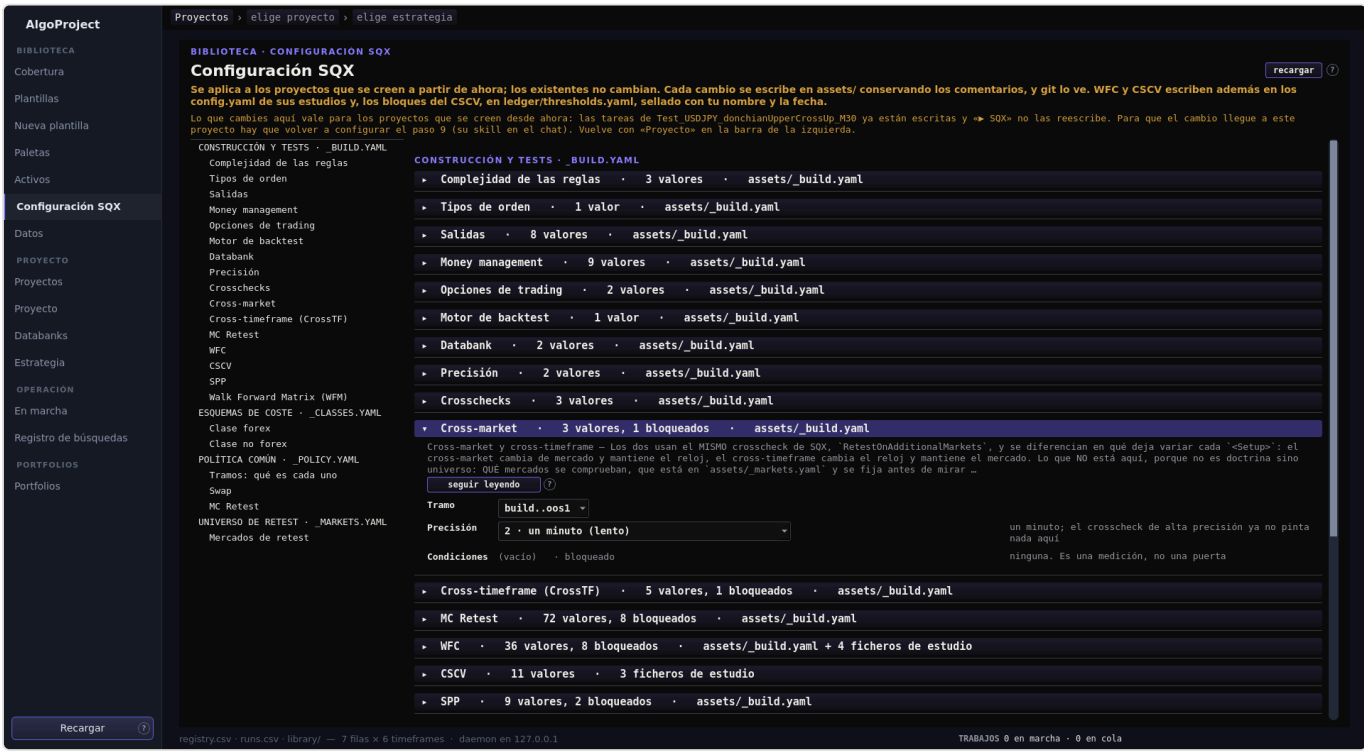
Si algo falla

- **«no se lanzó: … necesita la cosecha de este databank»**: falta la entrada; la frase dice con qué skill se hace (`/oos-gate`, `/export`, `/variants`…).
- **«elige en el panel el databank que lee»**: es un paso del 21 al 25; lánzalo desde el panel.
- **«el proyecto … no tiene plantilla en projects/registry.csv»**: el estudio firmaría en el ledger bajo una familia inventada; se corre a mano con `--family` si sabes cuál es.
- **«paso de SQX: la ventana no lo lanza desde el raíl…»**: el análisis cuelga de un paso de SQX; lánzalo desde el panel de databanks.
- **«Nada que correr: marca una prueba o pulsa el ▶ de un paso.»**: no había ninguna prueba marcada ni ninguna libre en ese paso.
- **✖ en rojo en una tarjeta de SQX**: un fallo de verdad — el databank que lee está vacío, o es un build y el proyecto no está en el custodio.
- **«· workflow corriendo»** en una tarjeta, y **«No disponible: el workflow ya está corriendo»** junto a «Lanzar en SQX»: SQX ya está trabajando (este proyecto u otro en ese install). No es un fallo: espera a que termine. Los textos son cortos a propósito (dueño, 2026-09-30); **el detalle completo — qué install, qué puerto, qué proceso — sale al pasar el ratón** por encima.
- **· en el color normal, no en rojo**: el paso simplemente no se lanza desde aquí — es el 5, el 10.5 o el 16.5, que no son una tarea suelta; no es un fallo.
- **«Correr workflow: No disponible: …»**: el primer motivo de «Lanzar en SQX», en pocas palabras (todos, al pasar el ratón), o «Nada que correr» cuando el primer paso pendiente ya es una parada.
- **«No se lanzó nada.»**: respondiste «no» a la pregunta de una prueba que lee oos2 o escribe en el ledger.
- **«Sin conexión: pulsa Recargar»**: la ventana perdió su demonio; si «Recargar» no basta, ciérrala y vuelve a abrirla.

El engranaje ⚙: dónde se edita la configuración de cada paso

El ⚙ de cada tarjeta te lleva directo a la pantalla donde se cambia lo que ese paso usa. Después vuelves con **Proyecto** en la barra de la izquierda y lo lanzas con su ▶.

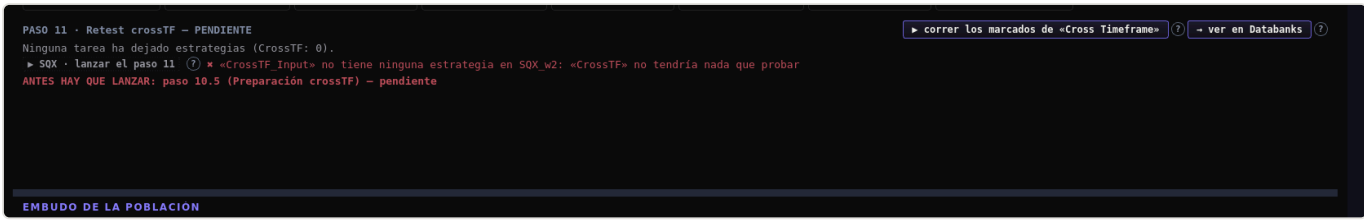
paso	el ⚙ abre
4 Preflight, 7 Retest OOS	**Activos**, en la ficha del activo del proyecto (costes y tramos)
6 Build	**Configuración SQX › Complejidad de las reglas** (debajo: tipos de orden, salidas, money management, opciones, motor, databank, precisión, crosschecks)
9 Retest crossmarkets	**Configuración SQX › Cross-market**
10.5 y 11 (crossTF)	**Configuración SQX › Cross-timeframe (CrossTF)**
13 MC Retest	**Configuración SQX › MC Retest**
15 SPP	**Configuración SQX › SPP**
16.5 y 17 (WFC)	**Configuración SQX › WFC**
18 CSCV	**Configuración SQX › CSCV**
19 WFM	**Configuración SQX › Walk Forward Matrix (WFM)**
5, 8, 10, 12, 14, 16, 18.5, 20-25	ninguna: la ventana **no tiene pantalla** para editar la configuración de esos estudios (vive en el `config.yaml` de cada uno). El ⚙ abre el paso en el cajón y lo dice; «configuración» la enseña, sólo lectura



Ojo — lo que cambias en Configuración SQX no llega solo a este proyecto. La propia pantalla lo avisa: vale para los proyectos que se creen desde ahora. Las tareas de un proyecto que ya existe están escritas en su `project.cfx`, y «▶ SQX» sólo enciende las del paso y arranca: **no las reescribe**. Para que el cambio llegue a este proyecto hay que volver a configurar ese paso en el chat (su skill: /crossmarket, /mcretest, /spp, /wfm…). Al llegar desde un ⚙, la línea ámbar de arriba de Configuración SQX te lo recuerda con el nombre del proyecto. La excepción son los valores de estudio de WFC y CSCV (pasos 17 y 18): se leen otra vez en la próxima corrida. Lo mismo con Activos: un coste cambiado vale para el preflight de hoy, pero la tarea OOS de un proyecto ya creado lleva los costes con que se creó.

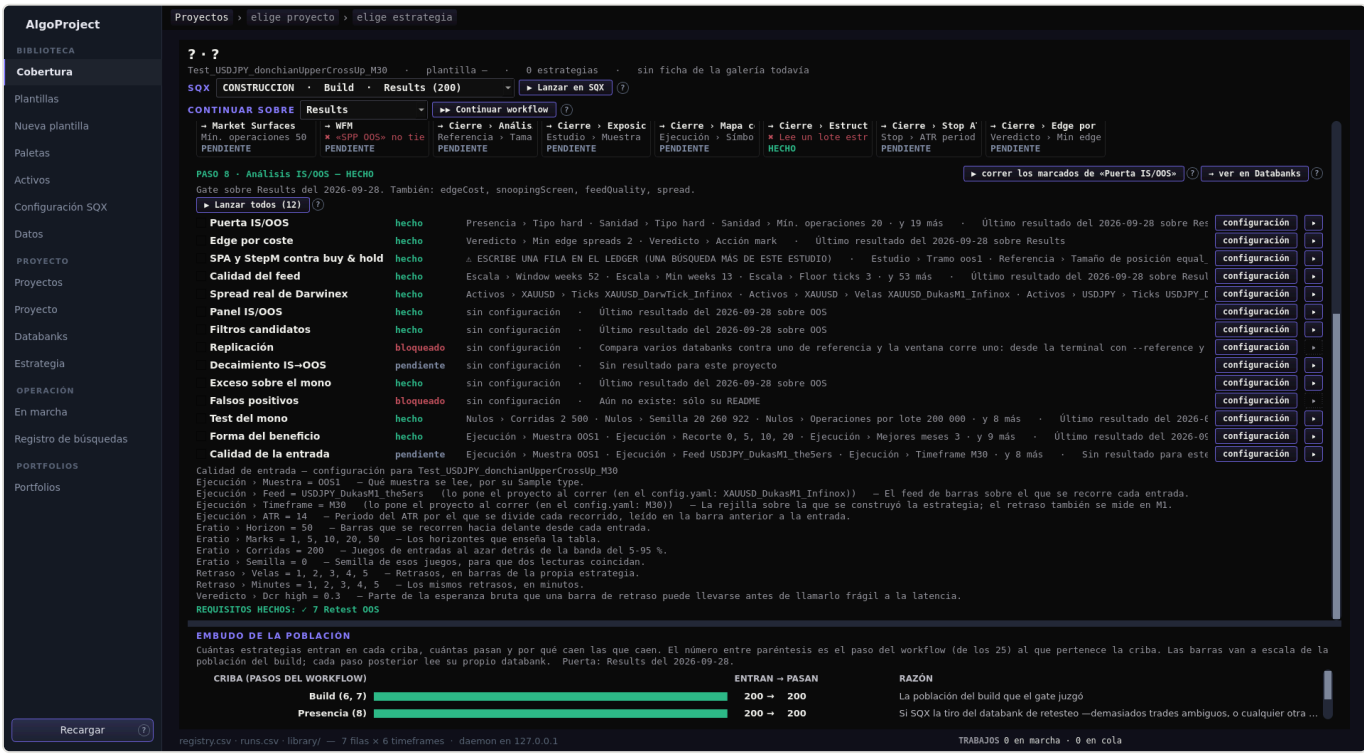
Qué hay que lanzar antes (abajo del cajón)

Cada paso dice, en la última línea del cajón, qué pasos tienen que estar hechos antes. Sale de los datos del proyecto, no de una lista fija: para un paso de SQX, el paso cuya tarea llena el databank de entrada de sus tareas y el paso de Python que juzga ese databank; para uno de Python, el paso de SQX cuyo databank lee; siempre, además, el paso anterior de la tabla. En rojo lo que falta, en verde lo hecho. Basta con mirar el primero que falta: abriendo ese, su cajón te dice a su vez el suyo.



Lanzar todos

En un paso con más de una prueba de Python (el 8, Puerta IS/OOS, tiene 14), **«▶ Lanzar todos (N)»** encima de ellas corre a la vez las N que se pueden correr aquí (no las bloqueadas ni las que ya están en marcha). Si alguna lee el oos2 o escribe en el ledger, primero sale la misma pregunta de siempre.

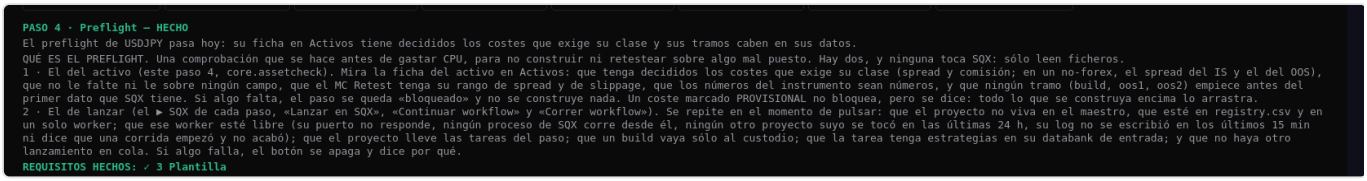


El preflight, explicado

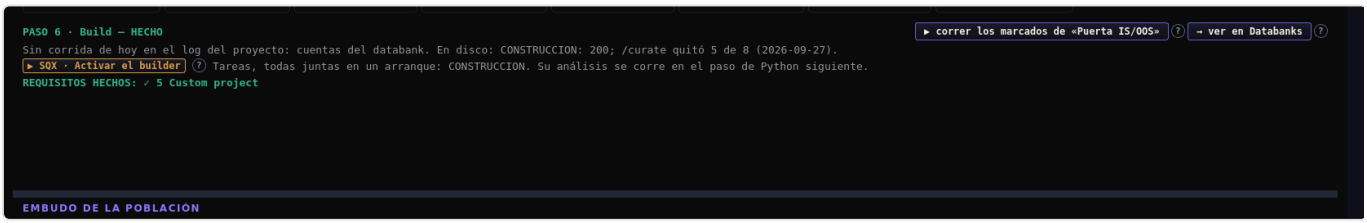
«El preflight de USDJPY pasa hoy» quiere decir que la **ficha del activo** está completa. Al abrir el paso 4, el cajón explica los dos preflights:

1. **El del activo** (paso 4): costes que exige su clase decididos (spread y comisión; en un no-forex, spread del IS y del OOS), ningún campo de más ni de menos, rangos de spread y slippage del MC Retest, números del instrumento que sean números, y ningún tramo que empiece antes del primer dato de SQX. Si falla, el paso queda **bloqueado**; un coste PROVISIONAL no bloquea pero se dice.

2. **El de lanzar** (▶ SQX, «Lanzar en SQX», «Continuar workflow», «Correr workflow»), repetido al pulsar: el proyecto no vive en el maestro, está en `registry.csv` y en un solo worker; el worker está libre (su candado de propietario, `user/log/OWNER`, sin otro dueño vivo — OPEN §32 —, puerto sin respuesta, ningún proceso de SQX, log quieto 15 min, ninguna corrida empezada sin acabar); el proyecto lleva las tareas del paso; un build sólo en el custodio; la tarea tiene estrategias en su databank de entrada; y no hay otro lanzamiento en cola. Que otro proyecto se tocara hace un rato ya no cuenta (OPEN §83): un worker libre acepta cualquier proyecto.



Paso 6: «Activar el builder»



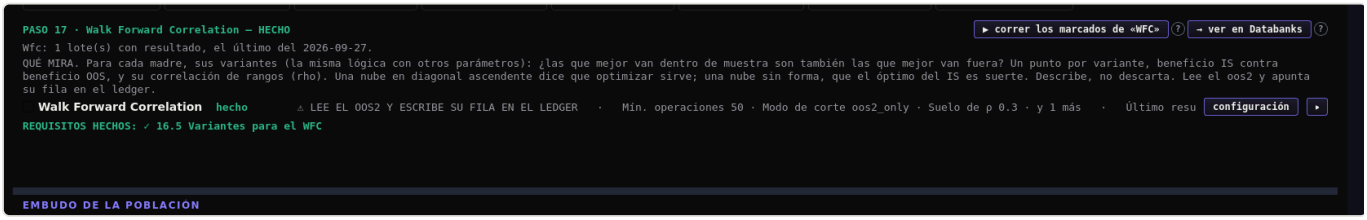
Lanza **sólo** la tarea de construcción (CONSTRUCCION). El retest OOS es el paso 7, con su propio ▶ SQX: puedes construir hoy y retestear otro día.

USDJPY y los XAUUSD del config.yaml

Los `config.yaml` de los estudios traen los valores del proyecto donante, que es XAUUSD. Hasta hoy «configuración» los enseñaba tal cual, y en un proyecto USDJPY salía «Feed XAUUSD_DukasM1_Infinox». Ahora enseña **lo que correrá**: donde la ventana pone el feed, el activo o el timeframe del proyecto al lanzar (Calidad de la entrada, Mapa condicional) sale el del proyecto, con «(lo pone el proyecto al correr (en el config.yaml: …))». Donde **no** lo cambia, sale con ⚠: la Nube de parámetros corre con `run.symbol = XAUUSD` aunque el proyecto sea USDJPY (hoy sin efecto: sólo se usa para cortar el oos2 a un agente autónomo, y los dos activos empiezan el oos2 en 2023). Los resultados guardados de este proyecto corrieron todos sobre `USDJPY_DukasM1_the5ers`.



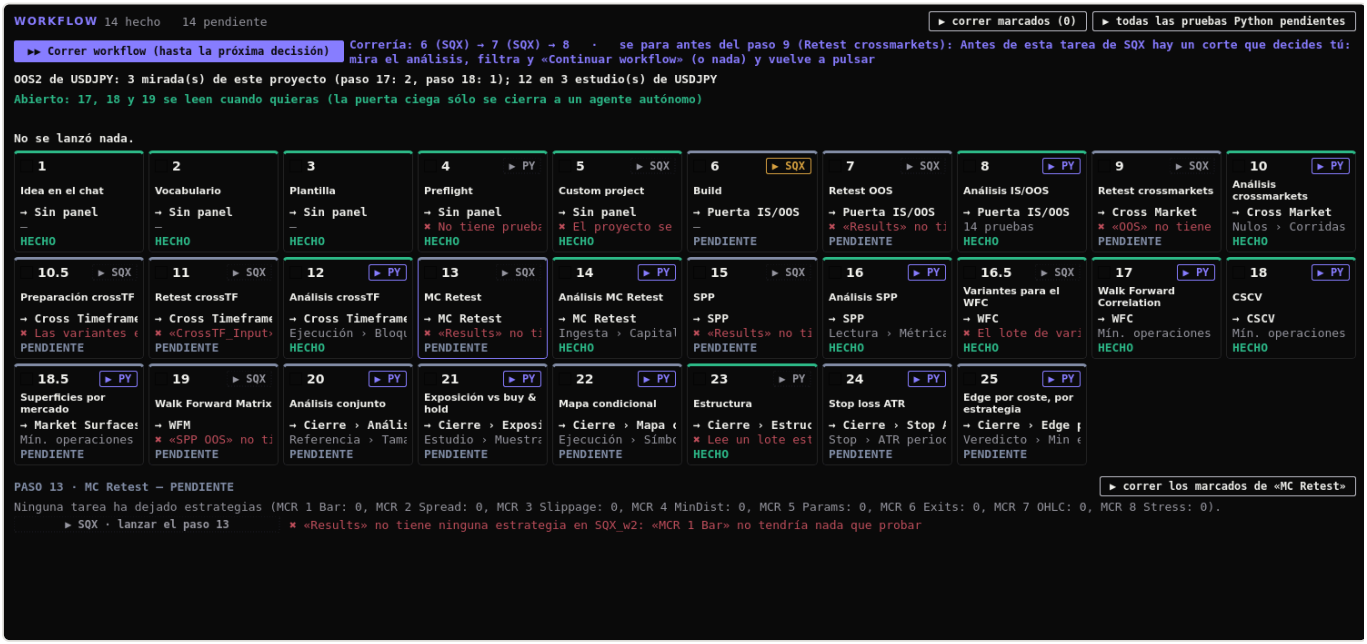
Walk Forward Correlation, en una frase



Para cada madre, sus variantes: ¿las que mejor van dentro de muestra son también las que mejor van fuera? Una nube en diagonal dice que optimizar sirve; sin forma, que el óptimo del IS es suerte.

Correr workflow: la cadena hasta la próxima decisión

Añadido el 28-09-2026. ▶▶ **Correr workflow (hasta la próxima decisión)** corre, uno detrás de otro y en el orden de `WORKFLOW.md`, cada paso pendiente del proyecto, de SQX y de Python, y **se para en el primer sitio donde tienes que decidir tú**. Para seguir, vuelves a pulsar el mismo botón.



A la derecha del botón está lo que haría ahora mismo. En la captura, con el build del proyecto cortado a medias (el log de SQX lo dejó empezado, sin ningún proceso vivo): «Correría: 6 (SQX) → 7 (SQX) → 8 · se para antes del paso 9 (Retest crossmarkets): antes de esta tarea de SQX hay un corte que decides tú…».

La regla de parada, que es lo único que hay que entender:

1. Corre los pasos pendientes en orden. Un paso de SQX lanza todas sus tareas juntas en un arranque del worker, que se para al acabar (nunca queda uno arrancado); un paso de Python corre las mismas pruebas que «todas las pruebas Python pendientes». Antes de cada paso de Python carga lo que leen sus pruebas (métricas, operaciones, cosecha), como al elegir un databank. Un paso de Python ya hecho no se toca (lo que le quede se corre con su ▶ PY).
2. Cuando ya ha corrido un paso de SQX en esta pulsación, lo que viene detrás se **rehace entero**, aunque tuviera resultado: su entrada ha cambiado (un build nuevo → el retest OOS y el paso 8 de nuevo).

3. **Se para SIEMPRE antes de la siguiente tarea de SQX si entre medias hay un paso de Python que juzga** (8, 10, 12, 14 o 16: los que dan veredicto por estrategia) y que no estaba hecho o se ha corrido en esta pulsación. Ahí miras el análisis, filtras y cortas con «Continuar workflow» (o no cortas nada) y **vuelves a pulsar**: sólo una pulsación que *empieza* en esa tarea de SQX, con su análisis ya hecho, la cruza. Si ese análisis no está hecho y la cadena no tiene pruebas suyas que correr, se para en él.
4. También se para en cualquier paso **en marcha** (de SQX o de Python), en lo que la ventana no lanza (5, 10.5, 16.5), en el **19** (el WFM lee el oos2: se lanza a mano con su ▶ SQX) y en el primer fallo.
5. **Nunca corre** lo que lee el oos2 o escribe en el ledger —WFC (17), CSCV (18), superficies por mercado (18.5), el análisis del WFM, el paso 20 y el SPA/StepM del paso 8— ni los pasos 21-25.
6. Corre **exactamente el plan que confirmaste**. Si antes de un paso algo cambió (un paso que estaba pendiente ya está hecho, o está en marcha), se para y lo dice: vuelve a mirar el raíl y pulsa otra vez.

Se niega a empezar, como «Lanzar en SQX», si el proyecto vive en el maestro, si su worker está ocupado (candado de propietario de otra sesión, puerto que responde, proceso de SQX vivo, log escrito hace menos de 15 min) o si ya hay un lanzamiento en marcha o en cola. Antes de nada enseña el plan entero y sólo **Sí** lo pone en marcha. Lo que se confirmó sobre el proyecto USDJPY (salida real):

```
«Correr workflow» en Test_USDJPY_donchianUpperCrossUp_M30 (SQX_w2), en este orden:
· paso 6 · Build — SQX en el custodio: CONSTRUCCION; el worker arranca, corre hasta «Project finished»
y se para
· paso 7 · Retest OOS — SQX en el custodio: OOS; el worker arranca, corre hasta «Project finished» y
se para
· paso 8 · Análisis IS/OOS — Python: gate, edgeCost, feedQuality, spread, isOos, filters, decay,
monkeyExcess, monkey, profitShape, entryQuality
Se para antes del paso 9 · Retest crossmarkets: antes de esta tarea de SQX hay un corte que decides
tú: …
```

Es **un solo trabajo** en **En marcha** («Correr workflow»), en el mismo carril que «Lanzar en SQX» y «Continuar workflow»: mientras corre, esos dos se niegan, los ▶ PY de ese proyecto se niegan («SQX trabaja en este proyecto…»: sus databanks cambian), y la carga de datos que pida la ventana espera a que acabe. Si una prueba de Python no encuentra su entrada lo dice y sigue; si falla, la cadena se para ahí. Si al pararse el worker sigue vivo («STILL RUNNING»), o si al ir a arrancarlo ya estaba arrancado, la cadena se para y lo dice; nunca toma ni para un worker que no arrancó ella.

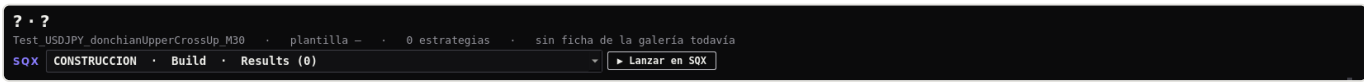
Desde la terminal, sin pantalla de confirmación: `python3 -m ui.daemon.launch.chain --project <P>`.

Cancelarlo desde **En marcha** ya no lo mata en seco: primero se le pide que acabe y pare el worker que arrancó él (puede tardar hasta 6 minutos; dice «cancelando: parando el worker», o sólo «cancelando» si estaba en un paso de Python) y sólo después se mata. La ventana **sólo para un worker que arrancó ese mismo trabajo**: nunca el de otra sesión. Si aun así queda arrancado, el trabajo lo dice en rojo, «⚠ el custodian sigue arrancado: bin/sqx-worker.sh --role custodian stop», también en el raíl.

Cerrar la ventana no lo cancela: el trabajo sigue solo hasta el final y para su worker. Al volver a abrir la ventana, el raíl dice en rojo «un lanzamiento de … sigue corriendo con el custodian arrancado», y no deja lanzar nada más hasta que acabe.

Lanzar en SQX: la tarea que tú elijas

Añadido el 28-09-2026. Bajo el título del proyecto hay una fila **SQX**: un desplegable con todas las tareas del proyecto y, al lado, ▶ **Lanzar en SQX**.



Cada tarea sale con su tipo y lo que hay ahora en disco: `OOS · Retest · Results (200) → OOS (0)` es un retest que leería 200 estrategias; `CONSTRUCCION · Build · Results (0)` es el build. Eliges una, y si el botón está gris la línea de al lado dice por qué. Se niega:

- si el proyecto vive en el **maestro** (nunca se toca tu install);
- si su **worker ya está arrancado** o su candado (`user/log/OWNER`) lo tiene otra sesión, o si su log de SQX se escribió hace menos de 15 min (puede ser otra sesión; nunca se para). Desde el 29-09 esa espera **no cuenta lo que escribió la propia ventana**: si lo último del log es de un trabajo suyo que ya soltó el worker, se lanza sin esperar;
- si es un **build** y el proyecto no está en el **custodio** (el conductor es para trabajos cortos);
- si es un retest y su databank de entrada está **vacío** en disco;
- si ya hay un «Lanzar en SQX», un «Correr workflow» o un «Continuar workflow» en marcha o en cola.

Al pulsar sale la frase a confirmar, por ejemplo: «Se va a lanzar «CONSTRUCCION» (Build) de Test_USDJPY_donchianUpperCrossUp_M30 en SQX_w2. Solo esa tarea quedará activa; el custodio arranca, corre hasta «Project finished» y se para. Un build ocupa todos los núcleos del custodio y puede durar horas…». Sólo **Sí** lo pone en marcha; su avance se sigue en **En marcha**.

Si la salida de una tarea **ya tiene estrategias**, la confirmación lo avisa: SQX añade las nuevas a su lado (una repetida se guarda como «Nombre(1)») salvo que una tarea «Clear databanks» la vacíe antes. Y si la tarea de un paso **está sin configurar** (la WFM, el SPP o el Cross Market con su cross-check apagado), lo dice y la configura antes de arrancar, igual que su skill; una tarea ya configurada no se toca.

Lo que hace, en orden: `python3 -m core.assets <SÍMBOLO>` (si falla, no se lanza nada); copia la carpeta del proyecto sin `log/` a `AlgoData/projects/snapshots/`; configura lo que esté sin configurar; deja activa **sólo** esa tarea; arranca el worker; espera a que SQX haya cargado sus databanks; manda `action=start`; mientras corre, sólo `action=status`; al «Project finished» para el worker. Después compara los databanks: si bajó alguno que la tarea no escribe, lo dice y **conserva la copia**; si no, la borra. Si SQX borró alguna estrategia al pararse por su propia carrera de sincronizaciones (el log dice «Cannot process strategy»), se **restaura sola** desde la copia. Un build apunta además su fila en `AlgoData/templates/runs.csv`. Lo que la tarea borre al empezar lo decide su configuración en SQX, que el botón no cambia.

Desde la terminal hace lo mismo `python3 -m ui.daemon.launch.run --project <P> --task <TÍTULO>` (`--task` se puede repetir), o todas las tareas de un paso con `--step 13`, pero sin pantalla de confirmación: úsalo sólo si sabes lo que lanzas.

Cancelar el trabajo con la ✕ de la franja de trabajos **pide confirmación** (desde el 29-09) y le deja parar su worker antes de matarlo (como en «Correr workflow»). Al pararse, el export automático saca también la ingesta del MC Retest, el SPP y la WFM, y sólo lo que cambió.

65b. El embudo (en Proyecto) y el panel de databanks (en Databanks)

Qué pregunta responde

- **¿Cuántas estrategias quedan y dónde se cayeron las demás?** El **embudo**, debajo del raíl en la zona Proyecto, pinta una barra por criba de la Puerta IS/OOS (verde lo que pasa, rojo lo que cae, gris las cribas blandas que solo marcan) y, debajo, una por cada paso de Python posterior con su entrada y su salida. Al lado de cada barra, escrito, **por qué** caen.

- **¿Qué dijo cada prueba de cada estrategia?** El **panel de databanks**, que ocupa toda la zona **Databanks**, al estilo de SQX: una fila por estrategia, sus métricas de SQX (IS y OOS) y una columna por prueba con su veredicto en color. Dos filas de pestañas: arriba la prueba (Puerta IS/OOS, Cross Market, Cross Timeframe, MC Retest, SPP, WFM, WFC, CSCV, Market Surfaces, Cierre), debajo sus sub-paneles (cada tarea del MC Retest, cada timeframe del cross-TF, cada composición del WFC…).
- **¿Cuánto gana el databank entero, con el spread de verdad?** A la derecha, la **equity agregada**: la suma diaria de todas las estrategias según SQX (violeta) y la misma con el spread y el slippage reales de Darwinex (ámbar), con la frontera IS/OOS.

Cuándo lo usas, y cuándo no

Lo usas para leer un proyecto entero de un vistazo: qué pasó cada criba, qué estrategias sobreviven y qué dijo de cada una cada prueba, sin abrir informes sueltos.

No sirve para mirar el WFC, el CSCV, el WFM ni el paso 20 antes de tiempo: mientras 17, 18 y 19 no estén los tres hechos, sus sub-paneles salen **BLOQUEADOS** con la frase del ledger, y sus columnas no aparecen en ninguna tabla. Es a propósito.

No hay pestaña de Monte Carlo de operaciones: irá con las carteras, en la zona **Portfolios** (decisión del dueño del 2026-09-23).

Antes de empezar

- La ventana abierta y el proyecto elegido en la galería.
- Las métricas salen de la **cosecha** del databank (paso 8, `/oos-gate`) o, si no la tiene, de su export de métricas, que la ventana hace sola al elegir el proyecto.
- La curva ámbar necesita el informe del estudio **spread** sobre ese databank (paso 8).

Cómo se ejecuta

No hay comando: son pestañas y botones.

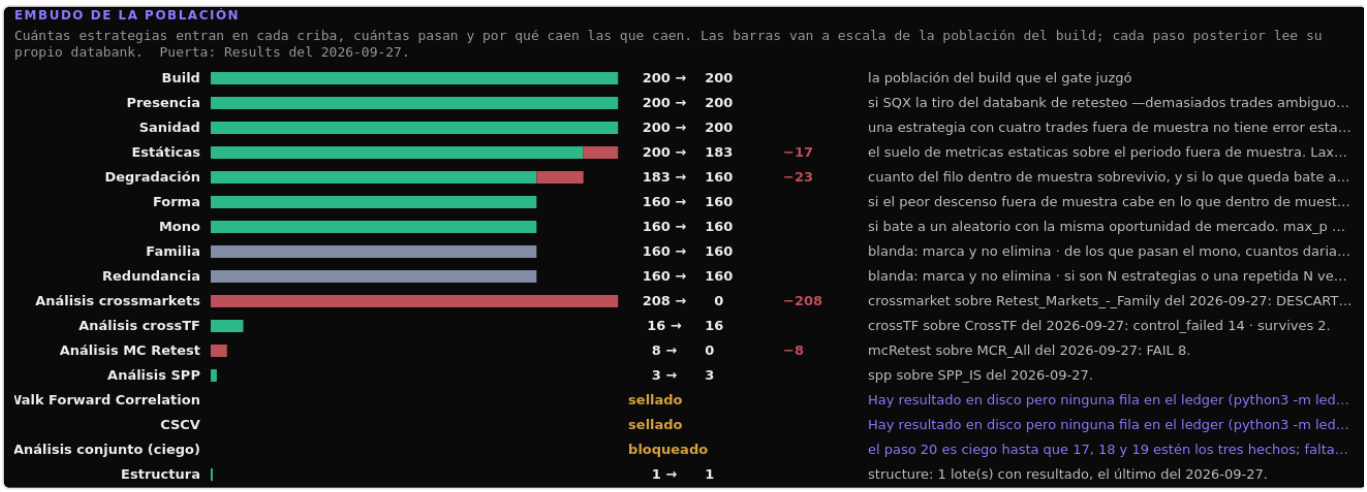
control	qué hace
pestaña de arriba	elige la prueba; debajo aparecen sus sub-paneles
sub-pestaña	elige qué columnas se ven (y, en SPP, qué databank: IS u OOS)
clic en una cabecera	ordena por esa columna (los números como números)
doble clic en una fila	abre la ficha de esa estrategia en «Estrategia»
arrastrar la banda entre el raíl y el embudo (zona Proyecto)	reparte la altura entre los dos
↻ **Recargar databank**	vuelve a leer del disco qué estrategias hay en el databank y pide cargar lo que le falte (métricas; operaciones y cosecha van al conductor, de una en una)
⚙ **Métricas**	(antes «☰ Columnas») abre la lista de métricas: quitar las de siempre, añadir cualquiera en IS, OOS1, OOS2 o IS+OOS1, o **Restaurar vista por defecto**. Se guarda **por databank** (ver «Métricas» más abajo)
arrastrar una cabecera a otro sitio	cambia el orden de las columnas; se guarda igual que la lista (el de las métricas, para todo el databank). El nombre de la estrategia vuelve siempre a la primera
arrastrar la banda entre la tabla y la equity (Databanks)	reparte el ancho: hacia la izquierda la curva crece, hacia la derecha la tabla. Ninguna de las dos baja de su mínimo (tabla 360 px, equity 300 px)
▶ **Correr marcados de este panel**	corre las pruebas marcadas en el raíl cuyo resultado vive en esta pestaña, sobre este databank (y solo sobre las filas seleccionadas, si hay)
casillas **SQX** y **spread y slippage reales**	enseñan u ocultan cada curva
tira **FILTROS** (Databanks) y ▶▶ **Continuar workflow** (Proyecto)	filtran el databank y lanzan la tarea siguiente de SQX: capítulo 69

Qué produce

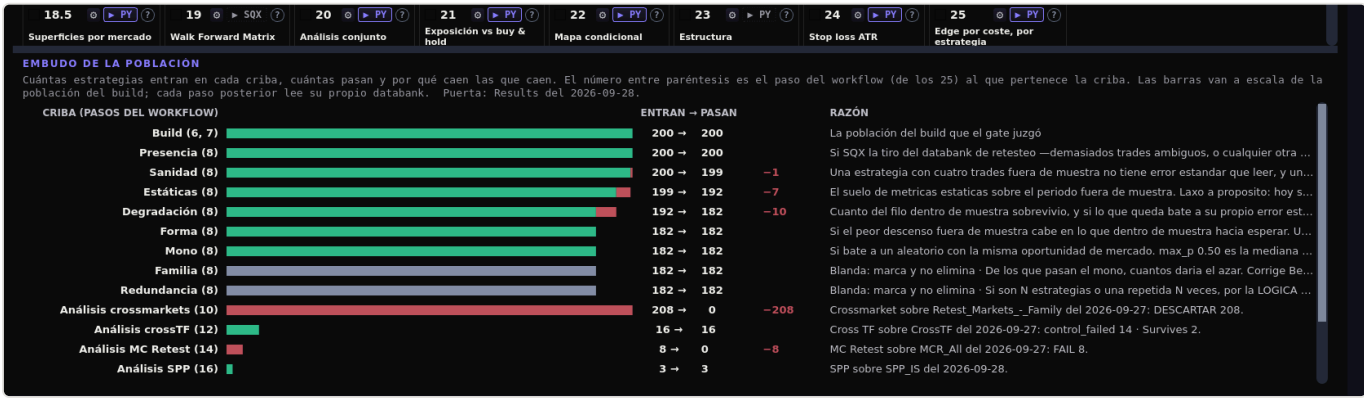
Mirar no escribe nada. «Recargar databank» encarga el mismo trabajo que la ventana encarga al elegir un proyecto. Elegir columnas escribe **solo tu elección**, en `AlgoData/filters/<proyecto>/<databank>/columns.json`, una entrada por tabla («pestaña › sub-panel»); ni una cifra, ni una fila del Ledger, ni nada en SQX. Borrar ese fichero es volver a las vistas por defecto. Lo que sí escribe —los filtros, en `AlgoData/filters/` y en el Ledger, y «Continuar workflow», en SQX— está en el capítulo 69.

Cómo se lee el resultado

El embudo del proyecto `Test_USDJPY_donchianUpperCrossUp_M30`:



Desde el 2026-09-28 (noche) el embudo lleva cabecera y el paso de cada criba:



- **Criba (pasos del workflow)**: el nombre de la criba y, entre paréntesis, a cuál de los 25 pasos pertenece. «Build (6, 7)» es la población que construyó el paso 6 y retesteó el 7; las cribas de la puerta (Presencia, Sanidad, Estáticas…) son todas del paso 8; las de abajo, de su propio paso.
- **Entran → pasan**: cuántas entraron y cuántas siguen; en rojo, las que cayeron.
- **Razón**: por qué caen, cada frase en mayúscula.
- 200 estrategias salieron del build; la criba **estáticas** tiró 17 y **degradación** 23: quedan 160. **Familia** y **redundancia** son blandas: marcan y no quitan.
- Debajo, cada paso posterior **lee su propio databank**, así que no es una cascada: el cross-market juzgó 208 (su databank) y las descartó todas porque no dispararon en ningún mercado; el MC Retest tiró las 8 que juzgó.
- (Captura tomada antes del 2026-09-28.) WFC y CSCV salían **sellados** y el paso 20 **bloqueado**; hoy eso sólo lo ve un agente autónomo.

La pestaña Puerta IS/OOS, sub-panel «Build + OOS1», ordenada por Beneficio neto OOS:



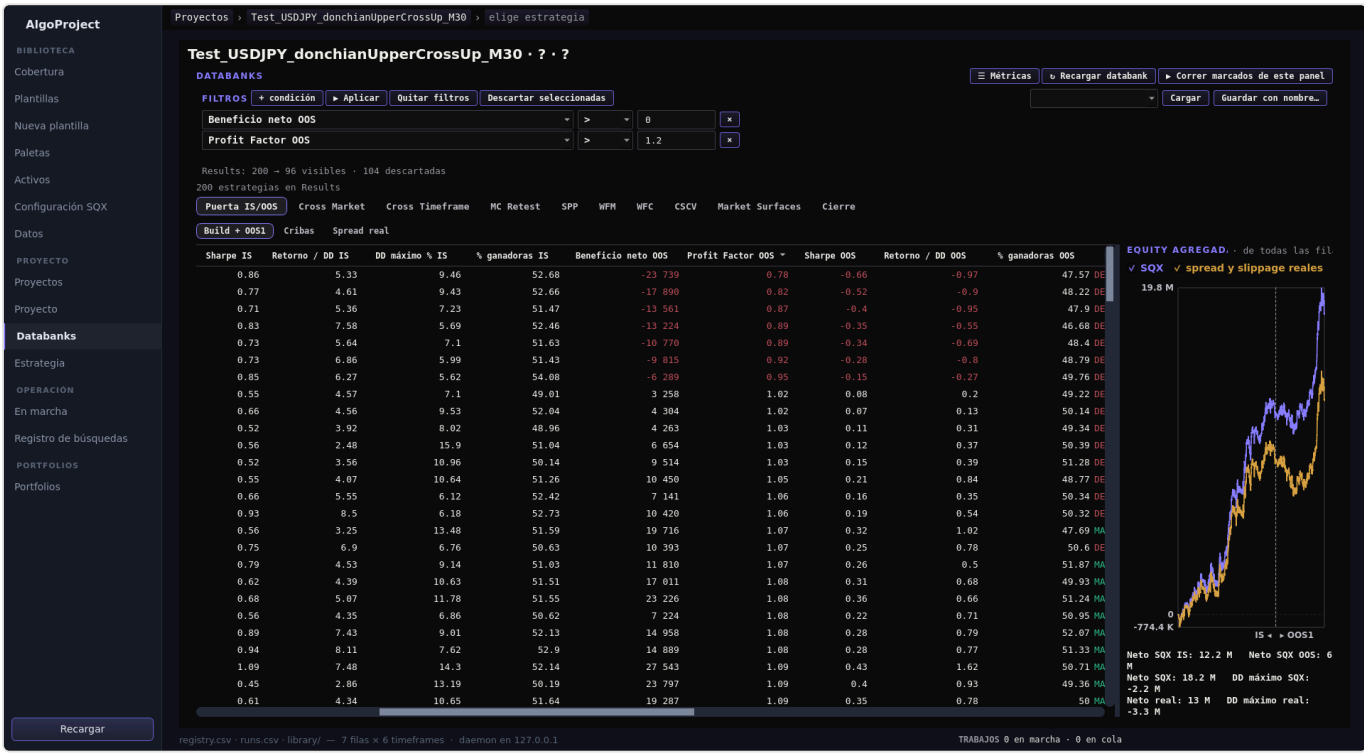
- Una fila por estrategia; las métricas de SQX en IS y luego en OOS; después una columna por prueba con su veredicto (verde pasa, rojo cae, ámbar dudoso).
- **En rojo, las cifras que dicen que la estrategia no gana dinero**, en IS, OOS y cualquier segmento: el Profit Factor por debajo de 1, y por debajo de 0 el beneficio neto, Sharpe, Sortino, esperanza, Retorno / DD, CAGR / DD, Calmar, SQN o rentabilidad anual (y, en las columnas de un estudio, un neto o un edge negativos o un PF por debajo de 1). El drawdown **nunca** va en rojo, ni el % de ganadoras ni el ZScore: no dicen si gana. Ordenar y los «–» funcionan igual.
- Arriba, en gris: cuántas filas hay, de dónde salen y qué está sellado.

- A la derecha, la equity agregada: 17,6 M según SQX y 13,1 M con el spread y el slippage reales; el DD máximo pasa de −2,1 M a −3,2 M. Es la **suma, día a día, del P/L de todas las estrategias visibles**, cada una con el tamaño con que SQX la probó y sin pesos: la curva del databank entero, no la de la fila elegida (un clic en una fila no la cambia; el doble clic abre su página en «Estrategia», donde está su curva). Desde el 2026-09-28 no lleva debajo la línea de origen («cosecha del … · spread real del …»); si falta la curva real, el motivo sale al pasar el ratón por su casilla «spread y slippage reales».

Con un filtro que oculta filas (los filtros del capítulo 69), la equity agregada se rehace **solo con las visibles** y lo dice junto al título: «· 180 visibles» en vez de «· de todas las filas». Aquí se ocultaron 20 estrategias y el neto SQX baja de 17,6 M a 15,6 M.



Las 200 filas, ordenadas por Profit Factor OOS de menor a mayor: las que pierden en OOS tienen en rojo el neto, el Profit Factor, el Sharpe y el Retorno / DD de OOS; el DD máximo sigue en blanco.



El MC Retest, sub-panel «Spread»: lo que hizo esa tarea en cada estrategia.



El cross-timeframe en H1: la lectura de cada estrategia en ese timeframe.



Lo que ve un **agente autónomo** en el WFC con 19 sin hacer: una sub-pestaña por composición, todas bloqueadas con su motivo. Tú ves sus cifras.



El paso 20 (Cierre › «Análisis conjunto (paso 20)») con 17, 18 y 19 sin hacer: la pestaña existe desde el principio, pero solo enseña el motivo, con la misma frase que el raíl, porque sale de la misma puerta del ledger. Un clic en la tarjeta 20 del raíl abre exactamente este sub-panel.



La nube de parámetros del lote de variantes: solo sale la madre que tiene lote.

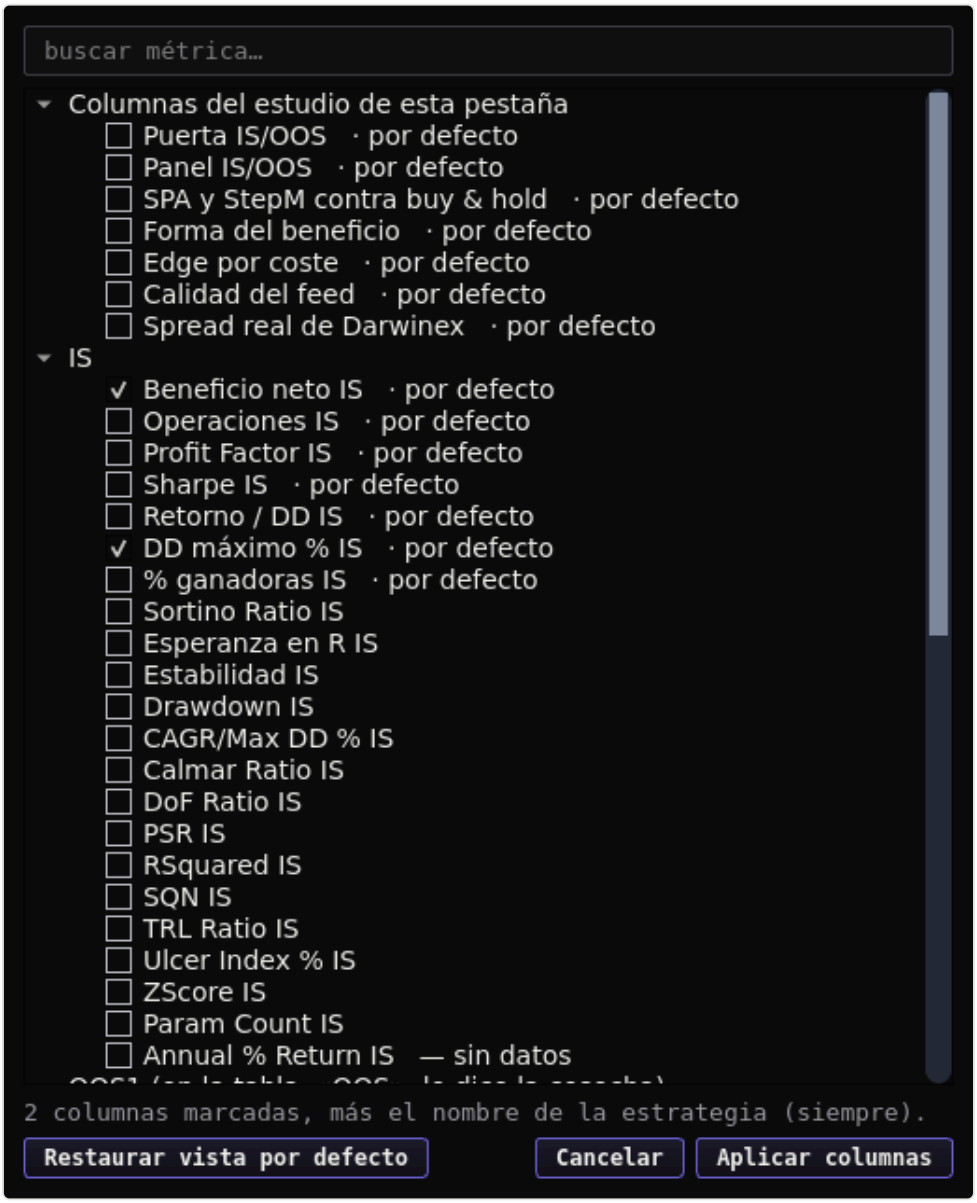


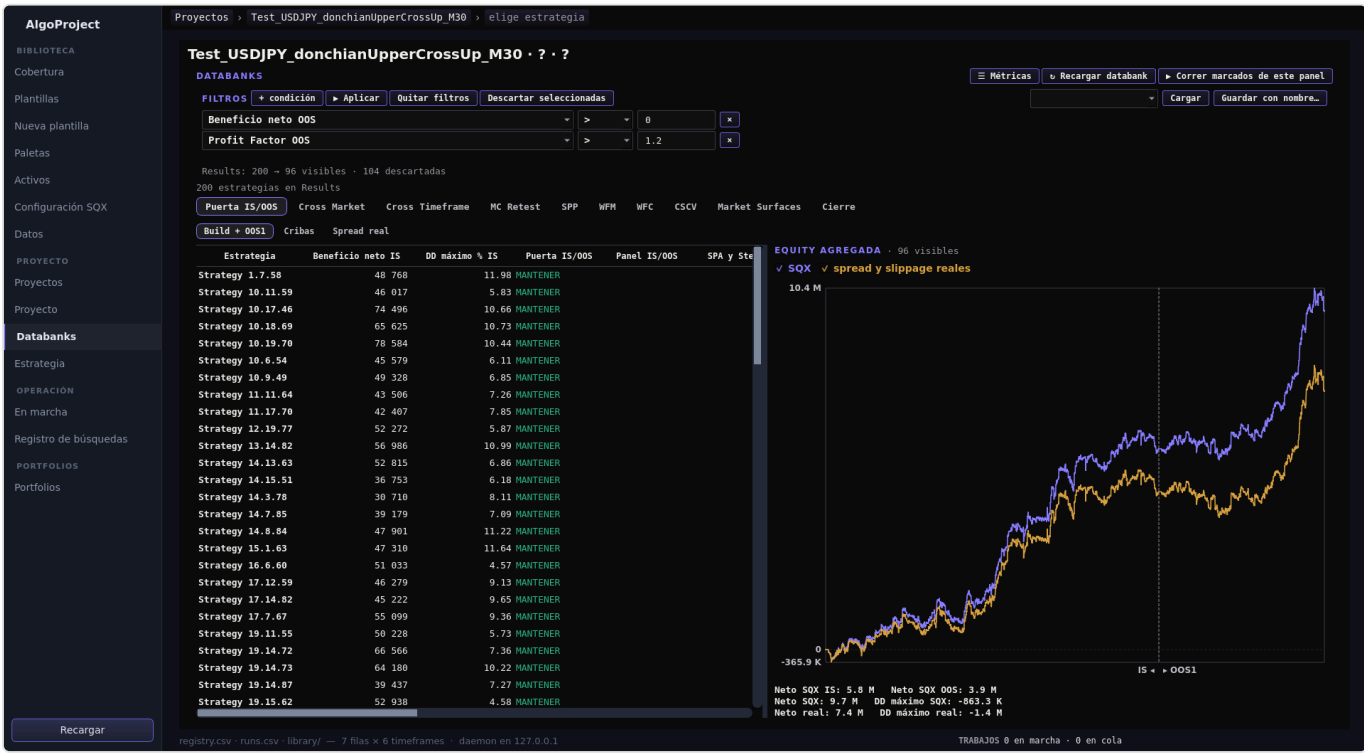
Métricas: qué enseña cada tabla

Cada tabla (cada sub-panel: «Build + OOS1», «Cribas», «Spread real», Cross Market, SPP › IS, cada tarea del MC Retest…) tiene su **vista por defecto** —las columnas de siempre—. Lo que eliges se guarda así, y se conserva al cerrar y volver a abrir la ventana:

- **Las métricas, por databank**: las que dejas en «Puerta IS/OOS › Build + OOS1» valen en todas las pestañas que leen `Results` (Cribas, Spread real, WFC…), y no tocan otro databank: Cross Market (`Retest_Markets_-_Family`), SPP, MC Retest… guardan cada uno la suya.
- **Las columnas del estudio, por pestaña**: quitar el veredicto de la Puerta en «Build + OOS1» no lo quita de «Cribas».

Por ejemplo, para ver en `Results` solo **Beneficio neto IS** y **DD máximo % IS**: ⚙ Métricas, desmarcar las demás métricas, **Aplicar columnas**. La tabla queda con esas dos y las columnas del estudio, en todas las pestañas de `Results`:

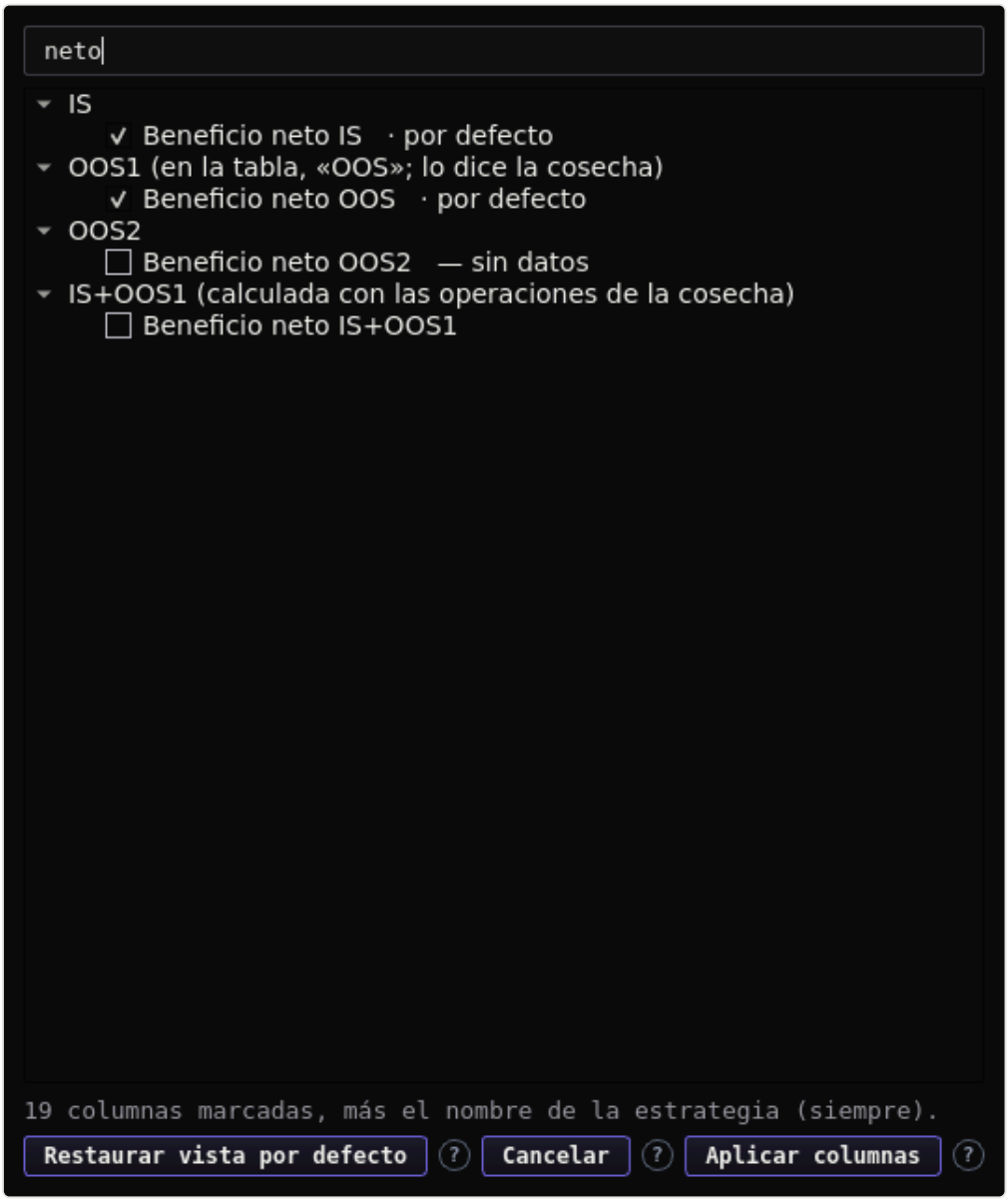

buscar métrica…
Columnas del estudio de esta pestaña
Puerta IS/OOS · por defecto
Panel IS/OOS · por defecto
SPA y StepM contra buy & hold · por defecto
Forma del beneficio · por defecto
Edge por coste · por defecto
Calidad del feed · por defecto
Spread real de Darwinex · por defecto
IS
Beneficio neto IS · por defecto
Operaciones IS · por defecto
Profit Factor IS · por defecto
Sharpe IS · por defecto
Retorno / DD IS · por defecto
DD máximo % IS · por defecto
% ganadoras IS · por defecto
Sortino Ratio IS
Esperanza en R IS
Estabilidad IS
Drawdown IS
CAGR/Max DD % IS
Calmar Ratio IS
DoF Ratio IS
PSR IS
RSquared IS
SQN IS
TRL Ratio IS
Ulcer Index % IS
ZScore IS
Param Count IS
Annual % Return IS — sin datos
2 columnas marcadas, más el nombre de la estrategia (siempre).
Restaurar vista por defecto
Cancelar
Aplicar columnas



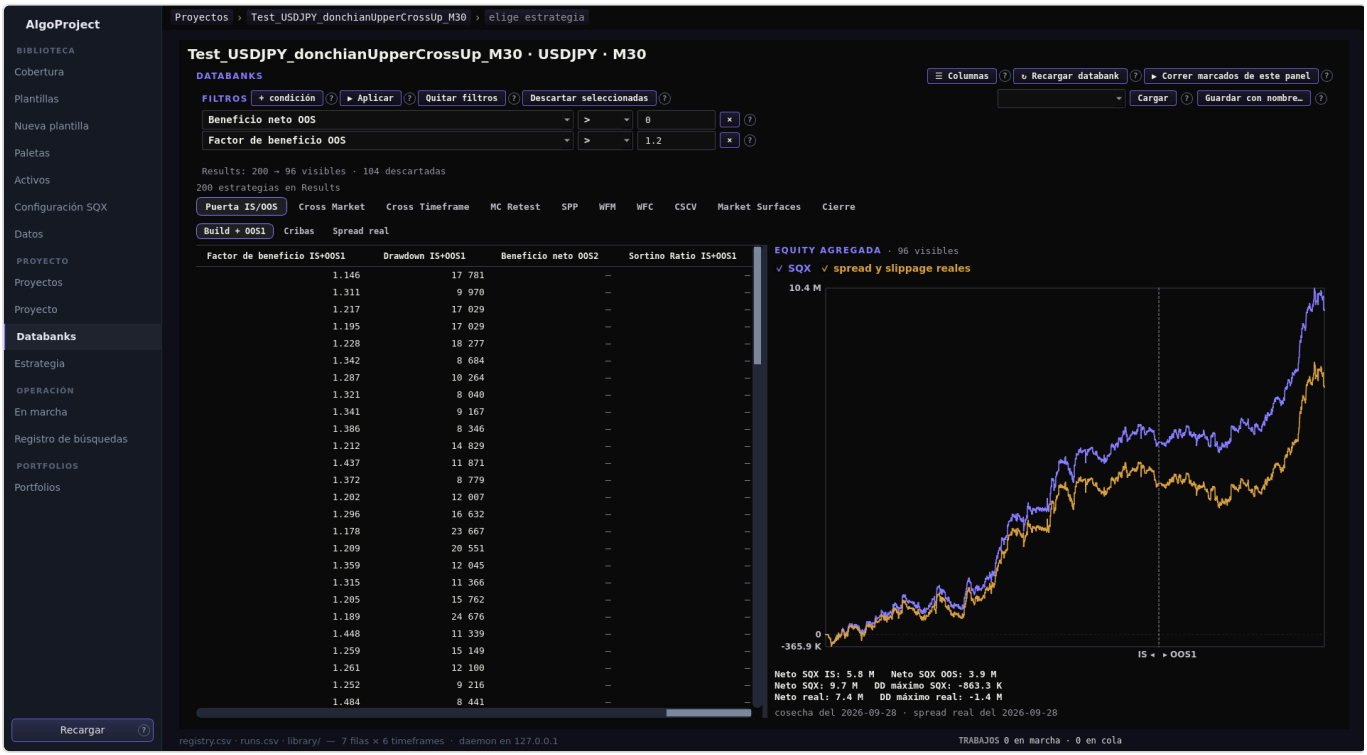
⚙ **Métricas** (se llamaba «☰ Columnas»; al pasar el ratón dice que se guarda por databank) abre la lista (la ventana la lee sin congelarse; mientras, la línea gris dice «Leyendo las columnas de …»), agrupada: las columnas del estudio de esa pestaña, y luego cada métrica en **IS**, **OOS**, **OOS2** e **IS+OOS1**. El grupo OOS dice qué tramo es cuando la cosecha lo sabe («OOS1 (en la tabla, «OOS»; lo dice la cosecha)», porque sus operaciones llevan escrito el tramo); en un databank sin cosecha (un export de métricas: SPP OOS, un retest sobre oos2, el WFM) dice «OOS (tramo de la tarea…)» y no presume que sea OOS1. Las marcadas se ven; «· por defecto» señala las de siempre; «— sin datos» dice que esa columna saldrá con «–» y, al pasar el ratón, por qué. **Aplicar columnas** deja en su sitio las que ya estaban y pone las nuevas al final; **Restaurar vista por defecto** olvida las métricas elegidas para ese databank (en todas sus pestañas) y las columnas del estudio de esa pestaña.



La caja de arriba busca sin mirar tildes ni mayúsculas: «neto» deja solo el beneficio neto en los cuatro segmentos.



La misma tabla con Sharpe, Retorno/DD y % ganadoras quitados y cinco columnas añadidas al final (desplazada a la derecha): el neto, el Profit Factor y el drawdown de **IS+OOS1** tienen cifra; el neto de **OOS2** y el Sortino de IS+OOS1 salen con «–».



Qué hay en cada segmento (medido el 2026-09-28 en `Test_USDJPY_donchianUpperCrossUp_M30`):

segmento	de dónde sale	qué métricas	qué sale con «–»
IS	la cosecha o el export de métricas de SQX	las 21 de la vista de SQX	Annual % Return (SQX no la da en IS)
OOS (OOS1 en `Results`)	lo mismo, bloque OOS	13: neto, PF, Sharpe, Sortino, Ret/DD, % ganadoras, PSR, SQN, Ulcer, ZScore, CAGR/DD, Calmar, Annual % Return	operaciones, drawdown, DD %, estabilidad, esperanza en R… (SQX no las exporta en OOS)
OOS2	una cosecha con OOS2	ninguna todavía: no existe ese export	todas
IS+OOS1	**calculada** con las operaciones IS y OOS1 de la cosecha, juntas; solo si sus operaciones OOS dicen «OOS1»	neto, operaciones, Profit Factor, % ganadoras, drawdown y Ret/DD	todas las demás (Sharpe, Sortino, SQN…: no se calculan en ningún sitio)

- Las seis de IS+OOS1 reproducen la definición de SQX: comprobadas contra su columna IS, salen iguales (operaciones y drawdown exactos; el resto dentro del redondeo de SQX).
- Solo tienen métricas los databanks con **cosecha** (`Results`) o con **export de métricas** (`SPP_IS`). En los de retest (`Retest_Markets_-_Family`, `CrossTF`, `MCR_All`, `WFM`) la lista solo ofrece las seis de IS+OOS1, y salen con «–».
- Una estrategia sin operaciones en OOS1 sale con «–» en IS+OOS1, y al pasar el ratón dice «sin operaciones OOS»: su «unión» sería solo su IS. Todo «–» explica su motivo al pasar el ratón.
- Al ordenar por una columna, los «–» quedan **siempre al final**, de mayor a menor y de menor a mayor.
- Mirar OOS2 aquí **no escribe nada en el Ledger**.

Se guarda lo que cambias, no la lista entera: qué columnas de siempre quitaste, cuáles añadiste y en qué orden. Si mañana una prueba nueva añade su columna a esa tabla, aparece sola (al final); y una columna que elegiste y que hoy ese databank no tiene se queda apuntada, sin verse, para cuando vuelva.

Filtros y columnas no se estorban: un filtro juzga todo el databank, esté la columna a la vista o no; y en un sub-panel de estudio (Cribas, MC Retest…) quitar las columnas del estudio no cambia qué filas salen — siguen siendo las que el estudio juzgó.

Solo la tabla, sin equity agregada

Ningún databank tiene equity agregada (dueño, 2026-09-29 — antes solo la Puerta IS/OOS estaba así, desde el 2026-09-28): cada pestaña de Databanks (Puerta IS/OOS, Cross Market, Cross Timeframe, MC Retest, SPP, WFM, WFC, CSCV, Market Surfaces, Cierre) muestra **solo la tabla**, a todo el ancho, con las métricas que elijas con ⚙ **Métricas**. El divisor arrastrable y la curva agregada de la cosecha que este capítulo describía antes ya no existen en ninguna pestaña.

Un ejemplo completo

Lo que el demonio devolvió para el databank `Results` de ese proyecto (200 filas, una por estrategia; 7 pruebas con columnas; los veredictos de la puerta y del MC Retest en la misma fila):

```
filas: 200 | origen filas: informes | métricas: cosecha
estudios: ['gate', 'isOos', 'crossTF', 'mcRetest', 'cloud', 'edgeCost', 'spread'] | sellado: ['wfc', 'cscv', 'wfm', 'blindJoint']
Strategy 17.28.58 67235.3 MANTENER None
con mcRetest: 8
sqx 4033 real 4033 split 2694
```

Pulsar ↻ **Recargar databank** en `Results`, que ya no está en ningún install: la línea gris lo dice y las filas siguen siendo las de la cosecha. En un databank que un install sí tiene, dice cuántas estrategias hay ahora en disco y qué se ha puesto a cargar.



La misma prueba sobre una copia de cuatro estrategias: se borró una y se tocó la carpeta.

```
filas antes: 4 | recarga 1: 4 estrategias en disco
recarga 2 tras touch: 3 estrategias en disco | filas después: 3
```

Qué NO te dice

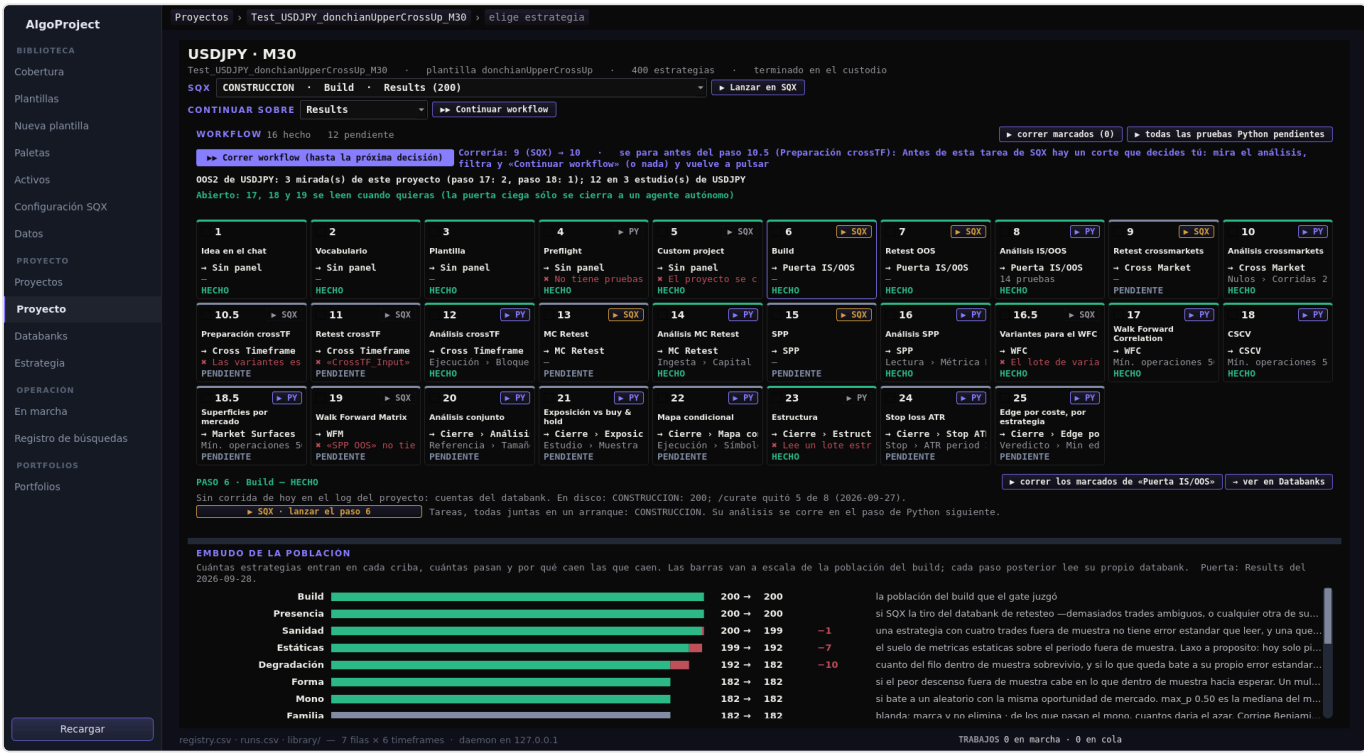
- **Un veredicto en la fila no es de este databank si la prueba corrió en otro**: el MC Retest y el cross-TF se emparejan con la puerta porque firman la misma identidad de la estrategia; el cross-market y el SPP no la firman y solo salen en su propio databank.
- **Las pruebas del lote de variantes** (nube, WFC, CSCV) se emparejan por el **nombre** de la madre, porque el lote no guarda su identidad.
- **«Origen filas: informes»** quiere decir que el databank ya no está en ningún install: las filas son las que recuerdan la cosecha y los informes. El doble clic abre una ficha que puede no encontrar la estrategia.
- **Una columna con «–» no es un cero**: es que ese dato no existe para ese segmento o esa estrategia.

Si algo falla

- **«ningún resultado de este estudio todavía»**: la prueba no ha corrido sobre ese databank; córrela desde el raíl.
- **«No se pudo leer … de …»**: un fichero que SQX o un estudio estaba escribiendo; pulsa **↻ Recargar databank**.
- **«paso de SQX: la ventana no lo lanza desde el raíl…»**: el análisis cuelga de un paso de SQX; lánzalo desde el panel de databanks.
- **«Nada que correr: marca una prueba o pulsa el ▶ de un paso.»**: no había ninguna prueba marcada ni ninguna libre en ese paso.
- **✖ en rojo en una tarjeta de SQX**: un fallo de verdad — el databank que lee está vacío, o es un build y el proyecto no está en el custodio.
- **«· workflow corriendo»** en una tarjeta, y **«No disponible: el workflow ya está corriendo»** junto a «Lanzar en SQX»: SQX ya está trabajando (este proyecto u otro en ese install). No es un fallo: espera a que termine. Los textos son cortos a propósito (dueño, 2026-09-30); **el detalle completo — qué install, qué puerto, qué proceso — sale al pasar el ratón** por encima.
- **· en el color normal, no en rojo**: el paso simplemente no se lanza desde aquí — es el 5, el 10.5 o el 16.5, que no son una tarea suelta; no es un fallo.
- **«Correr workflow: No disponible: …»**: el primer motivo de «Lanzar en SQX», en pocas palabras (todos, al pasar el ratón), o «Nada que correr» cuando el primer paso pendiente ya es una parada.
- **«No se lanzó nada.»**: respondiste «no» a la pregunta de una prueba que lee oos2 o escribe en el ledger.
- **«Sin conexión: pulsa Recargar»**: la ventana perdió su demonio; si «Recargar» no basta, ciérrala y vuelve a abrirla.
- **«No se pudieron leer las columnas de …: …»**: el demonio no dio tu elección o las cifras de IS+OOS1; la tabla enseña la vista por defecto o «–» con «no se pudo calcular: …». No se queda así: **⚙ Métricas** o **↻ Recargar databank** lo vuelven a pedir.
- **«Métricas cambiadas solo en esta sesión: …»**: la tabla ya enseña lo que elegiste, pero el demonio no pudo guardarlo; al reabrir la ventana volverá la vista anterior.
- **Una columna que elegiste ya no aparece**: el databank dejó de tener esa columna (por ejemplo la de un estudio que ya no está); se salta sin error. **Restaurar vista por defecto** la limpia.

69. Proyecto — filtros sobre un databank, y «Continuar workflow»

> **Desde el 2026-09-28 los filtros y «Continuar workflow» viven en zonas distintas.** La tira FILTROS está en la zona **Databanks**, encima de la tabla. «▶▶ Continuar workflow» está en la zona **Proyecto**, en la fila **CONTINUAR SOBRE**, debajo de «Lanzar en SQX», con **su propio desplegable de databank**: al entrar sigue al databank que tengas abierto en Databanks (si no, el primero con descartes); si eliges otro, se queda con el tuyo hasta que cambies de proyecto. La confirmación y sus comprobaciones son las mismas de siempre.



Qué pregunta responde

- **¿Qué queda si me quedo solo con las estrategias que cumplen esto?** Encima de la tabla del panel de databanks hay una tira **FILTROS**. Cada fila es una condición — métrica, operador y valor — y todas se combinan con **Y** (AND): una estrategia se queda solo si cumple todas. Al pulsar **▶ Aplicar**, la tabla esconde las que no pasan, la equity agregada de la derecha se recalcula **solo con las visibles** («183 visibles») y el embudo de la zona Proyecto gana una fila nueva («Filtro · Results 200 → 183»).
- **¿Y si me he pasado?** Cambia el valor y vuelve a pulsar **▶ Aplicar**. Cada «Aplicar» se calcula sobre **todas** las estrategias del databank y **sustituye** al filtro anterior: si pones «Sharpe OOS > 1» y te quedan 30 de 200, y lo aflojas a «> 0,5», vuelven a aparecer las que ahora pasan (110). Mientras escribes, una línea bajo las filas va diciendo cuántas pasarían, sin esconder nada todavía.
- **¿Y si quiero quitar a mano las que no me convencen?** Selecciona filas en la tabla y pulsa **Descartar seleccionadas**. Cuenta igual que un filtro, y esas **siguen escondidas** aunque cambies el filtro.
- **¿Cuántas búsquedas llevo en este estudio?** Cada filtro aplicado y cada borrado a mano se apunta **en el Ledger** como una búsqueda más (qué criterio, de cuántas a cuántas, en qué paso y tramo), y se ve en OPERACIÓN › **Registro de búsquedas** (capítulo 70). Mirar los datos y quedarte con una parte es buscar, y el Ledger existe para contarlo.

Cuándo lo usas, y cuándo no

Lo usas para cribar una población a mano mientras la miras: pruebas una regla, la aprietas o la aflojas hasta que el recorte te parece razonable, y borras a mano tres que no te gustan. **Manda el último filtro aplicado**: no se apilan. Si quieres dos reglas a la vez, ponlas como dos filas del mismo filtro (se combinan con Y). Los borrados a mano, en cambio, sí se acumulan y se respetan con cualquier filtro.

No borra nada en StrategyQuant X. Filtrar es una vista en Python: el databank de SQX sigue con todas sus estrategias (sus `.sqx` no se tocan) hasta que pulses «Continuar workflow», que es lo que de verdad las quita (segunda parte de este capítulo).

No se puede filtrar por métricas de **OOS2**: no aparecen en el desplegable. Tampoco los estudios que pueden leerlo: WFC, CSCV, superficies por mercado, WFM, paso 20, stop ATR y exposición (la lista sale de `assets/_policy.yaml`, a quién reserva el oos2 del activo, más esos fijos). Los databanks del WFC y del WFM se niegan enteros.

Las filas **sin identidad** (estrategias que solo nombra un informe antiguo) se ven en la tabla, pero ningún filtro las puede esconder: la tira y el embudo las cuentan aparte, «N sin identidad, no filtrables».

Antes de empezar

- La ventana abierta, en **Databanks**, con un databank en pantalla que tenga tabla (no un panel bloqueado).
- El proyecto tiene que tener **plantilla** en `AlgoData/projects/registry.csv`: el Ledger apunta cada búsqueda en el estudio `<símbolo>_<timeframe>_<plantilla>`. Sin plantilla no hay dónde apuntar, y el filtro **no se aplica** (la tira lo dice).
- Nada de SQX: ni hace falta el install arrancado ni se le envía ningún comando.

Cómo se ejecuta

No hay comando: es un botón. Paso a paso:

1. En la fila vacía, elige la **métrica**. El desplegable lista las columnas que ese databank tiene de verdad (las de SQX con su tramo, «Net profit (OOS)», y las de los estudios ya corridos, «Spread.operaciones [OOS]»), y al final las **distribuciones** guardadas («Distribución · Panel IS/OOS · Retorno por operación [USD/lote]»). Al pasar el ratón por una métrica dice cuántas estrategias tienen valor.
2. Elige el **operador**: `>`, `<`, `≥`, `≤`, `=`, `entre` (dos valores, «desde» y «hasta»). Una columna de texto (un veredicto) solo admite `=`. Una distribución admite «mediana dentro del intervalo» y «mediana fuera del intervalo».
3. Escribe el **valor**. Para una distribución, el valor es el intervalo en %: **50, 80, 90 o 98**.
4. **+ condición** añade otra fila; **×** la quita.
5. Mientras cambias una métrica, un operador o un valor, la línea «Con estas condiciones, sin aplicar: 200 → 110 pasan…» se recalcula sola (medio segundo después de dejar de escribir). No esconde nada ni se apunta en ningún sitio: es para ver cuánto aprietas antes de decidir.
6. ▶ **Aplicar**. La tabla cambia: se esconden las que no pasan y **vuelven** las que el filtro anterior escondía y este deja pasar.

botón	qué hace
▶ Aplicar	aplica todas las filas a la vez (Y) sobre **todo** el databank, **sustituye** al filtro anterior, apunta una fila en el Ledger y esconde lo que no pasa. Los borrados a mano siguen escondidos. Si aplicas el mismo filtro dos veces y el databank no ha cambiado entre medias, la segunda no se apunta; si SQX añadió o borró estrategias, se recalcula y sí se apunta. Sin ninguna condición escrita, quita solo el filtro (y no apunta nada)
Quitar filtros	quita el filtro **y** los borrados a mano: vuelve a enseñar todo. Lo apuntado en el Ledger **se queda**: lo que se miró, se miró
Descartar seleccionadas	esconde las filas seleccionadas y lo apunta en el Ledger como borrado a mano. Aguantan cualquier cambio de filtro
Guardar con nombre…	guarda las filas de la pantalla con un nombre, para reutilizarlas
Cargar	pone en las filas el filtro guardado elegido en el desplegable; **no** lo aplica

Al volver a abrir el databank (o tras «Recargar databank» o un «Continuar workflow»), las filas muestran las condiciones del filtro que está en vigor; si estabas escribiendo una, no se te borra. Tarda medio segundo en un databank de 200 estrategias (el recuento en vivo, unas décimas) (la primera vez que se abre un proyecto, unos 2 s en leer sus distribuciones).

El modo distribución, en concreto:

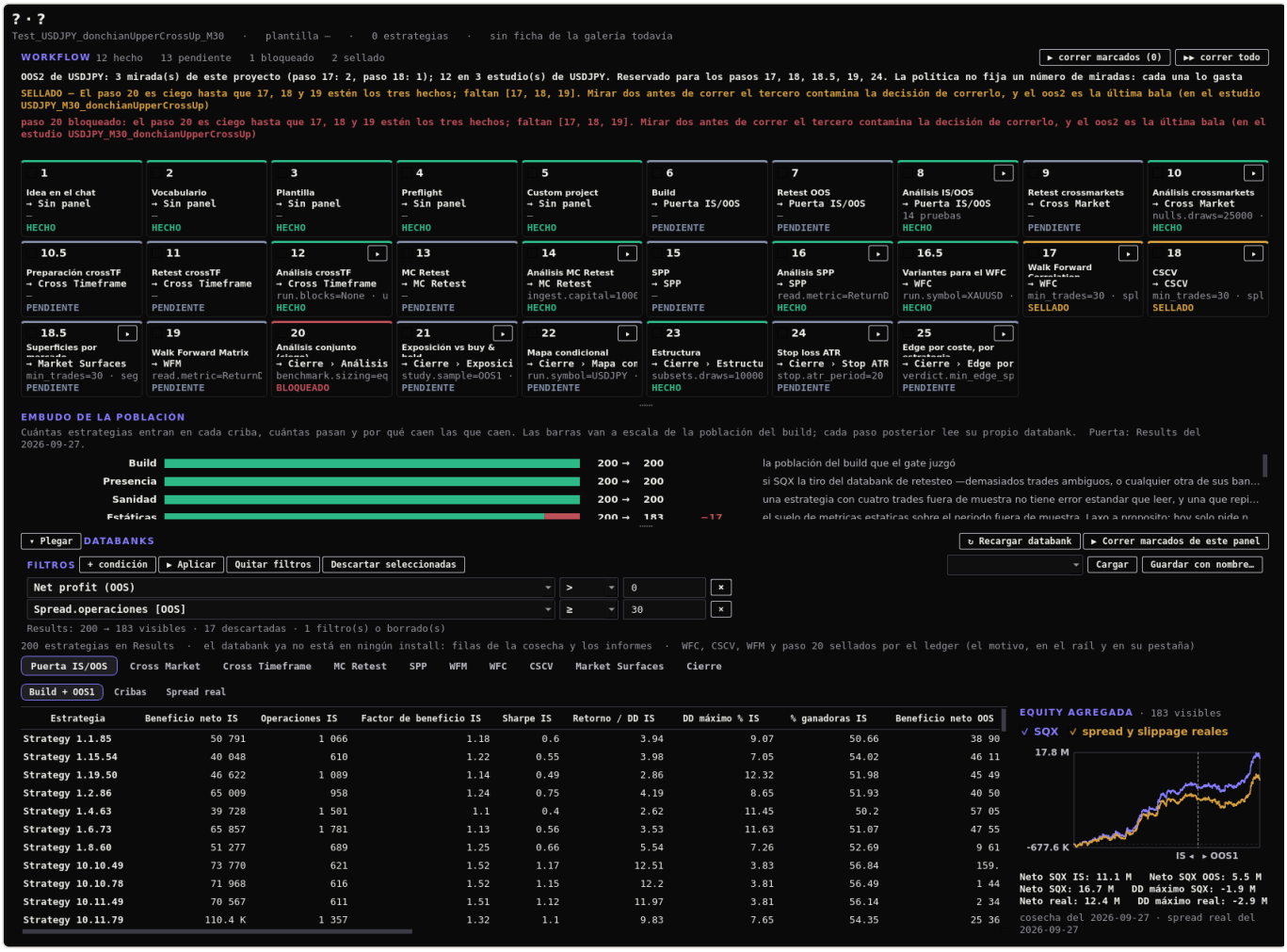
- Si la distribución tiene **dos muestras** (el panel IS/OOS: la de IS y la de OOS superpuestas), «mediana dentro del intervalo 90» significa: **la mediana de OOS cae dentro del 90 % central de IS** (entre su percentil 5 y su 95).
- Si tiene **una** (las re-ejecuciones del MC Retest), significa: **el valor real** (el backtest que ocurrió, la línea vertical) cae dentro del 90 % central de las re-ejecuciones.

Qué produce

ruta	qué es
`AlgoData/filters/<proyecto>/<databank>/discards.jsonl`	la lista de descartes: una línea por filtro o borrado (criterio, de cuántas a cuántas, hora) y una por estrategia escondida (su identidad, su nombre, si fue filtro o a mano), y las condiciones del filtro. Solo se añade: lo que se esconde ahora es lo del **último** filtro más todos los borrados a mano; «Quitar filtros» escribe una línea de corte y lo anterior queda como historia. Sobrevive a reiniciar la ventana
`AlgoData/filters/saved.yaml`	los filtros guardados con nombre
`AlgoData/ledger/<símbolo>_<timeframe>_<plantilla>.jsonl`	una línea más por filtro aplicado o borrado: paso, tramo, criterio, `n_in` (todo el databank), `n_out`, lanzado por «ventana». El recuento en vivo mientras escribes **no** escribe aquí

El paso es el de Python que lee ese databank (el databank `Results`, del build, es el **paso 8**, «Análisis IS/OOS»). El tramo es `build` si el filtro solo mira columnas IS, y `oos1` si mira algo fuera de muestra (una columna OOS, un estudio, una distribución). Un borrado a mano es `oos1`.

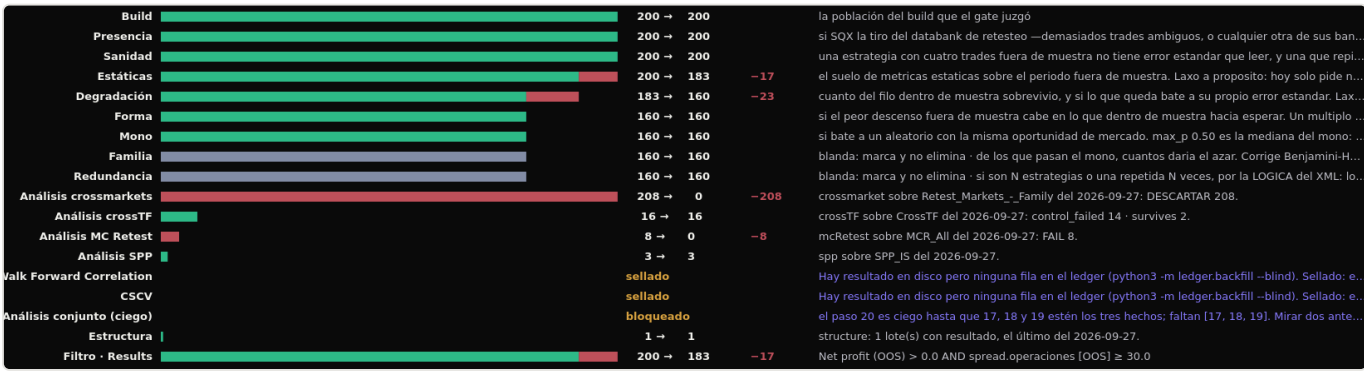
Cómo se lee el resultado



La línea bajo las filas dice cómo queda el databank: «Results: 200 → 183 visibles · 17 descartadas», y si alguna es un borrado a mano, cuántas: «· 20 descartadas (3 a mano)». Encima, mientras editas, la otra línea: «Con estas condiciones, sin aplicar: 200 → 110 pasan · se verían 107 contando los descartes a mano · ▶ Aplicar para que la tabla cambie». Al aplicar dice además dónde se apuntó: «200 → 183 · apuntado en el Ledger: USDJPY_M30_donchianUpperCrossUp, paso 8, oos1». Una estrategia **sin valor** en una métrica (una que el MC Retest no probó, por ejemplo) **no se esconde**: esa condición no la juzga y sigue visible. Se cuenta aparte, entre paréntesis y en el embudo: «(191 sin valor, siguen visibles)». Ejemplo real: «MC Retest · Parámetros fuera del intervalo 90 Y Factor de beneficio IS entre 1,1 y 3» sobre las 200 de `Results` → 200 → 198, 2 escondidas y 191 sin valor (solo 8 pasaron por el MC Retest).



En el embudo, el filtro en vigor es una fila al final, con su criterio como explicación, y detrás una fila por cada borrado a mano (contando solo las que el filtro no escondía ya). Al cambiar el filtro, su fila se sustituye; no se añade otra:



Un ejemplo completo

Proyecto `Test_USDJPY_donchianUpperCrossUp_M30`, databank `Results` (200 estrategias). Filtro: **Net profit (OOS) > 0 Y Spread.operaciones [OOS] ≥ 30** (**la cosecha no trae la columna «# of trades (OOS)» de SQX**, solo la de IS; hasta que la traiga, las operaciones fuera de muestra las cuenta el estudio de spread, `Spread.operaciones [OOS]`).

Respuesta del demonio al aplicar (2026-09-28):

```
n_in 200 · n_out 183 · blank 0
expression: Net profit (OOS) > 0.0 AND spread.operaciones [OOS] ≥ 30.0
ledger: USDJPY_M30_donchianUpperCrossUp · paso 8 · oos1
```

Y en el Ledger (`python3 -m ledger.report --study USDJPY_M30_donchianUpperCrossUp`):

```
USDJPY_M30_donchianUpperCrossUp · 6 búsquedas
-- el embudo, paso a paso
                      ts  step   criterion                                                segment
n_in  n_out  kept
2026-09-27T20:30:36+00:00  18.5   marketSurfaces: 10 mercados (…) x 150 variantes, NetProfit…   build
150    150 1.000
…
2026-09-28T07:18:12+00:00   8.0   Net profit (OOS) > 0.0 AND spread.operaciones [OOS] ≥ 30.0    oos1
200    183 0.915
2026-09-28T07:24:02+00:00   8.0   snoopingScreen/stepm                                         oos1
200    200 1.000
de 150 entraron a 200 supervivientes
```

(La fila del filtro es la de las 07:18:12. La de las 07:24 es otra búsqueda del mismo paso 8, el estudio snoopingScreen, lanzada desde el raíl.) Ojo con la última línea: el informe resume «de la primera fila a la última» por hora, y las filas del paso 8 se apuntaron después que las del 18.5. Lee la tabla, no ese resumen. Desde el 2026-09-28 los valores de las condiciones llegan al Ledger como números (`30`, no `"30"`), aunque en la ventana se escriban como texto.

La tabla pasa de 200 a 183 filas visibles; tras cerrar y abrir la ventana (o reiniciar el demonio) siguen escondidas las mismas 17.

Apretar y aflojar, mismo databank (recuentos reales del demonio, 2026-09-28):

condición	pasan	escondidas
Sharpe OOS > 1	30	170
Sharpe OOS > 0,5 (aplicado después)	110	90 — vuelven 80
Beneficio neto OOS > 0 Y Factor de beneficio OOS > 1,2	93	107

Cada fila es un «Aplicar» y una fila del Ledger; el segundo no se aplica sobre las 30 del primero, sino sobre las 200.

Qué NO te dice

- **Que las 183 sean buenas.** Un filtro elegido mirando el OOS **gasta el OOS**: cuantos más pruebes, más fácil es que las supervivientes lo sean por suerte. Por eso cada uno se apunta en el Ledger, y el informe del Ledger descuenta el Sharpe por todas las búsquedas.
- **Que SQX las haya borrado.** Siguen todas en el databank de SQX hasta «Continuar workflow».
- **Que el recuento en vivo sea gratis.** No se apunta en el Ledger (solo «Aplicar» cuenta como búsqueda), pero ver cuántas pasan con cada valor **también es mirar el OOS**. Si pruebas veinte valores antes de aplicar uno, el Ledger ve una búsqueda y tú has hecho veinte.
- **Nada con O (OR).** Solo Y; para un O, aplica un filtro y luego el otro (el segundo sustituye al primero) — y los dos quedan apuntados.

Si algo falla

- «el proyecto … no tiene plantilla en projects/registry.csv…»: sin plantilla no hay estudio del Ledger; no se aplica.
- «ninguna tarea de … escribe …»: el `project.cfx` del install ya no dice qué tarea escribe ese databank (o ningún install tiene el proyecto): no se sabe su paso, no se apunta, no se aplica.
- ««X» no es una columna de este databank (o es de OOS2…)»: la métrica no existe ahí o está prohibida. Mira el desplegable: solo lista las que hay.
- «WFC_OOS2 es de la etapa wfc, que lee OOS2: no se filtra»: ese databank no se filtra.
- «el filtro «…» es de antes de que los filtros se recalcularan y no está en vigor»: ese databank se filtró con la versión anterior (antes del 2026-09-28), en la que cada filtro se apilaba sobre el anterior y no se podía aflojar. Esos filtros ya no esconden nada; las filas muestran las condiciones del último, y la línea en vivo dice cuántas pasarían. Pulsa ▶ **Aplicar** para volver a aplicarlo sobre todo el databank. En `Test_USDJPY_donchianUpperCrossUp_M30` / `Results` el último era «Beneficio neto OOS > 0 Y Factor de beneficio OOS > 1,2»: al aplicarlo quedan 93.

«Continuar workflow» — borrar de verdad los descartes y lanzar la tarea siguiente

Qué pregunta responde

Ninguna: **ejecuta lo que ya decidiste** con los filtros. Los filtros solo esconden; este botón es el que **borra en StrategyQuant X** las estrategias descartadas de ese databank y, a continuación, **lanza la tarea siguiente** del proyecto, para que esa tarea lea solo las supervivientes. Es el único botón de la ventana que cambia lo que hay dentro de SQX.

Borra **exactamente lo que la tabla esconde en ese momento**: lo que deja fuera el último filtro aplicado más los borrados a mano. Lo que un filtro anterior escondía y el actual deja pasar no se borra. El recuento en vivo sin aplicar no cuenta: si cambias un valor, pulsa ▶ Aplicar antes.

Cuándo lo usas, y cuándo no

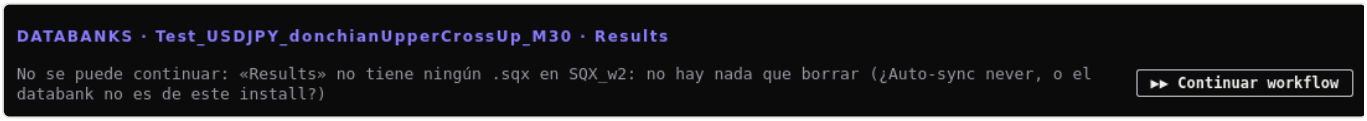
Lo usas cuando has terminado de filtrar un databank y quieres seguir el workflow: por ejemplo, cribaste `OOS` en el paso 8 y quieres que el retest de cross markets (paso 9) corra solo con lo que queda.

No lo usas para probar un filtro: para eso están los filtros, que no tocan nada. Y **no** sirve para un proyecto del maestro: la ventana nunca toca tu install (`SQX`), solo los workers (conductor `SQX_w1` , custodio `SQX_w2`).

Antes de empezar

El botón está **apagado** y dice al lado por qué, hasta que se cumplen todas estas condiciones (se comprueban en este orden):

1. El proyecto está en `AlgoData/projects/registry.csv` y vive en un **worker**. Si vive en el maestro se niega siempre, antes de mirar nada más.
2. **Nadie usa ese worker**: su candado de propietario (`user/log/OWNER` , OPEN §32) no lo tiene otra sesión viva, su puerto no responde y no hay ningún proceso de SQX trabajando desde su carpeta. Si el worker está arrancado, **se niega y no lo para** (parar un worker mata la ejecución de quien lo esté usando): páralo tú si sabes que está ocioso. Que otro proyecto de ese worker se tocara hace un rato ya no cuenta (OPEN §83): un worker libre acepta cualquiera.
3. El log de SQX de hoy de ese worker lleva **15 minutos** sin escribirse y no dice que un proyecto haya empezado sin terminar.
4. Hay **descartes** en ese databank (un filtro aplicado o estrategias descartadas a mano).
5. El databank tiene sus `.sqx` **en disco** en ese worker.
6. Existe una **tarea siguiente**: la primera tarea, en el orden del workflow después del paso que escribe ese databank, que lo **lee**. Para `Results` es `OOS` ; para `OOS` , `Retest Markets - Family` ; para `SPP OOS` , `WFM` . Se activa **solo esa tarea**.



Cómo se ejecuta

No hay que teclear nada: es un botón de la zona Proyecto, en la fila CONTINUAR SOBRE.

1. Filtra el databank en la zona Databanks (primera parte de este capítulo).
2. Vuelve a **Proyecto**, mira que el desplegable de CONTINUAR SOBRE diga ese databank (lo sigue solo) y pulsa ▶▶ **Continuar workflow**. Aparece la pantalla de confirmación con esta frase literal: «Se van a borrar **N** estrategias de en sobre , y después se lanzará ». Si SQX guarda alguna estrategia repetida con otro nombre, la frase lo añade al final («(19 ficheros: SQX guarda algunas repetidas con otro nombre, y se borran todas las copias)»).
3. **No** cancela sin tocar nada. **Sí** encola un trabajo, que sigues en la franja de trabajos y en OPERACIÓN › «En marcha» (capítulo 70).

El trabajo es un comando que también se puede lanzar desde la terminal (hace las mismas comprobaciones y se niega igual):

```
python3 -m ui.daemon.advance.run --project Test_UiContinuar_USDJPY --databank Results
```

flag	obligatorio	qué hace
`--project`	sí	el proyecto, tal y como aparece en `registry.csv`
`--databank`	sí	el databank cuyos descartes se borran; vale con espacios o con guiones bajos

Lo que hace, en este orden:

1. Vuelve a comprobarlo todo (dos clics no pueden pisarse: solo corre uno a la vez).
2. **Copia** los `.sqx` que va a borrar a `AlgoData/projects/discards/<proyecto>/<databank>/<fecha-hora>/`, con un `verdict.csv` (estrategia, `DESCARTAR`, identidad, el filtro que la descartó).
3. Los **borra** del databank con el worker parado, con el mismo código que `/curate` (`sqx.curate.apply_verdict`), que antes comprueba que cada fichero es la estrategia que juzgaste.
4. Apunta una fila en el **Ledger** (el borrado, de cuántas a cuántas) y marca los descartes como gastados en su lista (una línea `cut`).
5. Deja **activa solo la tarea siguiente** en el `project.cfx`.
6. **Arranca** el worker, espera a que responda y lanza el proyecto.
7. Mientras corre, solo le pregunta «¿cómo vas?» (`-project action=status`): ninguna otra orden.
8. Cuando el log de SQX dice «Project finished», **para** el worker. Lee también el log de ayer, así que una tarea que cruza la medianoche se sigue igual.

Tiene dos topes, para que el trabajo nunca se quede colgado con el worker arrancado: si SQX no escribe «Starting project» en los **2 minutos** siguientes a lanzarlo, o si la tarea pasa de **48 horas** sin terminar, para el worker y falla diciendo cuál de los dos fue. Si el arranque del worker falla a medias, también lo para.

Cuánto tarda: lo que tarde la tarea siguiente en SQX, más ~1 minuto de arrancar y parar.

Qué produce

ruta	qué es
`AlgoData/projects/discards/<proyecto>/<databank>/<fecha-hora>/*.sqx`	copia de cada estrategia borrada, por si hay que recuperarla
`…/<fecha-hora>/verdict.csv`	qué se borró y por qué
`AlgoData/reports/<proyecto>/<databank>/<día>/curate/before-HHMMSS.csv`, `rejected-HHMMSS.csv`	lo que había antes del borrado y las métricas de lo borrado (como `/curate`)
el databank en el worker	con N estrategias menos — **esto no se deshace solo**: para recuperar, copia los `.sqx` de vuelta con el worker parado
`AlgoData/ledger/<símbolo>_<timeframe>_<plantilla>.jsonl`	una fila más: el borrado
las salidas de la tarea siguiente	su databank en el worker, como si la hubieras lanzado tú

Cómo se lee el resultado

En «En marcha» el trabajo `advance` va diciendo por dónde va: «copiando 17 descartes», «borrando 17 de Results (SQX_w1)», «activando solo OOS», «arrancando el conductor», «OOS lanzada», el porcentaje de la tarea, y al final «OOS: done; Results 200 → 183». Si se niega, la última línea dice por qué y no ha tocado nada.

Un ejemplo completo

Aún no se ha corrido contra un install de verdad: la prueba en vivo (en el conductor, con el proyecto `Test_UiContinuar_USDJPY`) espera tu autorización. Lo que sí es real es la negativa del 2026-09-28 sobre el proyecto del custodio:

```
GET /api/advance/preflight?project=Test_USDJPY_donchianUpperCrossUp_M30&databank=Results
{"ok": false,
 "reasons": ["«Results» no tiene ningún .sqx en SQX_w2: no hay nada que borrar (¿Auto-sync never,
              o el databank no es de este install?)"],
 "install": "SQX_w2", "task": "OOS", "n": 17}
```

Había 17 descartes y la tarea siguiente era `OOS`, pero el databank `Results` de ese proyecto no tiene sus `.sqx` en el disco del custodio: no hay nada que borrar, y el botón se queda apagado.

Qué NO te dice

- **Que el corte sea bueno.** Borra lo que tú descartaste; el juicio es el de los filtros.
- **Que la tarea siguiente haya ido bien.** Solo que terminó: mira su databank y sus estudios.
- **No lanza varias tareas.** Si el databank lo leen varias (a `Results` lo leen `OOS`, los ocho MC Retest y `SPP IS`), solo lanza la primera del workflow.

Si algo falla

- **«install ocupado: el puerto … responde (lo tiene … desde …)»** — el worker está arrancado, y el candado dice quién lo arrancó y cuándo. Si eres tú en otra terminal, páralo con `bin/sqx-worker.sh --role conductor stop`; si es otra sesión, espera o confirma con ella — `bin/sqx-worker.sh --role conductor stop --force` solo si estás seguro de que ya no está.
- **«N de M descartes ya no están en «Results»»** — el databank cambió desde que filtraste; el botón se queda apagado. Quita los filtros y vuelve a filtrar. (Tras un «Continuar» que sale bien no pasa: el corte deja una línea en la lista de descartes y los da por gastados; el siguiente filtro empieza de lo que queda.)
- **«SQX no escribió «Starting project …» en 120 s»** o **«SQX rechazó `action=start`»** — SQX no arrancó la tarea. El worker ya está parado y el databank ya está cortado (la copia sigue en `projects/discards/`): lánzala a mano o vuelve a filtrar.
- **Cancelaste el trabajo a mitad** — el worker se queda arrancado: páralo con `bin/sqx-worker.sh --role <conductor|custodian> stop`.

68. Estrategia — la ficha de una estrategia: curvas, estadísticas, tests, metadatos y «Archivar»

Qué pregunta responde

- **¿Cómo le fue a esta estrategia, con los costes de SQX y con los de verdad?** Arriba, la curva de equity de SQX (violeta) y la misma con el **spread y el slippage reales** de Darwinex (naranja), cada una con su interruptor. Cada curva cambia de tono en OOS1 — violeta claro la de SQX, roja la real — y la leyenda dice cuál es cuál; la línea discontinua separa IS de OOS1 y el eje de abajo marca cada año.
- **¿Cómo se reparten sus operaciones?** Las estadísticas básicas de cada tramo — IS, OOS1, IS+OOS1 (los dos juntos) y OOS2 —: operaciones, beneficio neto (de SQX y con costes reales) y Profit Factor en letra algo mayor, % ganadoras, DD máximo, Sharpe y, del **retorno por operación**, media, desviación típica, asimetría y **curtosis** en forma corta («20.98 $/lote»). Al lado, el histograma, y debajo sus percentiles 1%, 5%, 10%… 99% en dólares.
- **¿De qué está hecha?** El botón pequeño **metadatos**, arriba a la derecha, abre en una ventana aparte: dirección, la señal de entrada leída como frase, cada condición con sus parámetros, las órdenes (SL, PT, salida por barras), el money management, el cierre de los viernes, el activo, el último test que guarda el `.sqx` y **sus costes**, con un aviso ámbar si una tarea del proyecto hoy carga otros. Ya no es una columna fija (dueño, 2026-09-30): ocupaba el ancho que necesitan los resultados de los tests.
- **¿Cómo se ve IS al lado de OOS?** La primera pestaña de abajo, **Ficha**, pone IS a la izquierda y OOS a la derecha, con los mismos colores que la curva de arriba: el **P&L acumulado** (SQX y con spread), el **Drawdown** máximo de cada tramo y el **P&L por año** en columnas. «Excluir top X% de trades» añade, discontinuas, las curvas sin sus mejores operaciones.
- **¿Qué dijeron de ella los estudios?** Debajo, las pestañas de siempre (Ficha, Cribado, Transferencia, Rotura, Optimización, Cierre, Lecturas, Datos), que se abren en la familia del panel desde el que llegaste: desde «Puerta IS/OOS», en **Cribado**; desde «MC Retest», en **Rotura**; desde «SPP» o «WFC», en **Optimización**. **Fuera de la Ficha, el panel básico se pliega solo** (dueño, 2026-09-29): las curvas de precios y las estadísticas son datos generales que solo hacen falta en la Ficha, y en las demás pestañas ese hueco es del estudio — vuelve a desplegarse al volver a Ficha. **El resultado de cada test ocupa todo el ancho**: sus parámetros se abren con ⚙ **Configuración** y sus corridas anteriores con **Historial**; el mismo botón los vuelve a cerrar (dueño, 2026-09-30).
- **¿Qué tests tienen datos en este databank?** Cada databank guarda lo de su paso: Results la puerta IS/OOS, `Retest Markets` el cross-market, `MCR_All` el MC Retest, `WFM` la Walk Forward Matrix… Una pestaña que aquí no tiene resultado **ni** se puede correr lleva **«· sin datos»** (y su familia también, si le pasa a todos sus tests). Abierta, dice por qué y, si el databank de ese test tiene una estrategia **con el mismo nombre**, ofrece → **abrir en …** para ir allí.

Cuándo lo usas, y cuándo no

Lo usas cada vez que una estrategia te llama la atención en el panel de databanks: doble clic en su fila y se abre su ficha. Y cuando decides que merece guardarse, **Archivar**.

No sirve para juzgar la estrategia: la ficha describe, no filtra. Las cribas y sus veredictos viven en las pestañas de abajo, cada una con su estudio.

OOS2 está abierto (dueño, 2026-09-28: la regla de no mirarlo antes de tiempo ya no se aplica a humanos). Se lee de una cosecha que traiga la muestra oos2; si no hay ninguna, la ficha dice «OOS2 abierto, sin export: exporta el retest oos2» y no se inventa otra fuente. Sólo un agente autónomo (`ALGO_AUTONOMOUS=1`) lo encuentra apagado, con «reservado: se abre tras los pasos 17, 18 y 19».

Antes de empezar

- La ventana abierta (`bin/algoui`), el proyecto elegido en la galería y un databank con **cosecha** (paso 8, `/oos-gate`): las curvas y las estadísticas salen de ella. Sin cosecha la ficha lo dice en rojo con el comando que la hace.
- La curva real necesita el informe del estudio **spread** de ese databank (paso 8). Si no existe, en su sitio aparece «spread y slippage reales: no calculado» con dos botones.
- Los metadatos se leen del `.sqx`: primero del install que tiene el databank; si el databank está vacío en el install (lo normal cuando el proyecto terminó), de la copia que dejó un export en `AlgoData/raw/…/strategies/`; y si tampoco, de la última versión archivada. Sin ninguna, la ventana de metadatos dice «sin .sqx en ningún install».
- Para «Archivar», el proyecto tiene que tener **plantilla** en `AlgoData/projects/registry.csv`: con ella se apunta la cuenta del ledger en el archivo.

Cómo se ejecuta

No hay comando: se abre con doble clic en una fila del panel de databanks (zona Databanks), o desde la paleta (Ctrl+K) eligiendo una estrategia.

botón	dónde	qué hace
SQX / spread y slippage reales	encima de la curva	enciende o apaga cada curva; el eje se ajusta a las que quedan
IS / OOS1 / IS+OOS1 / OOS2	estadísticas	el tramo que se enseña (IS+OOS1: los dos seguidos, sin Sharpe de SQX); OOS2 apagado sólo si no hay cosecha que lo traiga
USD por lote / USD por operación	estadísticas	en qué se cuenta el retorno de cada operación; por lote quita el tamaño de la posición
calcular	junto a un «no calculado»	encola el estudio que falta para **esta** estrategia (el spread real: `studies.data.spread.report --strategy …`)
todo el databank	junto a un «no calculado»	lo encola para todas las estrategias del databank, en paralelo
Excluir top X% de trades	pestaña Ficha	dibuja también cada curva de P&L acumulado (SQX y spread, IS y OOS) sin su X % de operaciones mejores; X empieza en 5 y se cambia en la casilla
Drawdown en % / $	pestaña Ficha	el drawdown en % del saldo en el pico (por defecto) o en dólares
Archivar	arriba a la derecha	pide el paso y una nota y congela la estrategia en `AlgoData/archive/`
▾ plegar panel básico	arriba a la derecha	esconde curvas y estadísticas a mano; solo aparece en la pestaña **Ficha**
metadatos	arriba a la derecha, el último	abre los metadatos del `.sqx` en una ventana aparte; se cierra con la ✕ de la ventana
⚙ Configuración	a la derecha de la barra de correr de cada test	despliega los parámetros del test (lo que cambias vale solo para la próxima corrida, como `--set`); otra pulsación lo pliega
Historial	al lado de ⚙ Configuración	despliega las corridas anteriores del test, para ver una vieja o comparar dos, o esta estrategia contra otra
→ abrir en …	en un test «· sin datos», junto a su explicación	abre la estrategia del mismo nombre en el databank de ese test, en esa misma pestaña

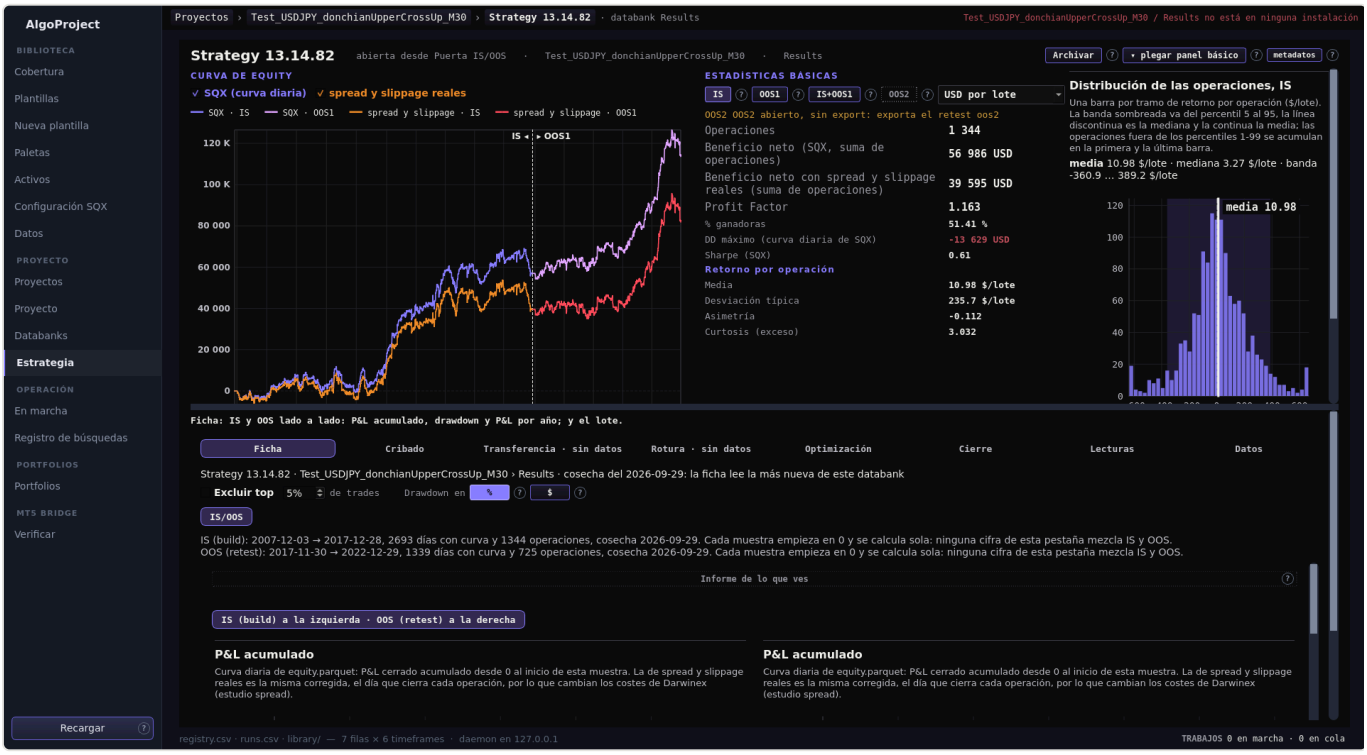
Los botones «calcular» no tocan SQX: son estudios de Python en la cola de la ventana. La ficha se redibuja sola cuando terminan. «Archivar» tampoco toca SQX: lee ficheros y tarda un par de segundos.

Qué produce

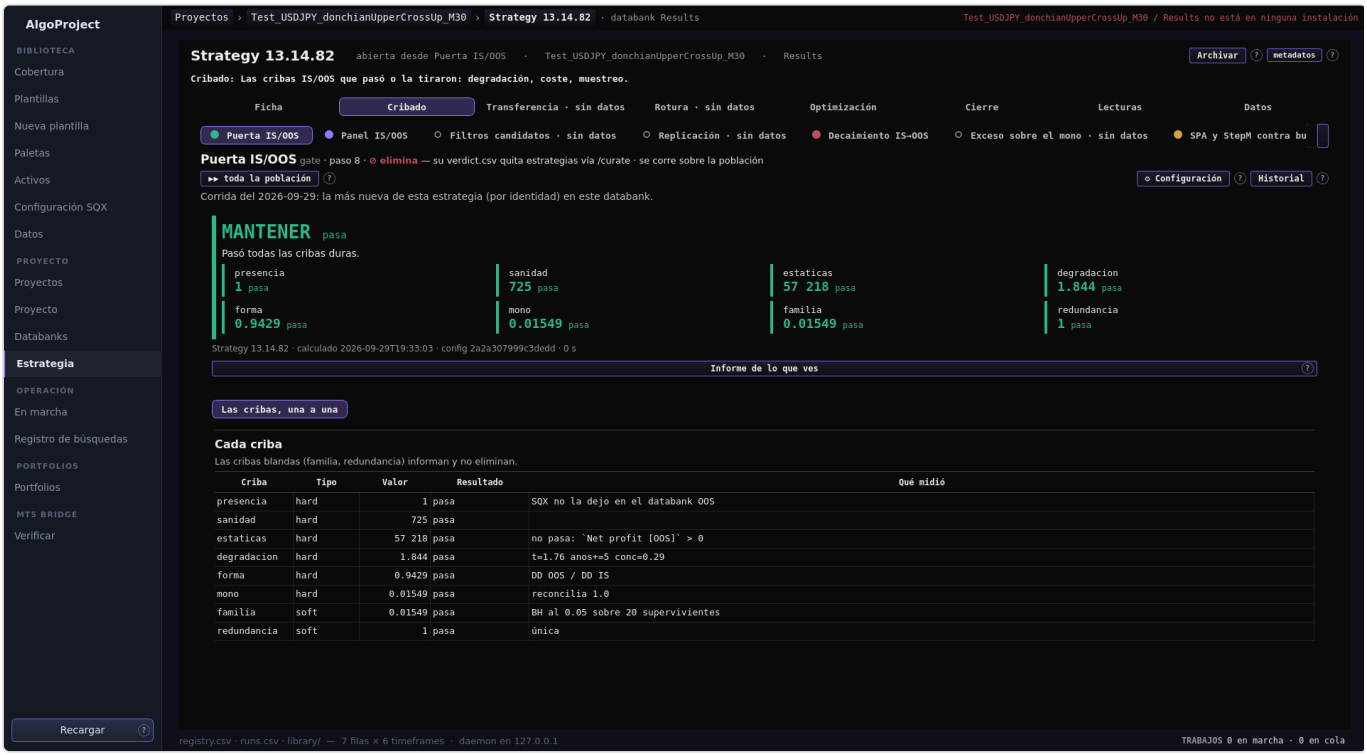
Mirar no escribe nada. Dos botones sí:

- **calcular / todo el databank** — el informe del estudio en `AlgoData/reports/<proyecto>/<databank>/<día>/spread/` (`estrategias/<nombre>.json` y `.trades.parquet` para una; `spread.json`, `trades.parquet` y `verdict.csv` para todo el databank).
- **Archivar** — una versión nueva en `AlgoData/archive/<identidad>/<AAAA-MM-DDTHHMM>/` con `manifest.json` (paso, nota, de dónde salió el `.sqx`, cada estudio archivado y lo que no se pudo archivar), `strategy.sqx`, `view.json`, las filas de la cosecha, los informes, la ficha de costes del activo y `meta.json`. Se sella de solo lectura; **nunca sobrescribe**: el mismo minuto dos veces se niega.

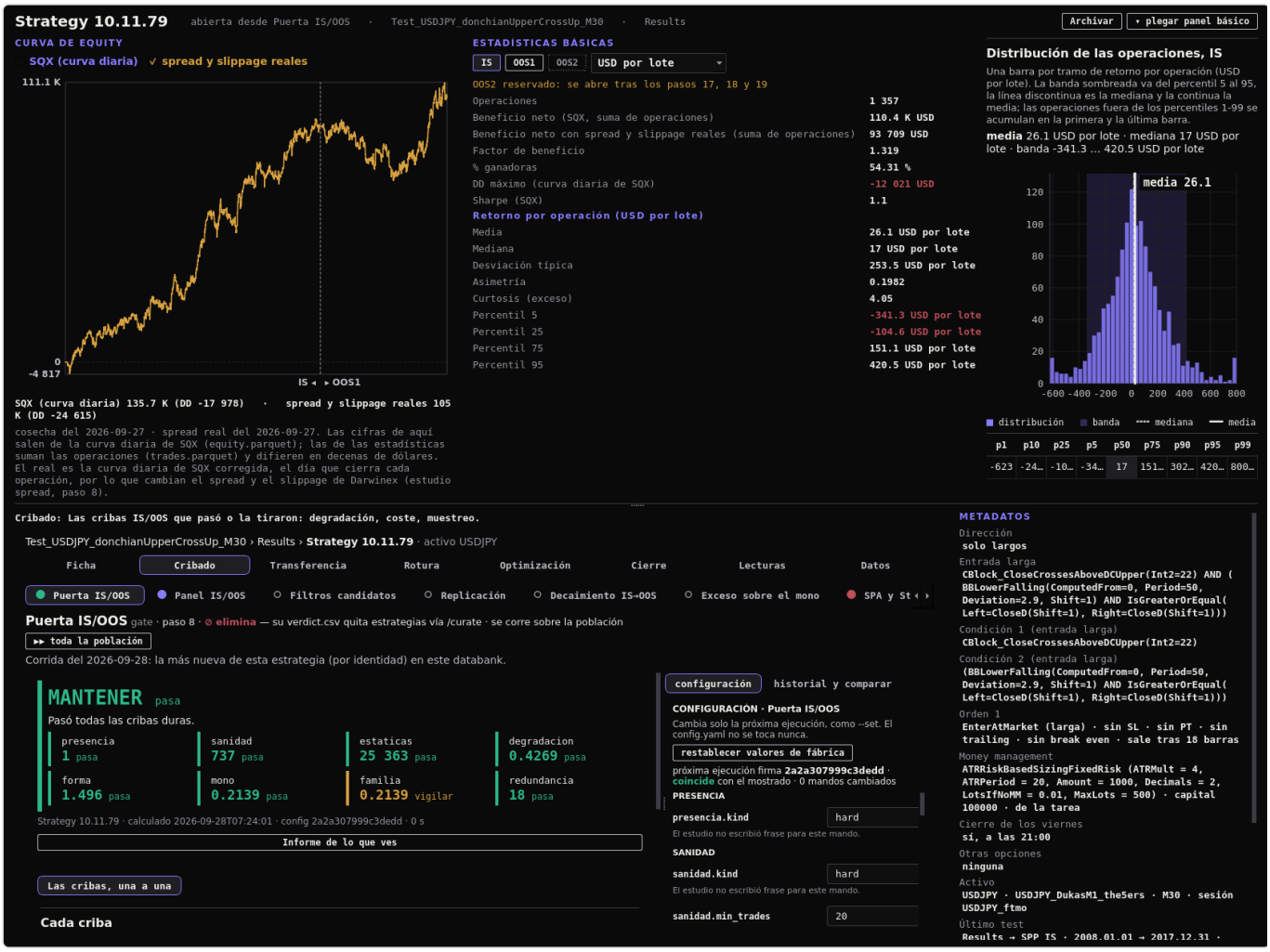
Cómo se lee el resultado



Strategy 13.14.82 de `Test_USDJPY_donchianUpperCrossUp_M30`, databank Results, a 1920 × 1050. A la izquierda, las dos curvas: la de SQX y, por debajo, la misma con el spread y el slippage reales de Darwinex; la distancia entre las dos es lo que SQX no cobró. En el centro, IS: 1 344 operaciones, 56 986 USD de SQX (suma de operaciones) y **39 595 con costes reales**, Profit Factor 1.163, 51.41 % ganadoras, DD máximo −13 629. El retorno por operación tiene media 10.98 $/lote, asimetría −0.112 y **curtosis (exceso) 3.032**: colas más gordas que una normal (que da 0). A la derecha, el histograma: la banda es del percentil 5 al 95, la discontinua la mediana y la línea blanca la media. Abajo, las pestañas: **Transferencia** y **Rotura** llevan «· sin datos» porque en Results no hay nada de sus pasos. Ya no hay columna de metadatos: el botón **metadatos** está arriba a la derecha.

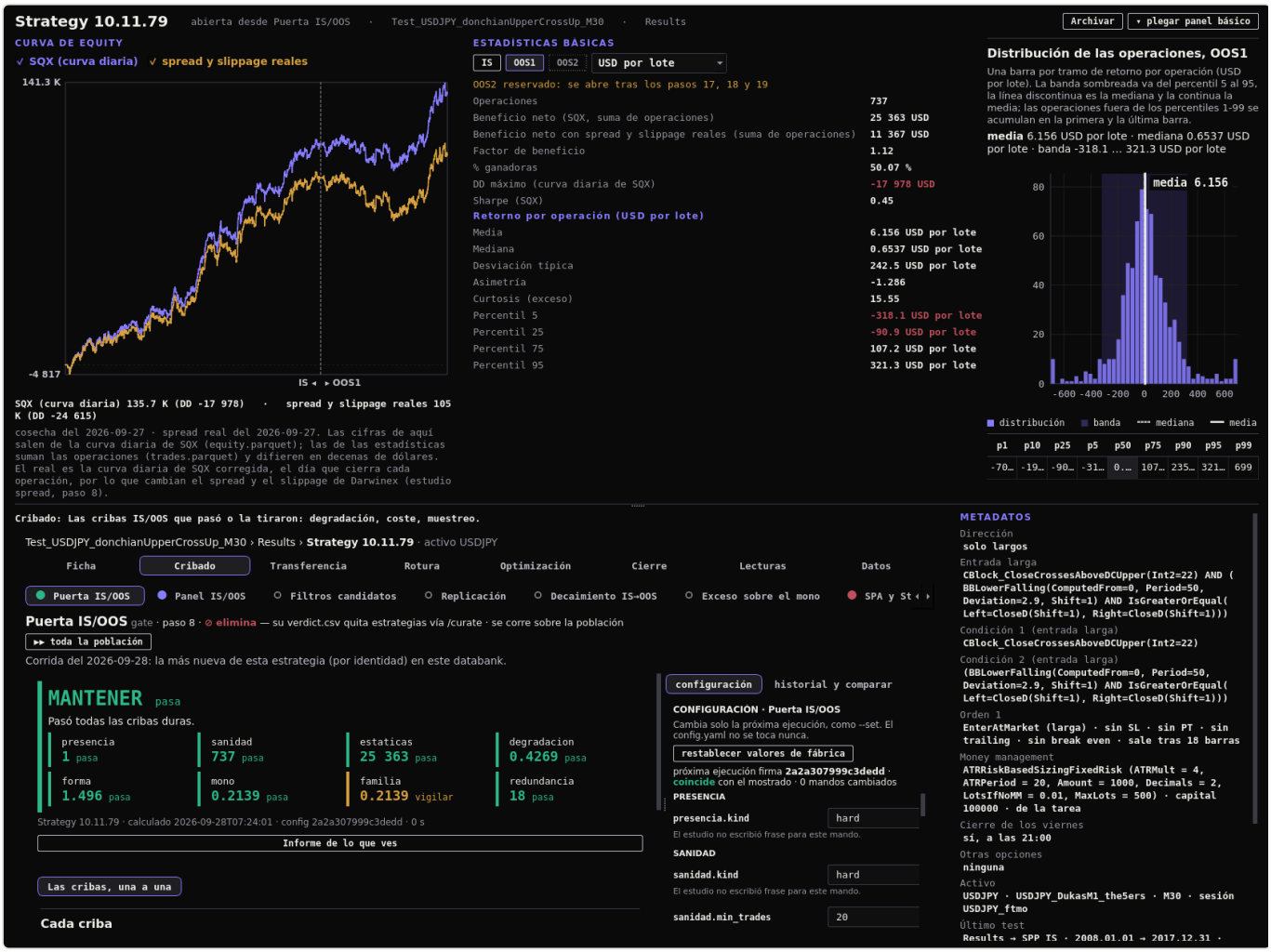


La misma estrategia en **Cribado › Puerta IS/OOS**: MANTENER, con las ocho cribas en tarjetas y en tabla, usando todo el ancho de la ventana. La configuración y el historial están plegados, en los dos botones de la derecha de la barra de correr. En la fila de tests, «Filtros candidatos», «Replicación» y «Exceso sobre el mono» llevan «· sin datos»: aquí no tienen resultado ni se pueden correr.



Esta captura y la siguiente son del 2026-09-28, de Strategy 10.11.79, que ya no está en la cosecha: muestran la distribución anterior, con la columna de metadatos y el panel de configuración siempre abiertos. Lo que explican — la curva y las estadísticas — no ha cambiado.

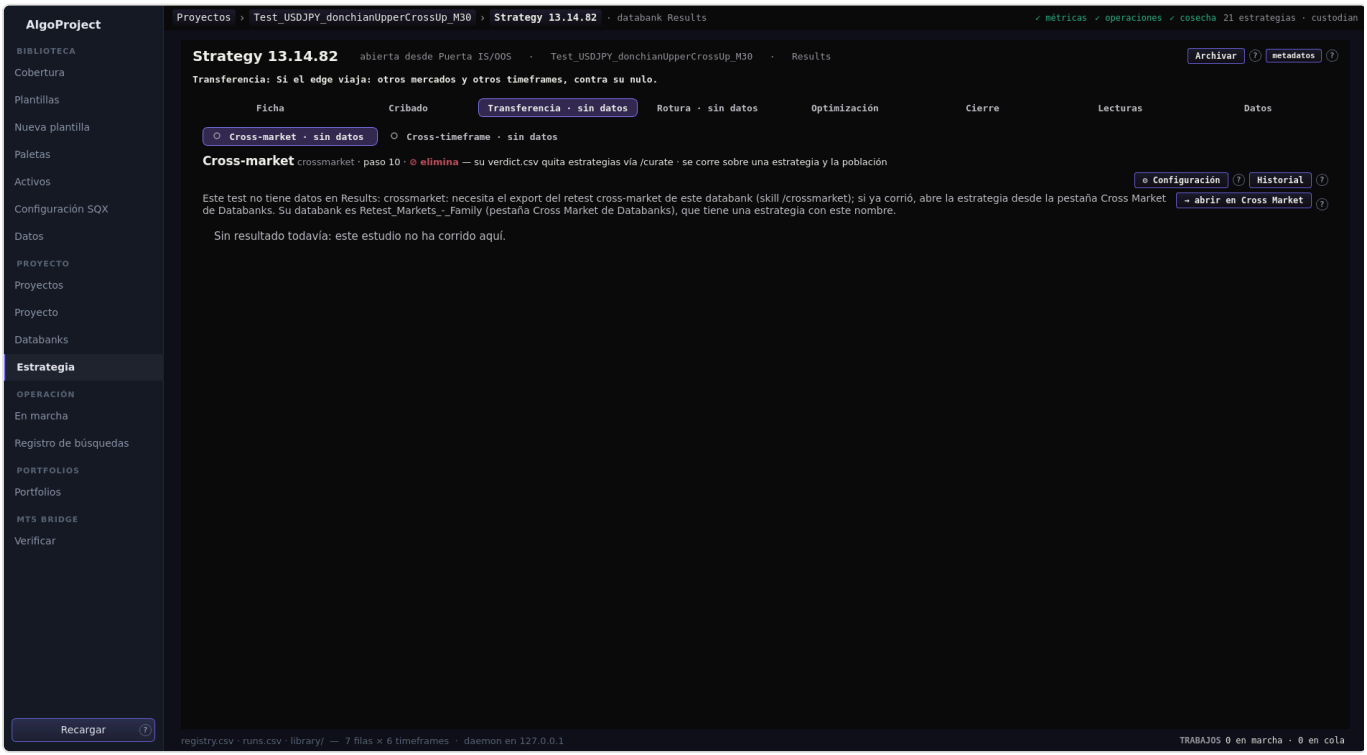
Con SQX apagada queda solo la curva real y el eje se reajusta a ella (111.1 K arriba).



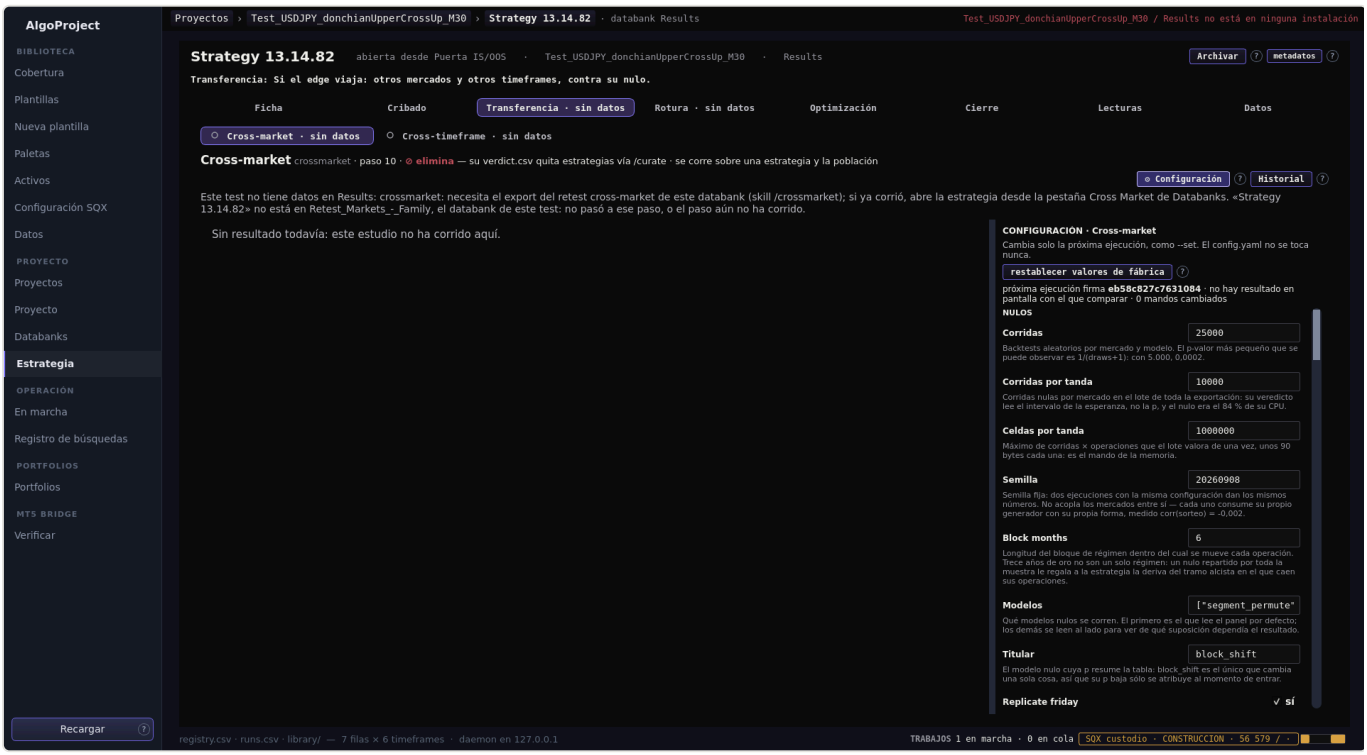
OOS1: 737 operaciones, 25 363 de SQX pero **11 367 con costes reales** — el coste real se come más de la mitad del beneficio fuera de muestra. La curtosis sube a 15.55 y la asimetría es −1.286: fuera de muestra la estrategia tiene más operaciones perdedoras grandes que ganadoras grandes.



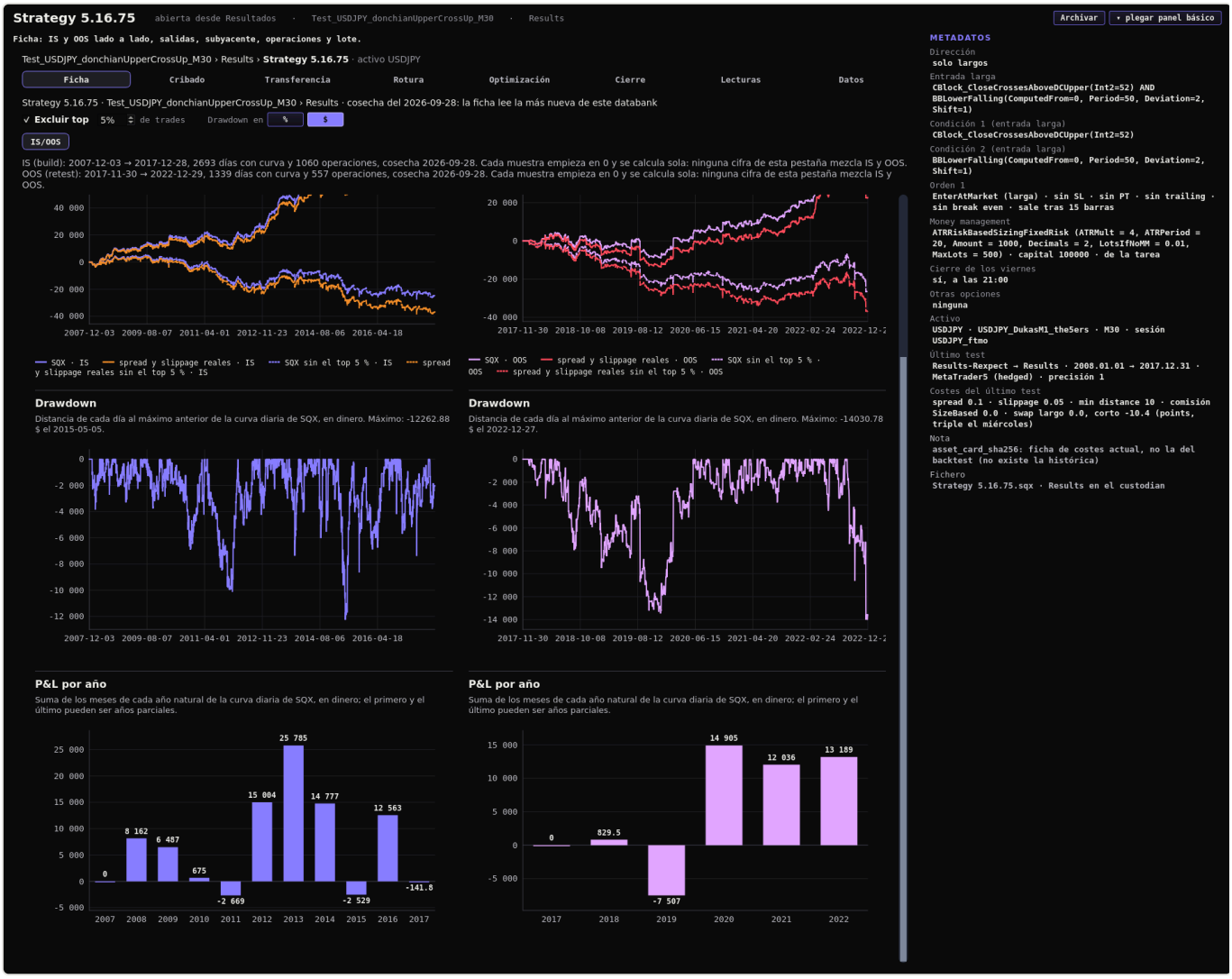
La ventana que abre **metadatos**, de Strategy 13.14.82: solo largos; la entrada es `CBlock_CloseCrossesAboveDCUpper(Int2=60) AND MomRising(…Period=20…)`; la orden, a mercado sin SL ni PT y sale tras 15 barras; el money management, riesgo fijo por ATR (1 000 por operación sobre 100 000); cierre de los viernes a las 21:00; el activo y la sesión; el último test (Results → SPP IS, 2008–2017) y sus costes: spread 0.1, slippage 0.05, swap corto −10.4 puntos con triple el miércoles. La última línea dice de dónde salió el `.sqx`: aquí, la copia que dejó el export del SPP, porque el databank del install ya no la tiene. Por eso el «último test» es el del SPP y no el del build.



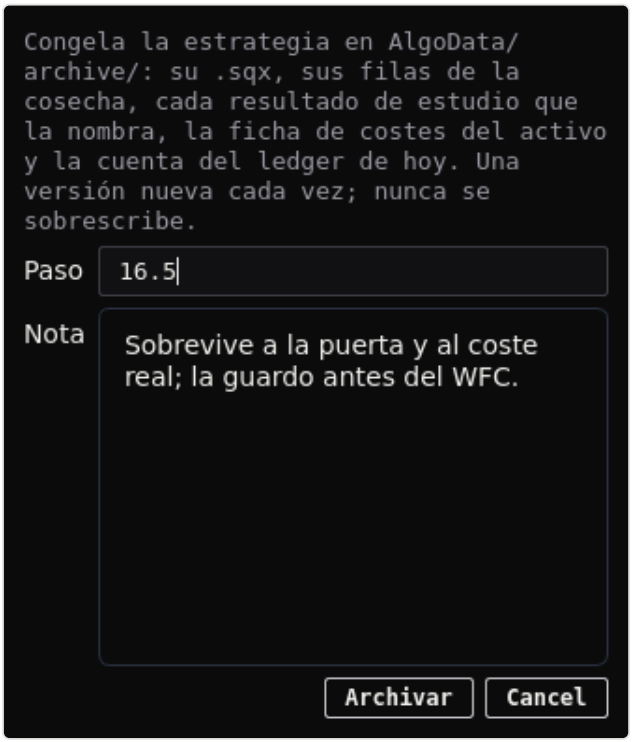
La ficha abierta desde Results, pestaña **Transferencia › Cross-market**. Los dos tests de la familia llevan «· sin datos» y la barra de correr no aparece: el cross-market necesita el export del retest en varios mercados, que vive en `Retest_Markets_-_Family`, no en Results. La frase lo dice, y como ese databank tiene una «Strategy 13.14.82», ofrece → **abrir en Cross Market**. Al pulsarlo, la ficha pasa a esa estrategia de ese databank, en la misma pestaña, y ahí sí aparecen «▶ esta estrategia», «▶▶ toda la población» y «↻ solo» con sus mercados. **El salto empareja por nombre**: dentro de un proyecto la identidad de una estrategia cambia de databank a databank, así que la ventana no lo hace sola — lo eliges tú.



⚙ **Configuración** pulsado: a la derecha se despliegan los parámetros del cross-market (corridas, tamaño de tanda, semilla, meses de bloque, modelos nulos…), cada uno con su frase. Arriba dice qué firma tendría la próxima corrida y si coincide con la del resultado en pantalla. Otra pulsación del mismo botón lo pliega y el resultado recupera el ancho.



La pestaña **Ficha** de Strategy 5.16.75 (cosecha del 2026-09-28), IS a la izquierda y OOS a la derecha. Arriba, el P&L acumulado con «Excluir top 5% de trades» encendido: las curvas continuas son SQX y la de spread; las discontinuas, las mismas sin su 5 % de operaciones mejores, y acaban en negativo en los dos tramos — casi todo el beneficio lo hacen unas pocas operaciones. En medio, el drawdown de la curva diaria de SQX en dólares (máximo −12 263 en IS, −14 031 en OOS); con **%** se lee en % del saldo en el pico. Abajo, el P&L de cada año natural, una columna por año.



«Archivar» pide el **paso** del WORKFLOW en que está la estrategia (sin valor por defecto: el último paso «hecho» del raíl no tiene por qué ser el paso en que está la estrategia) y una nota, que se guarda tal cual.

Un ejemplo completo

1. En Databanks, pestaña Puerta IS/OOS de Results, doble clic en «Strategy 10.11.79».
2. Se abre la ficha: curvas, estadísticas IS, y abajo Cribado. Los metadatos, en el botón **metadatos** de arriba a la derecha.
3. Clic en **OOS1**: el neto real fuera de muestra es 11 367 frente a 25 363 de SQX.
4. Clic en **OOS2**: no hay todavía cosecha del retest oos2, y al lado lo dice: «OOS2 abierto, sin export: exporta el retest oos2».
5. Clic en **Archivar**, paso `16.5`, nota «Sobrevive a la puerta y al coste real; la guardo antes del WFC.», **Archivar**. Arriba aparece «archivada: versión 2026-09-28T0724 (copia exportada de SPP_IS del 2026-09-27)», y la carpeta `AlgoData/archive/4d679e0c…/2026-09-28T0724/` tiene su `manifest.json`.

Lo mismo por el demonio:

```
curl -s "127.0.0.1:8765/api/tearsheet?
project=Test_USDJPY_donchianUpperCrossUp_M30&databank=Results&identity=4d679e0c2ce2a63ee53bc6cb305830b
caf5a74997e6e6e2c1deeb4c30f307048&sample=OOS2"
{"blocked":"OOS2 abierto, sin export: exporta el retest oos2"}
```

Y una estrategia archivada se lee sin calcular nada añadiendo `source=archive` a cualquier ruta de la ficha (`/api/result`, `/api/tearsheet`, `/api/strategy/stats`, …): responde lo mismo que la viva el día que se archivó.

Qué NO te dice

- **La curva real no es un retest en Darwinex.** Cambia solo el spread (y el slippage, la mitad del spread en cada ejecución) de cada operación, al minuto en que la paga; el precio sigue siendo el de Dukascopy. Antes de octubre de 2017 el spread es de modelo, no medido (capítulo 59).

- **Las estadísticas no son un veredicto.** Una curtosis alta dice que hay colas, no que la estrategia sea mala; la decisión la toman las cribas de abajo.
- **El «último test» es el que guarda el `.sqx` que se encontró.** Si el `.sqx` es la copia de un export (SPP, WFC…), sus costes son los de esa tarea; la línea «Fichero» lo dice.
- **Los metadatos del archivo son los del día de archivar.** La ficha de costes del activo puede haber cambiado después.

Si algo falla

- **«El databank Results de … no tiene cosecha»** — falta el paso 8: `/oos-gate`.
- **«sin .sqx en ningún install»** en la ventana de metadatos — ni el install, ni un export, ni el archivo guardan esa estrategia. Exporta el databank (`/export`) o archívala mientras siga en el install.
- **«Falta el paso del WORKFLOW…» / «el proyecto … no tiene plantilla en projects/registry.csv»** al archivar — escribe el paso; y sin plantilla no hay familia con la que firmar la cuenta del ledger, así que no se archiva.
- **«· sin datos» en un test que sí corriste** — corrió en otro databank: ábrelo con → **abrir en …**, o desde su pestaña de Databanks. Si en vez del botón dice «no se puede mirar …», el databank de ese test no está en ningún install o SQX lo está escribiendo ahora; vuelve cuando acabe.
- **«El demonio no respondió…»** en rojo en cada panel — el demonio está caído o arrancando; «Recargar» cuando vuelva.

58. Estrategia — cómo se lee el resultado de un estudio

Todos los estudios entregan su resultado con la misma forma (pestañas, selectores, ocho tipos de gráfico, veredicto, avisos y glosario) y la ventana la pinta siempre igual. Este capítulo explica esa página: los histogramas IS/OOS superpuestos, las superficies de varios mercados lado a lado, la re-ejecución de una sola subprueba sin pisar lo guardado y el informe que copia lo que ves.

Qué pregunta responde

- **¿Se degradaron las operaciones fuera de muestra, y cuánto?** → las densidades IS/OOS con el desplazamiento de la mediana y el p del KS al lado.
- **¿La región buena de parámetros es la misma en otros mercados?** → dos o tres superficies lado a lado con la misma escala, θ_0 marcado, y el mapa de consenso debajo.
- **¿Qué pasa si repito solo esta subprueba (un mercado, una prueba de Monte Carlo)?** → «↻ solo …»: el resultado nuevo **al lado** del guardado, o fusionado en pantalla, sin tocar el fichero.
- **¿Puedo guardar exactamente lo que estoy viendo?** → «Informe de lo que ves».

Cuándo lo usas, y cuándo no

Lo usas para leer cualquier resultado sin abrir un HTML ni un CSV, para comparar la forma de las operaciones dentro y fuera de muestra, para ver si una meseta viaja entre mercados y para repetir una subprueba con otro mando sin perder la corrida entera.

No lo usas para decidir con una re-ejecución parcial. Una re-ejecución **nunca cambia el veredicto**: un veredicto sale de un análisis entero o de ninguno. Tampoco para comparar Net Profit o drawdown entre IS y OOS: crecen con la longitud del tramo y por eso no salen en las densidades.

Antes de empezar

- El estudio ya corrido: su resultado vive en `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<día>/<estudio>/estrategias/<estrategia>.json`.
- La app abierta con `bin/algoui`. Si estaba abierta antes de este cambio, ciérrala y ábrela: el demonio viejo no conoce `/api/study/screen` ni guarda las re-ejecuciones al lado.
- Para «↻ solo …» de Monte Carlo hace falta una corrida entera de Monte Carlo en ese databank: las pruebas que ofrece salen de ella.

Cómo se ejecuta

```
bin/algoui
```

En **Proyectos** elige el proyecto; en **Proyecto**, doble clic en la fila de la estrategia en el panel de databanks; en **Estrategia**, abre la pestaña de la familia (Cribado, Transferencia, Rotura…) y, dentro, el estudio. Esa página es la que describe este capítulo. La re-ejecución de una subprueba la lanza la ventana; por debajo corre este comando, que también puedes lanzar a mano:

```
python3 -m ui.daemon.results.rerun --study crossmarket \
  --project Test_USDJPY_donchianUpperCrossUp_M30 --databank Retest_Markets_-_Family \
  --asset USDJPY --day 2026-09-27 --strategy "Strategy 9.18.85(1)" \
  --only EURUSD_DukasM1_the5ers --set nulls.draws=2000
```

flag	obligatorio	qué hace
`--study`	sí	`crossmarket` (un mercado) o `monteCarlo` (una prueba)
`--project`, `--databank`	sí	dónde está el export
`--asset`	sí	el activo base, p. ej. `USDJPY`
`--day`	sí	la fecha del export `raw/` que se lee
`--strategy`	sí	la estrategia, tal y como la escribe SQX
`--only`	sí	el mercado (`EURUSD_DukasM1_the5ers`) o el nombre de la prueba tal y como sale en el selector «Prueba» de Monte Carlo (`Costes hasta el doble`)
`--set`	no	cambia un mando solo para esta corrida, como el cajón de configuración

Tarda lo que tarda esa subprueba sola: un mercado de cross-market con 2.000 sorteos, **11 s**. No toca SQX.

Qué produce

- La re-ejecución: `reports/<P>/<D>/<día>/<estudio>/parciales/<fecha-hora>_<subprueba>/estrategias/<estrategia>.json` y su `.html`, **al lado** del resultado entero del día más nuevo que lo tenga. El resultado entero **no se sobrescribe** (antes sí: OPEN.md #54).
- «Informe de lo que ves»: `reports/<P>/<D>/<día>/<estudio>/pantalla/<estrategia>-<fecha-hora>.html`, y la ventana lo abre en el navegador.

Cómo se lee el resultado

Densidades IS y OOS

Este estudio describe y no juzga: no hay veredicto.

Strategy 1.1.85 · calculado 2026-09-27T20:27:09 · config 5c618829b201ac86 · 0.02 s

Informe de lo que ves

IS contra OOS, operación por operación

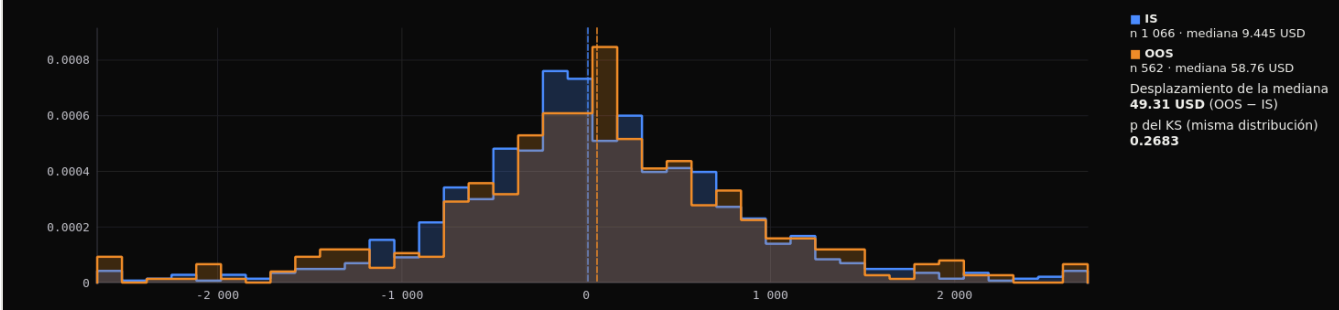
Las mismas cuatro medidas por operación dentro y fuera de muestra. Sólo medidas por operación: el beneficio neto y el drawdown crecen con la longitud de la ventana y no se comparan así. El múltiplo R espera a que la estrategia lleve stop (paso 24).

Unidad del dinero **USD**

Retorno por operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista ✓ **IS (n 1 066)** ✓ **OOS (n 562)**



Percentiles de la unión

p1	p5	p10	p25	p50	p75	p90	p95	p99
-2 281	-1 251	-819.9	-374.5	26.03	495.9	1 001	1 370	2 310

Duración de cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista ✓ **IS (n 1 066)** ✓ **OOS (n 562)**



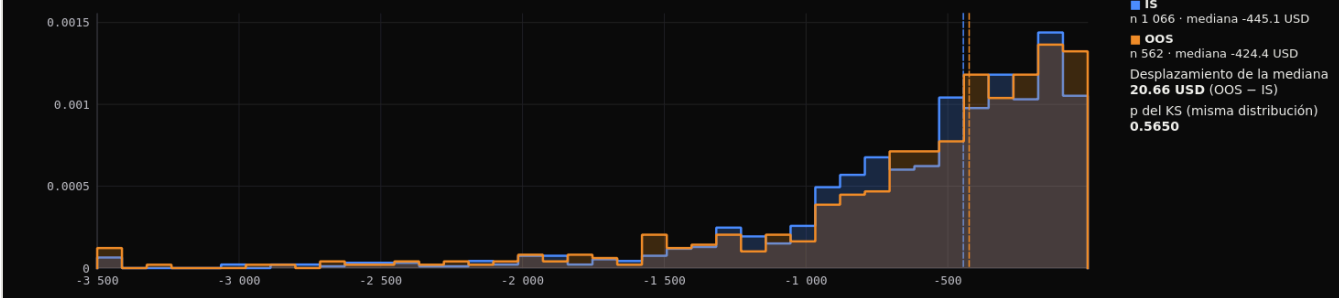
Percentiles de la unión

p1	p5	p10	p25	p50	p75	p90	p95	p99
1.5	5	9	9	9	9	9	9	9.5

MAE — lo peor que llegó a ir cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista ✓ **IS (n 1 066)** ✓ **OOS (n 562)**



Percentiles de la unión

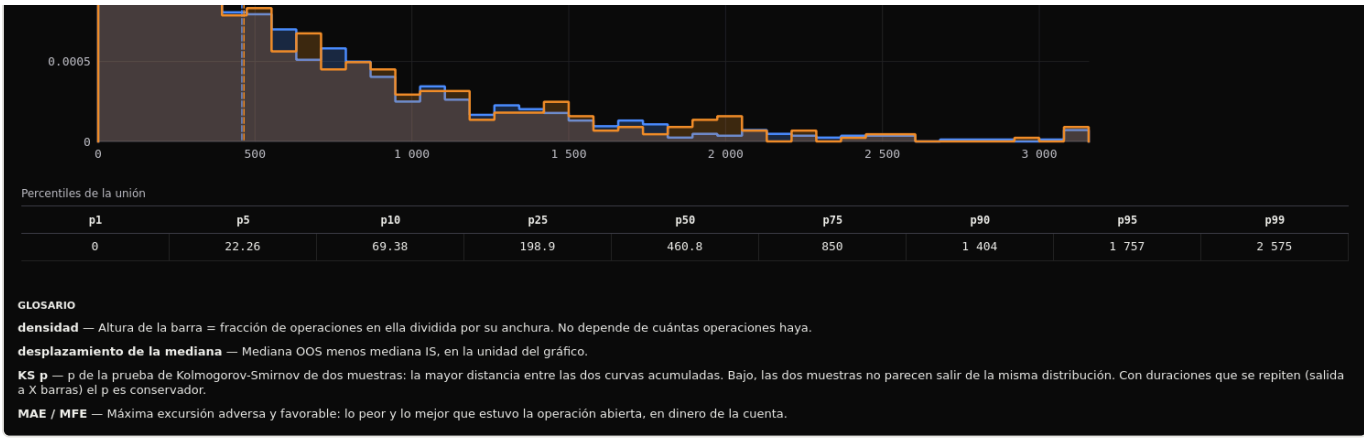
p1	p5	p10	p25	p50	p75	p90	p95	p99
-2 859	-1 715	-1 260	-784.8	-434.7	-212.5	-93.78	-46.32	-9.993

MFE — lo mejor que llegó a ir cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista ✓ **IS (n 1 066)** ✓ **OOS (n 562)**





Cada muestra es una **densidad**: el área bajo cada curva vale 1, así IS (1.066 operaciones) y OOS (562) se comparan por su forma, no por cuántas hay. Encima, una casilla por muestra para enseñarla u ocultarla. A la derecha:

- cada muestra con su **n** y su **mediana** (la línea discontinua de su color);
- **desplazamiento de la mediana**: mediana OOS menos mediana IS, en la unidad del gráfico. En el retorno por operación de la captura, +49,31 USD: fuera de muestra la operación típica ganó más;
- **p del KS**: la probabilidad de ver una diferencia de forma así de grande si las dos muestras vinieran de la misma distribución. **Pequeño (< 0,05) = las operaciones cambiaron de verdad.** Aquí 0,27 en el retorno: no hay degradación que se distinga del azar.

Debajo, la fila de **percentiles de la unión** de las dos muestras. El ratón sobre una barra dice el tramo y qué parte de las operaciones de cada muestra cae en él.

Los histogramas recortan el eje X. Todos los histogramas de la ventana dibujan solo del **percentil 0,5 al 99,5** de sus valores, para que un puñado de valores extremos no aplaste el resto en una sola barra. La nota del pie dice **cuántos valores se quedaron fuera**, y el recuento de las barras es el de los que se dibujan.

Superficies de varios mercados, lado a lado

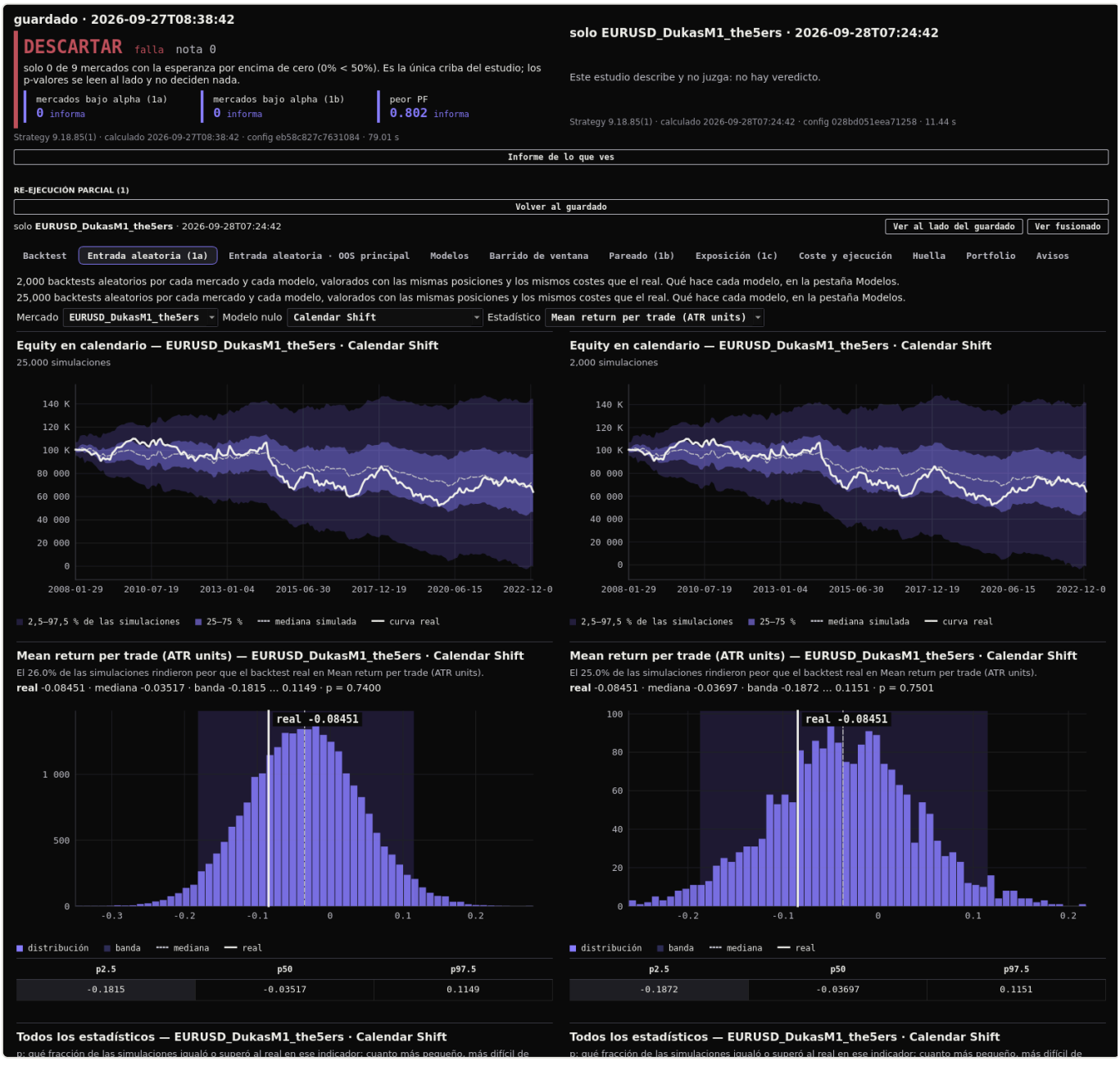


En la pestaña de superficies por pareja de parámetros de Market Surfaces:

- **«Mercados a la vista (hasta 3)»**: un botón por mercado. Pulsar un cuarto suelta el más antiguo; nunca quedan menos de uno. No recalcula nada.
- Las tres superficies comparten **la misma escala de color** (la leyenda dice «Escala común a las rejillas»): un mismo azul es el mismo beneficio en los tres mercados.
- **θ₀**, la celda de los parámetros de la madre, va recuadrada en blanco en cada una.

- Debajo, el **consenso de todos los mercados**: en cuántos de los diez cada celda está en su 10 % superior. Una región que viaja se ve como una mancha alta y continua; θ_0 debería caer dentro. En la captura cae en 1 de 10: la madre no está en la meseta compartida.
- Si la celda es demasiado estrecha para su número, el número sale al pasar el ratón.

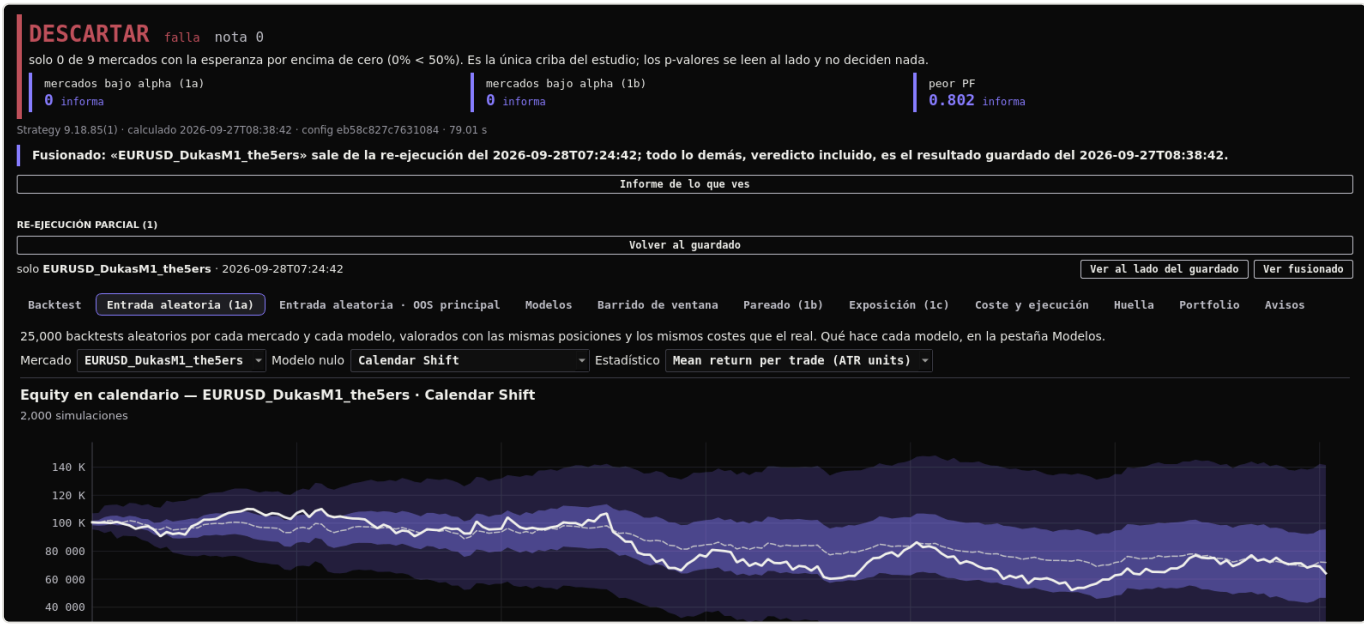
Re-ejecutar una subprueba: al lado y fusionado



Cuando una estrategia tiene re-ejecuciones parciales, sale la tira **RE-EJECUCIÓN PARCIAL (n)** con una línea por cada una:

- **Ver al lado del guardado**: el resultado entero a la izquierda y la re-ejecución a la derecha, gráfico con gráfico, abiertos en la misma subprueba (el mismo mercado o la misma prueba). En la captura, 25.000 sorteos guardados contra 2.000 nuevos: el valor real es el mismo y el p pasa de 0,74 a 0,75.
- **Ver fusionado**: el resultado entero con esa subprueba tomada de la re-ejecución. Una línea bajo el veredicto dice qué parte es nueva y de cuándo. El veredicto sigue siendo el guardado. Una re-ejecución vista sola no tiene veredicto: la cabecera dice que sale del análisis entero.

- **Volver al guardado**.



Un mercado que la corrida entera no llegó a ver entra en el selector al fusionar; uno que no está ni en la corrida ni en ninguna re-ejecución no aparece en el selector.

Lo que ya estaba y sigue igual

- **La distribución con el valor real marcado**, su banda, su mediana y la fila de percentiles debajo (la casilla donde cae el real, resaltada): el dibujo base de todos los tests.
- **El cono de equity con la curva real encima**, en cada estudio que lo trae (Monte Carlo, cross-market, estrés).
- **Los selectores filtran**, no recalculan: el estudio guardó todas las combinaciones.
- **Los avisos colorean y nunca eliminan**: los iguales se cuentan (×N), ninguno se esconde.
- **El glosario** va en la misma página, al final, con cada término que usa el estudio.

El informe de lo que ves

«Informe de lo que ves» escribe una página HTML con exactamente lo que hay en pantalla: cada pestaña con la combinación de selectores elegida (dicha en la nota de la pestaña), los mercados lado a lado que elegiste, los avisos y el glosario. Sale del mismo resultado y del mismo dibujante que los informes del estudio (`core.study.render`), no de una segunda copia. En una comparación van las dos columnas, una detrás de otra.

Un ejemplo completo

1. `bin/algoui` . Proyecto `Test_USDJPY_donchianUpperCrossUp_M30` , databank `Retest_Markets_-_Family` , estrategia `Strategy 9.18.85(1)` ; en Estrategia, pestaña **Transferencia**, estudio **Cross-market**. Sale el resultado del 27-09: `DESCARTAR` , 0 de 9 mercados.
2. En la barra de correr, «↻ solo …» → `EURUSD_DukasM1_the5ers` , con el cajón en `nulls.draws=2000` . A los 11 s el trabajo termina: `-> …/crossmarket/parciales/20260928-072442_EURUSD-DukasM1-the5ers/estrategias/Strategy 9.18.85(1).json` .
3. El resultado vuelve a salir con la tira «RE-EJECUCIÓN PARCIAL (1)». «Ver al lado del guardado»: izquierda el guardado (25.000 sorteos), derecha EURUSD con 2.000. «Ver fusionado»: el resultado entero, EURUSD del 28-09, veredicto `DESCARTAR` intacto.
4. «Informe de lo que ves» → se abre `…/crossmarket/pantalla/Strategy 9.18.85(1)-20260928-072535.html` .

Qué NO te dice

- **Una re-ejecución parcial no es un veredicto.** Ni al lado ni fusionada cambia la etiqueta; para eso se vuelve a correr el estudio entero.
- **El p del KS no dice si la estrategia sirve**: dice si la forma de las operaciones cambió. Un OOS mejor que el IS también da p pequeño.
- **La misma escala de color en las superficies depende del coste** (provisional en Market Surfaces): compara formas entre mercados, no niveles.
- **Monte Carlo ofrece las pruebas de su última corrida entera en ese databank.** Si una estrategia tiene menos operaciones, algún tamaño de bloque puede no existir para ella y la re-ejecución falla con ese motivo en el log.
- **MC Retest (paso 14) no tiene «↻ solo …»**: cada subprueba es una tarea de SQX, y una re-ejecución parcial nunca lanza SQX. Lo único que la ventana lanza en SQX es la tarea siguiente del workflow, con «Continuar workflow» (capítulo 69).

Si algo falla

- **« no sabe correr una sola subprueba: córrelo entero»**: solo cross-market y Monte Carlo re-ejecutan una subprueba, y solo sobre una estrategia.
- **«monteCarlo: el export de este databank es cross-market…»**: Monte Carlo lee la estrategia en su propio mercado; elige el databank del retest de un solo mercado.
- **«Este resultado no viene de un informe guardado»** al pulsar el informe: la vista no sabe dónde escribir la página (por ejemplo, un resultado de un lote de variantes).
- **La tira de re-ejecuciones no sale**: el demonio es anterior a este cambio; cierra y abre la app.

70. OPERACIÓN — «En marcha», «Registro de búsquedas» y la franja de trabajos

El grupo OPERACIÓN de la barra lateral responde a dos preguntas distintas: **¿qué está corriendo ahora y por dónde va?** («En marcha») y **¿cuánto he buscado ya?** («Registro de búsquedas», que enseña el Ledger). Abajo a la derecha, en la barra de estado, la franja de trabajos enseña cada trabajo de la ventana con su barra de 0 a 100 %.

Qué pregunta responde

- **«En marcha»**: qué proyecto de SQX corre, qué tarea, cuántas estrategias lleva de cuántas, y cómo va la máquina mientras tanto (memoria del JVM del custodio contra su techo, CPU, RAM libre, cuántos backtests caben todavía). El pulso y la tabla de tareas son **del mismo proyecto y en la misma pantalla**: ya no hay que cruzar dos zonas.
- **«Registro de búsquedas»**: cuántas veces se ha mirado cada estudio (activo + timeframe + familia) para reducir una población, con qué criterio, cuántas entraron y cuántas salieron, qué tramos del histórico se han leído, y qué pasos de 17, 18 y 19 están hechos. Los filtros que aplicas en «Proyecto» salen aquí como una búsqueda más.
- **La franja de trabajos**: qué análisis de Python lanzó la ventana, por dónde va cada uno (barra), cuáles esperan turno, y qué proyecto de SQX corre en los workers.

Cuándo lo usas, y cuándo no

Lo usas mientras corre un paso largo en el custodio, para no abrir SQX y no mandarle nada más que la línea de estado; al volver a una sesión, para ver qué acabó; y antes de decidir otro filtro o otra búsqueda, para ver en el Registro cuánto llevas gastado en ese estudio.

No lo usas para lanzar ni parar SQX: aquí no hay botón, a propósito. Arrancar y parar sigue siendo `bin/sqx-worker.sh` y las skills; lo único que la ventana lanza en SQX es la tarea siguiente, con «Continuar workflow» en la zona Proyecto (capítulo 69), y ese trabajo se sigue aquí. El ✕ de la franja cancela un trabajo **de Python** de la ventana; los chips de SQX no tienen ✕. El Registro tampoco se edita: es de solo lectura.

Antes de empezar

Nada. Vale con el maestro abierto (a él no se le pide nada nunca) y con el custodio en mitad de un trabajo: lo único que la ventana le pide es `-project action=status`, y solo mientras su log dice que un proyecto corre (es el único comando que el custodio puede recibir entre `start` y `collect`, porque no sincroniza nada).

Cómo se ejecuta

```
bin/algoui --zone "En marcha"
bin/algoui --zone "Registro de búsquedas"
```

O desde la barra lateral, grupo OPERACIÓN. La franja de trabajos está siempre a la vista.

control	dónde	qué hace
install	«En marcha»	`custodian` (SQX_w2), `conductor` (SQX_w1) o `master` (solo se lee)
proyecto	«En marcha»	cualquier proyecto de ese install. Al abrir se elige solo el que corre en un worker (el custodio primero); si nada corre, el proyecto elegido en la ventana; si tampoco, el primer `Test_` / `Trade_` / `Research_` del custodio. Cuando lo eliges tú, la zona deja de elegir
Leer ahora	«En marcha», pulso	una lectura del pulso sin esperar al minuto
estudio	«Registro de búsquedas»	activo + timeframe + familia; por defecto el último escrito
×	franja	cancela ese trabajo de Python (y todos sus procesos hijos)
Recargar	pie de la barra lateral	vuelve a leer las zonas que se cargan una vez; su tooltip dice cuáles

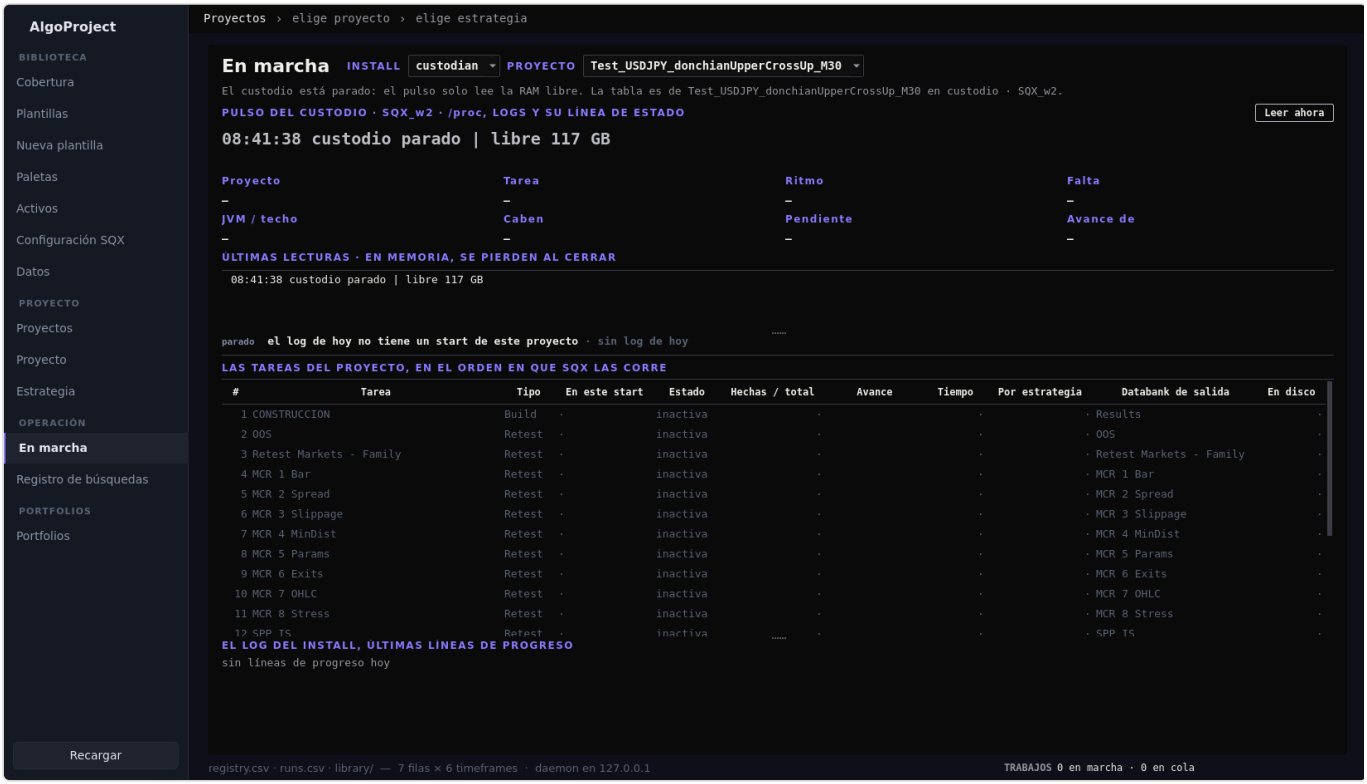
El tooltip de **Recargar** dice, literalmente: «Vuelve a leer del demonio: Cobertura, Plantillas, Paletas, Activos, Proyectos, En marcha. Las demás zonas leen solas al abrirlas o al cambiar la selección, y no necesitan este botón.»

«En marcha» refresca la tabla cada tres segundos y el pulso cada minuto, **solo mientras está a la vista**, y **cada 30 segundos vuelve a buscar** proyectos e installs nuevos, sin pulsar «Recargar». La franja lee los trabajos cada dos segundos y los proyectos de SQX cada dieciséis.

Qué produce

Nada. No escribe ni un fichero: lee `/proc`, el log de SQX de cada install, sus `project.cfx` y databanks, los logs de los trabajos de la ventana (`AlgoData/logs/ui/`) y el ledger (`AlgoData/ledger/*.jsonl`).

Cómo se lee el resultado



Arriba, el pulso del custodio. La línea grande es la que el dueño eligió el 2026-09-25: `13:38:32 3987 de 15000 | JVM 63.2 GB | CPU 6845% | libre 55 GB | caben 900 más`. Cada cifra de la rejilla tiene un tooltip que dice de dónde sale. En rojo, los avisos: RAM libre bajo 15 GB, el JVM al 90 % de su techo, más backtests pendientes de los que caben, o el log de SQX callado más de diez minutos. La cifra **«avance de»** dice de dónde sale la cuenta:

avance de	qué significa
un log de trabajo (`…-variants.log`)	el run lo lanzó la ventana y cuenta el **run entero** (`PROGRESS n de N`); hay ritmo y minutos que faltan
`estado del custodio · <tarea>`	el run se lanzó **fuera** de la ventana (terminal, otra sesión): la cuenta es de la **tarea** en curso, del estado del worker. No hay ritmo ni «falta», porque la cuenta de una tarea sobre los minutos del proyecto no sería un ritmo
`—` y la línea dice `avance ?`	nada da cuenta: ni un trabajo de la ventana ni el estado del worker

La frase bajo el título dice si el pulso y la tabla son el mismo run. En ámbar, cuando no: el custodio corre un proyecto y tú miras otro (o miras el conductor). Elige arriba el que corre para verlos juntos.

Abajo, las tareas del proyecto, en el orden en que SQX las corre. Cada `start` de un proyecto es un paso del workflow: `stage` deja activa solo la tarea del paso y SQX marca las demás como saltadas.

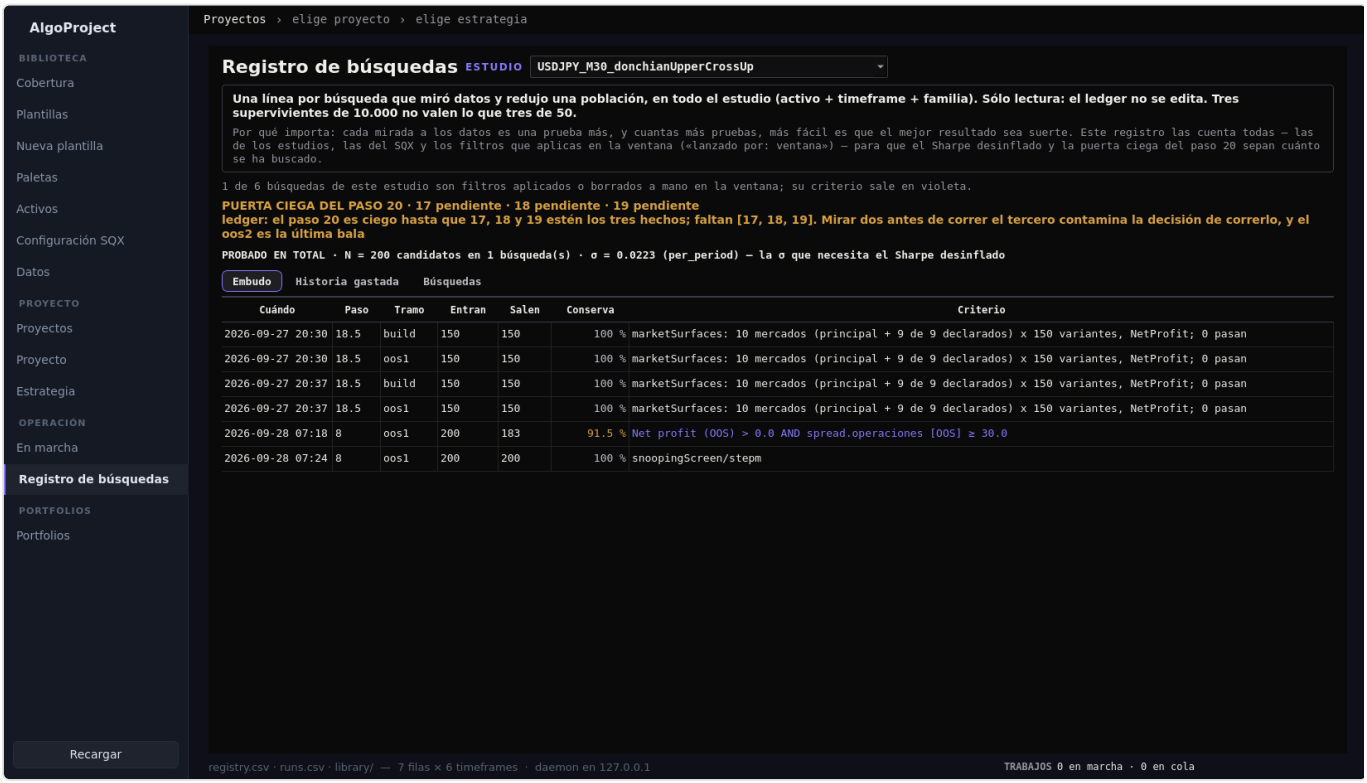
estado	qué significa
en curso	la última tarea que escribe en el log; en negrita, con su barra ámbar en «Avance»
hecha	el log vio `Task finished` en este start; barra verde llena
hecha antes	inactiva en este start, pero su databank tiene estrategias: la hizo un start anterior
en cola	activa en este start y aún no empezada
saltada	inactiva en este start y con el databank vacío
inactiva	apagada, sin start en marcha

La barra de la tarea en curso es el porcentaje que SQX escribe en su log cuando lo da (las tareas walk-forward); si no, hechas ÷ total. Un build no tiene total: su barra solo se mueve, sin cifra.

La línea «GENÉTICA». Mientras corre un Build genético, bajo la línea de estado sale una línea con la **población inicial** (estrategias que hay en el databank «Initial population» contra población × islas, el tamaño que tiene que llenarse antes de que el databank reciba nada), las **generaciones** como máximo que permite el proyecto y la **última generación** (estrategias en el databank «Last generation»). Las dos cuentas son **ficheros en disco**: SQX los escribe al sincronizar, así que van con retraso. **La generación actual no la publica SQX** (ni su estado ni su log la traen), por eso la línea dice «la actual no se publica». No sale en un Build que no sea genético ni cuando no corre ninguno. «En disco» son ficheros `.sqx`: SQX los escribe al sincronizar, así que la salida de la tarea en curso va con retraso.



La franja de trabajos: `N en marcha · N en cola`, y un chip por trabajo con su barra. La barra es la última línea `PROGRESS <0-100>` que imprimió el trabajo, y 100 % al acabar bien. **Un estudio que no imprime PROGRESS se queda en 0 % hasta que acaba** (es el caso de `profitShape`, que tarda uno o dos segundos por estrategia); los que sí lo imprimen (la puerta, el cribado de snooping, isOos, las superficies…) mueven la barra. Los que corren más avanzados van primero; más de cuatro se resumen en «+N». Un chip ámbar `SQX custodio · <tarea> · 7 / 20` es un proyecto de SQX en un worker, con la misma barra que «En marcha»: no se cancela desde aquí.



El Registro de búsquedas abre con lo que es y por qué importa, en un recuadro: cada mirada a los datos es una prueba más, y cuantas más pruebas, más fácil que el mejor resultado sea suerte. Tres supervivientes de 10.000 no valen lo que tres de 50. Debajo:

- cuántas de las búsquedas del estudio son **filtros de la ventana** (`lanzado por: ventana`); su criterio sale en violeta en «Embudo» y la fila entera en «Búsquedas»;
- **pasos 17, 18 y 19**: cuáles están hechos. Para ti la línea es verde y dice «abierta»: la puerta ciega sólo se cierra a un agente autónomo (`ALGO_AUTONOMOUS=1`);
- **probado en total**: N candidatos y la σ que necesita el Sharpe desinflado, o por qué no hay;
- tres pestañas: **Embudo** (entran, salen y qué parte conserva cada búsqueda: rojo si conserva menos de la mitad, ámbar si conserva menos del todo), **Historia gastada** (cada tramo, su ventana, cuántas veces se ha leído y, en la última columna, a qué pasos limita la reserva a un agente autónomo: verde si sigue virgen) y **Búsquedas** (cada fila completa; al pasar el ratón, sus umbrales y su nota).

Un ejemplo completo

La sesión de las capturas, el 2026-09-28, tal cual:

1. `bin/algoui --zone "En marcha"` . Ningún worker corría nada y no había proyecto elegido, así que la zona eligió el custodio y su primer proyecto `Test_` : `Test_USDJPY_donchianUpperCrossUp_M30` . La frase: «El custodio está parado: el pulso solo lee la RAM libre. La tabla es de Test_USDJPY_donchianUpperCrossUp_M30 en custodio · SQX_w2.»
2. Pulso: `07:22:14 custodio parado | libre 116 GB` ; todas las cifras del JVM en «—». (La captura de arriba se rehízo a las 08:41 sobre la ventana definitiva: `08:41:38 custodio parado | libre 117 GB` .)
3. Tabla: las 27 tareas del proyecto (CONSTRUCCION, OOS, Retest Markets - Family, MCR 1 Bar… MCR 8 Stress, SPP IS…), todas «inactiva», porque ningún start de hoy las tocó.
4. Desde el raíl del proyecto se lanzó `profitShape` (paso 8) sobre el databank `OOS` : tres trabajos, uno por estrategia. La franja: `TRABAJOS 3 en marcha · 0 en cola` y tres chips `profitShape` con su barra a `0 %` (profitShape no imprime PROGRESS); a los dos segundos, los tres a 100 % y fuera de la franja.

5. `Registro de búsquedas`, estudio `USDJPY_M30_donchianUpperCrossUp`: «1 de 6 búsquedas de este estudio son filtros aplicados o borrados a mano en la ventana». En el Embudo, cuatro filas del paso 18.5 (marketSurfaces, 150 → 150, 100 %) y la del filtro de la ventana `2026-09-28 07:18 · 8 · oos1 · 200 → 183 · 91.5 % · Net profit (OOS) > 0.0 AND spread.operaciones [OOS] ≥ 30.0`, en ámbar porque conserva menos del todo. Pasos 17, 18 y 19: «17 pendiente · 18 pendiente · 19 pendiente». La sexta fila, `07:24 · 8 · snoopingScreen/stepm · 200 → 200`, es el cribado SPA/StepM lanzado desde el raíl.

Qué NO te dice

- **Cuánto falta con exactitud** en un run lanzado fuera de la ventana: la cuenta es de la tarea en curso, no del proyecto, y hechas × tiempo por estrategia es una estimación.
- **De quién es el start.** El log nombra el proyecto solo cuando el start llega por la API; un start desde la GUI del maestro no se atribuye.
- **Si el resultado es bueno.** «En marcha» cuenta; el Registro cuenta miradas. Ninguno juzga.
- **Las búsquedas que nadie apuntó.** Un análisis corrido a mano que no escribe su fila no está en el Registro; el WFC y el CSCV escriben la suya desde el 27-09-2026.

Si algo falla

- **«avance ?» con el custodio corriendo**: ni un trabajo de la ventana ni el estado del worker dan cuenta — por ejemplo, un build (no tiene total y su estado no siempre trae «Strategies generated») o un proyecto que el log nombra pero ya no está en disco.
- **La frase en ámbar**: no es un error. Miras un proyecto distinto del que corre el custodio.
- **«sin log de hoy»**: el install no ha escrito nada hoy. La zona solo lee el fichero del día.
- **Un trabajo se queda en 0 % y de golpe pasa a 100 %**: ese estudio no imprime PROGRESS.
- **«demonio no responde»**: el demonio de la ventana se ha caído; ciérrala y ábrela otra vez.

71. Portfolios — las estrategias archivadas, e «Importar»

La zona **Portfolios** de la ventana, el cuarto grupo de la barra lateral. Hoy hace una cosa: enseña las estrategias que has guardado con «Archivar» y, con **«Importar»**, abre cualquiera de ellas en la página de Estrategia **tal como se guardó**, sin recalcular nada. Es la puerta por la que las estrategias entrarán en una cartera; la cartera en sí todavía no está construida.

Qué pregunta responde

- **¿Qué estrategias tengo guardadas «como oro en paño»?** Una fila por estrategia: su nombre, el activo, el timeframe, en qué paso del workflow estaba, cuántas versiones tiene, cuándo se archivó la última y **la cuenta del Ledger** de ese día.
- **¿Qué guarda cada versión y qué no?** Al elegir una versión, a la derecha: los estudios que trae por databank, lo que no se pudo archivar y por qué, y su procedencia (la ficha de costes del activo de ese día, el código con que se hizo, de dónde salió el `.sqx`).
- **¿Cómo se veía esta estrategia el día que la archivé?** «Importar» abre su página de Estrategia con las dos curvas, las estadísticas, los metadatos y cada estudio archivado, leídos del archivo.

Cuándo lo usas, y cuándo no

Lo usas para volver a mirar una estrategia que ya pasó la secuencia sin rehacer ningún test (el monkey de 25 000 corridas no se vuelve a correr), para comparar dos versiones de la misma estrategia archivadas en pasos distintos, y, cuando exista el módulo de cartera, para elegir qué entra en ella.

No lo usas para calcular nada. Una estrategia importada **no corre ningún estudio ni encola ningún trabajo**: los botones «calcular» de la ficha contestan «archivada: aquí no se calcula nada». Para calcular, abre la estrategia desde su proyecto, en Proyectos. Tampoco importa desde un databank vivo: por ahora solo desde el archivo (decisión del dueño).

Antes de empezar

- La ventana y su demonio (capítulo 35). No hace falta SQX ni ningún worker: todo sale de `AlgoData/archive/`.
- Al menos una estrategia archivada. Se archiva con el botón «Archivar» de la página de Estrategia (capítulo 68) o con `python3 -m core.archive archive …` (capítulo 64).

Cómo se ejecuta

Dentro de la ventana: **PORTFOLIOS → Portfolios**. Elige una estrategia en la tabla, una versión en la lista de la derecha y pulsa **«Importar esta versión»**.

O `bin/algoui --zone Portfolios`. Para verla sin la ventana entera, o sacar sus capturas sin pantalla, se abre sola en su propia ventana, con el demonio arrancado (`bin/algoui` lo arranca):

```
python3 -m ui.desktop.portfolios.preview
python3 -m ui.desktop.portfolios.preview --shot /tmp/portfolios --version 2026-09-27T2039
```

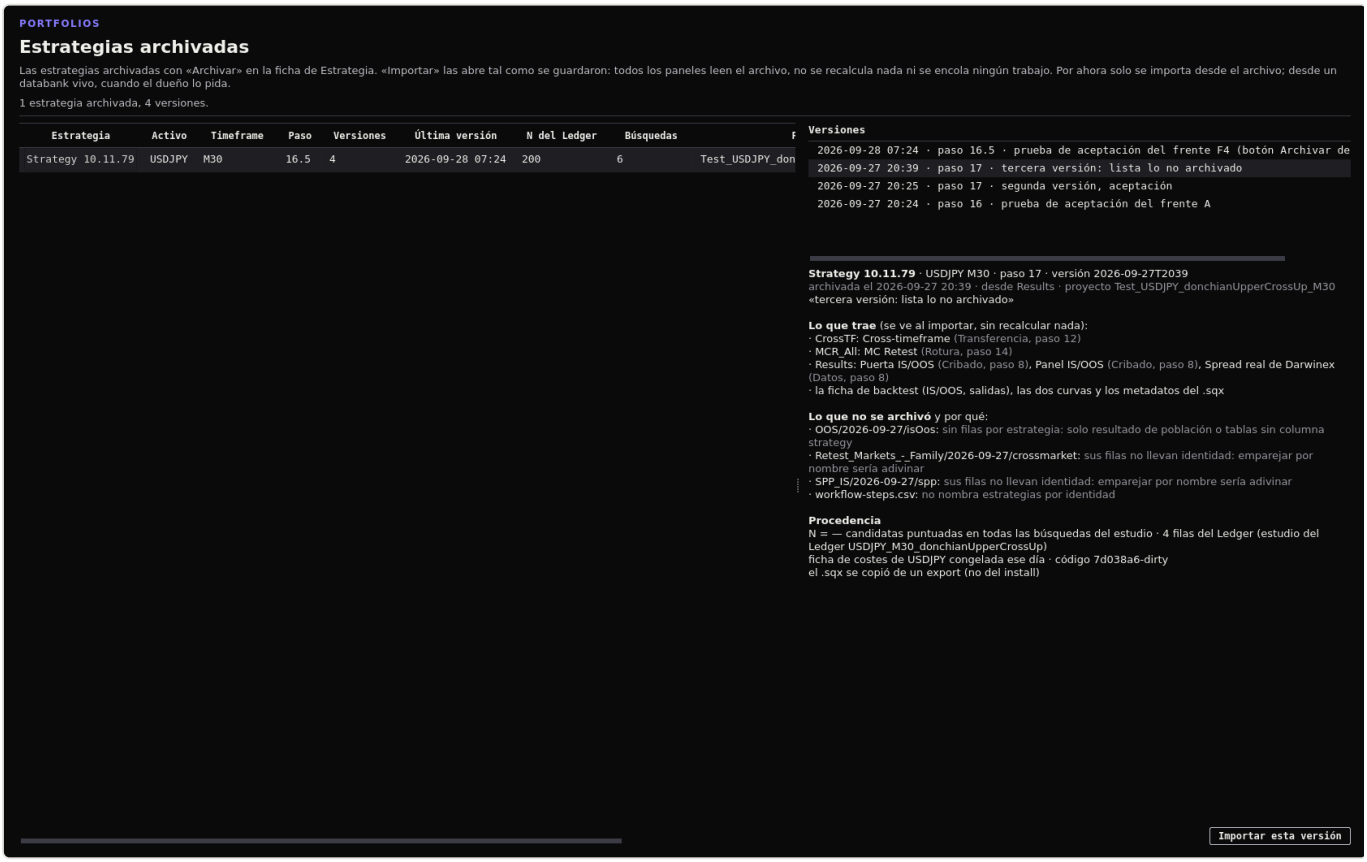
flag	obligatorio	qué hace
`--shot DIR`	no	guarda la lista (`1-portfolios.png`) y la estrategia importada (`2-importada.png`) en `DIR` y sale
`--identity`	no	la estrategia que importa la captura; sin él, la primera de la tabla
`--version`	no	la versión que importa la captura; sin él, la más nueva
`--port`	no	el puerto del demonio; sin él, el de la ventana (8765)

Tarda un par de segundos. No toca SQX: se puede abrir con cualquier install arrancado.

Qué produce

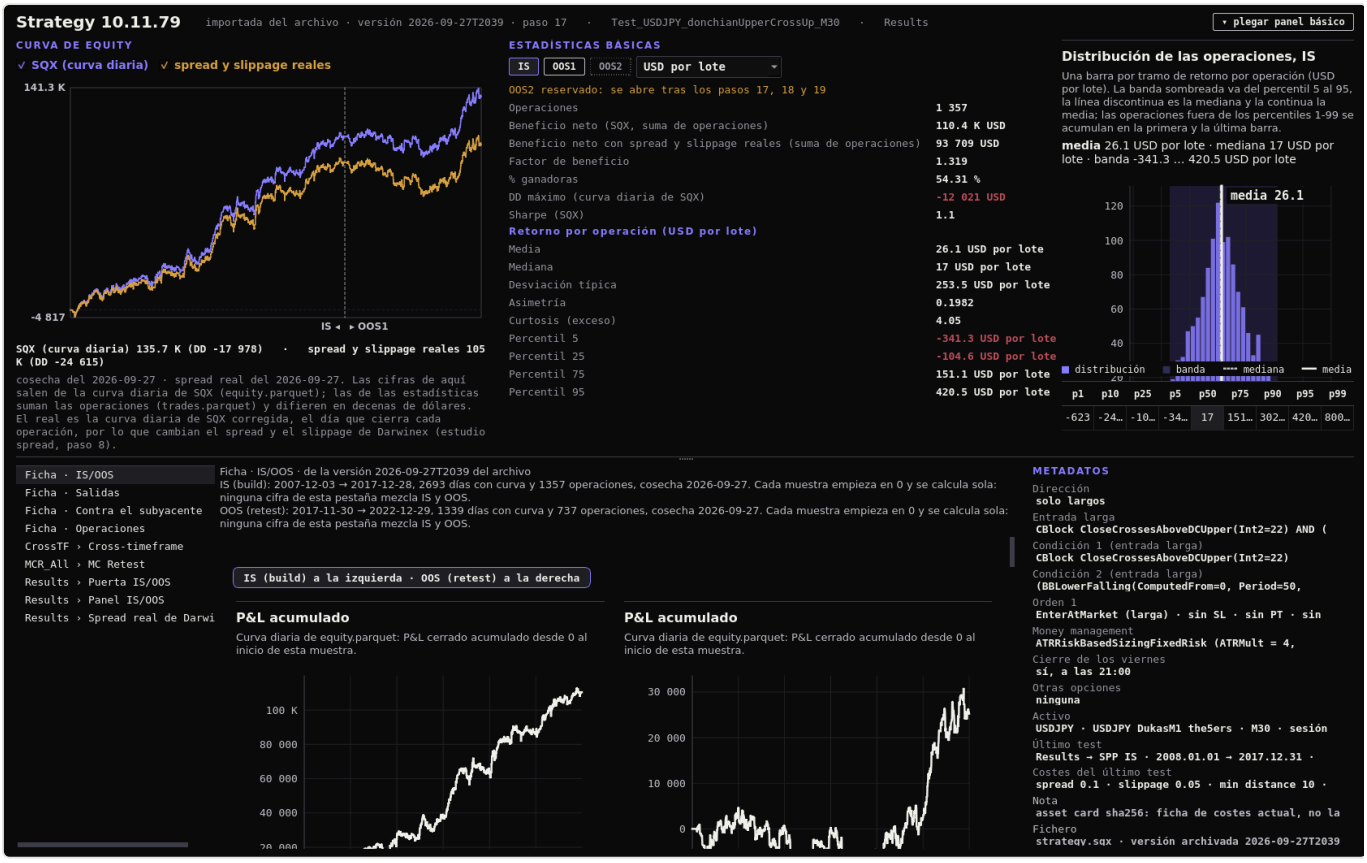
Nada en disco. Lee `AlgoData/archive/<identidad>/<versión>/` a través del demonio (`/api/archive/list` , `/api/archive/show` y las rutas de la ficha con `source=archive`). El archivo es de solo lectura: importar no lo cambia.

Cómo se lee el resultado



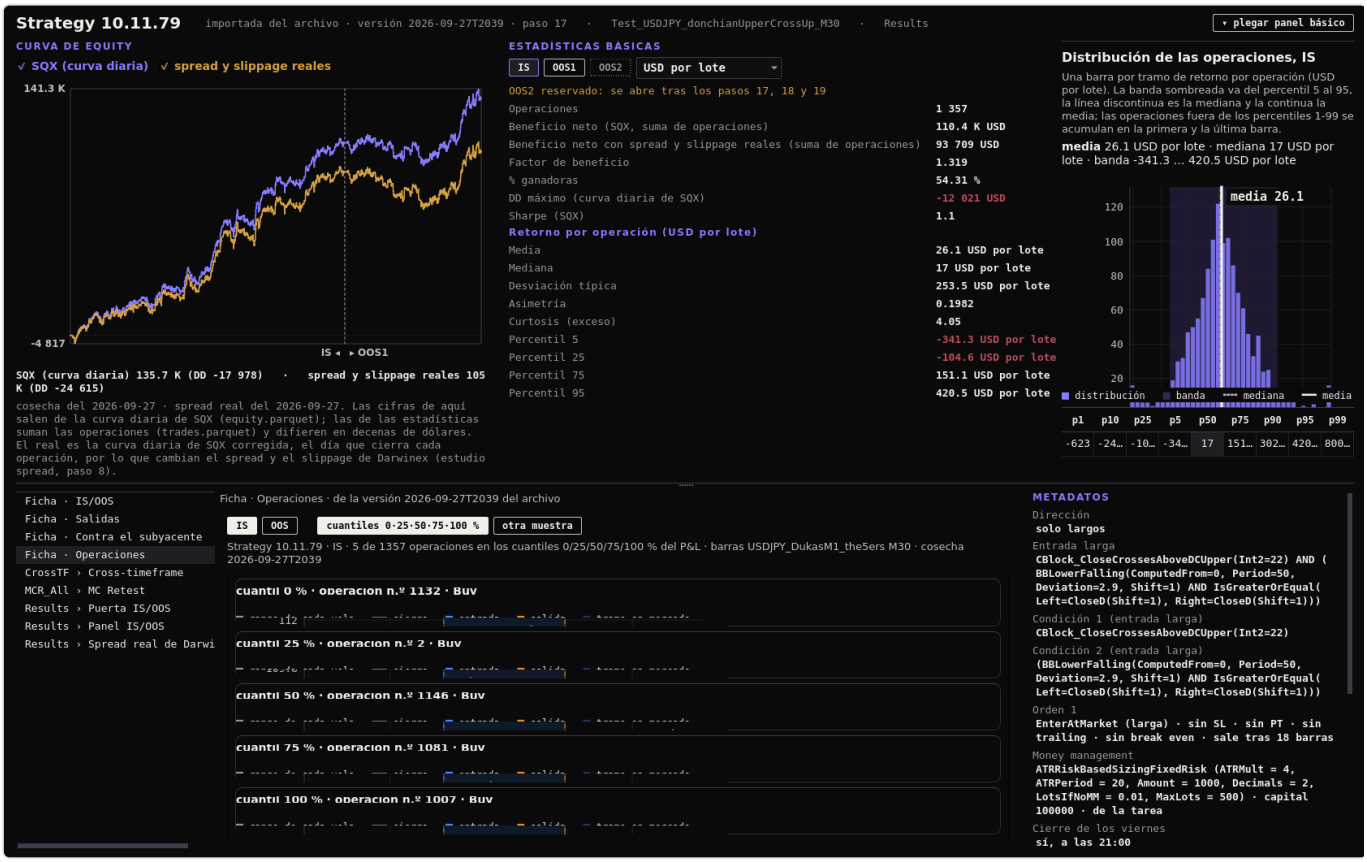
- **Tabla de la izquierda.** Una fila por estrategia. **N del Ledger** es cuántas candidatas se puntuaron en **todas** las búsquedas del estudio del Ledger (la plantilla en ese activo y timeframe), no solo en la búsqueda que encontró esta estrategia, según el Ledger el día que se archivó su última versión. Es lo que hará falta para deflactar su Sharpe (un Sharpe alto entre 200 probadas vale menos que entre 5). **Búsquedas** son todas las filas del Ledger de ese estudio: cada build, retest o filtro que miró sus datos, apuntara candidatas o no. Si hay búsquedas pero ninguna apuntó candidatas, N sale como **«—»**, no como 0. La identidad de la estrategia no se imprime: está en el tooltip del nombre.

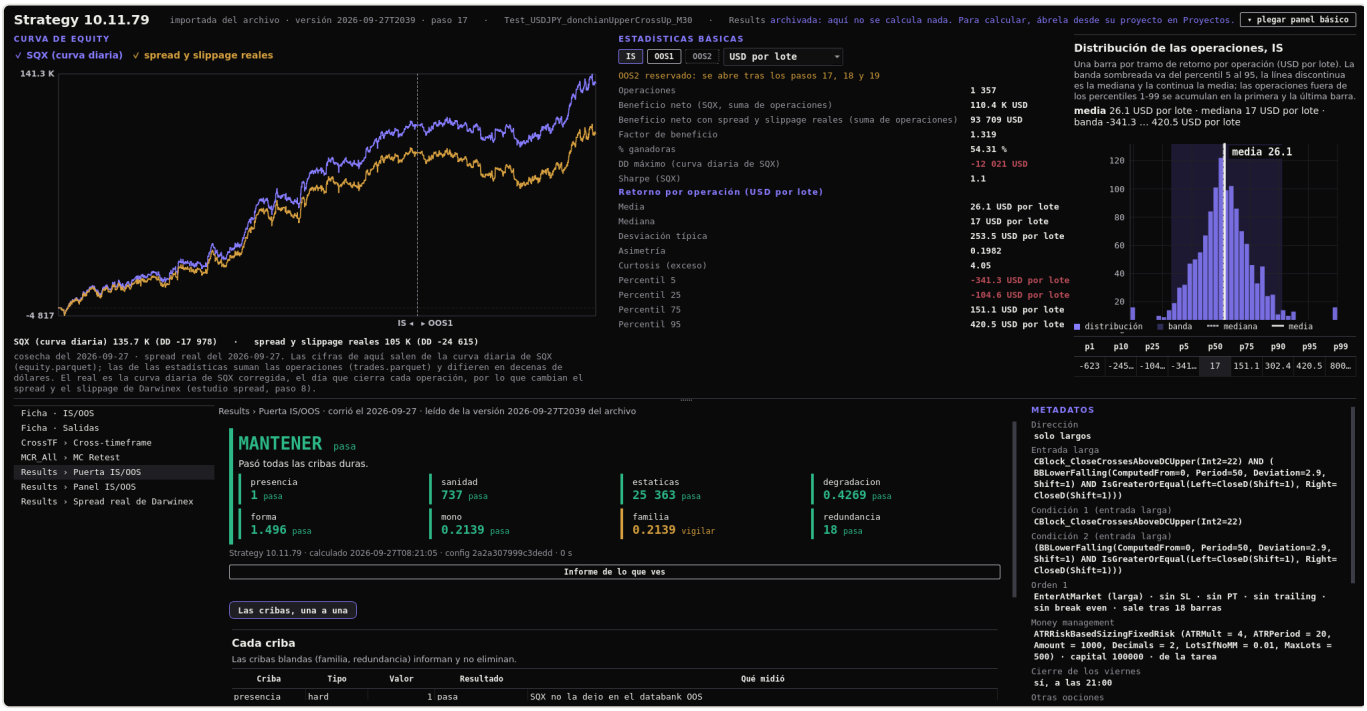
- **Versiones.** La más nueva arriba, con su fecha, el paso y la nota que escribiste al archivar. Una versión nunca se sobrescribe: archivar otra vez crea otra.
- **Lo que trae / Lo que no se archivó.** Los estudios congelados por databank, y los que se quedaron fuera con su motivo (por ejemplo, un crossmarket cuyas filas no llevan identidad: emparejarlas por nombre sería adivinar). Una versión antigua, anterior a esa lista, lo avisa.
- **Procedencia.** La cuenta del Ledger de esa versión (puede ser distinta de la de la tabla), la ficha de costes congelada y el código.



La página importada es la de Estrategia (capítulo 68) con tres diferencias: la cabecera dice **«importada del archivo · versión … · paso …»**; no hay botón «Archivar»; y debajo del panel básico, en vez de las pestañas de familias, hay una lista con la **Ficha** (IS/OOS, Salidas, Contra el subyacente y Operaciones) y cada estudio archivado, por databank. Un estudio que solo escribe el resultado del databank entero (el spread real, el cross-timeframe) enseña ese, y la línea de arriba lo dice.

Lo que no sale en una estrategia importada: las **pestañas de familias** de la página viva con sus **puntos de estado**, el **historial** de corridas, **«comparar»** (dos corridas o dos estrategias) y la **barra de correr**, porque todas leen el proyecto vivo — y con ellas, la **«Lote»** de la pestaña Walk Forward Correlation (las variantes de la madre se leen en vivo por su nombre y el archivo no las congela; movida ahí desde IS/OOS el 2026-09-29, ver capítulo 68); y el botón **«Informe de lo que ves»**, que escribiría su página en la carpeta `reports/` viva.





Un ejemplo completo

La estrategia `Strategy 10.11.79` del proyecto `Test_USDJPY_donchianUpperCrossUp_M30` tiene cuatro versiones archivadas (pasos 16, 17, 17 y 16.5). Se elige la de 2026-09-27 20:39 (paso 17) y se pulsa «Importar esta versión». La prueba de que nada se recalculó y de que se ve lo mismo que en vivo, hecha el 2026-09-28 comparando cada ruta con `source=archive` contra la misma ruta en vivo:

```
IGUAL    /api/result CrossTF crossTF
DISTINTO /api/result MCR_All mcRetest   diferencias: $.result.partials solo en vivo
DISTINTO /api/result Results gate       diferencias: $.result.partials solo en vivo
DISTINTO /api/result Results isOos      diferencias: $.result.partials solo en vivo
IGUAL    /api/result Results spread
IGUAL    /api/tearsheet Results IS
IGUAL    /api/tearsheet Results OOS
{"jobs":[]}      ← ningún trabajo nuevo en el demonio
hijos: []        ← el demonio no lanzó ningún proceso
```

La única diferencia es `partials`, una clave que la ventana empezó a servir el 28 de septiembre, después de archivar: el archivo guarda lo que se veía **ese día**. La versión de 2026-09-28 07:24, archivada ya con esa clave, sale igual en todas las rutas.

Qué NO te dice

- **No te dice si la estrategia sigue valiendo hoy.** Enseña lo que se calculó el día de archivar, con la configuración y los costes de ese día. Si un estudio se ha vuelto a correr después, o la ficha de costes del activo ha cambiado, la versión archivada no lo sabe.
- **No compone una cartera.** Correlaciones entre curvas, drawdown agregado, reglas de cuenta fondeada: nada de eso existe aún.
- **La cuenta del Ledger es la de ese día**, por estudio (plantilla), no por estrategia. Si el fichero del Ledger se movió, cuenta 0 búsquedas; el manifiesto guarda su huella para verlo.

Si algo falla

- **«El archivo está vacío»**: no hay ninguna estrategia archivada. Archiva una desde su ficha.
- **«No hay demonio en el puerto 8765»** (la vista previa): arranca la ventana con `bin/algoui` o el demonio con `python3 -m ui.daemon.serve`.
- **«No pude leer el archivo: …»**: una carpeta de `AlgoData/archive/` se tocó a mano o quedó a medias. Las carpetas a medias empiezan por punto y no se listan; cualquier otra cosa, dilo.

35. La ventana — cómo se abre, el mapa de sus zonas, y las plantillas

La ventana. No es una pestaña del navegador ni un panel más: es la aplicación de escritorio que envuelve SQX y Python, la única de esta máquina. Este capítulo cuenta cómo se abre y las cuatro primeras zonas de BIBLIOTECA, las de las **plantillas**: Cobertura, Plantillas, Nueva plantilla y Paletas. El mapa entero de la barra lateral está al final, en «Las demás zonas».

Qué pregunta responde

Tres, y las tres se contestaban hasta hoy abriendo dos CSV en Excel y una carpeta en el explorador:

- **¿Qué plantillas tengo, de qué tipo, y en qué mercados y timeframes las he probado?**
- **¿De qué tengo poco?** Qué arquetipo no he tocado, qué timeframe está vacío, qué idea probé una sola vez y no volví a mirar.
- **¿Cómo empiezo una plantilla nueva sin olvidarme de ninguna de las preguntas?** La ventana te entrevista, escribe el borrador del brief y te deja el comando listo para pegar en Claude Code.

Cuándo lo usas, y cuándo no

Lo usas para mirar la librería, para decidir qué construir a continuación, para anotar el veredicto de una corrida en cuanto lo tengas, y para redactar la idea de una plantilla nueva.

No lo usas para construir nada. Estas zonas **no arrancan SQX, no instalan bloques y no emiten ningún** `.sqx`. Todo eso lo hace `/sqx-strategy-template` y `/template-run`, que escriben en instalaciones de verdad y son el carril del conductor y el del custodio. Una ventana que escribiera en `SQX_w1` por su cuenta rompería la tabla de carriles de `sqx/CLAUDE.md` en cuanto hubiera dos sesiones abiertas.

Tampoco es donde se editan los CSV. Escribe tres cosas y solo tres: el **estado** de una plantilla, su **nota**, y el **veredicto** de una corrida.

Antes de empezar

- `pip install -r requirements.txt` — la app añade PySide6, FastAPI, uvicorn y httpx.
- Nada más. No hace falta SQX abierto ni cerrado, no hace falta ningún worker arrancado, y no toca ninguna instalación. Si `~/Desktop/AlgoData/templates/` no existe todavía, la ventana abre vacía en vez de fallar.

Cómo se ejecuta

Tres formas, las tres hacen exactamente lo mismo:

dónde	qué haces
el escritorio	doble clic en el icono **AlgoProject**
el menú de aplicaciones	buscas *AlgoProject*
cualquier terminal, desde cualquier carpeta	`algoui`

```
algoui
```

Las tres acaban en `bin/algoui`, que no hace más que llamar a `python3 -m ui.desktop.launch` desde la raíz del proyecto. El icono es la propia matriz de cobertura en miniatura, y es el mismo que sale en la barra de tareas y en el alt-tab.

`algoui` funciona desde cualquier sitio porque `~/.local/bin/algoui` es un enlace al del repositorio y el script resuelve el enlace antes de buscar la raíz. Si alguna vez mueves la carpeta del proyecto, rehaz el enlace y la línea `Exec=` de `~/.local/share/applications/algoproject.desktop`: las dos llevan la ruta escrita.

En Windows la ventana funciona igual, sin SQX: doble clic en `bin\algoui.cmd`, o `bin\algoui.cmd` desde una consola. Hace lo mismo que `bin/algoui` y además activa el modo UTF-8 de Python, sin el cual los textos en español se leen en cp1252. Lo único que cambia sin SQX es la ficha de una plantilla: sus huecos atados a un grupo salen como *«atado a un grupo que esta instalación no tiene»*, porque los nombres de los grupos viven en el `blockGroups.xml` de la instalación. Cómo montarla en un PC nuevo está en el capítulo de GitHub.

flag	obligatorio	qué hace
`--zone`	no	la zona que se abre, por su nombre en la barra lateral (`algoui --zone Proyectos`); sin él, Cobertura
`--shot`	no	guarda una captura PNG de la ventana en esa carpeta y se cierra (capítulo 60)
`--port`	no	el puerto del demonio; sin él, `ui_port` de `config/machine.yaml` (8765). Para pruebas y capturas usa **otro** puerto: el 8765 es el de tu ventana (capítulo 60)

Si algo va mal y la ventana no abre, el demonio se arranca por separado para ver su error:

```
python3 -m ui.daemon.serve
```

Dos procesos, no uno. `bin/algoui` levanta el demonio si no está ya escuchando y abre la ventana. Al cerrar la ventana, el demonio se apaga **solo si lo arrancó ella**: dos ventanas comparten uno, y matarlo le quitaría la librería a la otra. El demonio escucha en 127.0.0.1 y en ningún sitio más.

Qué produce

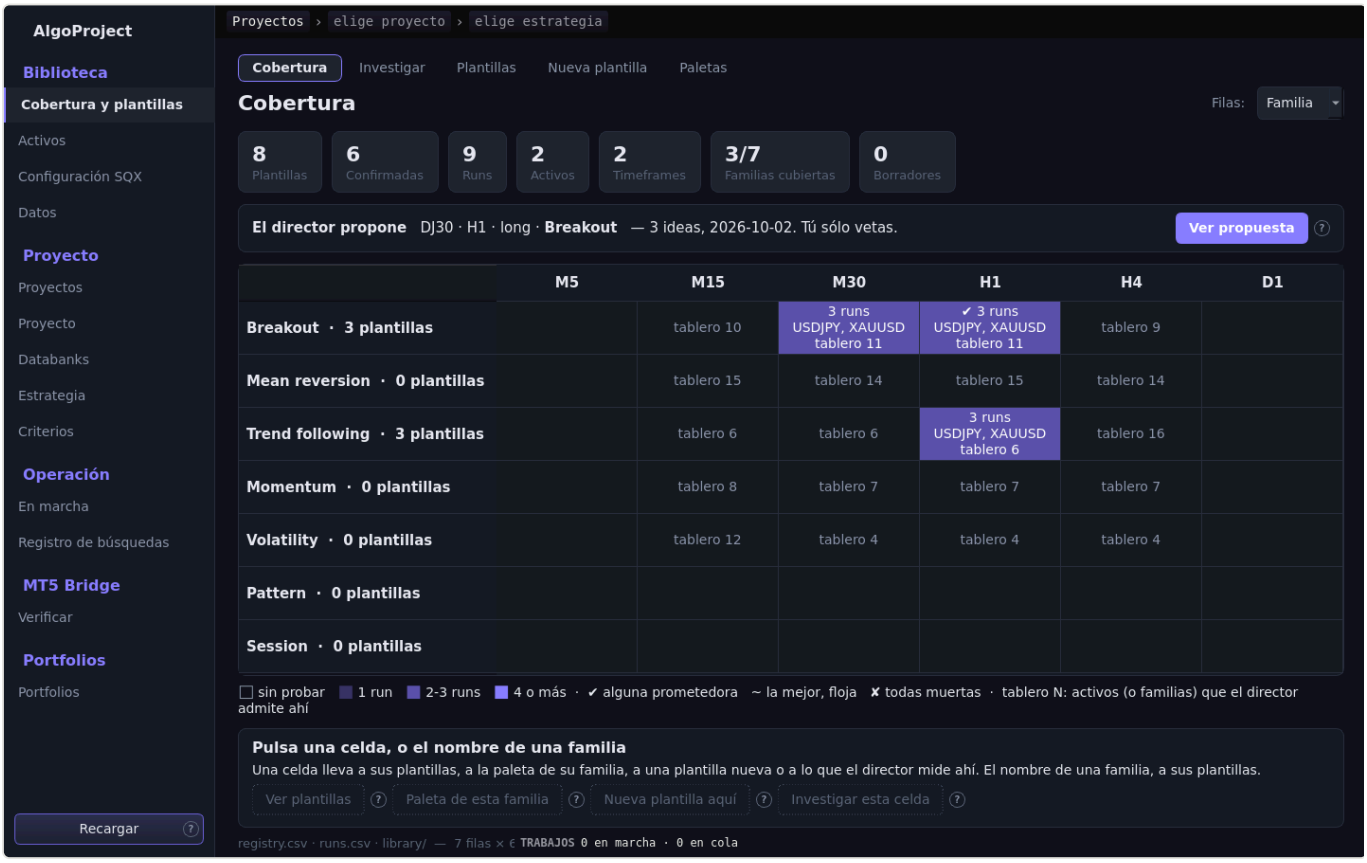
ruta	qué es	cuándo se escribe
`AlgoData/templates/registry.csv`	la fila de la plantilla, con su `status` y su `notes`	al pulsar **Guardar** en la ficha
`AlgoData/templates/runs.csv`	la fila de la corrida, con su `verdict`	al cambiar el desplegable de veredicto
`AlgoData/templates/drafts/<nombre>.json`	el borrador del brief que redacta el chat	al terminar la entrevista
`sqx/blocks/palettes/<familia>.yaml`	la paleta de bloques de un arquetipo	al pulsar **Guardar** en Paletas

Los dos CSV los escribe `sqx/templates/registry.py`, **la misma función que usa la línea de comandos**. La ventana no tiene una segunda forma de escribir un CSV: dos escritores para un fichero es como se descuadra una columna.

⚠ **La ventana escribe paletas, nunca la taxonomía.** `sqx/blocks/taxonomy.yaml` lleva las etiquetas y lo rellena otra persona (`docs/encargos/6-taxonomia-bloques.md`); desde aquí solo se escriben los `overrides` de una paleta. Son dos ficheros distintos a propósito.

Cómo se lee el resultado

Cobertura — la primera pantalla



Cobertura es **la portada de la biblioteca**. En la barra lateral es una sola entrada, «Cobertura y plantillas», y arriba de la página hay una tira con sus cinco páginas, en el orden del workflow: **Cobertura · Investigar · Plantillas · Nueva plantilla · Paletas**. Todas siguen abriéndose por su nombre con Ctrl+K.

Arriba, siete contadores. **Todos explican qué cuentan al pasar el ratón por encima**, porque un número que nadie sabe definir es un número con el que nadie debería decidir. El que más dice es *Familias cubiertas*: `3/7` significa que de las siete familias que el catálogo conoce, has tocado tres. Esa distancia **es** la respuesta a «de qué tengo poco».

Debajo, **lo que propone el director de investigación** en una línea (activo, timeframe, dirección y familia) con el botón «Ver propuesta»: sus tres ideas, que tú sólo vetas. Si no hay propuesta, el botón lleva al tablero para pedirle una.

Después, la rejilla. Las columnas son siempre los timeframes. Las filas las eliges arriba a la derecha: **Familia**, **Activo** o **Plantilla**. Las familias se escriben igual en toda la ventana: Breakout, Mean reversion, Trend following, Momentum, Volatility, Pattern y Session; al lado de cada una, cuántas plantillas tiene.

Cada celda dice, en hasta tres líneas: cuántos runs hay, en qué activos, y **«tablero N»**: en cuántos activos el director admite esa familia en ese timeframe (por tu prior Alta o porque lo medido pasa los filtros; sólo trabaja de M15 a H4). Una celda sin probar con «tablero 15» es un hueco donde merece la pena mirar; una sin nada, uno donde el director no propondría.

fondo de la celda	significa
oscuro	**sin probar**
violeta apagado	1 run
violeta medio	2-3 runs
violeta vivo	4 o más

La marca delante del número es el **mejor veredicto** que hay dentro, no un promedio: `✔` algún run prometedor, `~` el mejor quedó flojo, `✘` todos muertos; sin marca, sin veredicto escrito.

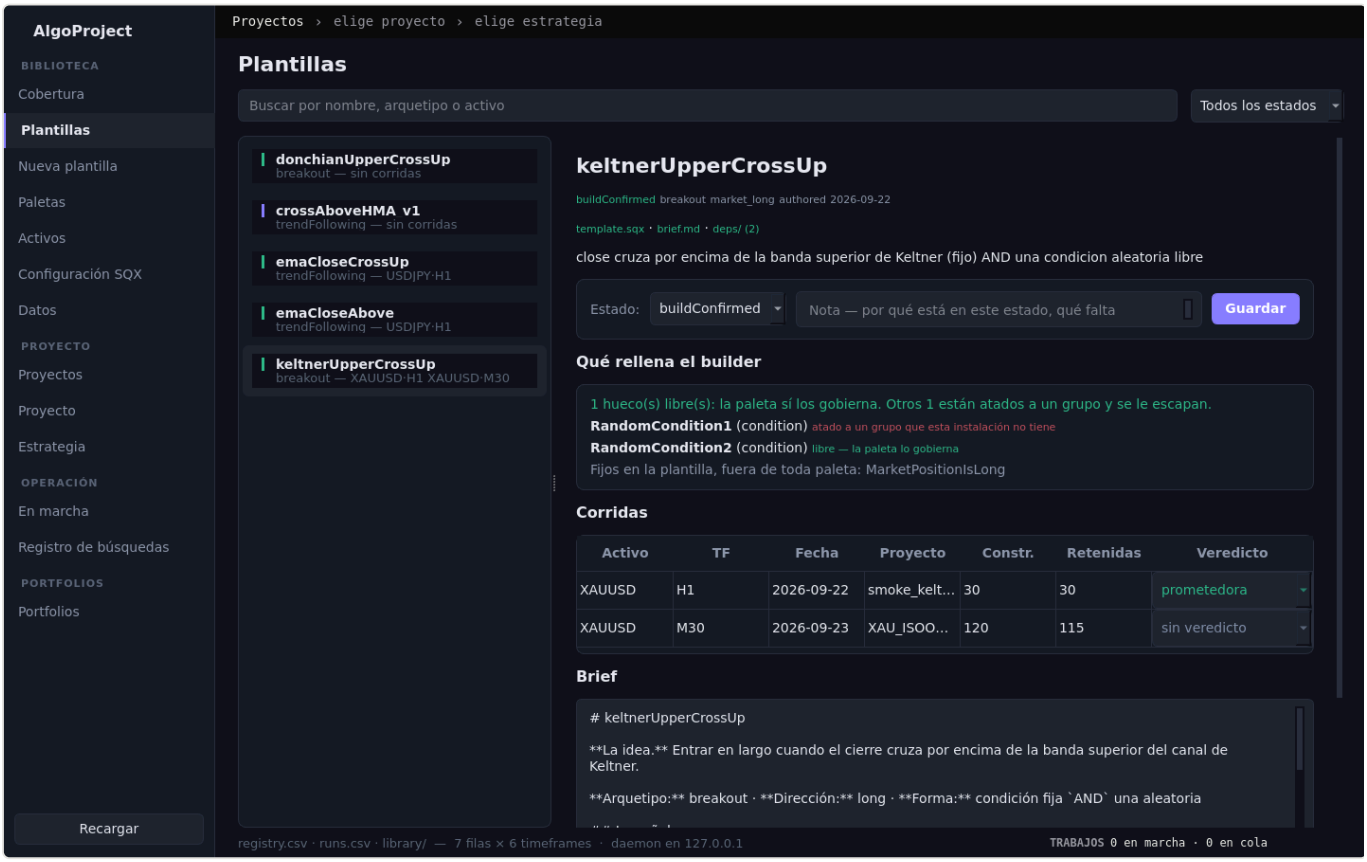
De una celda se va a todo lo demás. Pulsar el nombre de una familia abre Plantillas con sólo las suyas. Pulsar una celda la elige, y el panel de abajo dice qué hay en ella y ofrece cuatro botones:

botón	abre
Ver plantillas	Plantillas, filtrada por esa familia (doble clic en la celda hace lo mismo)
Paleta de esta familia	Paletas, en la paleta por defecto de la familia
Nueva plantilla aquí	la entrevista, con la familia ya contestada
Investigar esta celda	el mapa de Investigar en el activo que el tablero pone primero para esa celda

Las filas de arquetipo que nadie ha tocado **siguen apareciendo**, vacías. Si se generaran a partir de los datos desaparecerían, y con ellas justo la información que buscas.

Un clic en una celda con datos abre esa plantilla.

Plantillas — la ficha

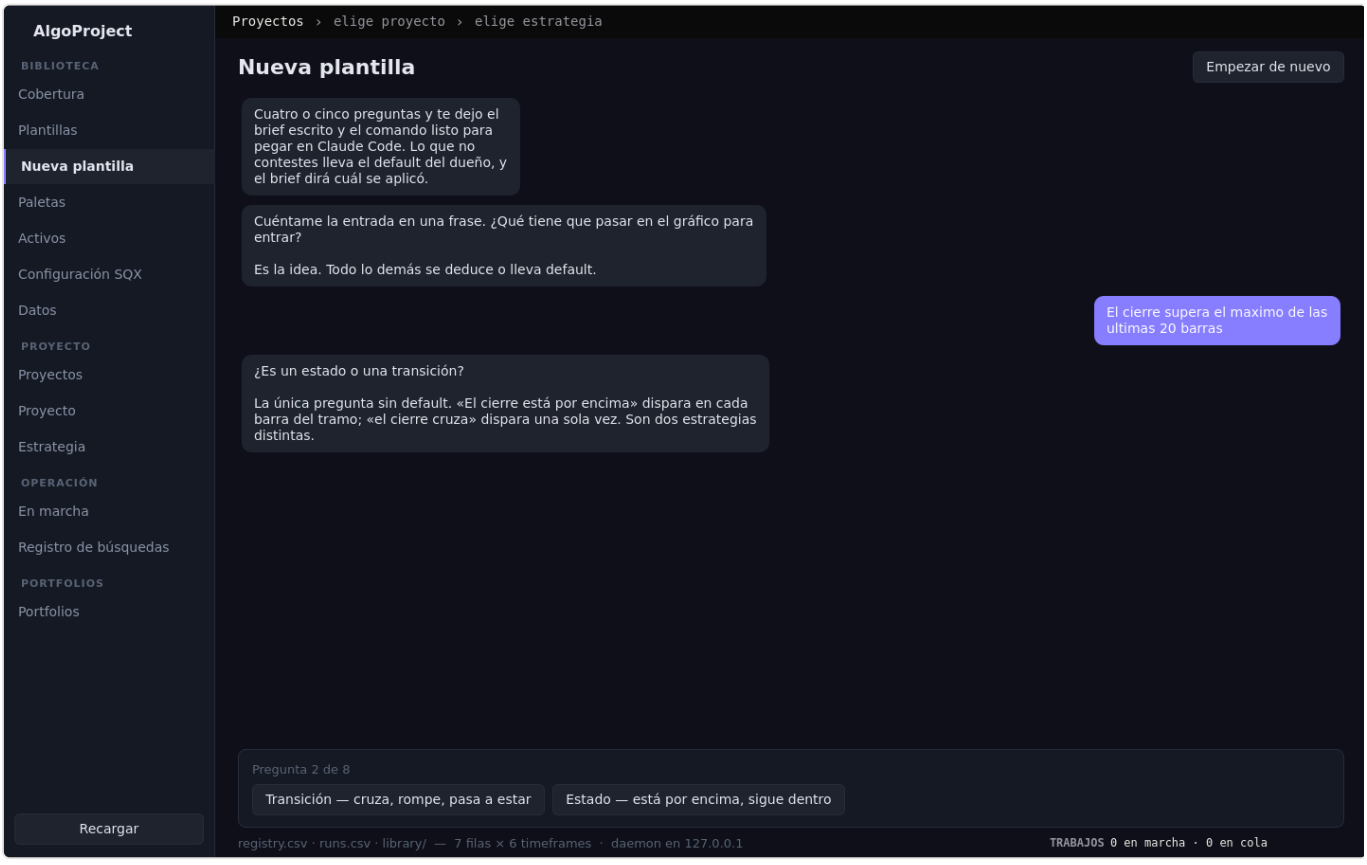


A la izquierda la lista, con buscador (nombre, familia o activo) y filtros por familia y por estado. Los borradores del chat aparecen marcados como tales: **no son plantillas**, no tienen `.sqx`, y por eso no cuentan en la matriz.

A la derecha la ficha. En orden:

- **Las etiquetas**: estado, arquetipo, forma, origen y fecha. El estado explica su significado al pasar el ratón: `draft` es XML sin comprobar, `validated` es XML que resuelve, `buildConfirmed` es el único que prueba algo — una construcción real sacó estrategias y llevan de verdad su bloque.
- **La línea de disco**: `template.sqx`, `brief.md`, `deps/`. En verde lo que está, en rojo lo que falta. El registro es una **afirmación**; esta línea es la **prueba**. Importa porque SQX no da error con una plantilla cuyos bloques no están instalados: la quita del builder en silencio.
- **Estado y nota**, con su botón Guardar.
- **Qué rellena el builder**: qué huecos de la plantilla son libres y cuáles están atados a un grupo (se explica más abajo, en Paletas).
- **Corridas**: una fila por mercado y timeframe, con las estrategias construidas y retenidas, y el veredicto como desplegable. El veredicto se escribe **al cambiarlo**, sin botón: es un juicio tuyo, no un formulario.
- **Brief**: el texto en castellano tal cual lo escribió quien la autoró. La ventana no lo resume y no lo deja editar por detrás del fichero.

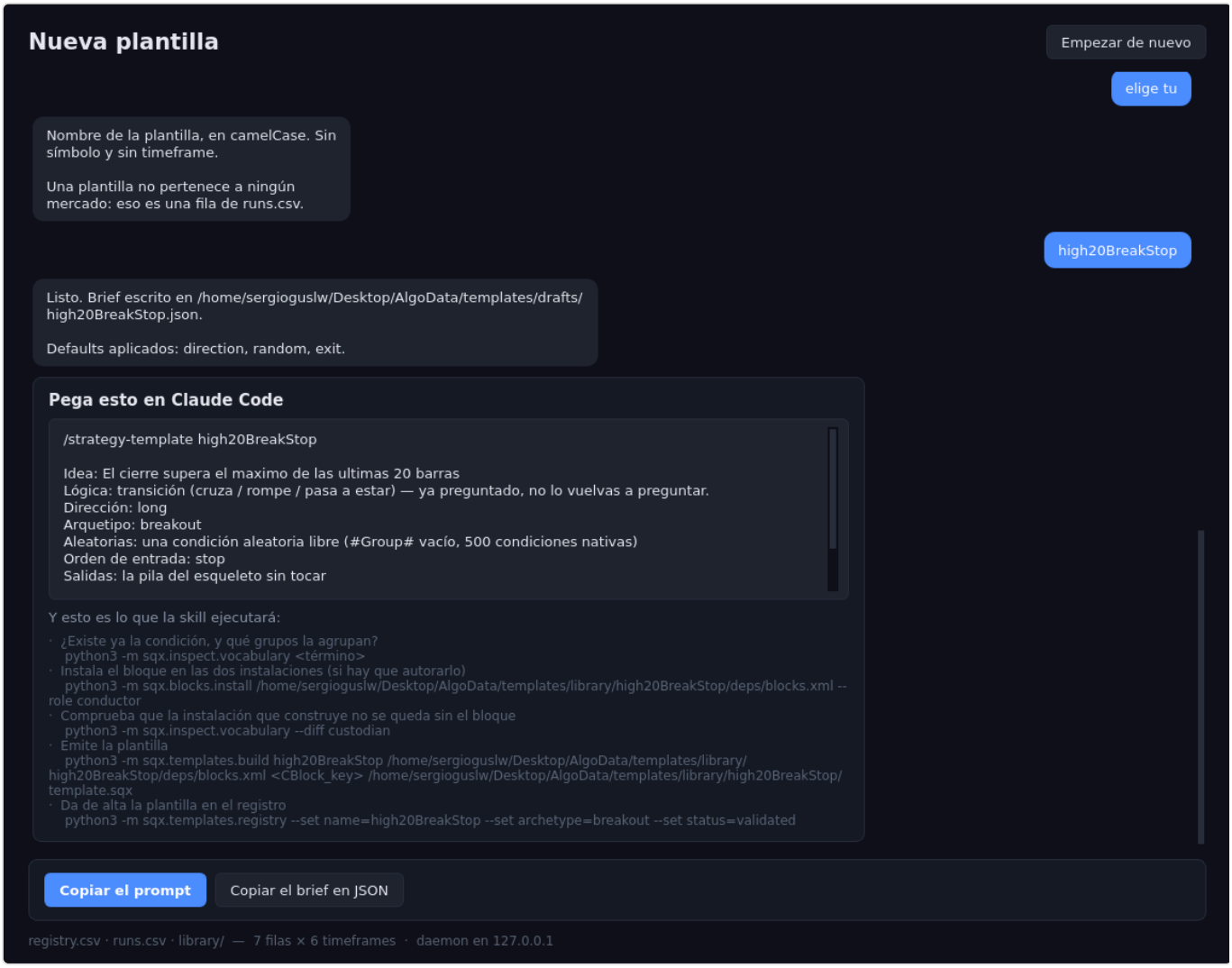
Nueva plantilla — el chat



Ocho preguntas como mucho. Las que llevan default del dueño traen un botón **Elige tú** que lo aplica y lo deja escrito en el brief; las que no lo llevan, no se pueden saltar.

Una pregunta no tiene default y nunca lo tendrá: estado contra transición. «El cierre *está* por encima de la banda» dispara en cada barra del tramo; «el cierre *cruza* la banda» dispara una sola vez. Son dos estrategias distintas, no dos maneras de decir lo mismo, y adivinarla gasta CPU para construir lo que no pediste.

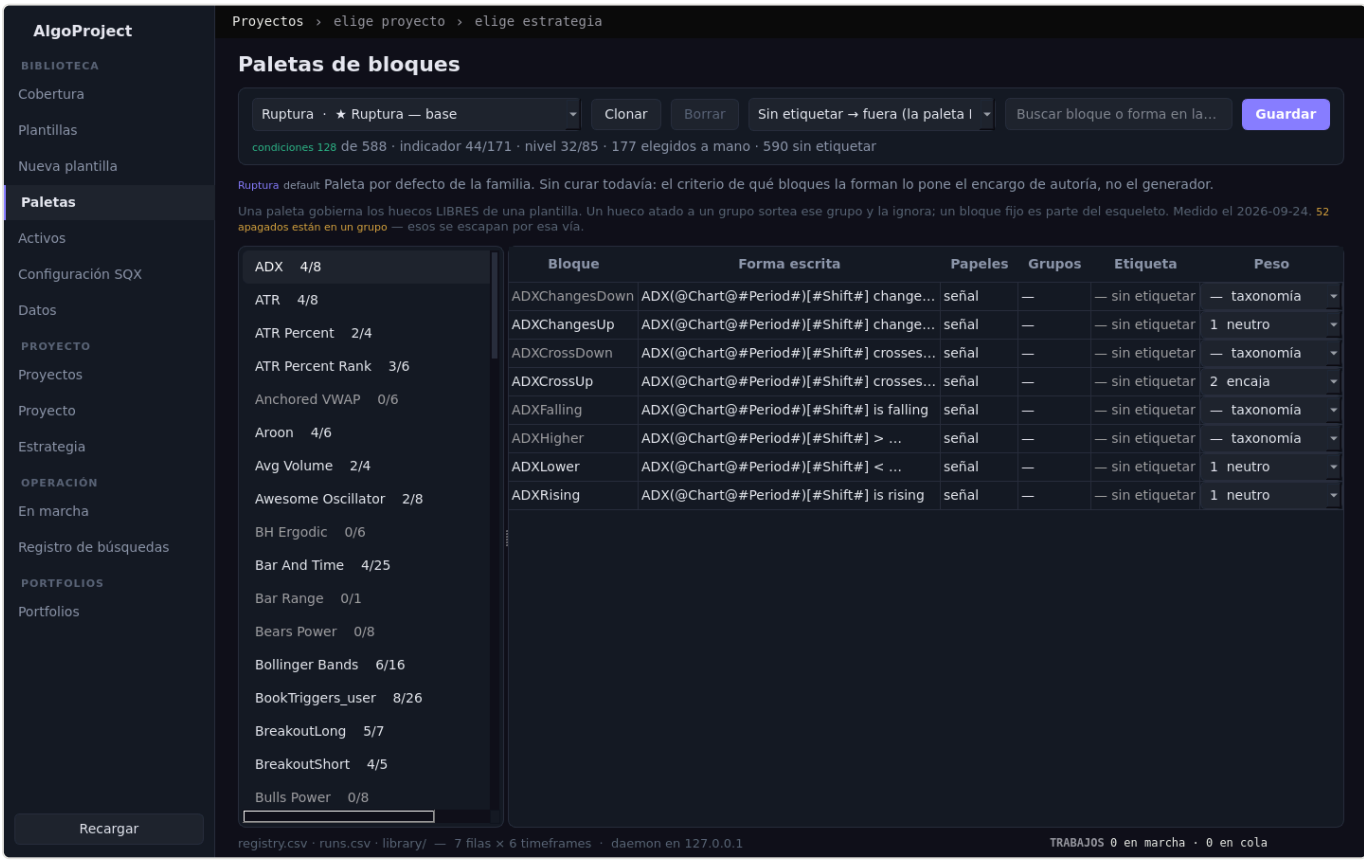
Al terminar:



Te quedan tres cosas: el **borrador del brief** escrito en disco, el **prompt** para pegar en Claude Code —que ya lleva todas las respuestas, para que la skill no vuelva a preguntar lo que ya contestaste—, y **la lista de comandos que la skill va a ejecutar**, para que sepas lo que viene. La ventana no ejecuta ninguno.

Fíjate en la línea *Defaults aplicados: direction, random, exit*. El brief deja constancia de qué preguntas te saltaste y qué default las rellenó. Un default aplicado en silencio y nunca escrito es como se pierde una decisión.

Paletas — qué puede sortear el builder



Esta es la vista que estrecha el vocabulario del builder, y es una **librería**: no hay tres paletas fijas, hay las que haya. Vienen tres por defecto, una por familia, marcadas con ★, y encima de ellas se clona todo lo demás.

Arriba, de izquierda a derecha:

- **El selector de paleta**, agrupado por familia: `Ruptura · ★ Ruptura — base`. Las ★ son las de por defecto.
- **Clonar**: copia la paleta abierta con el nombre que le des y la abre. Es el camino normal — parte de una por defecto y personalízala.
- **Borrar**: solo para las tuyas. En una por defecto está deshabilitado y dice por qué.
- **Qué hacer con lo no etiquetado**: `entra a peso 1` (la paleta hereda la taxonomía y tú solo corriges) o `fuera` (**la paleta ES su lista**: solo entra lo que hayas elegido). La segunda es la que hace útil una paleta curada a mano sin esperar a que la taxonomía esté etiquetada.
- **El buscador**, que ignora la categoría: encuentra un bloque sin saber en cuál de las 83 lo archivó SQX.
- **Los recuentos**, en su propia línea. El primero, **condiciones**, va en verde o en rojo según caiga dentro del **90–170 que pide el dueño** por build: por debajo el builder no tiene de dónde combinar, por encima vuelve el sobreajuste. Si una paleta se sale, la ficha de debajo lo dice y además apunta hacia dónde — faltan bloques o sobran.

Debajo, la ficha de la paleta abierta: su familia, si es por defecto o tuya, de cuál es clon, y su nota. Y si no ha elegido nada todavía, un aviso en ámbar diciéndolo con todas las letras — porque una paleta vacía y una paleta que deja pasar todo se ven igual, y no son lo mismo.

A la izquierda, las categorías con cuántos bloques tienen encendidos. A la derecha, los bloques: su clave, **su forma escrita** —que es lo único que dice de verdad qué comprueba—, sus papeles, los grupos que lo agrupan, su etiqueta y el peso.

La lista de bloques es siempre la misma — son los 767 del vocabulario del builder — y eso no cambia entre familias. Lo que cambia por familia son las dos últimas columnas y los recuentos.

- **Etiqueta**: lo que `taxonomy.yaml` dice de ese bloque **para la familia que tienes elegida**. Es la columna que hace visible el cambio de familia. `— sin etiquetar` significa que nadie ha dicho nada todavía, y entonces las tres familias lo tratan igual.
- **Peso**: lo único editable, y es un *override*. `— taxonomía` significa «haz lo que diga la etiqueta»; cualquier otro valor la pisa **solo para esta paleta**. `0` apaga el bloque y lo pone en gris, `3` lo prefiere. Se guarda con el botón; hasta entonces el recuento dice cuántos llevas sin guardar.

Un bloque apagado se ve en gris en toda su fila, y el contador de su categoría baja: `ADX 8/8` pasa a `ADX 5/8`. Ésa es la forma rápida de comprobar que una paleta está estrechando de verdad.

⚠ La línea gris de debajo repite el límite, y hay que tomárselo en serio: **una paleta gobierna los huecos LIBRES de una plantilla.** Un hueco atado a un grupo sortea ese grupo y la ignora, y un bloque fijo es parte del esqueleto. Está medido el 2026-09-24 y está en `knowhow/authoring/builder-block-switches.md`. Cuando un bloque apagado está en un grupo, su celda de **Grupos** se pone en ámbar y lo dice.



Por eso la ficha de cada plantilla lleva ahora **Qué rellena el builder**: un hueco por línea, diciendo si es libre —y entonces la paleta lo gobierna— o a qué grupo está atado, y debajo los bloques fijos, que están fuera de toda paleta. Si esa línea dice que no hay huecos libres, la vista de paletas no puede hacer nada por esa plantilla y hay que tocar el grupo.

En la captura, `RandomCondition1` sale en rojo, *«atado a un grupo que esta instalación no tiene»*, y no es un fallo de la captura. La ficha resuelve los grupos con el `blockGroups.xml` **del maestro**, y el grupo de esta plantilla (`keltnerUpperCrossUpSignal`) se instaló solo en los workers, conductor y custodio, que son los que construyen. Comprobado el 28-09-2026: contra el conductor o el custodio ese hueco se lee «atado a keltnerUpperCrossUpSignal». El rojo quiere decir «el maestro no lo tiene», no «ningún install lo tiene».

Las demás zonas

La barra lateral de la izquierda (se ve en la captura de Cobertura) va en cinco grupos, y cada zona tiene su capítulo en este mismo PDF:

grupo	zonas	capítulo
Biblioteca	Cobertura y plantillas (la tira: Cobertura, Investigar, Plantillas, Nueva plantilla, Paletas)	este y el 84
	Activos	38
	Configuración SQX	66
	Datos	67
PROYECTO	Proyectos (la galería)	61
	Proyecto (lanzar en SQX, «Continuar workflow», el raíl y el embudo)	65
	Databanks (el panel de databanks: filtros, pestañas, tabla y equity)	65b y 69
	Estrategia (la ficha)	68, y 58 para leer el resultado de un estudio
	Criterios (qué exige cada paso para pasar solo)	95
OPERACIÓN	En marcha, Registro de búsquedas y la franja de trabajos	70
MT5 Bridge	Verificar	76
PORTFOLIOS	Portfolios (el último grupo)	71

Arriba de todas, la barra de contexto `Proyectos › proyecto › estrategia` dice qué hay elegido, y cada miga abre su zona. **Ctrl+K**, desde cualquier sitio, abre la paleta de comandos: escribe el nombre de una zona, de un proyecto, de un databank o de una estrategia y Enter la abre. Cómo se lee todo esto de un vistazo —las etiquetas, los números, el modo lectura— está en el capítulo 60.

Un ejemplo completo

Idea: *«el cierre supera el máximo de las últimas 20 barras»*.

1. `bin/algoui`. Abre en Cobertura. El 28-09-2026: `2/7` arquetipos, cuatro corridas, dos activos (XAUUSD y USDJPY).
2. **Nueva plantilla**. Escribes la idea. Contestas **Transición** — quieres que dispare al romper, no en cada barra por encima. Arquetipo **breakout**. Orden de entrada **Stop**. Dirección, aleatorias y salidas: **Elige tú** en las tres. Nombre: `high20BreakStop`.
3. La ventana escribe `~/Desktop/AlgoData/templates/drafts/high20BreakStop.json` y te da el prompt.
4. **Copiar el prompt** y pegarlo en Claude Code. Ahí sí: `/sqx-strategy-template` mira el vocabulario de la instalación, autora el bloque si no existe, emite el `.sqx` y da de alta la fila.
5. Vuelves a la ventana, **Recargar**. La plantilla ya está en la lista, el borrador ha desaparecido, y la fila `breakout` de la matriz sigue sin corrida nueva — porque autorar no es construir. Eso es `/template-run`.
6. Cuando esa corrida termine y la hayas mirado, abres su ficha y le pones el veredicto.

Qué NO te dice

- **No te dice si una plantilla es buena.** `buildConfirmed` significa que SQX la acepta y que las estrategias llevan su bloque. No dice que gane dinero. Eso lo deciden los pasos 8 a 20 del workflow, que se leen en las zonas Proyecto y Databanks, no en estas.
- **El veredicto de una corrida es tuyo, no suyo.** La ventana lo guarda y lo pinta; no lo calcula. Una celda verde significa «tú dijiste que era prometedora», nada más.
- **La matriz cuenta corridas, no calidad ni tamaño.** Una celda con `1` puede ser una construcción de humo de 30 estrategias y 29 segundos. El número de estrategias está en la ficha, no en la celda.

- **No sabe lo que hay dentro de un `.sqx`.** La línea de disco dice si el fichero existe, no si su contenido es el que el registro afirma. Para eso está `sqx/inspect/template_check.py`. La excepción es el panel de huecos, que sí lo abre y lee sus bloques aleatorios.
- **Una paleta guardada todavía no llega a SQX.** La ventana escribe el YAML; quien lo tiene que aplicar al `<BuildingBlocks>` de la tarea Build es el builder, y esa pieza no está escrita. Hasta entonces una paleta es una decisión anotada, no un efecto.
- **La paleta no etiqueta.** Los pesos salen de `taxonomy.yaml`, que está casi entero sin etiquetar: el 28-09-2026 la línea de recuentos de la paleta Ruptura dice `590 sin etiquetar` de los 767. Lo no etiquetado lo tratan igual las tres familias, así que lo que hoy distingue una paleta de otra son sus pesos puestos a mano (`177 elegidos a mano` en esa misma línea), no la taxonomía.

Si algo falla

- **`el demonio no respondió en el puerto 8765`** — hay otra cosa ocupando el puerto, o el demonio se cayó al arrancar. Arráncalo a mano con `python3 -m ui.daemon.serve` y lee el error; si el puerto está ocupado, cámbialo en `ui_port` de `config/machine.yaml`.
- **La ventana abre vacía y los contadores están a cero** — `AlgoData/templates/` no existe o no tiene CSV. No es un fallo: es una librería sin nada dentro.
- **Una plantilla aparece en rojo en la línea de disco** — su fila está en el registro pero su carpeta no está completa. Suele pasar al traer un registro de otra máquina sin traer `library/`.
- **`qt.qpa.plugin: could not load the Qt platform plugin "xcb"`** — **pasó en esta máquina**, el 2026-09-24, la primera vez que se abrió la ventana de verdad. El plugin `xcb` de Qt necesita cuatro librerías que Ubuntu no trae puestas: `libxkbcommon-x11.so.0`, `libxcb-icccm.so.4`, `libxcb-keysyms.so.1` y `libxcb-xkb.so.1`. El mensaje que da Qt culpa a `libxcb-cursor0`, que **sí estaba** — o sea que el mensaje miente sobre cuál falta. Quién falta de verdad lo dice esto:

```
bash ldd ~/.local/lib/python3.10/site-packages/PySide6/Qt/plugins/platforms/libqxcb.so | grep "not found"
```

Como `sudo` pide contraseña aquí, no se instalan con `apt install`: se bajan y se extraen en el home, que es lo que ya está hecho en `~/.local/lib/qt-xcb/`.

```
bash apt-get download libxkbcommon-x11-0 libxcb-icccm4 libxcb-keysyms1 libxcb-xkb1 for d in *.deb; do dpkg -x "$d" x; done mkdir -p ~/.local/lib/qt-xcb && cp -P x/usr/lib/x86_64-linux-gnu/*.so* ~/.local/lib/qt-xcb/
```

`bin/algoui` añade esa carpeta al `LD_LIBRARY_PATH` él solo, así que una vez hecho esto no hay que acordarse de nada. En un ordenador nuevo con `sudo` a mano, `sudo apt install libxkbcommon-x11-0 libxcb-icccm4 libxcb-keysyms1 libxcb-xkb1` hace lo mismo y la carpeta del home sobra.

84. La ventana: Investigar — el mapa, la memoria, el director y su propuesta

Qué pregunta responde

¿Qué es cada mercado, qué se ha probado ya, y qué tres ideas lanzo ahora? La zona **Investigar** (BIBLIOTECA, debajo de Plantillas) junta en cuatro pestañas el perfil de mercado (cap. 82), la memoria de resultados (cap. 81), el tablero y el director (cap. 83) y la propuesta, con el botón que la manda a SQX.

Cuándo lo usas, y cuándo no

Para mirar el mapa y la memoria, siempre: no cuestan nada. «Proponer investigación» cuando quieras tres ideas nuevas (10-30 $). «Crear plantillas y lanzar en SQX» cuando hayas leído la propuesta y vetado lo que no quieras: ocupa el custodio varias horas por idea.

No la uses para lanzar una plantilla que ya existe (eso es Proyecto › «+ Nuevo proyecto») ni para seguir un build al detalle (eso es «En marcha»).

Antes de empezar

El perfil tiene que estar medido. Para lanzar: el custodio parado y libre, ningún otro lanzamiento de la ventana en marcha, y Claude Code instalado (`claude_bin` en `config/machine.yaml`).

Cómo se ejecuta

```
bin/algoui --zone Investigar
```

Los dos botones encolan estos trabajos del demonio; no hace falta lanzarlos a mano:

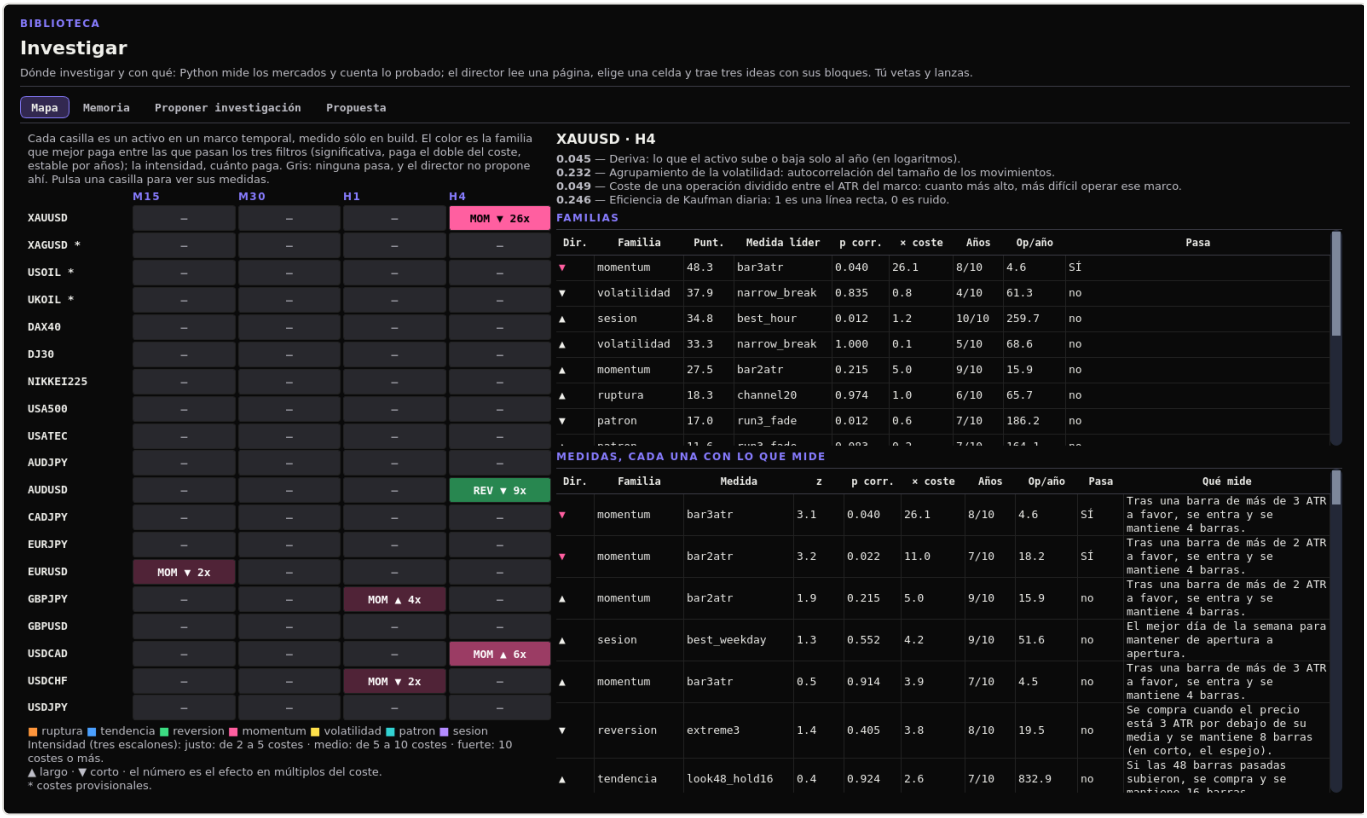
botón	trabajo	qué hace
«Proponer investigación»	`python3 -m ui.daemon.research.direct`	corre `/research-direct` sin nadie delante; termina con una propuesta
«Crear plantillas y lanzar en SQX»	`python3 -m ui.daemon.research.queue --proposal <id>`	la cola: una idea detrás de otra en el custodio

Los dos piden confirmación con su coste. Los proyectos de la cola se llaman siempre `Research_<ACTIVO>_<plantilla>_<marco>` (los proyectos del director de investigación; tu decisión del 2026-10-02): no se elige prefijo.

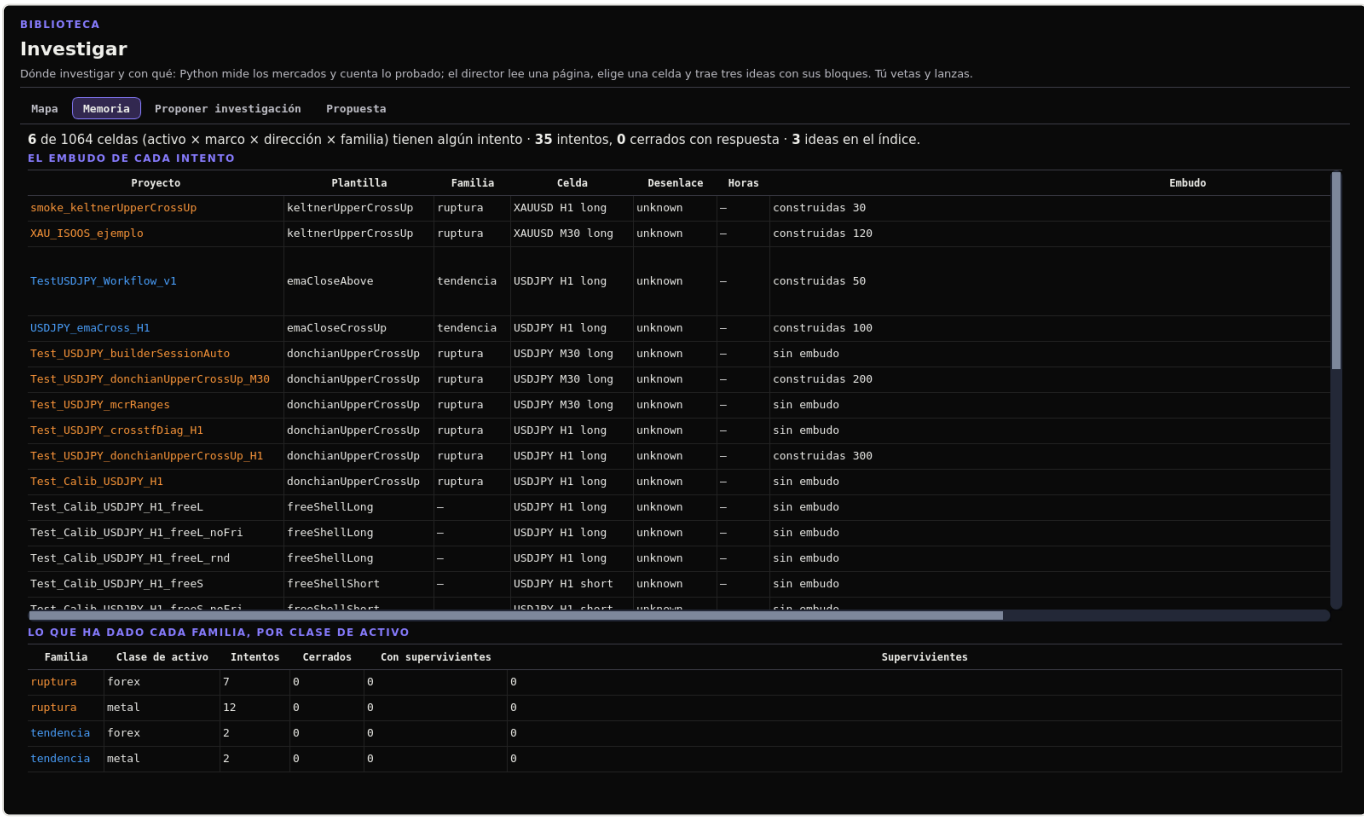
Qué produce

qué	dónde
la propuesta, con tus vetos y tus respuestas	`AlgoData/research/proposals/<id>.json` y `.md`
el estado de la cola	`AlgoData/research/proposals/queues/<id>.json`
por cada idea lanzada	una plantilla en la librería, una paleta en `sqx/blocks/palettes/`, un proyecto en el custodio, su run del autopiloto en `AlgoData/autopilot/` y su fila cerrada en `templates/runs.csv`

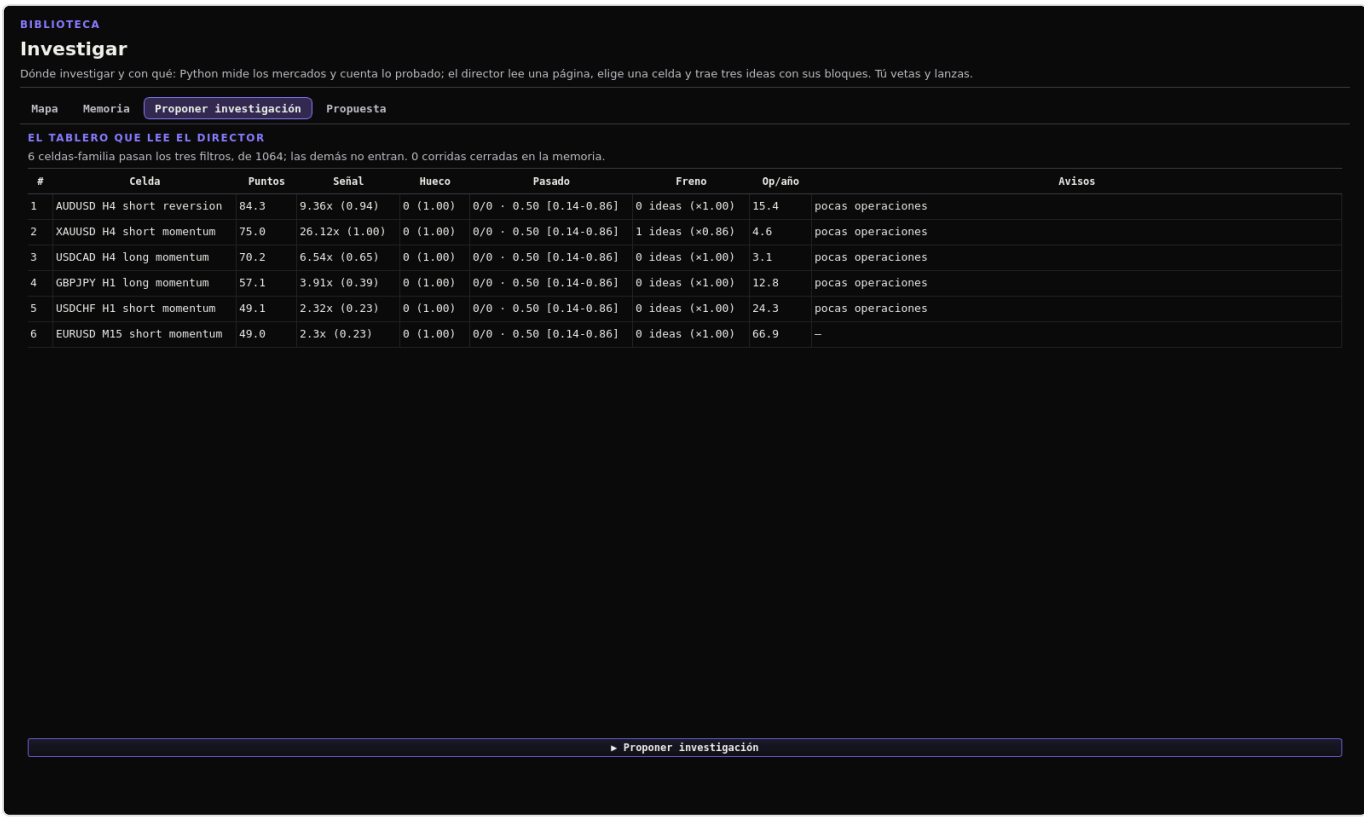
Cómo se lee el resultado



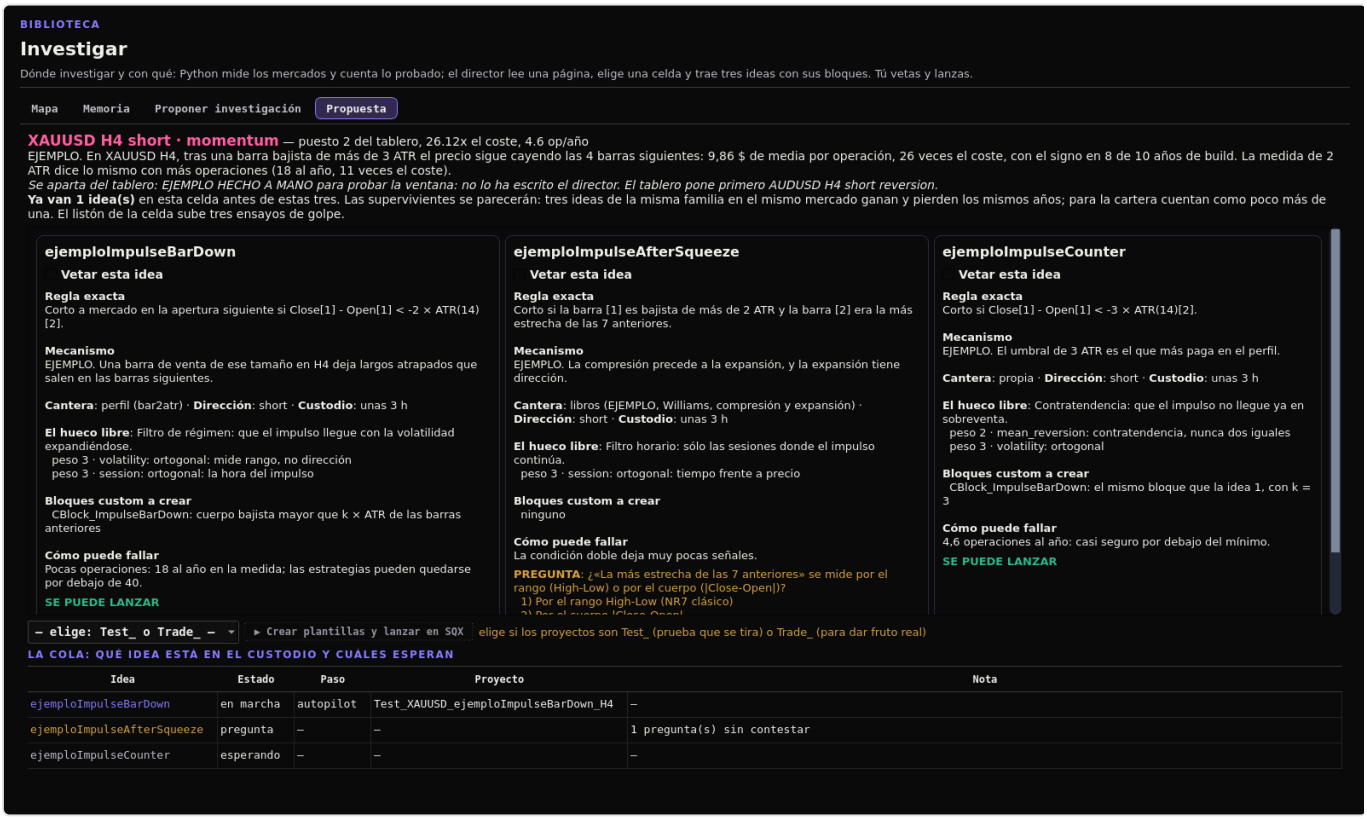
Mapa. Activos en filas, marcos en columnas. El color es la familia que mejor paga entre las que pasan los cuatro filtros; la intensidad tiene tres escalones (de 2 a 5 costes, de 5 a 10, 10 o más); ▲ largo, ▼ corto, y el número es el efecto en múltiplos del coste. Gris: ninguna pasa. Al pulsar una casilla, a la derecha salen sus familias y todas sus medidas, cada una con lo que mide; pasa el ratón por la cabecera de una columna para ver qué es cada cifra. El asterisco marca costes provisionales.



Memoria. Cuántas celdas tienen algún intento, el embudo de cada intento (construidas → tras cada paso → supervivientes) y lo que ha dado cada familia por clase de activo.



Proponer investigación. El tablero que va a leer el director (cap. 83) y el botón. Mientras trabaja se ven sus cinco pasos y en cuál está: leer el tablero, elegir la celda, pedir tres ideas, preparar las paletas, escribir la propuesta.



Propuesta. Arriba, el diagnóstico de la celda, cuántas ideas van ya ahí y los dos costes de concentrar (las supervivientes se parecerán; el listón sube tres ensayos). Debajo, las tres ideas lado a lado: regla exacta, mecanismo, de qué cantera viene (perfil, libros o propia), el hueco libre y sus familias de bloques, los bloques custom que hay que crear, las horas de custodio y cómo puede fallar. Cada una tiene su casilla **Vetar esta idea**. Una idea con una **PREGUNTA** en ámbar no se lanza hasta que escribas la respuesta y pulses Enter; las otras dos sí pueden ir. Abajo, la cola: qué idea está en el custodio, en qué paso (activos → plantilla → proyecto → paleta → autopilot) y cuáles esperan. La propuesta de la captura es **un ejemplo hecho a mano** para probar la ventana, no una propuesta del director.

Un ejemplo completo

1. Abre Investigar › Mapa y pulsa XAUUSD · H4: momentum en corto, 26 veces el coste, 4,6 op/año.
2. Proponer investigación › «▶ Proponer investigación» › Sí. Al terminar, la ventana salta a Propuesta.
3. Lee las tres ideas. Veta la tercera. Contesta la pregunta de la segunda.
4. Pulsa «▶ Crear plantillas y lanzar en SQX», lee la frase (qué ideas, qué proyectos, cuántas horas, cuántos dólares) y confirma.
5. La cola pasa la primera idea por sus cinco pasos; al acabar su autopiloto empieza la segunda.

Qué NO te dice

- Que una idea sea buena: la propuesta es una hipótesis medida sólo en build.
- Cuándo acabará la cola con precisión: las horas son una estimación del director.
- Si el templateArchitect vuelve con una pregunta, esa idea queda en «pregunta» y sigue la siguiente: la respuesta se le da en una sesión, no aquí.

Si algo falla

- El botón de lanzar está apagado: la frase ámbar a su derecha dice por qué (el custodio está ocupado, hay otro lanzamiento en marcha, el proyecto ya existe).

- Una idea en «falló»: la cola se para y las siguientes se quedan en «esperando». El motivo está en la columna Nota y el detalle en el log del trabajo, en «En marcha».
- «El demonio no respondió a /api/research/…»: el demonio es anterior a esta zona; cierra y abre la ventana.

38. Activos — costes, tramos, MC Retest y Cross Market de cada instrumento

La zona **Activos** de la ventana. Enseña **cada instrumento de `assets/` en una ficha propia**, dos por fila cuando la ventana es ancha: lo que cuesta operarlo, entre qué fechas se construye y se testea, qué sortea su MC Retest y contra qué otros mercados se comprueba su edge — y todo eso se cambia desde ahí.

Qué pregunta responde

- **¿Qué activos tengo y cuáles están bloqueados?** Un activo con un coste obligatorio sin pactar no deja escribir ni una plantilla ni un proyecto (regla dura 5). El índice de la izquierda lo pinta en rojo, y la ficha dice exactamente qué campo falta y qué pasa por eso.
- **¿Qué le estoy cobrando de verdad a este instrumento, y en qué unidad?** Spread, comisión, slippage y los dos swaps, cada uno con su unidad y al lado de lo que SQX lleva hoy, que casi nunca está en la misma unidad.
- **¿Qué datos hay y entre qué fechas va cada tramo?** Las fechas con datos (desde / hasta) y los tramos Build, OOS 1 y OOS 2, cada uno con su selector de fechas.
- **¿Contra qué mercados compruebo que el edge sobrevive?** El Check de Cross Market, fila a fila, agrupado en Family y Structural.

Cuándo lo usas, y cuándo no

Lo usas cuando el bróker te da una cifra de verdad y hay que sustituir una provisional; cuando das de alta un instrumento; cuando mueves la fecha de un tramo; cuando fijas los mercados del Cross Market **antes** de mirar ningún resultado; y antes de construir sobre un activo, para ver qué arrastra.

No lo usas para nada que hable con SQX. La ventana no abre el maestro, no arranca un worker y no escribe en ningún install: `SQX hoy` se lee del registro de instrumentos del **custodio** (`SQX_w2/user/data/data.db`) en disco, sin arrancarlo.

Tampoco para lo que decide para todos los activos a la vez — la doctrina de construcción, las dos clases de coste, qué es cada tramo, los múltiplos por defecto del MC Retest. Eso salió de aquí y vive en la zona **Configuración SQX** (su propio capítulo).

Y no sustituye a la preflight: antes de autorizar nada sigue siendo obligatorio `python3 -m core.assets <SÍMBOLO>`. La ventana enseña lo mismo, pero el código de salida — 2 bloqueado, 3 esquema roto — solo lo da el comando.

Antes de empezar

- `pip install -r requirements.txt` (la zona necesita `ruamel.yaml`, el YAML que conserva los comentarios).
- Nada más. No hace falta SQX, ni un worker, ni datos exportados.

Cómo se ejecuta

```
algoui
```

La ventana de siempre. En la barra de la izquierda, BIBLIOTECA → **Activos**. Para una captura sin pantalla: `python3 -m ui.desktop.launch --zone Activos --shot <carpeta>` con `QT_QPA_PLATFORM=offscreen`.

Qué produce

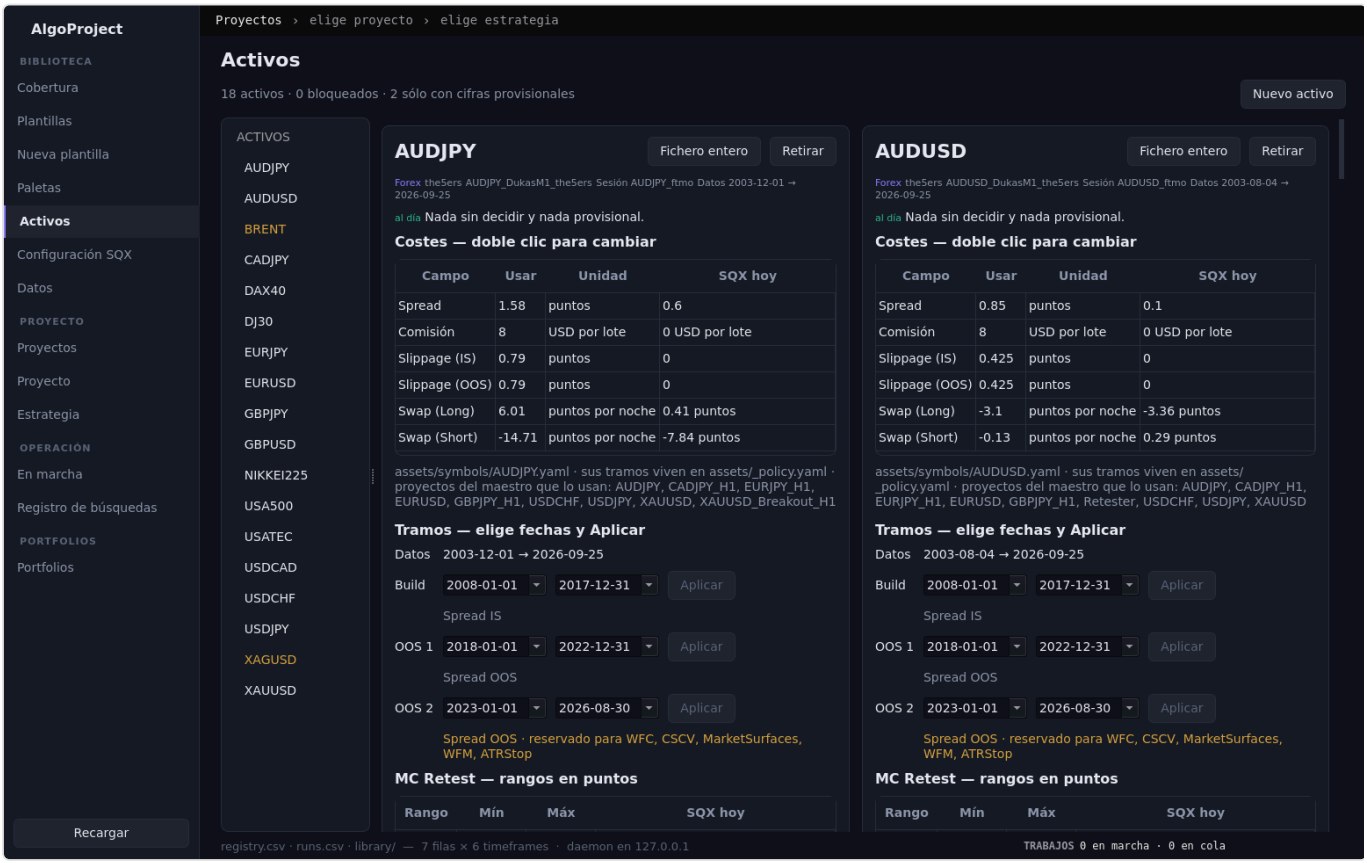
Escribe, y solo escribe, dentro de `assets/`, siempre a través de `core.assetwrite`:

lo que tocas	dónde acaba
un coste y su «por qué», un rango de MC Retest, cualquier valor de «Fichero entero»	`assets/symbols/<SÍMBOLO>.yaml`
la fecha de un tramo	`assets/_policy.yaml`, bajo `segments: <SÍMBOLO>:` — **una línea**
añadir o quitar un mercado del Check de Cross Market	`assets/_markets.yaml` — **una línea** (dos si la categoría estaba vacía)
dar de alta un activo	un `.yaml` nuevo en `symbols/` **y** su bloque en `_policy.yaml`
retirar un activo	se mueve a `assets/symbols/_retired/<SÍMBOLO>.yaml`

Los comentarios se conservan. Cambiar una fecha cambia esa línea y nada más; añadir un mercado añade su línea y quitarlo la quita, sin tocar los párrafos que explican por qué está cada uno. Después de cada escritura se regenera `assets/INDEX.md`.

Cómo se lee el resultado

La zona



A la izquierda, **el índice**: pulsar un activo lleva su ficha a la vista. El color es su estado y el tooltip lo deletrea:

color	qué significa
rojo	bloquea la autoría: falta un coste obligatorio, o el esquema está roto
ámbar	funciona, pero alguna cifra es provisional y todo resultado con coste la arrastra
normal	nada que decidir

Arriba, la cuenta: cuántos activos, cuántos bloqueados, cuántos solo con cifras provisionales. Los retirados ya no se enseñan.

Una ficha

XAUUSD

Fichero entero | Retirar

No forex Infinox XAUUSD_DukasM1_Infinox Sesión XAUUSD_ftmo Datos 2003-05-05 → 2026-09-25

RANGOS MC RETEST SIN DECIDIR Min Distance — No bloquea: sólo vuelve ininterpretable esa tarea del MC Retest.

Costes — doble clic para cambiar

Campo	Usar	Unidad	SQX hoy
Spread (IS)	16.37	puntos	5
Spread (OOS)	22.54	puntos	10
Spread (OOS 2)	37.49	puntos	—
Comisión	0.001778	% del nocional	8 USD por lote
Slippage (IS)	8.19	puntos	0
Slippage (OOS)	11.27	puntos	0
Slippage (OOS 2)	18.75	puntos	—
Swap (Long)	-7	% anual	-73.42 puntos
Swap (Short)	-7	% anual	38.76 puntos

assets/symbols/XAUUSD.yaml · sus tramos viven en assets/_policy.yaml · proyectos del maestro que lo usan: AUDJPY, Builder, EURUSD, Retester, USDJPY, XAUUSD, XAUUSD_Breakout_H1

Tramos — elige fechas y Aplicar

Datos 2003-05-05 → 2026-09-25

Build 2008-01-01 2017-12-31 Aplicar

Spread IS

OOS 1 2018-01-01 2022-12-31 Aplicar

Spread OOS

OOS 2 2023-01-01 2026-08-30 Aplicar

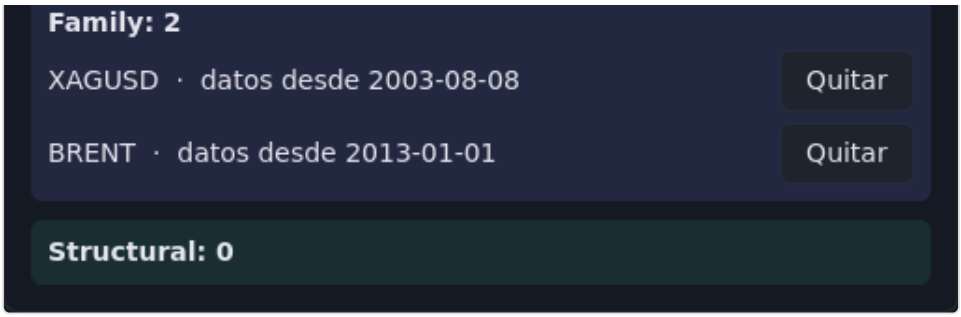
Spread OOS 2 · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Mín	Máx	SQX hoy
Spread	8.7	31.98	1 – 5 · 5 – 12
Slippage	5	20	0 – 5 · 0 – 10
Min Distance	0	0	0 – 10

Check de Cross Market

Añadir



De arriba abajo:

1. **El nombre**, con dos botones: **Fichero entero** despliega todos los valores del YAML del activo, cada uno con el comentario que el propio fichero lleva al lado — entre ellos `mt5`, el símbolo del activo en cada empresa de fondeo (FTMO, Hantec), que lee MT5 BRIDGE › Verificar y que se cambia ahí mismo con doble clic; **Retirar** lo saca de la librería.
2. **Las fichas**: clase, bróker, feed, sesión (en rojo si está sin decidir) y el rango de datos que SQX tiene.
3. **La preflight**, en el orden en el que se actúa: lo que rompe, lo que bloquea, lo que solo mancha el resultado. Solo la etiqueta y los campos; el porqué de cada aviso sale al pasar el ratón (dueño, 2026-09-30: nada de frases de relleno en la página).
4. **Costes**, con nombres legibles — Spread (IS), Spread (OOS), Spread (OOS 2), Slippage (IS)…, **uno por tramo en todos los activos, forex incluido** (dueño, 2026-09-30; antes un par tenía un solo spread medio) — su unidad y lo que el **custodio** lleva hoy en su registro de instrumentos, en todos los activos y nunca lo del maestro (dueño, 2026-09-30). Pasa el ratón por una fila para leer **por qué** vale lo que vale; doble clic para cambiarla.
5. **Tramos**: primero **Datos** (lo que hay, no se edita), luego Build, OOS 1 y OOS 2 con su selector de fechas. Debajo de cada tramo, qué spread usa; para qué pruebas está reservado va en la ayuda al pasar el ratón.
6. **MC Retest**: el rango (mínimo y máximo, en puntos) que sortea cada método — Spread, Slippage y Min Distance — al lado de lo que lleva el **custodio** (`SQX_w2`), que es quien de verdad corre el MC Retest, leído de sus propios proyectos en disco sin arrancarlo; si el custodio no tiene todavía ningún proyecto de ese activo, la columna dice «—» (nunca lo del maestro). **Min Distance sin decidir se lee 0**: sin perturbación. El fichero sigue diciendo `null`, y por eso la tarea `MCR 4 MinDist` no se escribe y la preflight lo sigue avisando.
7. **Check de Cross Market** (antes «universo de retest»): los mercados agrupados por categoría, cada grupo con su color y su cuenta («Family: 2», «Structural: 0»).

Una ficha con trabajo pendiente se ve así — BRENT tiene cifras, pero provisionales, y ningún tramo con fechas:

BRENT

Fichero entero | Retirar

No forex ftmo BRENTCMDUSD_ftmo Sesión SIN DECIDIR Datos 2013-01-01 → 2026-09-24

COSTES PROVISIONALES Spread (IS), Spread (OOS), Comisión, Slippage (IS), Slippage (OOS), Swap (Long), Swap (Short) — Hay cifra y no bloquea, pero no está pactada con el bróker: TODO resultado con coste producido con ella arrastra esa advertencia.
TRAMOS SIN FECHAS Build, OOS 1, OOS 2 — No bloquea, pero `window()` se niega a inventar una ventana, así que una tarea sobre ese tramo falla al configurarse.

Costes — doble clic para cambiar

Campo	Usar	Unidad	SQX hoy
Spread (IS)	16	puntos	32
Spread (OOS)	32	puntos	32
Comisión	0	% del nocional	0 USD por lote
Slippage (IS)	8	puntos	0
Slippage (OOS)	16	puntos	0
Swap (Long)	0.0111	% anual	0.03 puntos
Swap (Short)	-9.537	% anual	-25.76 puntos

assets/symbols/BRENT.yaml · sus tramos viven en assets/_policy.yaml · proyectos del maestro que lo usan: XAUUSD

Tramos — elige fechas y Aplicar

Datos 2013-01-01 → 2026-09-24

Build sin decidir sin decidir Aplicar

Spread IS

OOS 1 sin decidir sin decidir Aplicar

Spread OOS

OOS 2 sin decidir sin decidir Aplicar

Spread OOS · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Mín	Máx	SQX hoy
Spread	sin decidir	sin decidir	1 – 5
Slippage	sin decidir	sin decidir	0 – 5

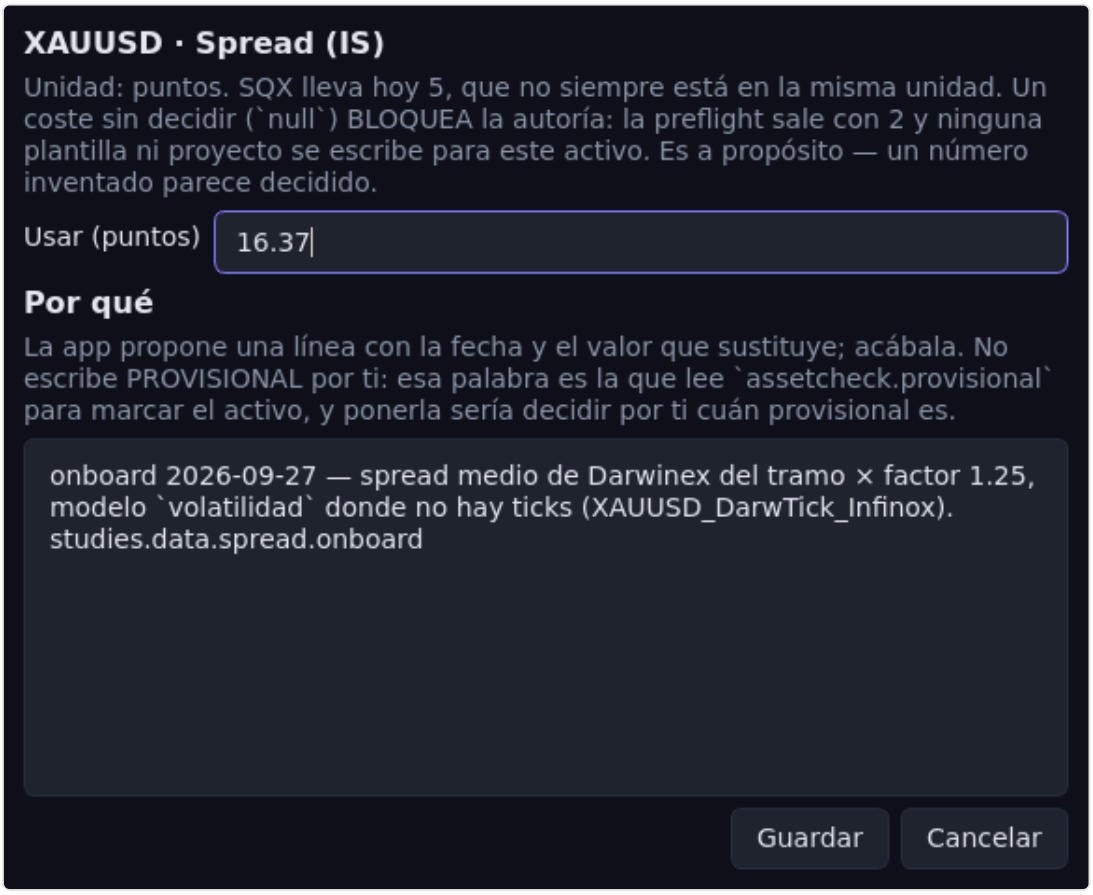
Check de Cross Market — este activo no es un main declarado en _markets.yaml

Cambiar la fecha de un tramo



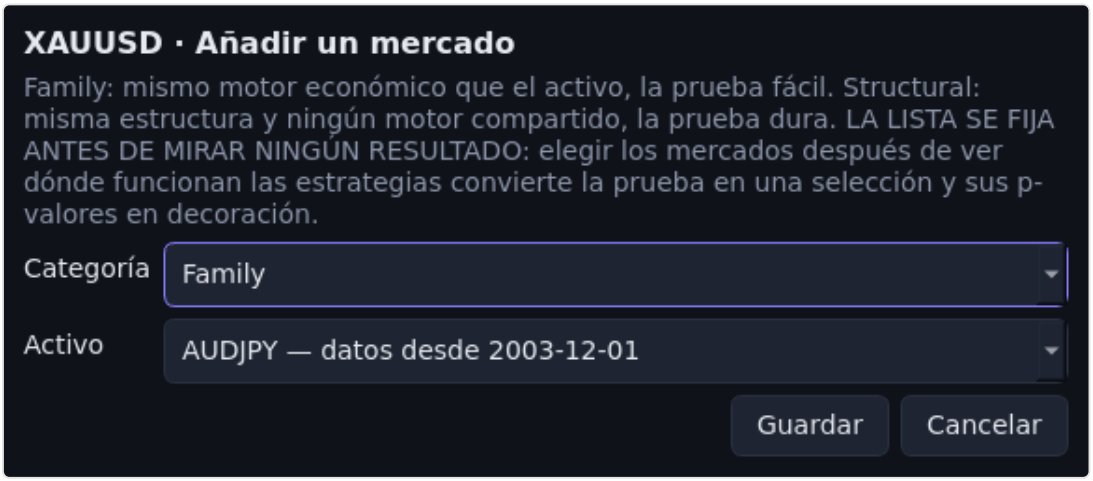
Elige el día en el selector (tiene calendario, o se teclea). **Aplicar** solo se enciende cuando algo cambió y el «desde» no queda detrás del «hasta»; al pulsarlo escribe **solo el extremo que se movió**. Un 1 de enero como inicio o un 31 de diciembre como final se guardan como el año entero (`2018`), que es la convención del fichero; cualquier otro día, como fecha (`2022-06-30`). Los dos extremos están siempre INCLUIDOS. Bajar el selector hasta el fondo lo deja en «sin decidir» (`null`).

Cambiar un coste



Doble clic en la fila. El valor y su «por qué» se escriben juntos o no se escribe ninguno. Vacío es `null`: el campo vuelve a «sin decidir» y, si es obligatorio, bloquea la autoría a propósito. Al salir del valor la app propone un «por qué» con la fecha y la cifra que sustituye; no escribe PROVISIONAL por ti.

Añadir y quitar un mercado del Cross Market

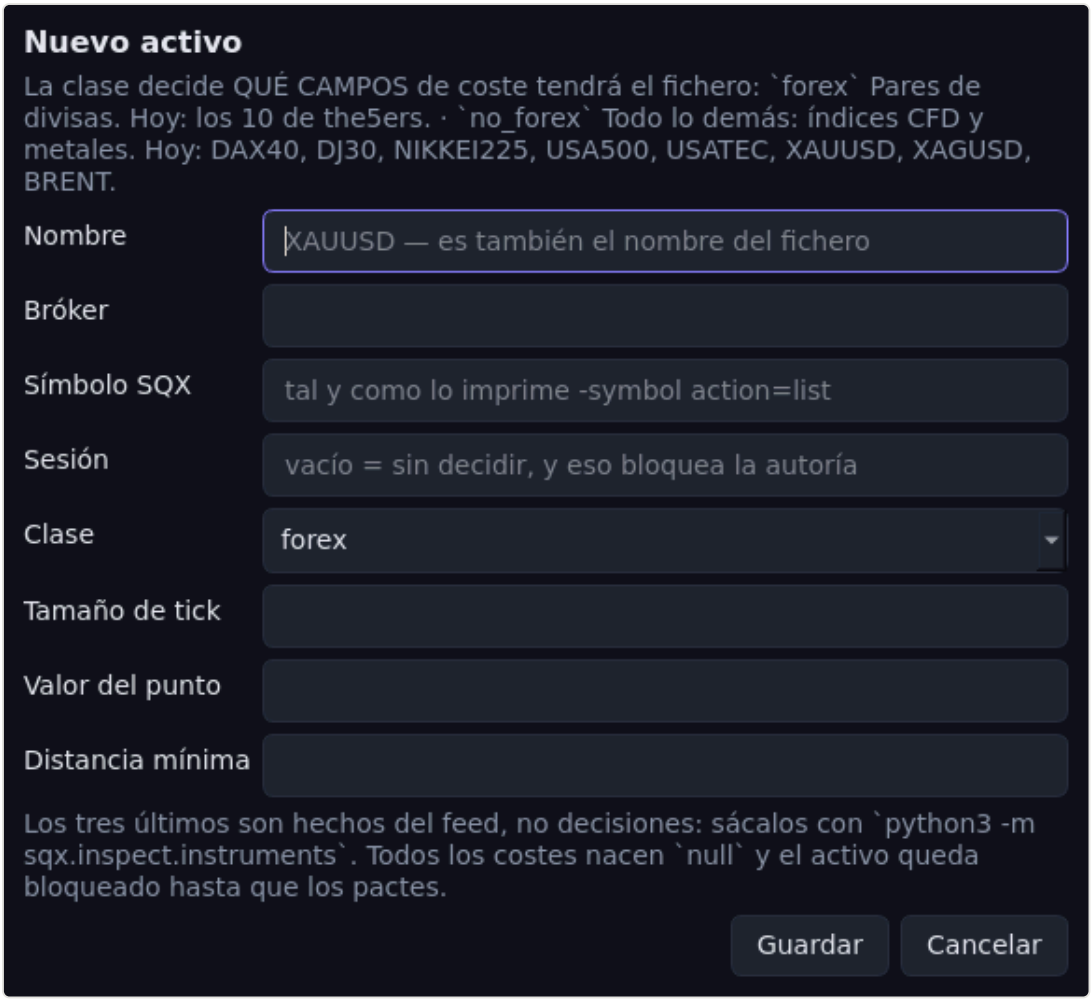


Añadir abre la caja: primero la categoría (Family / Structural), después el activo, por su nombre y con la primera fecha con datos. Solo ofrece activos que aún no están en ninguna de las dos. **Quitar**, en cada fila, pregunta antes. Las dos cosas recuerdan lo mismo: **la lista se fija antes de mirar ningún resultado**; elegirla después convierte la prueba en una selección.

Añadir el primer mercado a una categoría vacía son **dos líneas** en el diff: `structural: []` pasa a `structural:` y debajo la fila. Quitarlo después la devuelve a `[]`, sin rastro.

Solo un activo declarado como *main* en `_markets.yaml` tiene Check de Cross Market; en los demás la ficha lo dice y ofrece **Declarar como main** en vez de Añadir. Desde el 2026-09-29 (dueño: todo activo con ficha es un main, no solo el que alguien ya curó) los 18 activos lo son: XAUUSD y USDJPY llevan su lista propia de siempre, y los otros 16 se declararon con todos los demás activos en Family de una vez — una clasificación de partida, no mercado a mercado, con `structural` vacío.

Dar de alta y retirar



Nuevo activo, arriba a la derecha. La clase decide qué campos de coste tendrá el fichero. Los tres hechos del instrumento se teclean (sácalos de `python3 -m sqx.inspect.instruments`) y **todos los costes nacen `null`**: el activo entra bloqueado, que es lo que se quiere. **Retirar**, en la ficha, mueve el fichero a `symbols/_retired/`; su bloque de tramos se queda en `_policy.yaml`.

Un ejemplo completo

Quieres acortar el OOS 1 de AUDJPY para que acabe a mitad de 2022.

1. `algoui` → **Activos** → `AUDJPY` en el índice.
2. En Tramos, fila **OOS 1**, el segundo selector: `2022-06-30`. Se enciende **Aplicar**.
3. Aplicar. La ficha se redibuja con la fecha nueva.
4. En la terminal, lo que cambió:

```
$ git diff assets/_policy.yaml
   AUDJPY:
     data: {from: 2003-12-01, to: 2026-09-25}   # AUDJPY_DukasM1_the5ers, 8.533.415 barras M1
     build: {from: 2008, to: 2017}
-    oos1: {from: 2018, to: 2022}
+    oos1: {from: 2018, to: 2022-06-30}
     oos2: {from: 2023, to: 2026-08-30}
```

Una línea, y el comentario de al lado intacto. Antes de construir nada sobre AUDJPY: `python3 -m core.assets AUDJPY`, que ya imprime el tramo nuevo en milisegundos.

Lo mismo con el Cross Market: **Añadir** → Family → USA500 en la ficha de XAUUSD deja en `_markets.yaml` exactamente una línea nueva, `- {feed: USA500_DukasM1_Infinox, data_from: 2012-01-19}`, y **Quitar** la borra sin dejar rastro.

Qué NO te dice

- **No dice si una cifra es correcta**, solo si está decidida. Eso lo dice su «por qué», que es texto y no lo valida nadie.
- **No sabe qué lleva un proyecto concreto.** `SQX hoy` es el registro de instrumentos del custodio tal como está guardado en disco; cada tarea lleva sus propios costes.
- **No comprueba que los tramos no se solapen** ni que caigan dentro de los datos; la preflight (`python3 -m core.assets`) sí avisa cuando un tramo pide datos que no existen.
- **No propaga nada.** Cambiar un coste o una fecha no reconfigura ningún proyecto de SQX ya escrito: los costes y las ventanas viven dentro de cada tarea.
- **«Declarar como main» no le pone timeframe.** Le abre un bloque vacío (`family` / `structural` a `[]`) para que **Añadir** funcione; el `timeframe` se queda `null` hasta que ese activo corra de verdad un Cross Market con una plantilla concreta — no se inventa aquí.

Si algo falla

- **`ModuleNotFoundError: ruamel`** — `python3 -m pip install --user ruamel.yaml==0.18.16`.
- **La zona se queda con datos viejos después de editar un YAML a mano** — el botón **Recargar** de abajo a la izquierda relee todo.
- **`KeyError` al abrir un activo copiado a mano** en `symbols/` — le falta su bloque en `_policy.yaml`. Los que se dan de alta desde la ventana lo llevan.
- **Una sola ficha por fila** — la ventana es estrecha: por debajo de unos 880 píxeles de ancho útil las fichas se apilan, porque dos no caben legibles.
- **El calendario de un tramo «sin decidir» abre en 1970** — es el suelo del selector; teclea el año directamente.

66. Configuración SQX — los ajustes con que nacen los proyectos nuevos

La zona **Configuración SQX** de la ventana, en BIBLIOTECA. Reúne en un sitio todos los ajustes con los que se construye y se prueba un proyecto nuevo de SQX —precisión, motor, tipos de orden, salidas, money management, crosschecks, cross-market, CrossTF, MC Retest, WFC, CSCV, SPP y WFM—, cada test en su sección, y deja cambiarlos con un desplegable allí donde los valores posibles son fijos. WFC y CSCV enseñan además **todo lo que los alimenta**, aunque no viva en `assets/`: cuántas variantes se fabrican por madre, la semilla, los suelos de operaciones, las reglas y los bloques del CSCV.

Qué pregunta responde

- **¿Con qué ajustes se va a construir y a probar el próximo proyecto?** Cada valor que `sqx/projects/` escribe en las tareas, leído de `assets/_build.yaml`, con la frase que el propio fichero pone encima para explicarlo.
- **¿Qué timeframes lee el CrossTF, con qué precisión corre la WFM, cuántas simulaciones lleva cada MC Retest?** Sin abrir el YAML: un desplegable por timeframe, uno por precisión, una caja por número.
- **¿Cuántas variantes lleva cada madre, y cuántas combinaciones prueba el CSCV?** En las secciones **WFC** y **CSCV**: el máximo y el mínimo de variantes por madre, el reparto por estratos, los bloques del CSCV con las particiones que dan, y el resto de sus entradas.
- **¿Qué es común a todos los activos?** Los dos esquemas de coste (`_classes.yaml`), lo global de `_policy.yaml` —qué es cada tramo, el swap, los rangos del MC Retest— y el universo de retest de `_markets.yaml`, que aquí sólo se consulta. Antes vivían en Activos.

Cuándo lo usas, y cuándo no

Lo usas antes de crear un proyecto, para mirar o cambiar cómo va a salir: por ejemplo, para añadir un timeframe al CrossTF, bajar las simulaciones de una tarea del MC Retest cara o cambiar la precisión de un test.

No lo usas para cambiar un proyecto que ya existe. **Lo que se cambia aquí se aplica a los proyectos que se creen a partir de ahora; los existentes no cambian** — la zona lo dice arriba, en amarillo. Un `project.cfx` ya creado lleva sus ajustes dentro y esta zona no lo toca. Tampoco es el sitio de los costes, las fechas de los tramos o los mercados de un activo concreto: eso es **Activos** (capítulo 38). Y la zona **no habla con SQX**: sólo lee y escribe ficheros de `assets/` y, en WFC y CSCV, los `config.yaml` de sus estudios y una fila de `ledger/thresholds.yaml`.

Antes de empezar

- La ventana y su demonio (capítulo 35). No hace falta SQX ni ningún worker.
- Para verla sin la ventana entera (o sacar sus capturas sin pantalla), la zona también se abre sola, con el demonio ya levantado (`python3 -m ui.daemon.serve`, o la ventana abierta con `bin/algoui`):

```
python3 -m ui.desktop.sqxconfig.preview
```

Cómo se ejecuta

En la ventana: **BIBLIOTECA** → **Configuración SQX** (o `bin/algoui --zone "Configuración SQX"`). O suelta, con el comando de arriba.

flag de `preview`	obligatorio	qué hace
`--shot CARPETA`	no	guarda seis capturas (todo plegado, CrossTF, MC Retest, WFM, WFC, CSCV) en esa carpeta y sale
`--port P`	no	habla con el demonio de ese puerto; por defecto el de la ventana (8765). Para capturas, un demonio de pruebas: `python3 -m ui.daemon.serve --port 8751`

Abrir la zona tarda menos de un segundo: lee diez ficheros YAML. No toca SQX, así que se puede usar con la GUI del maestro abierta y con el custodio en mitad de una corrida.

Para cambiar un valor:

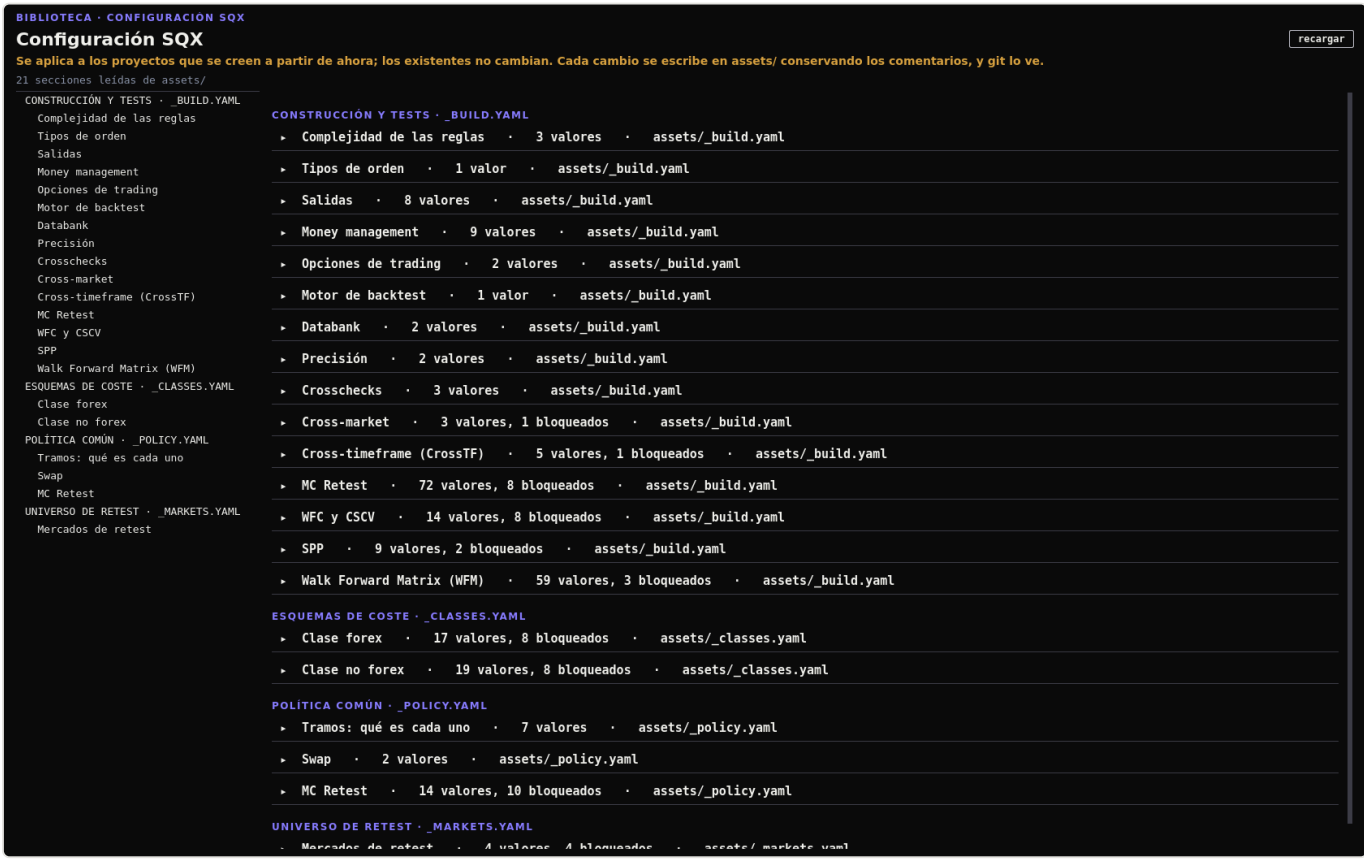
1. Pulsa en el índice de la izquierda (o en la cabecera ▸) la sección que quieres: se despliega.
2. Donde hay un desplegable, elige; se escribe al instante. Donde hay una caja, escribe el número y sal de la caja (tabulador o clic fuera); se escribe al salir.
3. En las listas ordenadas —los tipos de orden, los timeframes del CrossTF, los crosschecks— hay un desplegable por elemento, **+** añade al final la primera opción que la lista aún no lleva y **−** quita el último. Una lista no puede repetir un elemento.
4. La línea bajo el título dice, en verde, qué se ha escrito; o, en rojo, por qué no.

Qué produce

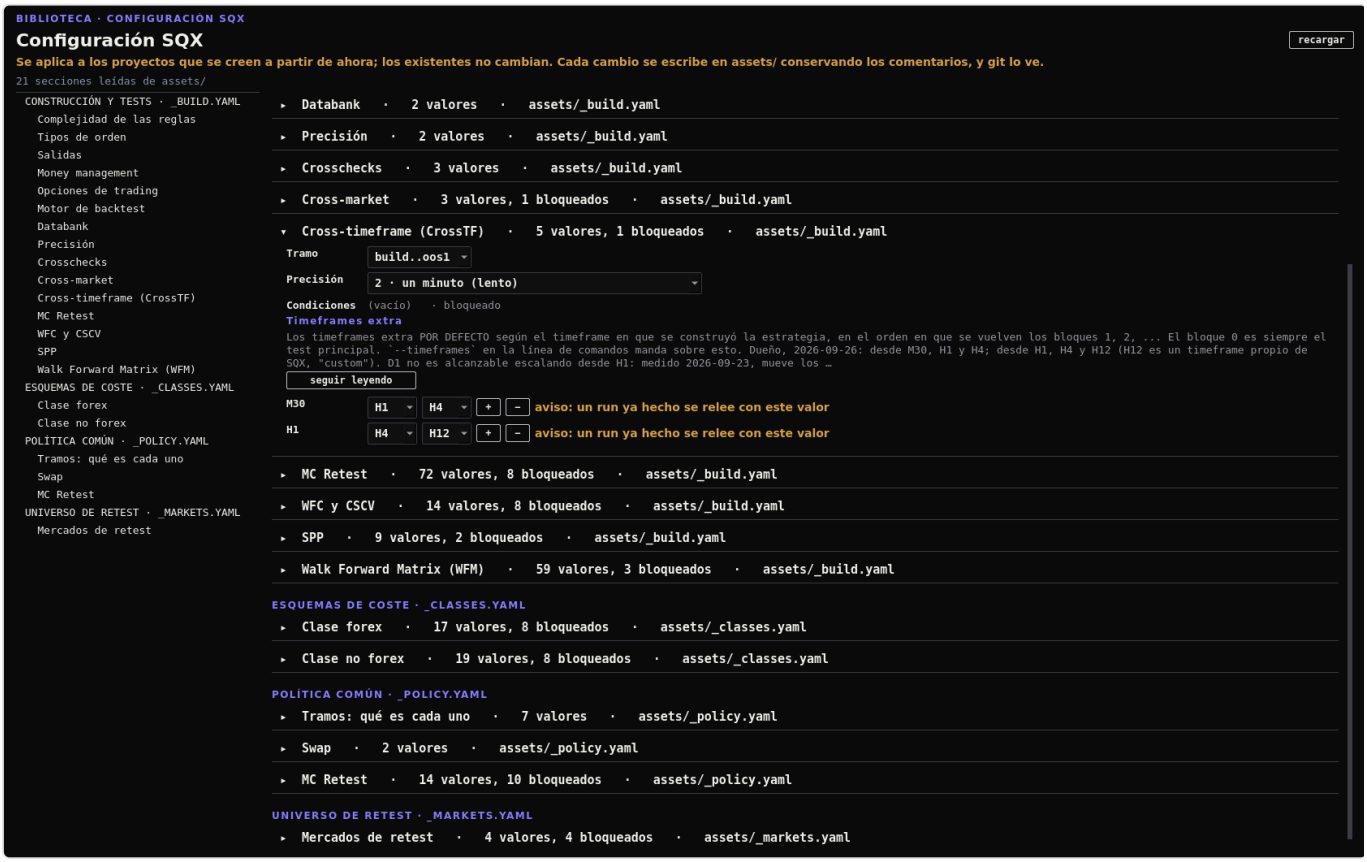
Cada cambio escribe **una línea** en el fichero que lo guarda — `assets/_build.yaml`, `assets/_classes.yaml` o `assets/_policy.yaml` — a través de `core.assetwrite`, el único que escribe en `assets/`. Los comentarios, las comillas y las listas en una línea (`[H4, H12]`) se quedan como estaban. Como `assets/` está en git, el cambio aparece en `git diff`; se deshace desde la propia ventana, volviendo a elegir el valor de antes.

WFC y CSCV escriben también fuera de `assets/`, igual, una línea y los comentarios intactos: en `studies/breakage/spp/config.yaml` (el diseño del lote), `sqx/variants/config.yaml` (la fábrica), `engines/variants/config.yaml` (lo que comparten los dos estudios), `studies/optimisation/wfc/config.yaml`, `studies/optimisation/cscv/config.yaml` y, los bloques del CSCV, en `ledger/thresholds.yaml`.

Cómo se lee el resultado

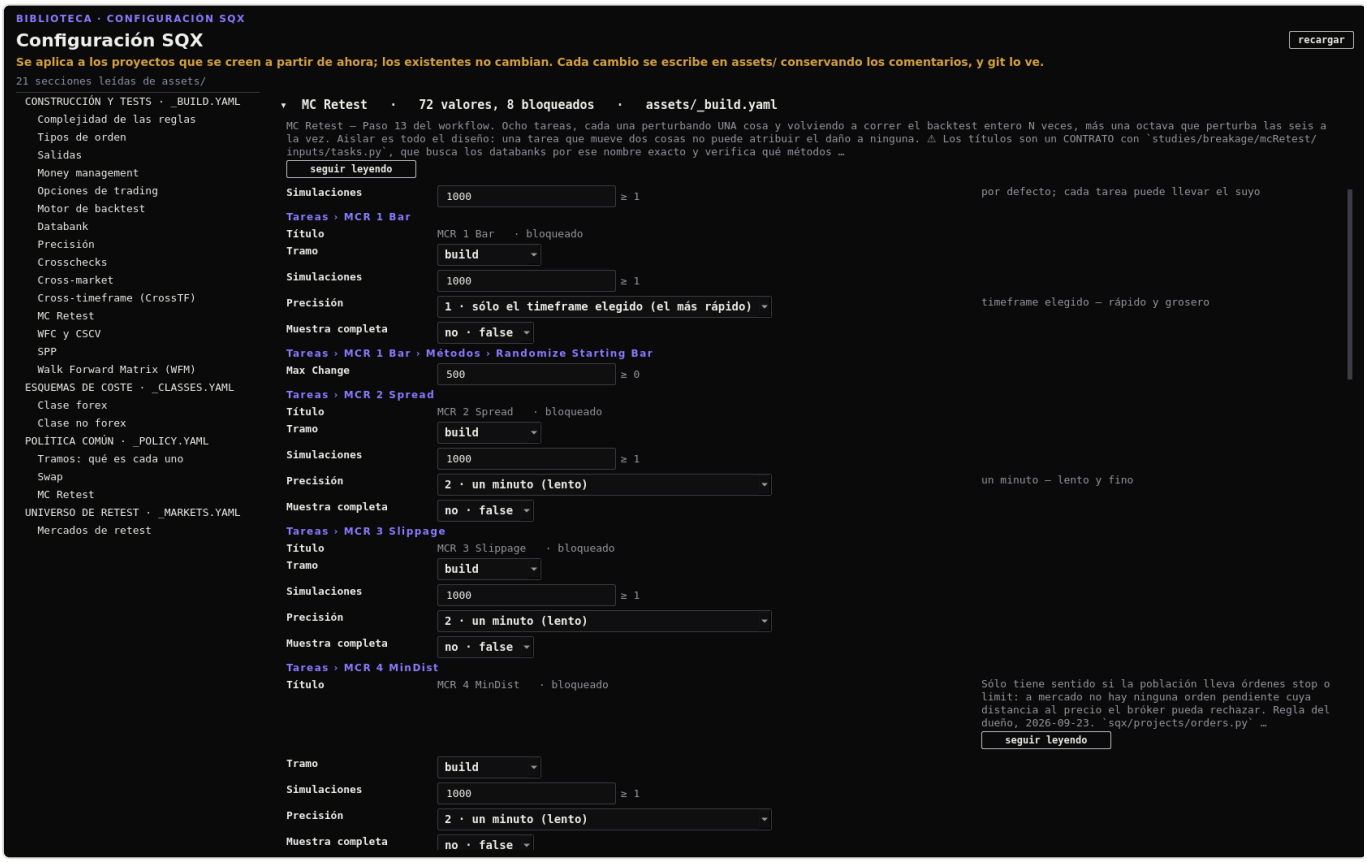


A la izquierda, el índice: primero **Construcción y tests** (`_build.yaml`, una sección por test), luego **Esquemas de coste** (`_classes.yaml`), **Política común** (lo global de `_policy.yaml`) y **Universo de retest** (`_markets.yaml`). Cada cabecera dice cuántos valores tiene la sección y cuántos están **bloqueados**.

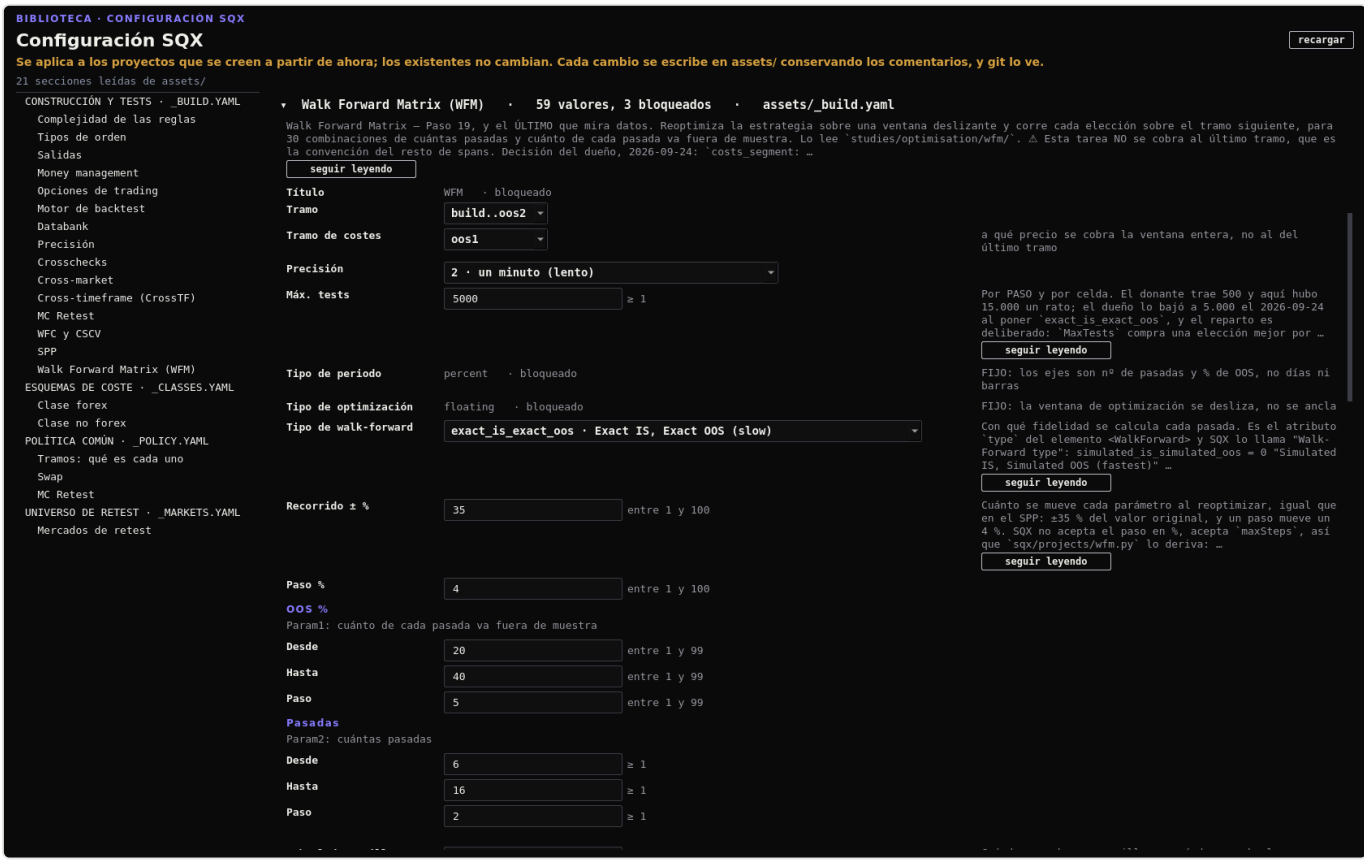


Cada fila es un valor: su nombre, su editor y, si el fichero le pone un comentario, ese comentario a la derecha. Los comentarios largos se cortan y **seguir leyendo** los abre enteros. Un valor **bloqueado** se ve pero no se cambia desde aquí, y al pasar el ratón dice por qué:

- **Títulos y nombres de databank** (MCR 1 Bar, WFC_Build…): los busca por ese nombre exacto el análisis de Python; cambiarlos aquí lo rompería en silencio.
- **Tipo de periodo y tipo de optimización de la WFM**: fijos por decisión tuya del 24-09.
- **Condiciones de aceptación** del cross-market, el CrossTF y el WFC: son estructuras, no valores, y vacías significan «ninguna», por decisión tuya.
- **Los campos `fields`, `field` y `unit` de los esquemas de coste**: `core/assetdata.py`, `core/assets.py` y `core/assetwrite.py` los leen por su nombre y cada fichero de activo lleva esos campos; cambiarlos aquí rompería todos los activos.
- **Nombres y valores de fábrica de SQX** en los rangos del MC Retest, y **todo el universo de retest**, que se decide en Activos.

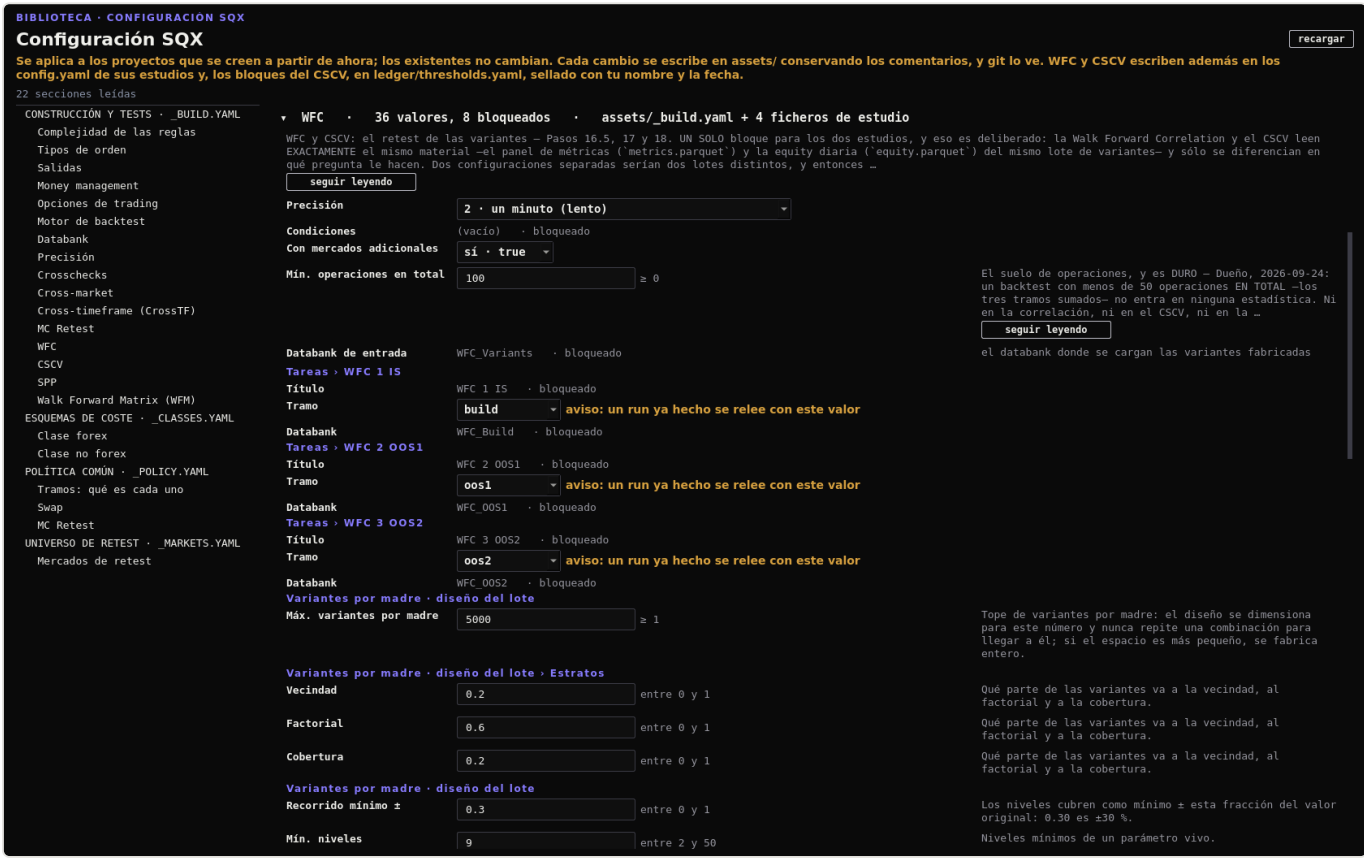


Dentro de una sección, los subtítulos violetas siguen la forma del fichero: **Tareas › MCR 1 Bar › Métodos › Randomize Starting Bar**. Cada tarea se nombra por su título.

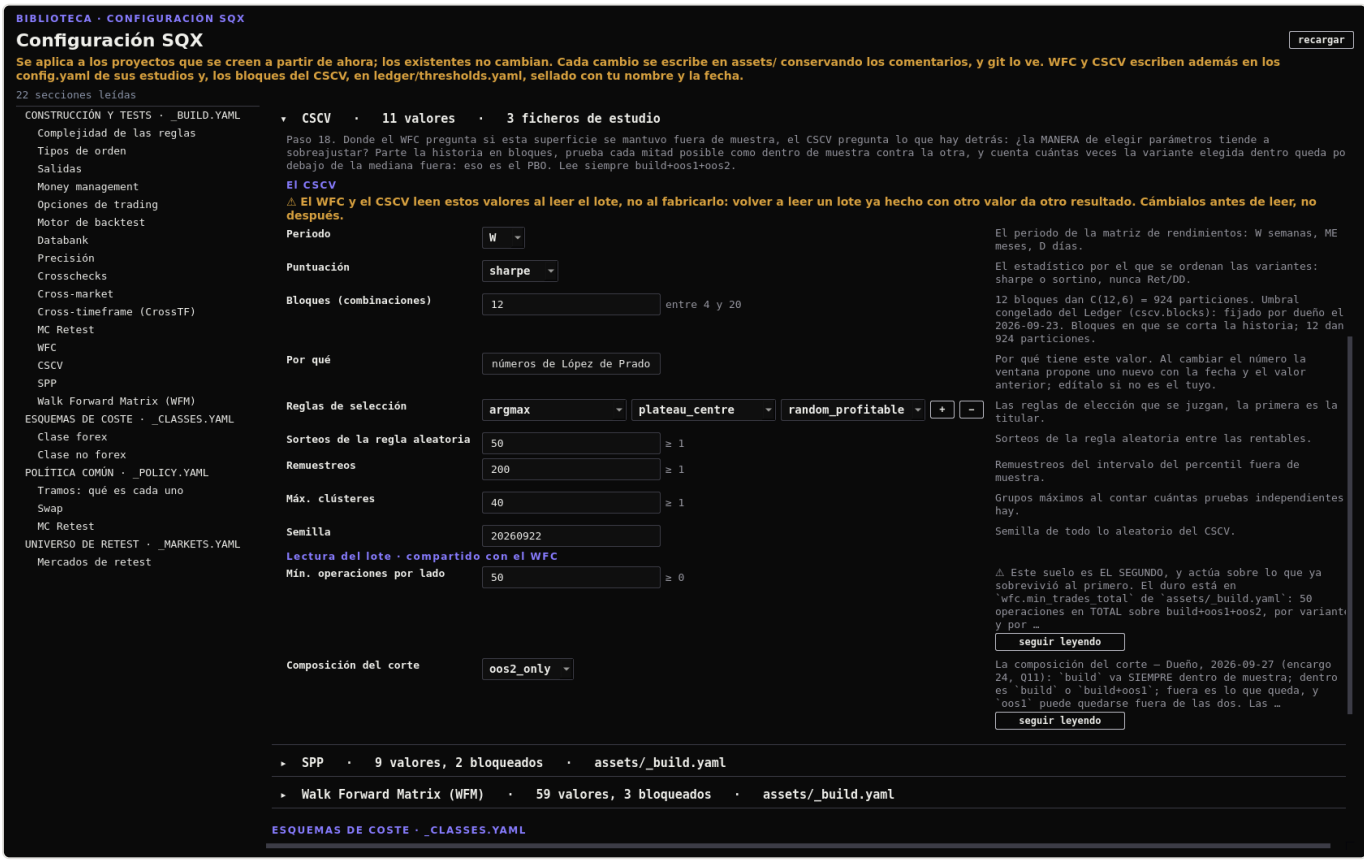


WFC y CSCV: todo lo que los alimenta

Añadido el 28-09-2026, porque faltaban el número máximo de variantes por madre y las combinaciones del CSCV. Bajo las tareas de SQX del WFC vienen, cada grupo con su subtítulo violeta:



grupo	valores	fichero
Variantes por madre · diseño del lote	**Máx. variantes por madre** (5000, un tope, nunca una cuota: no repite una combinación para llegar), los tres **estratos** (vecindad 0,2, factorial 0,6, cobertura 0,2), recorrido mínimo ±, mín./máx. niveles, meseta	`studies/breakage/spp/config.yaml`
Mínimo de variantes por madre	**Mín. variantes por madre** (1000), cuánto se ensancha el recorrido por vuelta y hasta dónde	`sqx/variants/config.yaml`
Fábrica de variantes · semilla y rejillas	semilla, radio de la vecindad, niveles mínimos del factorial, recorrido y pasos de los congelados	`sqx/variants/config.yaml`
Canarios de control	cuántos, y si lleva pares inertes	`sqx/variants/config.yaml`
Lectura del lote · compartido con el CSCV	mín. operaciones por lado, composición del corte (`oos2_only` u `oos1_oos2`)	`engines/variants/config.yaml`
Lectura del WFC	suelo de ρ, filas por extremo de la tabla	`studies/optimisation/wfc/config.yaml`



La sección **CSCV** trae periodo (D, W, ME), puntuación (sharpe o sortino), **bloques** —al lado dice las particiones que dan: 12 bloques, C(12,6) = 924—, reglas de selección, sorteos de la regla aleatoria, remuestreos, máx. clústeres, semilla, y repite la lectura del lote que comparte con el WFC (cambiarla en una sección la cambia en las dos: es el mismo valor).

Los bloques del CSCV son un umbral congelado del Ledger. Al cambiarlos, la ventana escribe en su fila de `ledger/thresholds.yaml` el valor nuevo, `set_by: dueño`, la fecha de hoy y un «por qué» propuesto que dice el valor anterior y sus particiones; la fila **Por qué** de debajo te deja reescribirlo. Así el Ledger sigue distinguiendo un número fijado antes de ver resultados de uno movido después. Un número impar de bloques se niega: el CSCV parte la historia en dos mitades iguales.

⚠ **Aviso amarillo bajo el subtítulo.** Todo lo de *Lectura del lote*, *Lectura del WFC* y el CSCV lo leen los estudios **al leer el lote**, no al fabricarlo: volver a leer un lote ya hecho con otro valor da otro resultado. Cámbialos antes de leer, no después. Lo de la fábrica y el diseño sólo cambia los lotes que se fabriquen a partir de ahora.

Los desplegables sólo ofrecen lo que se puede elegir:

valor	opciones
precisión de un test	las cuatro de SQX: 1 timeframe elegido, 2 un minuto, 3 y 4 tick real
precisión de una tarea del MC Retest	1 y 2, las dos que documenta `_build.yaml`
motor	los ocho motores de SQX (hoy se usa `MetaTrader5 (hedged)`)
tipos de orden	`EnterAtMarket`, `EnterAtStop`, `EnterAtLimit`, `EnterReverseAtMarket`
timeframes del CrossTF	M1, M5, M15, M30, H1, H4, H12, D1 — los que sabe reescalar la fábrica de hermanas —, **menos el de origen** (desde H1 no se ofrece H1)
método de money management	los seis que llevan las tareas del donante
condición de parada del databank	`databank-full`, `passed-count`, `time-limit`, `never`
métrica de una condición de la WFM	las cuatro columnas `special` si la condición lee `special`; si no, las que ya usan las demás (calibradas sobre las 150 casillas del maestro)
tramo de una tarea	`build`, `oos1` y `build..oos1`; `oos2` y sus rangos **sólo** en el WFC y la WFM, que son a los que `_policy.yaml` se lo reserva
tipo de walk-forward	los tres de SQX, con su nombre en SQX
composición del corte	`oos2_only` y `oos1_oos2`, las dos con nombre
periodo del CSCV	D días, W semanas, ME meses
puntuación del CSCV	`sharpe`, `sortino`
reglas de selección del CSCV	`argmax`, `plateau_centre`, `random_profitable`, sin repetir
sí / no	«sí · true» y «no · false»

⚠ Las listas de SQX —las cuatro precisiones, los ocho motores, las condiciones de parada— **se copiaron a mano el 27-09-2026** de la página de ajustes del propio SQX (`SQX/internal/web/RESULTS2/result2.js`). La ventana no lee ese fichero: si una versión nueva de SQX añade un motor o una precisión, no aparecerá aquí hasta que alguien edite `ui/daemon/sqxconfig/lists.py`.

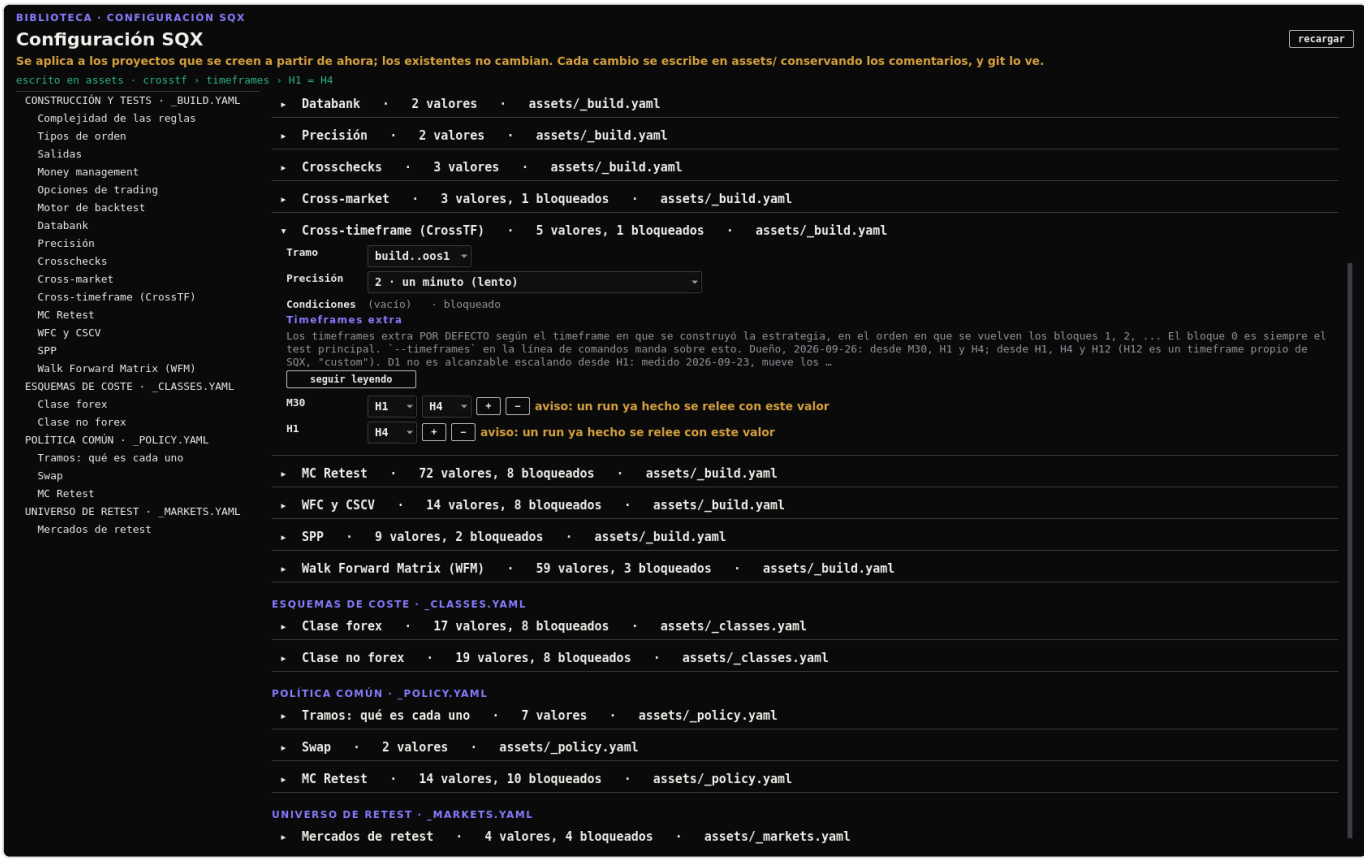
Los números también tienen su rango, cuando el fichero o la cantidad lo fijan: al lado de la caja se lee «≥ 1» o «entre 1 y 100». Un porcentaje no pasa de 100, el rectángulo de la WFM no pasa de 5 casillas (la matriz sólo tiene 5 columnas), y las horas mínimas de la salida por barras no pueden quedar por encima de las máximas. En WFC y CSCV: una cuenta (variantes, niveles, sorteos) no admite decimales, los mín. niveles no pueden pasar de los máx., los bloques van de 4 a 20 y pares, y si los tres estratos dejan de sumar 1 se escribe igual pero la línea verde avisa — cambiar uno descuadra la suma hasta que cambias el siguiente.

El aviso amarillo de fila. Dos valores llevan al lado *«aviso: un run ya hecho se relee con este valor»*: los **timeframes extra del CrossTF** y el **tramo de cada tarea del WFC**. Los dos estudios que los usan (`studies/transfer/crossTF` y `studies/readings/structure`) vuelven a leer `_build.yaml` cuando analizan un run, y no guardan el valor con el que corrió. Si cambias el orden de los timeframes después de un CrossTF, al analizarlo cada celda se leerá sobre las barras de otro timeframe, sin que nada avise. Cámbialos **entre runs, no después de uno**; la propuesta de que cada run guarde los suyos está en `OPEN.md` §80.

Un ejemplo completo

Quitar el H12 de los timeframes extra de las estrategias construidas en H1, y volver a ponerlo (prueba del 28-09-2026):

1. Índice → **Cross-timeframe (CrossTF)**. Se despliega; en **Timeframes extra › H1** hay dos desplegables, `H4` y `H12`, y al lado el aviso amarillo.
2. Pulsa **−** en la fila de H1. El segundo desplegable desaparece y la línea de estado se pone verde:




```
escrito en assets · crosstf › timeframes › H1 = H4
```

3. En la terminal, `git diff assets/_build.yaml` enseña una sola línea cambiada y los comentarios intactos:

```
@@ -143,7 +143,7 @@ crosstf:
   # equivocarlo lee cada celda sobre las barras de otro timeframe, en silencio.
   timeframes:
     M30: [H1, H4]
-    H1: [H4, H12]
+    H1: [H4]
```

4. Para volver a ponerlo: **+** en la fila de H1 añade un desplegable al final (con la primera opción que la lista no lleva, `M1`); elige en él **H12**. El fichero vuelve a decir `H1: [H4, H12]` y `git diff` ya no enseña nada.

La ventana también dice que no. Tres intentos de la prueba, y la respuesta de cada uno — en los tres el fichero quedó intacto:

intento	respuesta en rojo
escribir sobre `exits`, una sección entera	«no es un valor de este fichero: o no existe, o es una sección entera o un elemento suelto de una lista»
escribir en `crosstf › nope`, que no existe	la misma frase
motor `MetaTrader9`	««MetaTrader9» no es una de las opciones: [MetaTrader4, …]»

Qué NO te dice

- **No cambia ningún proyecto existente.** Si quieres que un proyecto ya creado corra con otro ajuste, hay que crear uno nuevo (capítulo 28) o reconfigurar esa tarea con su comando.
- **No valida que la combinación tenga sentido.** El desplegable garantiza que el valor existe en SQX, no que sea buena idea: D1 desde H1, por ejemplo, está en la lista y el fichero explica por qué no conviene. Lee el comentario de la fila antes de cambiar nada.
- **No sabe qué se ha construido con el valor anterior.** Git sí: `git log -p assets/_build.yaml`.
- **No enseña los ajustes por activo** (costes, tramos, mercados): eso es Activos.

Si algo falla

- **«El demonio no contesta»** en rojo bajo el título: el demonio no está levantado. Abre la ventana con `bin/algoui` o `python3 -m ui.daemon.serve`, y pulsa **recargar**.
- **«no escrito — … no es un número»**: la caja de un número recibió texto. El valor del fichero no se ha tocado y la caja vuelve a enseñarlo.
- **«no escrito — … no es una de las opciones»**: alguien cambió el fichero a mano con un valor que la zona no ofrece; el desplegable lo enseña como «(el del fichero, fuera de la lista)».
- **«no escrito — … repite un elemento»** o **«… está por encima del máximo»**: la lista llevaría dos veces el mismo timeframe, o el número se sale del rango que la caja dice al lado.
- **«no escrito — Contrato con …»** al tocar un valor bloqueado por otra vía: los títulos, los databanks y los campos `fields`, `field` y `unit` de los esquemas de coste los lee el código por su nombre, y sólo se cambian en el YAML y en el código a la vez.
- **Otro cambió el fichero mientras tenías la zona abierta**: pulsa **recargar** para volver a leer `assets/`.

67. Datos — AlgoData, velas, spread real y calidad del feed

La zona **Datos** de la ventana, en BIBLIOTECA. Enseña lo que guarda `~/Desktop/AlgoData` sin abrir ni un fichero a mano: qué carpetas hay, cuántos ficheros tienen y de cuándo son; las velas de cada activo; y los dos estudios del paso 4 que describen el activo antes de construir nada — el spread real de Darwinex y la calidad del feed M1.

Qué pregunta responde

- **¿Qué tengo en AlgoData y qué está viejo?** Cada carpeta hasta tres niveles, con sus ficheros, la fecha de lo último que se escribió y cuántos exports firmados (`manifest.json`) cuelgan de ella.
- **¿Qué pinta tiene el precio de este activo?** Sus velas a D1, H4 o H1, del último año (lo que se abre por defecto), de tres meses, de tres o diez años o de toda la historia.
- **¿Cuánto cuesta de verdad el spread aquí, y es constante?** El informe del spread real, tal y como lo escribió `studies.data.spread.scan` , y su banda para el MC Retest (`studies.data.spread.bands`).
- **¿Es fiable el feed M1 con el que construyo?** El informe de calidad del feed, tal y como lo escribió `studies.data.feedQuality.scan` : picos, congelados y huecos por año, y desde qué año es estable.

Cuándo lo usas, y cuándo no

Lo usas antes del paso 1 y en el paso 4: para ver si un activo tiene barras, ticks de Darwinex y calidad escaneada antes de montar un proyecto; para mirar qué exports hay y de qué fecha antes de pedir uno nuevo; y para saber qué carpeta está creciendo.

No lo usas para calcular nada. La zona **no corre ningún estudio, no exporta velas y no toca SQX**: lee lo que ya está en disco. Si un activo no tiene su informe de spread o de calidad, la zona dice qué comando lo escribe, y ese comando se corre aparte. Tampoco sustituye a la preflight (`python3 -m core.assets <SÍMBOLO>`), que es la que para el trabajo cuando algo falta.

Antes de empezar

- La ventana y su demonio (capítulo 35). No hace falta SQX ni ningún worker.
- Las velas salen de la librería de barras (`AlgoData/bars/<feed>/M1.parquet` , capítulo 13). Si un activo no está en ella, la pestaña Velas lo dice.
- El spread y la calidad salen de sus carpetas: `AlgoData/spread/<feed de ticks>/` y `AlgoData/feedQuality/<feed>/` . Los escriben los capítulos 59 y 57.

Cómo se ejecuta

```
algoui
```

En la barra de la izquierda, **BIBLIOTECA › Datos** (o `algoui --zone Datos`). Para verla sin la ventana entera, o sacar sus capturas sin pantalla, la zona se abre sola con el demonio ya arrancado (`python3 -m ui.daemon.serve`):

```
python3 -m ui.desktop.datazone.preview --asset XAUUSD --tab Velas
```

flag	obligatorio	qué hace
`--asset`	no	el activo que se abre; por defecto XAUUSD
`--tab`	no	la pestaña que se abre: `Catálogo`, `Velas`, `Spread real`, `Banda del spread`, `Calidad del feed`
`--shot DIR`	no	guarda una captura PNG de cada pestaña en esa carpeta y sale (para comprobar la zona sin pantalla, con `QT_QPA_PLATFORM=offscreen`)

Abrir la zona tarda menos de un segundo. La primera vez que pides un timeframe de un activo, el demonio lo remuestrea del M1 (~1 s) y lo guarda en `barsDerived/`; las siguientes lo lee de ahí. El H1 de toda la historia de XAUUSD son 140 000 velas: tarda medio segundo en llegar.

Qué produce

Nada propio. Lo único que se escribe es la caché de la librería de barras: un timeframe pedido por primera vez queda en `AlgoData/barsDerived/<feed>/<TF>-<huella>.parquet`, como cuando lo pide cualquier estudio. Esa carpeta se puede borrar entera: se regenera al leer.

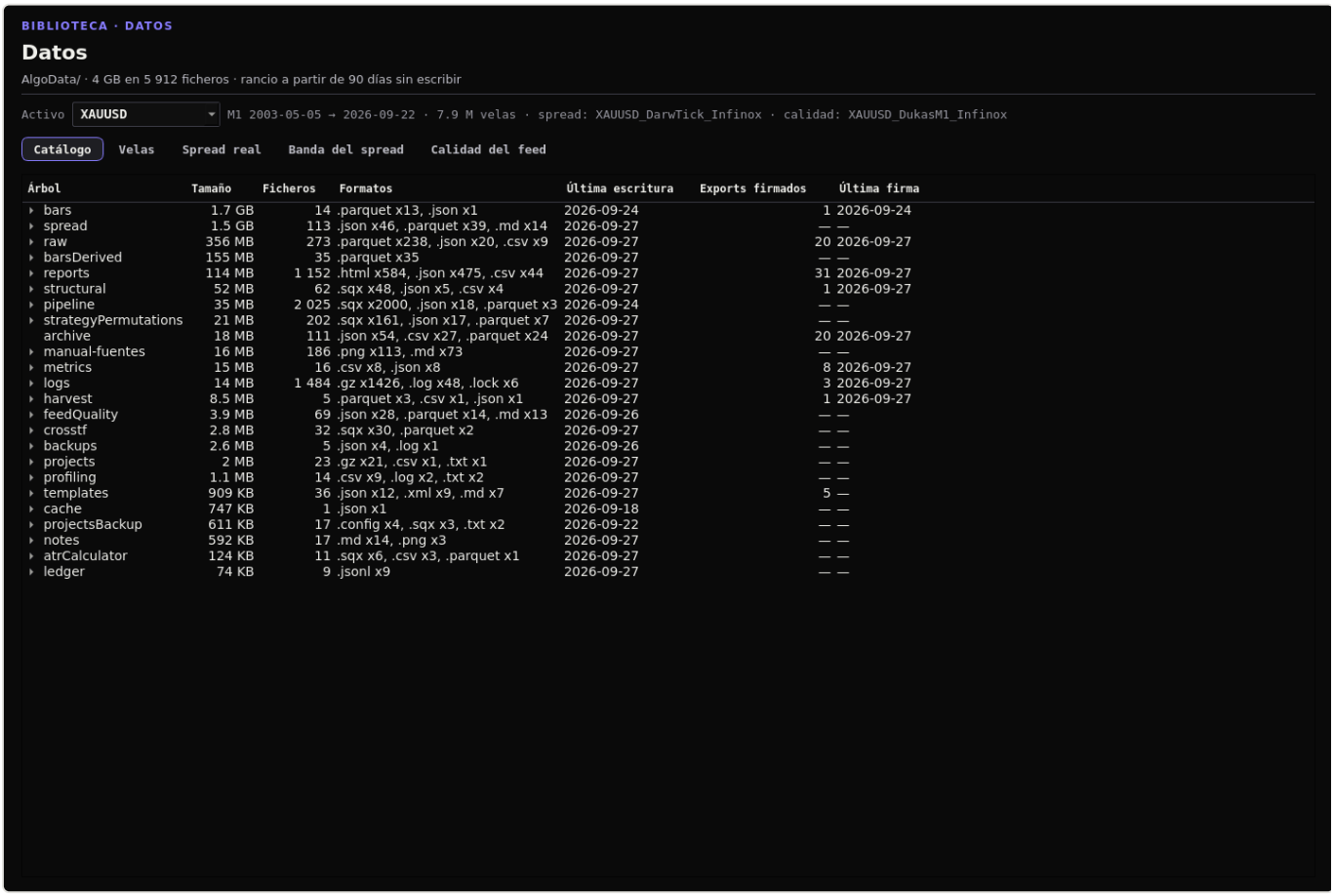
El demonio sirve cinco rutas, por si quieres mirarlas con `curl`:

ruta	qué devuelve
`/api/data/catalogue`	cada carpeta de AlgoData con ficheros, formatos, última escritura y exports firmados
`/api/data/assets`	cada activo con su feed de barras, de ticks y de calidad
`/api/data/bars?feed=…&tf=D1&since=AAAA-MM-DD`	las velas OHLC de un feed a D1, H4 o H1
`/api/data/spread?feed=…&part=spread` (o `band`)	el informe del spread real, o su banda
`/api/data/feedquality?feed=…`	el informe de calidad del feed

Cómo se lee el resultado

Arriba, siempre: `AlgoData/` y cuántos ficheros tiene (la ventana ya no enseña tamaños en disco), y el umbral de «rancio» (90 días sin escribir, de `perf/config.yaml`). Debajo, el selector de activo y una línea con lo que hay de él: desde cuándo hasta cuándo tiene M1 y cuántas velas, y qué feed usa para el spread y para la calidad («—» si no tiene).

Catálogo. Una fila por carpeta, anidadas; se despliegan con la flecha. «Última escritura» en ámbar quiere decir que nadie ha escrito ahí en más de 90 días. «Exports firmados» cuenta los `manifest.json` de debajo: una carpeta de datos sin ninguno no se puede reproducir. Al pasar el ratón por el nombre sale la ruta y, si la carpeta tiene su propio manifest, el comando que la produjo. Las carpetas del archivo de estrategias van sin desglosar: su nombre es la identidad de la estrategia, y la ventana nunca la enseña.



Velas. Se abre en «1 año», donde a D1 caben las velas. Cuando cada vela cabe en al menos tres píxeles se pintan velas: azul la que sube, naranja la que baja (verde y rojo están reservados a los veredictos). Cuando no caben — toda la historia a D1 son 6 088 velas — cada columna de la gráfica resume las suyas: la línea gris es su rango de máximo a mínimo y la blanca el cierre de la última. La leyenda de debajo dice cuál de los dos dibujos estás viendo. Al pasar el ratón sale la vela, o el tramo de velas de esa columna, con su apertura, máximo, mínimo y cierre.

BIBLIOTECA · DATOS

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo **XAUUSD** M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo | **Velas** | Spread real | Banda del spread | Calidad del feed

Timeframe **D1** Periodo **1 año**

XAUUSD_DukasM1_Infinox · D1 · 259 velas · 2025-09-22 00:00 → 2026-09-22 00:00 · remuestreadas del M1 de la librería, idénticas al export de SQX



vela que sube vela que baja

BIBLIOTECA · DATOS

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo **XAUUSD** M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo | **Velas** | Spread real | Banda del spread | Calidad del feed

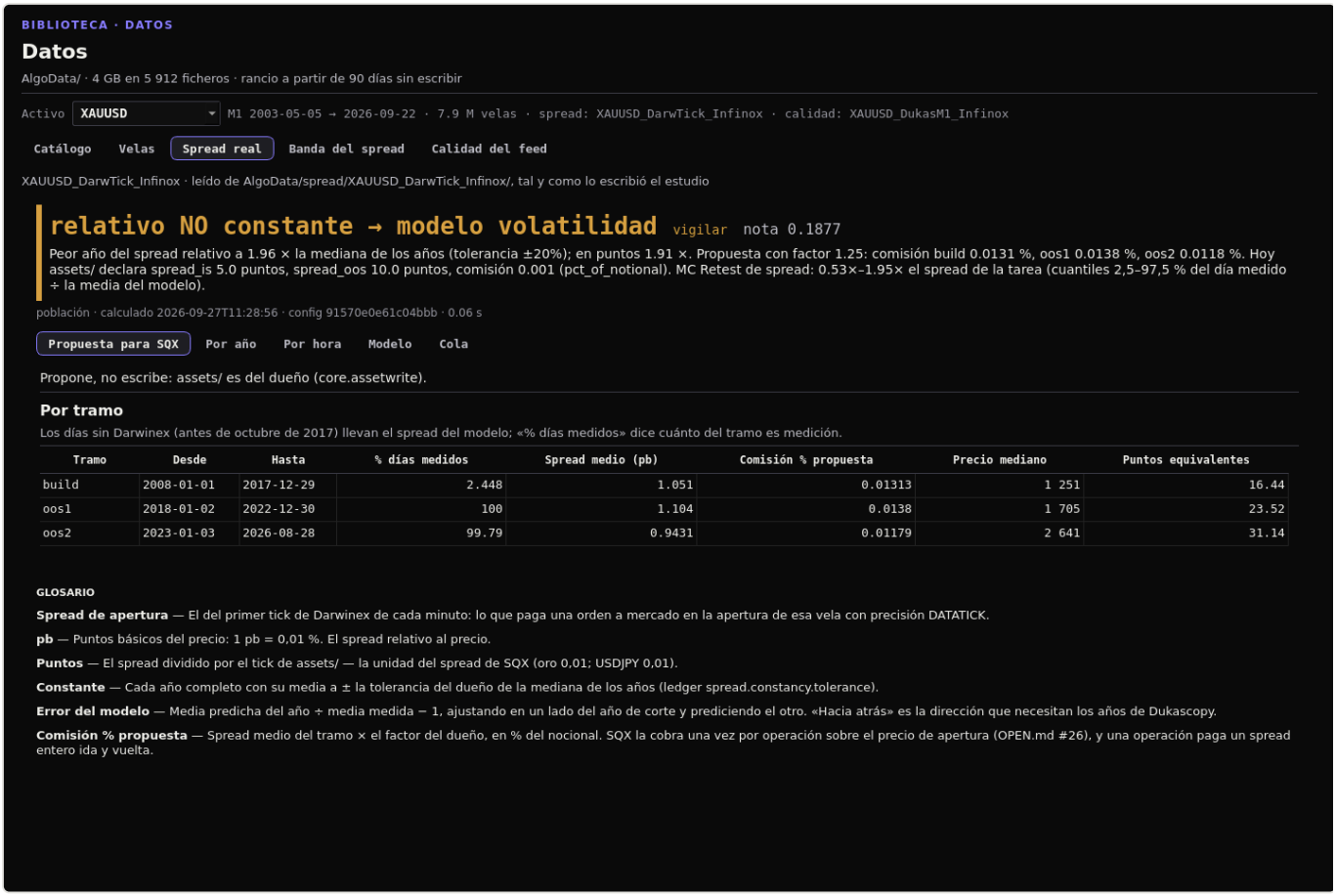
Timeframe **D1** Periodo **toda la historia**

XAUUSD_DukasM1_Infinox · D1 · 6 088 velas · 2003-05-05 00:00 → 2026-09-22 00:00 · remuestreadas del M1 de la librería, idénticas al export de SQX



rango máximo–mínimo de las velas de cada columna cierre de la última vela de la columna

Spread real, Banda del spread, Calidad del feed. El informe entero del estudio, con su veredicto arriba, sus pestañas, sus avisos y su glosario, pintado igual que un estudio en la ficha de Estrategia (capítulo 58). La línea de encima dice de qué carpeta se ha leído. Cómo se lee cada informe está en su capítulo: 59 (spread real y banda) y 57 (calidad del feed).



BIBLIOTECA · DATOS

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo **XAUUSD** M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo | Velas | Spread real | **Banda del spread** | Calidad del feed

XAUUSD_DarwTick_Infinox · leído de AlgoData/spread/XAUUSD_DarwTick_Infinox/, tal y como lo escribió el estudio

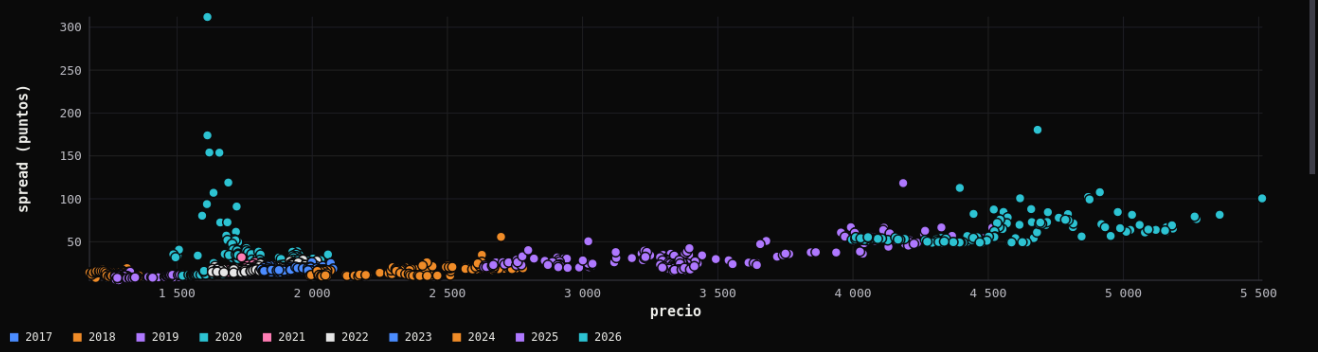
MC Retest de spread hoy: 17.8 – 83.4 puntos informa

Al último precio (4,274.5) la curva media da 49.2 puntos y la banda 0.025–0.975, 17.8 a 83.4. Exponente de la media b = 1.18 (1 = proporcional al precio, 0 = fijo en puntos). Peor año fuera de la banda por un lado: 18 % de sus días (lo esperado, 2,5 %).

población · calculado 2026-09-27T10:48:00 · config f2bb399635cae4ba · 0.01 s

Curva | Rangos para el MC Retest | Calibración

Spread medio de cada día contra el precio



Curvas ajustadas: media y banda



BIBLIOTECA · DATOS

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo **XAUUSD** M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo | Velas | Spread real | Banda del spread | **Calidad del feed**

XAUUSD_DukasM1_Infinox · leído de AlgoData/feedQuality/XAUUSD_DukasM1_Infinox/, tal y como lo escribió el estudio

estable desde 2014 pasa nota 16

K = 25; 16 anomalías marcadas por año (mediana desde 2013); huecos propios 11 al año (sesión bien deducida); 10 meses de episodio en congelados o huecos (0 desde el año estable) y 21 en el rollover, que sólo se cuentan.

población · calculado 2026-09-26T19:41:23 · config 6fdf8fe2dc54145b · 0.08 s

Por año | Por mes | Los más extremos | Calibración

Recuento por año

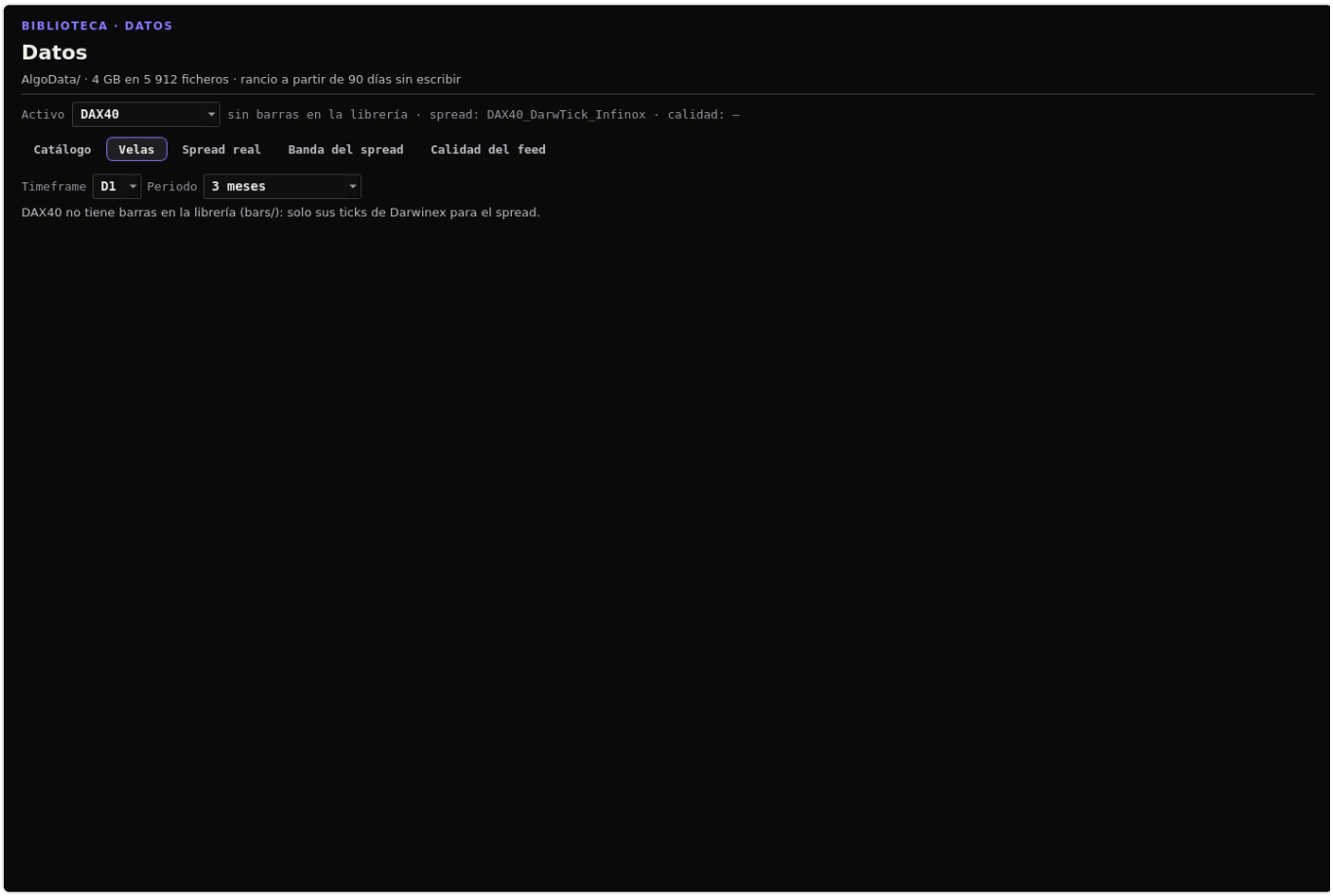
«marcadas» suma lo que la atribución del paso 8 lee.

Año	Cierre	Mecha	Cierre y mecha	Cierre y vuelta	Mecha y vuelta	Vuelta m=1	Vuelta m=5	Movimiento extremo
2003	0	0	0	0	0	0	0	
2004	0	0	0	0	0	0	0	
2005	0	0	0	0	0	0	0	
2006	0	0	0	0	0	0	0	
2007	0	0	0	0	0	0	0	
2008	26	4	0	2	2	1	1	
2009	0	0	0	0	0	0	0	
2010	0	0	0	0	0	0	0	
2011	27	0	0	2	0	0	6	
2012	8	2	1	0	0	0	0	
2013	172	18	5	16	2	11	18	
2014	37	7	2	0	0	0	0	
2015	36	5	2	1	3	0	2	
2016	57	10	2	7	2	4	8	
2017	10	5	1	2	1	1	0	
2018	4	1	0	0	1	0	0	

GLOSARIO

Un ejemplo completo

1. `algoui` → **Datos**. Arriba: «AlgoData/ · 5 909 ficheros · rancio a partir de 90 días sin escribir». En el Catálogo, `bars` y `spread` son las carpetas con más ficheros; `spread` no tiene ningún export firmado.
2. Activo **XAUUSD**: «M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox». Tiene las tres cosas.
3. **Velas**, D1, «1 año» (lo que abre): 259 velas del 2025-09-22 al 2026-09-22; el oro sube de 3 750 a 5 400 a finales de enero y vuelve a 4 370.
4. **Spread real**: «relativo NO constante → modelo volatilidad», en ámbar (vigilar). La pestaña «Propuesta para SQX» da la comisión por tramo: build 0.01313 %, oos1 0.0138 %, oos2 0.01179 %.
5. **Calidad del feed**: «estable desde 2014», en verde.
6. Cambio el activo a **DAX40**: «sin barras en la librería · spread: DAX40_DarwTick_Infinox · calidad: —». Velas dice que no hay barras; el spread sí se puede leer.



Qué NO te dice

- **Si los datos están al día con SQX.** Las velas son las de la última sincronización de la librería (`bars/manifest.json`); si SQX ha actualizado su historia después, aquí no se ve. Eso lo hace el capítulo 25.
- **Si un informe de spread o de calidad está caducado.** Se enseña el último que se escribió, con su fecha y su firma de configuración en la línea gris del veredicto; si se cambió la configuración del estudio después, hay que volver a correrlo.
- **Qué se puede borrar.** Una carpeta grande o rancia no es una carpeta sobrante. Para eso está `python3 -m perf.disk.report` (capítulo 12), que además tampoco borra nada.
- **Un «activo» es el nombre del feed hasta el primer guion bajo.** Si algún día hubiera dos feeds de barras del mismo símbolo, la zona usaría el primero en orden alfabético.

Si algo falla

- **«El demonio no respondió a /api/data/…»**: el demonio no está arrancado. `algoui` lo arranca solo; con `preview`, arráncalo antes con `python3 -m ui.daemon.serve`.
- **«XAUUSD_… no tiene spread.json todavía. Lo escribe: python3 -m studies.data.spread.scan --symbol XAUUSD»**: ese estudio no se ha corrido para ese activo. Corre el comando que dice.
- **«No pude leer las velas de … »**: la librería de barras no tiene ese feed o su M1 está roto. Mira el capítulo 13.

60. Cómo se lee la ventana — etiquetas, números, botones, identidad y modo lectura

Qué pregunta responde

- **¿Qué significa esta columna?** Toda etiqueta que la ventana enseña sale de un único glosario: «Beneficio neto», «Murió en», «Nulo · corridas», siempre con mayúscula inicial. Si un estudio nuevo trae una clave que el glosario aún no tiene, la ventana la humaniza («null.chunk3_traits» se lee «Nulo chunk3 traits») en vez de enseñarla cruda.
- **¿Cuánto es este número?** Ninguna cifra sale en notación científica. Se escribe entera, con los miles separados por un espacio («12 345»), y desde 100 000 con «K» o «M» («123.5 K», «2.4 M»). Un p-valor se lee «0.0123», y por debajo de 0.0001, «< 0.0001».
- **¿Qué se puede pulsar, y qué hace cada botón?** Desde el 28-09-2026 todo botón que se puede pulsar se ve encendido —borde violeta y un relleno que se aclara hacia arriba— y brilla más al pasar el ratón; uno que no se puede pulsar sale apagado, con el texto gris y el borde a rayas. Al lado de cada botón hay un **«?»**: al pasar el ratón por encima (o al pulsarlo) dice en una o dos frases qué hace ese botón, si escribe algo, si pregunta antes y si toca SQX.
- **¿Esta máquina puede lanzar algo en SQX?** Si no hay ningún install de SQX, arriba de todo sale en ámbar **«Esta máquina no tiene SQX: modo lectura»**, y el ↻ de la barra de carga queda desactivado diciendo por qué en su propio texto.

Cuándo lo usas, y cuándo no

No es un comando que se corra: es cómo se ve la ventana desde el 27-09-2026. Lo usas para saber qué esperar al leerla. Lo único que se «ejecuta» es la captura (`--shot`), que sirve para ver la ventana sin abrirla, por ejemplo desde una sesión sin pantalla.

No sirve para Windows: la ventana solo se usa en el servidor Linux (decisión del dueño, 27-09-2026). El modo lectura es un guarda por si falta SQX, no una versión para otro sistema.

Antes de empezar

Nada. La ventana se abre como siempre, con `bin/algoui` o el icono del escritorio.

Cómo se ejecuta

La ventana, como siempre:

```
bin/algoui
```

Una captura de la ventana sin abrirla (guarda un PNG y se cierra sola):

```
QT_QPA_PLATFORM=offscreen python3 -m ui.desktop.launch --zone "Registro de búsquedas" --port 8756 --
shot /tmp/capturas
```

flag	obligatorio	qué hace
`--zone`	no	la zona que se abre, por su nombre en la barra lateral; sin él, «Cobertura»
`--shot`	no	carpeta donde guardar la captura `<zona>.png`; la ventana se cierra a los 2,5 s
`--port`	no	el puerto del demonio; sin él, `ui_port` de `config/machine.yaml` (8765)

Tarda unos 5 segundos. No toca SQX: solo lee del demonio. Si el demonio no está levantado, lo arranca y lo para al terminar.

⚠ **Para capturas y pruebas, pasa siempre otro `--port`** (por ejemplo `--port 8756`). El 8765 es el demonio de tu ventana: si el que está ahí corre código más viejo y está ocioso, la ventana nueva lo sustituye, y una captura sin `--port` te lo cambiaría por debajo.

Para saber qué installs ve el demonio:

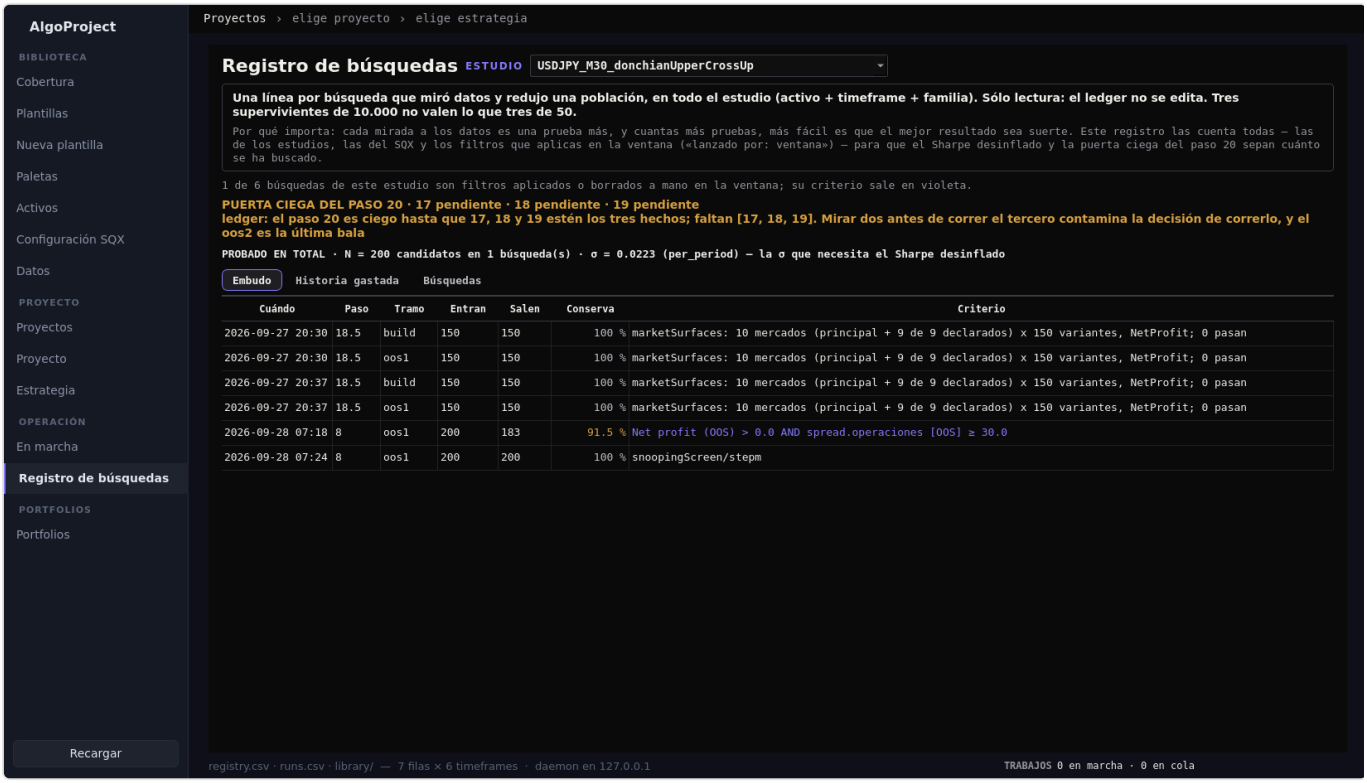
```
curl -s 127.0.0.1:8765/api/health
```

Qué produce

- La captura: `<carpeta>/<zona>.png` (los espacios del nombre pasan a `_`: el Registro de búsquedas se guarda como `Registro_de_búsquedas.png`). Sobrescribe una captura anterior de la misma zona en la misma carpeta.
- `/api/health` devuelve, además de lo de siempre, el bloque `sqx`: `{"installs": {"conductor": true, "custodian": true, "master": true}}` — cada install de `config/machine.yaml` y si su carpeta existe en esta máquina.

Cómo se lee el resultado

Las pestañas activas son un recuadro redondeado con el borde violeta, no una raya debajo. El violeta es el color de acento, y no significa nada: verde, ámbar y rojo siguen siendo pasa, vigilar y muere.



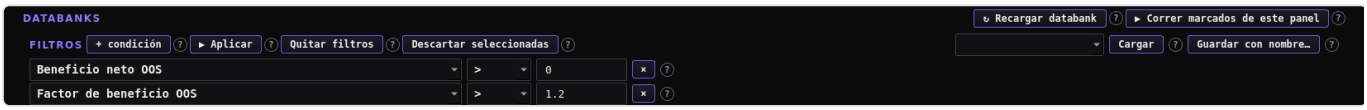
Las etiquetas salen del glosario. En el panel de la zona Databanks las columnas dicen «Estrategia», «Beneficio neto IS», «Operaciones IS», «Factor de beneficio IS», «% ganadoras IS»; en la ficha de una estrategia, las métricas de SQX salen en castellano y el nombre original de SQX aparece al pasar el ratón.



Los botones. Un botón que se puede pulsar lleva el borde violeta y un relleno algo más claro arriba que abajo, como si tuviera luz; al pasar el ratón el borde y el relleno se aclaran, y al pulsarlo se oscurece. El puntero se vuelve una mano. Los botones de color propio (los ▶ PY y ▶ SQX de las tarjetas del workflow, «Correr workflow») conservan su color, con la misma idea. Un botón que ahora no se puede pulsar pierde el violeta: texto gris, fondo vacío y borde a rayas — como los «Aplicar» de los tramos de un activo mientras no mueves ninguna fecha.

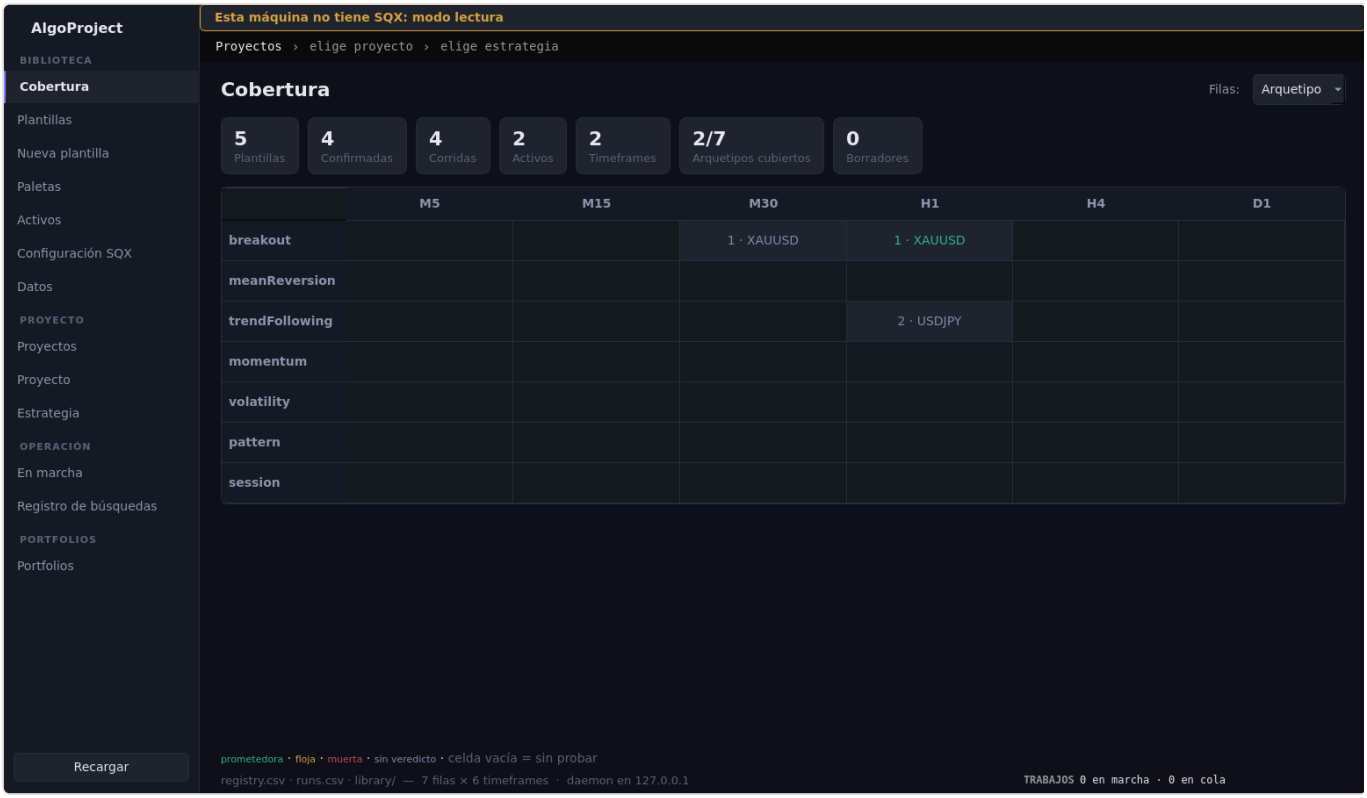


El **«?»** redondo va justo a la derecha del botón; si ahí no cabe (otro control pegado), va sobre el borde derecho del propio botón cuando le sobra sitio, y si tampoco, no sale y queda la nota del propio botón al pasar el ratón. Nunca ensancha un botón. Al pasar el ratón por encima se pone violeta y sale la explicación; si además el botón trae su propia nota —por qué está apagado, qué valor elegiría «Elige tú»— sale debajo, salvo que diga casi lo mismo: entonces sale una sola vez. Pulsar el «?» enseña la misma explicación y **nunca** pulsa el botón. No llevan «?» los botones de la barra lateral ni las migas de arriba (solo navegan), las cabeceras plegables de Configuración SQX, ni los botones de los cuadros de diálogo.



La identidad (la huella SHA-256 de la estrategia) no se enseña en ningún texto de la ventana: ni en la barra de contexto, ni en la cabecera de un estudio, ni en las tablas. Sigue ahí por debajo — con ella se emparejan los resultados — y se ve al pasar el ratón sobre el nombre de la estrategia en la barra de contexto.

El modo lectura es la franja ámbar de arriba. Todo se sigue leyendo; el ↻ de la barra de carga (que reintenta cargar operaciones en el conductor) queda desactivado, con el texto «↻ sin SQX: modo lectura». Ninguna zona se pone gris. El otro botón que llega a SQX, «Continuar workflow», no depende de la franja: su propia comprobación (capítulo 69) se niega si el proyecto no vive en un worker de esta máquina.



Un ejemplo completo

El 27-09-2026 se probó el modo lectura con un `config/machine.yaml` de prueba que apuntaba los tres installs a carpetas que no existen (y el demonio a otro puerto, para no molestar al de siempre). El demonio respondió:

```
{"ok":true,"module":"templates","code":"0cec582189c36056","pid":2236596,"busy":0,
 "sqx":{"installs":{"conductor":false,"custodian":false,"master":false}}}
```

Con el `machine.yaml` real:

```
{"ok":true,"module":"templates","code":"0cec582189c36056","pid":2236895,"busy":0,
 "sqx":{"installs":{"conductor":true,"custodian":true,"master":true}}}
```

y la franja no aparece. La captura del modo lectura de arriba se rehízo el 28-09-2026 sobre la ventana definitiva, pasándole a la ventana esa misma respuesta —los tres installs en `false`— sin tocar el `machine.yaml` de verdad.

Qué NO te dice

- **Que un install esté vivo.** «true» solo dice que su carpeta existe. Un worker parado sale igual de «true»; saber si responde es cosa de la zona **En marcha** (capítulo 70).
- **Que el «?» lo sepa todo.** Explica qué hace el botón, no qué pasará con tus datos concretos; lo de «¿Confirmas?» de los botones que lanzan en SQX sigue diciéndolo su propia pantalla de confirmación. Las frases viven en `ui/text/buttonhelp_texts.py`; un botón nuevo sin frase ni nota propia sale sin «?», y la prueba `tests/test_ui_helpmark.py --port <puerto>` lo señala.

- **Que todas las claves estén ya en el glosario.** Una clave que un estudio nuevo trae y el glosario aún no tiene sale humanizada («Nulo chunk3 traits»), no traducida; se arregla añadiéndola al glosario (`ui/text/glossary.py`).

Si algo falla

- **La captura sale en negro o a medio pintar:** alguna zona tardó más de 2,5 s en leer del demonio. Repite la captura; la segunda vez el demonio ya tiene la caché caliente.
- **«el demonio no respondió en el puerto 8765»:** arráncalo a mano con `python3 -m ui.daemon.serve` para ver el error.

72. Crear desde la ventana — la plantilla, el proyecto, y el export al terminar cada tarea

Qué pregunta responde

Cómo se pasa de una idea escrita a un proyecto de SQX listo para construir **sin salir de la ventana**, y por qué, al acabar cualquier tarea de SQX, sus estrategias aparecen ya exportadas en el databank sin que pulses nada.

Cuándo lo usas, y cuándo no

- **«▶ Crear la plantilla con Claude»** (al final del chat de *Nueva plantilla*): cuando has contestado las ocho preguntas y quieres que la plantilla se cree sola. No sirve si la idea tiene dos lecturas: el trabajo no elige, termina con una línea `PREGUNTA:` y te toca contestarla en una sesión de Claude.
- **«+ Nuevo proyecto»** (arriba en *Proyectos*): cuando la plantilla ya está en la librería y quieres probarla en un activo y un timeframe. No arranca SQX: deja el proyecto creado en el custodio y la construcción se lanza después con «▶ Lanzar en SQX» o «▶ SQX» del paso 6.
- **El export al terminar** no se usa: ocurre solo, cada vez que un worker se para.

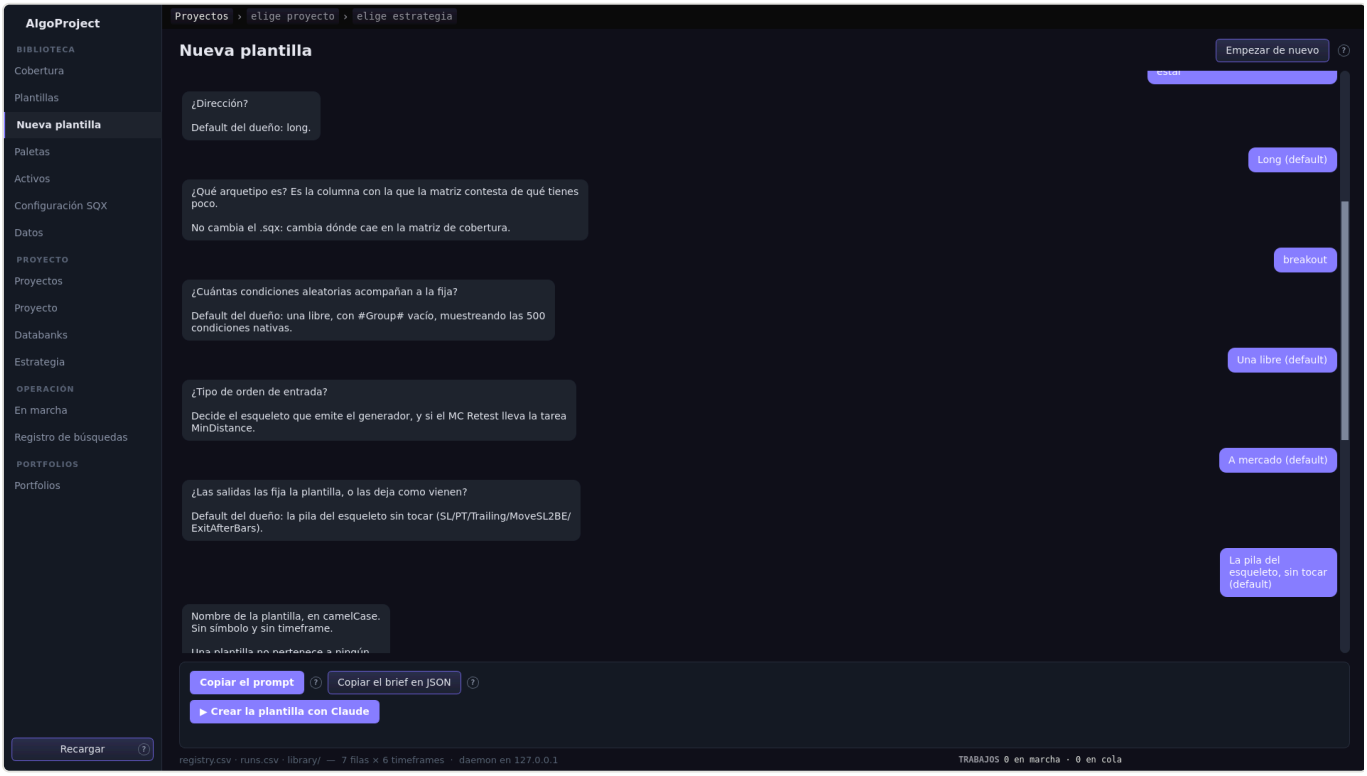
Antes de empezar

- Para la plantilla: el chat terminado (su borrador está en `AlgoData/templates/drafts/`). Conductor y custodio parados: la skill instala bloques en los dos.
- Para el proyecto: la plantilla en la librería con su `template.sqx`, y el activo con sus costes decididos — antes de crear nada corre `core.assets` (regla 5) y, si no pasa, no crea nada.

Cómo se ejecuta

La plantilla. *Nueva plantilla* → contesta las preguntas → ▶ **Crear la plantilla con Claude** → «Sí». Por dentro:

```
python3 -m ui.daemon.create.author --name donchianUpperBreakUp
```

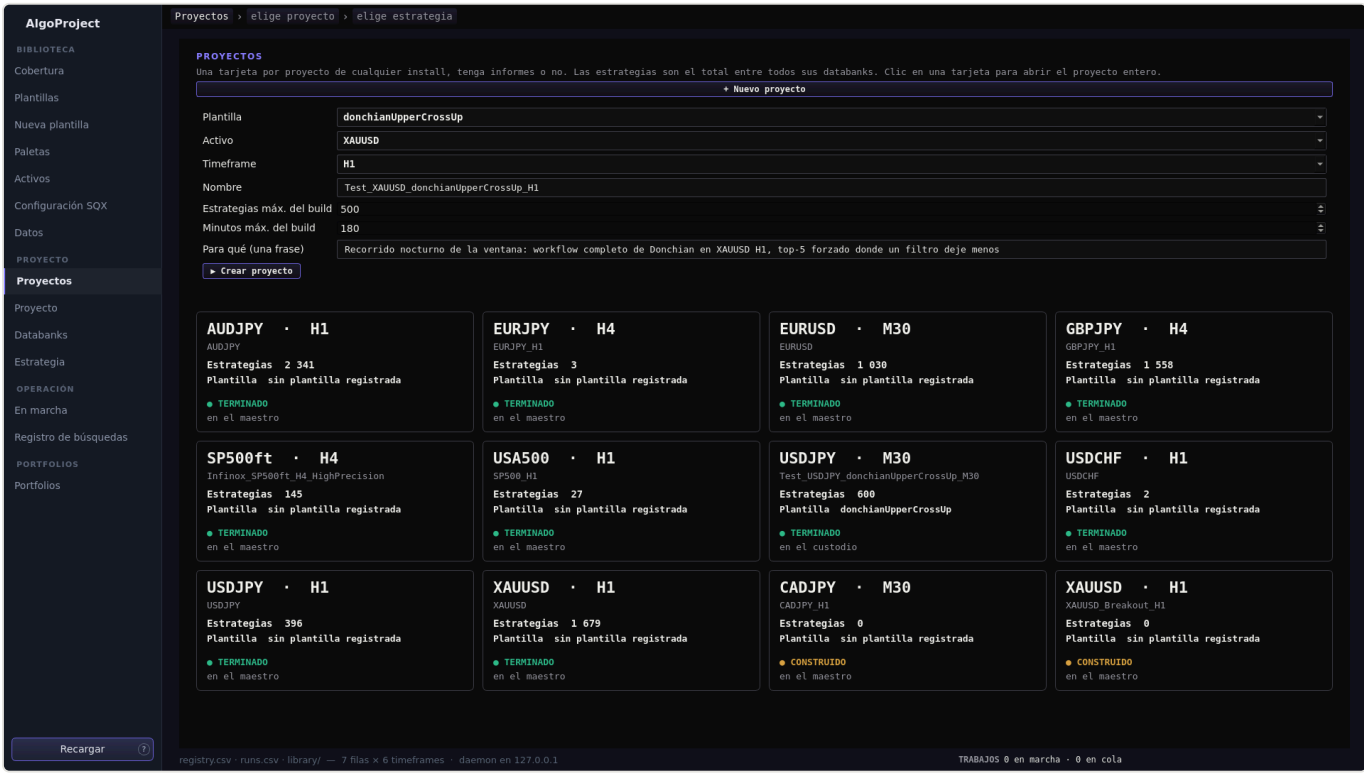


Corre Claude Code sin nadie delante (`claude -p … --permission-mode auto`) con la skill `/sqx-strategy-template`. Tarda de 2 a 10 minutos. Bajo el botón aparece el estado del trabajo y, al terminar, una de estas líneas:

línea	qué significa
`PLANTILLA: <nombre>`	la plantilla que queda en la librería
`YA EXISTE: …`	la idea es la misma que una plantilla que ya tienes: no se duplica y se usa esa
`PREGUNTA: …`	había dos lecturas; el trabajo sale con código 2 y no crea nada

El proyecto. *Proyectos* → **+ Nuevo proyecto** → plantilla, activo, timeframe (el nombre se propone solo: `Test_<ACTIVO>_<plantilla>_<TF>`), estrategias máximas del build (500), minutos máximos (180) y una frase de para qué → **▶ Crear proyecto** → «Sí».

```
python3 -m ui.daemon.create.project Test_XAUUSD_donchianUpperCrossUp_H1 --template
donchianUpperCrossUp \
  --symbol XAUUSD --timeframe H1 --max-strategies 500 --minutes 180 --purpose "..."
```



campo	obligatorio	qué hace
Plantilla	sí	una de la librería con `template.sqx`
Activo	sí	uno de `assets/symbols/`
Timeframe	sí	M15, M30, H1 o H4
Nombre	sí	`Test_…` (prueba, se retira), `Trade_…` (de verdad) o `Research_…` (la cola de investigación; se queda hasta que tú digas); otro nombre se rechaza
Estrategias / minutos	sí	los topes del build
Para qué	sí	una frase; va a `registry.csv`

Tarda unos segundos y no toca SQX.

El export al terminar. Cada vez que un worker se para (`bin/sqx-worker.sh stop` , lo lance la ventana, una skill o un agente), corre:

```
python3 -m ui.daemon.loader.afterrun --role custodian
```

Exporta de cada proyecto `Test_` / `Trade_` / `Research_` de ese install, databank a databank, lo que falte o esté viejo: métricas, operaciones y la cosecha. Mientras exporta, la ventana dice que SQX está escribiendo ese install y no pide lo mismo dos veces. `ALGO_NO_EXPORT=1` lo salta; una cancelación desde la ventana también lo salta, para que el worker se pare a tiempo.

Qué produce

- La plantilla: `AlgoData/templates/library/<nombre>/` y su fila en `registry.csv` , y los bloques instalados en conductor y custodio. El log del trabajo en `AlgoData/logs/ui/…-crear plantilla.log` .
- El proyecto: `SQX_w2/user/projects/<nombre>/` con **todas** las tareas del workflow (solo Build y OOS activas) y su fila en `AlgoData/projects/registry.csv` .
- El export: `AlgoData/metrics/<P>/<D>/` , `raw/<P>/<D>/<día>/` y `harvest/<P>/<D>/<día>/` , lo mismo que al elegir un databank en la ventana.

Cómo se lee el resultado

Verde bajo el botón = terminó bien. Rojo = falló, y las últimas líneas del log dicen por qué. Un `YA EXISTE` no es un fallo: sigue con la plantilla que nombra.

Errores típicos

- **«no hay borrador»**: el chat no se terminó; contéstalo hasta el final.
- **El nombre del proyecto se rechaza**: no empieza por `Test_` / `Trade_` / `Research_` o lleva espacios.
- **«core.assets salió con 1»**: el activo tiene costes sin decidir. Arréglalo en *Activos*.
- **«▶ Lanzar en SQX» no se deja pulsar tras crear el proyecto** — el custodio está ocupado: su candado de propietario (`user/log/OWNER` , OPEN §32) lo tiene otra sesión, su puerto responde o hay un proceso de SQX vivo. El mensaje dice quién lo tiene y desde cuándo; espera o confirma con esa sesión. Que otro proyecto se haya tocado hace un rato ya no bloquea nada (OPEN §83): en cuanto esa tarea termina, lanzar la tuya está bien.

Ver también

Capítulos 61 (Proyectos), 65 (Proyecto), 35 (Plantillas) y 28 (builder).

95. CRITERIOS — qué tiene que cumplir una estrategia en cada paso para pasar sola

Qué pregunta responde

¿Con qué reglas decide el autopilot que una estrategia sigue adelante, y puedo cambiarlas sin abrir un fichero? La vista **Proyecto › Criterios** enseña las reglas de cada paso del workflow (las que están en `pipeline/autopilot/criteria.yaml`) y te deja añadir, cambiar o borrar una. No es un sistema aparte: es una ventana sobre lo que ya juzga.

Cuándo la usas, y cuándo no

La usas para ver de un vistazo qué exige cada paso, o para endurecer o aflojar una regla. No sirve para tocar los filtros del Build (paso 6): esos los escribe el builder y los aplica SQX; aquí solo se leen.

Antes de empezar

Nada: no necesita SQX ni lanza nada. Se abre desde la barra lateral, en Proyecto.

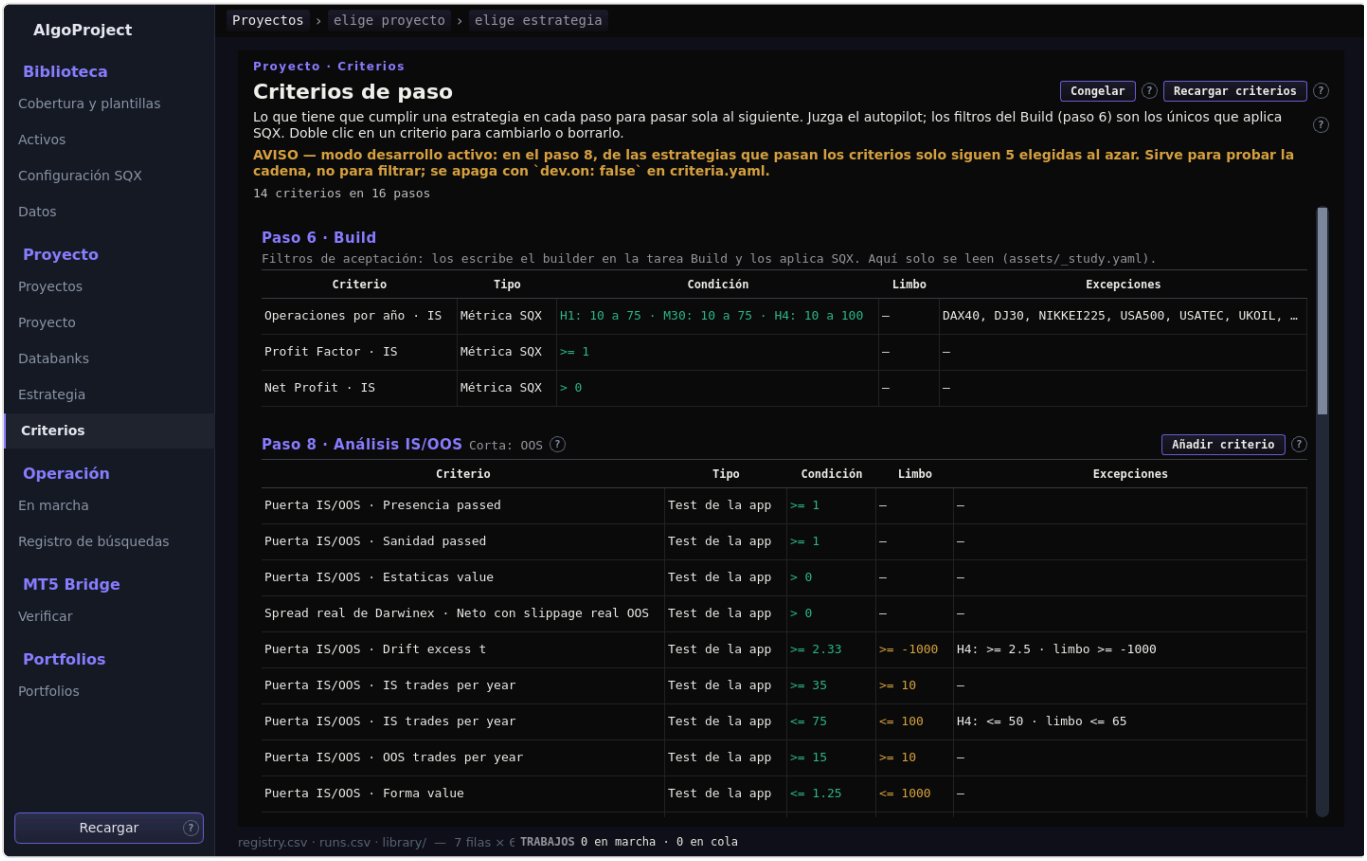
Cómo se ejecuta

```
bin/algoui --zone Criterios
```

Qué produce

Cada cambio añade una línea a `ledger/criteria-history.jsonl` (cuándo, quién, qué paso, qué había y qué hay ahora) y reescribe `pipeline/autopilot/criteria.yaml` conservando sus comentarios.

Cómo se lee el resultado



Captura real. El aviso naranja de «modo desarrollo» es el estado real de la configuración el día de la captura.

Arriba: el botón **Congelar**, **Recargar criterios**, un aviso si el modo desarrollo está activo (solo siguen N estrategias elegidas al azar: sirve para probar la cadena, no para filtrar) y el recuento de criterios y pasos.

El paso 6 (Build) va solo de lectura: son los filtros de aceptación que SQX aplica a la construcción.

Un bloque por paso. Cada uno dice dónde corta («Corta: OOS» es el databank que se queda sin las rechazadas) y su tabla: *Criterio* (el estudio y la cifra), *Tipo* (Métrica SQX o Test de la app), *Condición* (lo que pasa), *Limbo* (la banda en la que no se elimina) y *Excepciones* (reglas distintas por timeframe o activo). Un paso sin criterios dice «Sin criterios: pasan todas». Cada cabecera tiene su «?».

Cómo añadir, cambiar o borrar un criterio

Doble clic en una fila para cambiarla o borrarla; **Añadir criterio** (arriba a la derecha de cada paso) para uno nuevo. El diálogo pide:

campo	qué es
Tipo	Métrica SQX, Test de la app, Veredicto o Compuesto
Dato	el menú del paso (se puede escribir la clave a mano)
Umbral	comparador y número; opcionalmente una banda de limbo
Cerca de la mediana	la regla pasa si el valor está a ±% de la mediana de la población
Veredicto	pasa si el estudio dice Aprobado (o Suspenso)
Compuesto	varias condiciones juntas, un nivel de profundidad; cada una se define con el mismo diálogo
Si no cumple	**Fallo** (elimina) o **Limbo** (nunca elimina: la estrategia sigue y se anota; es el «else: limbo»)

Al guardar, la ventana comprueba la regla y rechaza la que el autopilot no sabría leer, con la frase de por qué.

El historial

Debajo de los pasos, **Historial de cambios**: las últimas 30 líneas de `ledger/criteria-history.jsonl`. Todo cambio queda ahí, con lo que había antes y lo que hay después.

Congelar

Congelar escribe `frozen` al principio de `criteria.yaml`, con la fecha y un sello del contenido. Desde ese momento la ventana **rechaza cualquier cambio**, y no hay botón para descongelar: **desde la ventana es irreversible**. Levantarlo es borrar esa línea del fichero, y git lo enseña. Si alguien edita el fichero a mano tras congelar, la vista avisa de que ya no coincide con el sello.

Qué NO te dice, y lo que aún no hace

- No dice si un criterio es *bueno*: solo qué se exige.
- Todavía no se pueden editar desde la ventana: las **excepciones** (`by`: por timeframe o activo; se conservan tal cual al editar la regla), el modo **desarrollo**, la **banda de corte** de cada paso ni los filtros del **paso 6**.

Si algo falla

- **«Añadir criterio» apagado:** el criterio está congelado, o ese paso no tiene `cut`; el «?» lo dice.
- **Una regla se rechaza al guardar:** el umbral o el compuesto no es algo que el juez sepa leer; la frase de error dice cuál.